UNIVERSIDAD AUTONOMA GABRIEL RENE MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

GRUPO 21

FASHIONSTORE: PLATAFORMA INTELIGENTE DE COMERCIO ELECTRÓNICO WEB Y MÓVIL BAJO EL MODELO SAAS MULTITENANT PARA TIENDAS DE ROPA CON GESTIÓN MULTISUCURSAL, OPERACIÓN SIN CONEXIÓN, SINCRONIZACIÓN DE DATOS Y REPORTES GENERATIVOS MEDIANTE INTELIGENCIA ARTIFICIAL

Integrantes	Carrera	Registro
- Contili Villarroel Alejandro	Ing. en Sistemas	220153361
- Delgado Rivero Klara Crisel	Ing. en Sistemas	220032572

Materia : INF412 – Sistemas de Información II

Grupo : SA

Docente : MSc. Ing. Cuellar Garzón Angelica

Semestre : 2 – 2026

Santa Cruz de la Sierra – Bolivia

Gestión 2 – 2026

Contenido

1. CAPITULO I. ASPECTOS GENERALES 6
1.1. Introducción 6
1.2. Planteamiento del problema 7
1.3. Objetivos del Proyecto 9
Objetivo General 9
Objetivos Específicos 9
1.4. Justificación 11
1.5. Alcance y limitaciones 12
1.6. Metodología de desarrollo ICONIX 15
2. CAPITULO II. ANÁLISIS DE REQUISITOS 16
2.1. Descripción del Sistema Propuesto 16
2.2. Identificación de actores 17
2.3. Requisitos funcionales y no funcionales 18
Requisitos Funcionales 18
Requisitos No Funcionales 20
Reglas de Negocio 21
2.4. Diagrama UML de casos de uso 22
Especificación de casos de uso 24
3. CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX 39
3.1. Modelo de dominio 39
3.2. Diagramas de robustez 39
CU01 - Gestionar acceso y cuenta 39
CU02 - Administrar empresas, planes y suscripciones 40
CU03 - Administrar personal, roles y permisos 40
CU04 - Gestionar ciudades y sucursales 41
CU06 - Administrar catálogo de prendas 42
CU07 - Gestionar proveedores, compras y recepciones 42
CU08 - Consultar inventario y disponibilidad 43
CU09 - Explorar catálogo y asistencia de compra 43
CU10 - Registrar movimientos de inventario 44
CU11 - Realizar compra y pago en línea 44

CU12 - Gestionar reservas de prendas 45
CU13 - Registrar venta presencial 45
CU14 - Obtener comprobantes de compra 46
CU15 - Gestionar cambios y reembolsos 46
CU16 - Registrar y consultar operaciones offline 46
CU17 - Sincronizar operaciones y resolver conflictos 47
CU18 - Obtener reportes comerciales 47
CU19 - Preparar y consultar analítica inteligente 47
CU20 - Consultar auditoría y trazabilidad 47
3.3. Diagramas de secuencia UML 47
CU01 - Gestionar acceso y cuenta 47
CU02 - Administrar empresas, planes y suscripciones 49
CU03 - Administrar personal, roles y permisos 51
CU04 - Gestionar ciudades y sucursales 53
CU05 - Gestionar fichas de clientes 55
CU06 - Administrar catálogo de prendas 57
CU07 - Gestionar proveedores, compras y recepciones 59
CU08 - Consultar inventario y disponibilidad 61
CU09 - Explorar catálogo y asistencia de compra 63
CU10 - Registrar movimientos de inventario 65
CU11 - Realizar compra y pago en línea 65
CU12 - Gestionar reservas de prendas 66
CU13 - Registrar venta presencial 67
CU14 - Obtener comprobantes de compra 68
CU15 - Gestionar cambios y reembolsos 69
CU16 - Registrar y consultar operaciones offline 69
CU17 - Sincronizar operaciones y resolver conflictos 71
CU18 - Obtener reportes comerciales 71
CU19 - Preparar y consultar analítica inteligente 71
CU20 - Consultar auditoría y trazabilidad 71
3.4. Diagrama de clases UML 71
3.5. Diseño de la base de datos 73

3.6. Diseño de interfaces de usuario 75
4. CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN 76
4.1. Herramientas y tecnologías utilizadas 76
4.2. Diseño de la arquitectura del sistema 79
4.3. Implementación de la base de datos 80
4.4. Desarrollo de los módulos del sistema 198
4.5. Integración de componentes....198
5. CAPÍTULO V. PRUEBAS Y RESULTADOS 198
5.1. 5.1. Plan de pruebas....198
5.1.1. Objetivos del aseguramiento de calidad y alcance del sistema 198
5.1.2. Entornos de prueba y aislamiento de datos....199
5.1.3. Herramientas, marcos de trabajo y versiones tecnológicas....200
5.1.4. Criterios de aceptación, suspensión y reanudación de pruebas 201
5.1.5. Asignación de responsabilidades de verificación....201
5.1.6. Matriz de cobertura prevista por Caso de Uso ICONIX (CU01 a CU20) 202
5.2. 5.2. Pruebas funcionales 206
5.2.1. Metodología de derivación de pruebas desde casos de uso ICONIX 206
5.2.2. Verificación de capacidades comerciales (CU01 a CU15)....206
5.2.3. Verificación de capacidades offline, sincronización e idempotencia (CU16 y CU17) 207
5.2.4. Verificación de reportería, analítica y vestidor virtual (CU18 a CU20) 208
5.2.5. Matriz consolidada de pruebas funcionales ejecutadas....209
5.3. 5.3. Pruebas de integración 211
5.3.1. Arquitectura de integración y contratos REST / OpenAPI....211
5.3.2. Integración Web: Angular 20 ↔ FastAPI ↔ PostgreSQL RLS 211
5.3.3. Integración Móvil: Flutter ↔ FastAPI mediante túnel seguro ADB reverse 212
5.3.4. Integración de almacenamiento local persistente: IndexedDB (Dexie v2) y SQLite (sqflite 3.44.5) 212
5.3.5. Integración de servicios complementarios: Mailpit, pasarela Stripe y streaming WebRTC 213
5.4. 5.4. Corrección de errores 213
5.4.1. Registro cronológico de incidencias mayores (C1 a A6)....213
5.4.2. Correcciones de autorización y permisos POS (C1 y A1)....215

5.4.3. Corrección de sincronización y aislamiento por sucursal (A2) 215
5.4.4. Implementación y activación de traspasos de inventario de dos patas (A3) 215
5.4.5. Recuperación SQLite multicuenta y compilación de APK en Android 16 (A4) 215
5.4.6. Reconciliación en lotes de 50 y avance parcial ante desconexión (A5) 216
5.4.7. Recuperación de la suite general Angular y eliminación de 30 errores TS (A6) 216
5.4.8. Síntesis de no regresión e integridad referencial transversal 217
5.5. 5.5. Resultados de las pruebas 217
5.5.1. Resumen métrico consolidado de ejecución sin doble conteo 217
5.5.2. Desglose de la suite completa Angular (116/116 pruebas aprobadas) 218
5.5.3. Desglose de la suite completa Backend (298/298 pruebas aprobadas) 220
5.5.4. Desglose de la suite completa Móvil Flutter (70/70 pruebas aprobadas) 220
5.5.5. Resultados de pruebas de integración HTTP en Docker local (196/196 aprobadas) 220
5.5.6. Resultados de compilación y empaquetado productivo (Web y APK) 221
5.5.7. Limitaciones técnicas reconocidas y estado de aceptación 221
5.6. 5.6. Casos de prueba 222
5.6.1. Inventario formal de Casos de Prueba del Sistema (CP-CU01 a CP-CU20) 222
5.6.2. Fichas técnicas de pruebas automatizadas críticas 223
5.6.3. Casos de prueba manuales ejecutados en dispositivo físico Android (A5-AND01 a A5-AND05) 224
5.6.4. Casos de prueba de persistencia y migración en dispositivo físico (M01 a M12 de A4) 226
5.6.5. Matriz de trazabilidad integral: Requisito → Caso de Uso → Componente → Prueba → Evidencia → Resultado 227
6. CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES 228
6.1. Conclusiones 228
6.2. Recomendaciones 228
7. BIBLIOGRAFIA 228
8. ANEXOS 228
8.1. Enlace y QR de Github 228
8.2. Enlace y QR de la Página https://fashionstore.aledevcv.me/login 229

1. CAPITULO I. ASPECTOS GENERALES

1.1. Introducción

La comercialización de prendas de vestir requiere coordinar productos, variantes, inventarios, clientes y transacciones entre los canales físicos y digitales de una empresa. Cuando estas actividades se distribuyen entre varias sucursales, disponer de información organizada resulta necesario para atender solicitudes, controlar existencias y conocer el desempeño del negocio.

FashionStore surge como una plataforma web y móvil que integra estos procesos comerciales. Su primera versión estableció una base funcional para administrar una cadena de tiendas de ropa, incorporando catálogo, compras, inventarios, reservas, ventas y herramientas de asistencia al cliente. A partir de ese desarrollo, el presente proyecto propone ampliar la solución para que pueda ser utilizada por diferentes empresas mediante un modelo de software como servicio, conocido como SaaS.

Esta evolución implica atender nuevas necesidades de administración, seguridad y continuidad operativa. Cada empresa deberá gestionar sus recursos y sucursales dentro de un entorno independiente, con funcionalidades determinadas por su suscripción. Asimismo, las aplicaciones deberán permitir consultar información descargada y registrar determinadas operaciones durante interrupciones de Internet, conservándolas hasta que puedan ser verificadas y sincronizadas con el servidor.

La propuesta también comprende reportes solicitados mediante lenguaje natural y un módulo de analítica inteligente que permita explorar indicadores, predicciones

y agrupaciones de clientes. Estas herramientas buscan facilitar la interpretación de los datos comerciales, respetando los permisos y la separación de información entre empresas.

El proyecto se desarrollará tomando ICONIX como referencia metodológica y utilizando UML para representar los requisitos, las responsabilidades y las interacciones del sistema. Se mantendrá una relación verificable entre los casos de uso, el código, las pruebas y la documentación, con el propósito de obtener una plataforma demostrable y coherente con las necesidades planteadas.

1.2. Planteamiento del problema

La primera versión de FashionStore fue concebida para integrar los procesos comerciales de una sola cadena de tiendas de ropa. Aunque esta solución constituye una base reutilizable, su utilización por empresas independientes introduce necesidades que superan la administración de múltiples sucursales. Compartir una plataforma exige distinguir a qué organización pertenece cada recurso, qué operaciones puede realizar cada usuario y cuáles son las condiciones del servicio contratado.

Administración de empresas y aislamiento de información.

La incorporación de diferentes negocios dentro del mismo sistema puede generar accesos indebidos si los usuarios, productos, inventarios y documentos comerciales no se encuentran correctamente asociados a su empresa. También resulta necesario separar la administración general de la plataforma de la gestión interna de cada negocio. Sin esta distinción, un administrador empresarial podría

disponer de facultades que corresponden únicamente al responsable del servicio SaaS.

Dependencia de la conectividad.

El registro de ventas y reservas puede verse interrumpido cuando una sucursal pierde conexión con el servidor. Recurrir a anotaciones externas durante estas interrupciones dificulta reconstruir las operaciones y aumenta el riesgo de omisiones o registros duplicados. Por ello, la continuidad operativa requiere almacenar información en el dispositivo y conservarla después del cierre o reinicio de la aplicación.

Consistencia de las operaciones e inventarios.

La información disponible en un dispositivo desconectado puede diferir de la existente en el servidor. Mientras una sucursal registra una venta local, otro usuario puede vender las mismas unidades, modificar un precio o cancelar una reserva. En consecuencia, recuperar la conexión no basta para incorporar automáticamente todos los registros: es necesario comprobar su validez, evitar duplicaciones y comunicar los conflictos que requieran intervención.

Acceso e interpretación de la información comercial.

Las consultas administrativas pueden requerir distintas combinaciones de fechas, sucursales, categorías y productos. Un conjunto limitado de reportes predefinidos dificulta atender estas necesidades. Al mismo tiempo, incorporar inteligencia artificial sin controles podría producir respuestas sin respaldo en los datos o

exponer información ajena. Se necesita un mecanismo que transforme las solicitudes de los usuarios en consultas autorizadas y presente resultados comprensibles.

Estas necesidades plantean el siguiente problema central:

¿Cómo ampliar FashionStore para prestar servicios a diferentes empresas de prendas de vestir, mantener determinadas operaciones durante interrupciones de Internet y proporcionar información comercial útil, preservando la seguridad y la consistencia de los datos?

1.3. Objetivos del Proyecto

Objetivo General

Desarrollar una versión ampliada de FashionStore que permita a empresas independientes de prendas de vestir gestionar sus operaciones mediante suscripciones, utilizar servicios web y móviles con funcionamiento parcial sin conexión y consultar reportes y resultados analíticos, manteniendo la separación de información y la consistencia de las transacciones comerciales.

Objetivos Específicos

- Analizar los componentes existentes de FashionStore y las necesidades de su ampliación, identificando los procesos que pueden reutilizarse y aquellos que requieren modificaciones.
- Especificar los actores, casos de uso, requisitos y reglas de negocio de la nueva versión, estableciendo su relación con los componentes del sistema y los criterios de aceptación.

- Diseñar la organización de los datos y los mecanismos de autorización para diferenciar cuentas, empresas, roles y sucursales, manteniendo la independencia de la información comercial.
- Implementar la administración de empresas, planes y suscripciones, aplicando los límites y las funcionalidades correspondientes al servicio contratado.
- Adaptar las aplicaciones Angular y Flutter para consultar información previamente descargada y registrar reservas y ventas de contado sin conexión, mediante almacenamiento local persistente.
- Desarrollar la sincronización de operaciones pendientes, incorporando validación central, prevención de duplicaciones y tratamiento de conflictos relacionados con existencias, precios y permisos.
- Integrar la gestión de cambios de prendas, contemplando la validación de solicitudes, la recepción de artículos, las diferencias de precio y los movimientos de inventario asociados.
- Construir reportes comerciales mediante solicitudes escritas o por voz, utilizando datos autorizados y permitiendo su presentación en tablas, gráficos y documentos exportables.
- Desarrollar y evaluar modelos de aprendizaje supervisado y no supervisado, integrando sus resultados en un dashboard que permita interpretar predicciones, segmentos e indicadores.

- Verificar el funcionamiento de la plataforma mediante pruebas de autorización, aislamiento, persistencia, sincronización e integridad comercial, incluyendo la regresión de funcionalidades anteriores.
- Documentar y desplegar la solución, relacionando los modelos UML con la implementación y estableciendo procedimientos reproducibles de instalación, respaldo y recuperación.

1.4. Justificación

La ampliación de FashionStore se justifica por la posibilidad de aprovechar una base tecnológica existente para atender a diferentes empresas del mismo sector comercial. El modelo SaaS permite organizar la prestación del servicio mediante planes y suscripciones, mientras cada negocio conserva la administración de sus productos, empleados, sucursales y operaciones. Esto ofrece una alternativa a mantener desarrollos separados para organizaciones que comparten procesos similares.

Desde la perspectiva operativa, el funcionamiento parcial sin conexión busca disminuir las interrupciones en la atención de las tiendas. Conservar reservas y ventas localmente permite continuar determinadas actividades y disponer de un registro estructurado para su posterior validación. La sincronización y el tratamiento de conflictos complementan esta capacidad, evitando que la continuidad local comprometa la consistencia del inventario central.

La incorporación de reportes generativos y analítica inteligente se orienta a facilitar el aprovechamiento de la información comercial. Consultar datos

mediante lenguaje natural puede simplificar la preparación de reportes, mientras las predicciones y agrupaciones de clientes permiten explorar patrones que no siempre resultan evidentes mediante listados convencionales. Su utilidad deberá evaluarse a partir de los resultados obtenidos y de las limitaciones de los datos empleados.

En el ámbito técnico, el proyecto aborda problemas de autorización, aislamiento entre empresas, persistencia local, concurrencia y recuperación ante fallos. Resolverlos dentro de una misma plataforma permite aplicar conocimientos de desarrollo web y móvil, bases de datos, integración de servicios e inteligencia artificial.

Finalmente, su realización aporta una experiencia académica de evolución de software. El trabajo exige revisar decisiones anteriores, incorporar nuevos requisitos y demostrar mediante modelos y pruebas cómo se relacionan las necesidades del negocio con el comportamiento implementado.

1.5. Alcance y limitaciones

Alcance

El proyecto comprende la ampliación de FashionStore como plataforma de comercio electrónico para empresas de prendas de vestir. Se conservarán las capacidades comerciales existentes y se incorporarán las siguientes áreas:

1. Administración SaaS. Registro y gestión de empresas, configuración de planes, asignación de suscripciones y control de vigencia y límites. La demostración incluirá al menos dos empresas independientes.

2. Identidad y control de acceso. Autenticación mediante correo y contraseña, identificación del tipo de cuenta y aplicación de permisos por rol y empresa. Los clientes dispondrán de una cuenta global para operar en distintas tiendas, manteniendo separados sus contextos comerciales.

3. Gestión comercial multisucursal. Administración de catálogo, variantes, clientes, proveedores, compras, inventarios, reservas, ventas, pagos y comprobantes, adaptando los procesos heredados al aislamiento entre empresas.

4. Operación sin conexión. Consulta de catálogo descargado, registro de reservas y ventas de contado, y revisión de operaciones pendientes. La aplicación web utilizará capacidades PWA e IndexedDB; la aplicación móvil utilizará SQLite.

5. Sincronización y conflictos. Envío de operaciones pendientes, conservación de resultados, reintentos y prevención de duplicaciones. Los registros incompatibles con las condiciones centrales deberán identificarse y conservar su historial.

6. Cambios de prendas. Registro y seguimiento de solicitudes, validación de condiciones comerciales, recepción de artículos y gestión del reemplazo o devolución correspondiente.

7. Reportes e inteligencia artificial. Consultas comerciales convencionales y generativas, con filtros variables, datos de la empresa autorizada, visualización y exportación a PDF y Excel.

8. Analítica inteligente. Preparación de datos sintéticos, entrenamiento y evaluación de modelos supervisados y no supervisados, e integración de predicciones y segmentos en un dashboard.

9. Documentación, pruebas y despliegue. Especificaciones de casos de uso, modelos UML, evidencias de pruebas, manuales y publicación en un servidor propio accesible mediante HTTPS.

La solución utilizará FastAPI para el backend, Angular para la aplicación web, Flutter para la aplicación móvil y PostgreSQL como base de datos central.

Limitaciones

- El funcionamiento sin conexión abarcará únicamente las operaciones definidas para ese modo. El primer acceso, el registro de cuentas, los pagos electrónicos y las consultas a servicios de IA requerirán conectividad.

- Las existencias consultadas sin conexión corresponderán a la última información descargada. Una operación local podrá ser rechazada o quedar en conflicto al sincronizarse.

- Las ventas offline serán de contado. El registro local del efectivo recibido no equivaldrá a la confirmación definitiva de la venta en el servidor.

- La contratación y renovación de suscripciones se gestionarán administrativamente. No se contempla implementar cobro recurrente automático del servicio SaaS.

- La validación móvil inicial se concentrará en Android. No se incluye la distribución de una versión para iOS dentro de esta entrega.

- Se conservará el vestidor existente, sin desarrollar un nuevo motor de visualización o simulación de prendas.

- Los pagos se comprobarán en un ambiente de prueba. El envío real de correos, las notificaciones push y los servicios externos dependerán de la configuración y disponibilidad de sus proveedores.

- Los modelos analíticos iniciales utilizarán datos sintéticos. Sus resultados tendrán un propósito demostrativo y no acreditarán precisión sobre el comportamiento de empresas reales.

- La capacidad de atención simultánea estará condicionada por los recursos del servidor disponible y por los resultados de las pruebas de rendimiento.

1.6. Metodología de desarrollo ICONIX

El proyecto tomará ICONIX como referencia para organizar la evolución de FashionStore mediante casos de uso y modelos UML. Su aplicación se concentrará en comprender las necesidades del negocio, identificar las responsabilidades del sistema y mantener una relación verificable entre los requisitos y el software desarrollado.
El trabajo comprenderá las siguientes actividades:
1. Definición de requisitos y casos de uso. Se identificarán los actores, objetivos, reglas de negocio y condiciones de operación. Cada caso de uso describirá su flujo principal, alternativas, validaciones y resultados esperados.

2. Modelado del dominio. Se identificarán los conceptos del negocio y sus relaciones, distinguiendo elementos de plataforma, empresa, sucursal y operación comercial. Este modelo servirá para mantener un vocabulario común durante el desarrollo.
3. Análisis de responsabilidades. Los diagramas de robustez relacionarán las interfaces, los controles y las entidades que participan en los casos de uso, permitiendo revisar la distribución de responsabilidades.
4. Diseño de interacciones y estructura. Los diagramas de secuencia y de clases representarán la colaboración entre los componentes y los métodos involucrados. Se complementarán con el diseño de datos, interfaces y despliegue.
5. Implementación y verificación. El desarrollo se organizará por casos de uso. Cada funcionalidad será contrastada con sus criterios de aceptación y con las reglas comerciales afectadas, conservando las evidencias correspondientes.
Para registrar estas relaciones se utilizará una matriz de trazabilidad que vincule requisito, regla de negocio, caso de uso, componente, prueba y evidencia.
El equipo adoptará una adaptación del orden de trabajo: las especificaciones y decisiones iniciales se prepararán antes de implementar, mientras los diagramas técnicos finales se completarán y contrastarán con el código estabilizado mediante ingeniería inversa. Esta organización deberá identificarse como una adaptación de ICONIX, ya que difiere de su secuencia habitual de modelado previo a la implementación. Los modelos obtenidos se revisarán para comprobar que representen las responsabilidades e interacciones de la versión entregada.

2. CAPITULO II. ANÁLISIS DE REQUISITOS

2.1. Descripción del Sistema Propuesto

FashionStore se propone como una plataforma de comercio electrónico web y móvil dirigida a empresas que comercializan prendas de vestir. Su organización bajo el modelo SaaS permite prestar un servicio compartido a diferentes negocios, manteniendo espacios de operación separados para sus usuarios, sucursales y recursos comerciales.
La plataforma distingue la administración del servicio de la gestión interna de cada empresa. El superadministrador administra empresas contratantes, planes y suscripciones, mientras los administradores empresariales organizan su personal y sus operaciones. El acceso se realiza mediante correo y contraseña, obteniendo del servidor el tipo de cuenta y las autorizaciones correspondientes. Los clientes disponen de una cuenta común para interactuar con distintas tiendas, conservando la pertenencia empresarial de cada compra, reserva o solicitud.
El sistema integra la administración del catálogo, proveedores, compras, inventarios, reservas, ventas, pagos y cambios de prendas. Asimismo, conserva el

vestidor existente y las recomendaciones como herramientas de apoyo a la experiencia de compra.

Para mantener determinadas actividades durante interrupciones de Internet, las aplicaciones incorporan almacenamiento local. Los usuarios autorizados podrán consultar información descargada y registrar reservas o ventas de contado según su rol. Estas operaciones conservarán su identificación y estado hasta que puedan ser verificadas por el servidor. La sincronización permitirá conocer cuáles fueron confirmadas, rechazadas o requieren resolver un conflicto.

La solución también incorpora reportes solicitados mediante lenguaje natural y un módulo de analítica inteligente. Los reportes utilizarán datos comerciales autorizados, mientras los modelos supervisados y no supervisados permitirán presentar predicciones, segmentos e indicadores con sus respectivas limitaciones.

La propuesta se organiza en 20 casos de uso principales, vinculados con los requisitos, las reglas de negocio y las pruebas necesarias para demostrar su funcionamiento.

2.2. Identificación de actores

Los actores representan los papeles externos que interactúan con FashionStore. Una misma persona puede desempeñar distintos papeles cuando sus permisos lo permitan; su participación en un caso de uso no implica acceso a todas sus operaciones.

- **SuperAdministrador SaaS**: Administrar empresas, planes y suscripciones. Consultar información de plataforma sin obtener acceso comercial indiscriminado a las empresas.
- **Administrador de Empresa**: Organizar personal, permisos, sucursales, catálogo y operaciones de su empresa. Consultar las condiciones de su suscripción.
- **Encargado de sucursal**: Atender reservas, ventas, cambios y conflictos dentro de las sucursales y permisos asignados.
- **Vendedor o cajero**: Atender clientes, registrar ventas y utilizar las funciones offline y de sincronización autorizadas.
- **Responsable de inventario y compras**: Administrar proveedores, compras, recepciones y movimientos de existencias, además del catálogo que tenga autorizado.
- **Cliente**: Comprar, reservar y consultar sus propios comprobantes, datos y solicitudes en las tiendas donde opere.
- **Visitante**: Explorar las tiendas y el catálogo público. Requiere una cuenta para realizar operaciones privadas.
- **Pasarela de pagos**: Procesar pagos y reembolsos de prueba, proporcionando resultados verificables.
- **Servicio de inteligencia artificial**: Interpretar solicitudes y apoyar recomendaciones mediante operaciones controladas.

- **Servicios de correo y notificaciones push**: Transportar mensajes de recuperación y avisos generados por la plataforma.

2.3. Requisitos funcionales y no funcionales

Requisitos Funcionales

Los requisitos funcionales describen las capacidades que FashionStore deberá proporcionar.

- **RF01 - Gestionar acceso y cuenta**

  Registrar clientes, autenticar por correo y contraseña, recuperar acceso, identificar roles y revocar sesiones.

- **RF02 - Administrar empresas, planes y suscripciones**

  Aprovisionar empresas, configurar planes y administrar contratación, vigencia, renovación y cambios de suscripción.

- **RF03 - Administrar personal, roles y permisos**

  Mantener empleados, asignar roles y sucursales, y activar o suspender autorizaciones empresariales.

- **RF04 - Gestionar ciudades y sucursales**

  Administrar ubicaciones, responsables y estado de sucursales, respetando los límites del plan.

- **RF05 - Gestionar fichas de clientes**

  Mantener datos comerciales por empresa y permitir al cliente consultar su información e historial propio.

- **RF06 - Administrar catálogo de prendas**

  Gestionar categorías, temporadas, colecciones, prendas, variantes, atributos, precios y ofertas.

- **RF07 - Gestionar proveedores, compras y recepciones**

  Administrar proveedores y adquisiciones, confirmando recepciones con sus detalles y entradas de inventario.

- RF08 - Consultar inventario y disponibilidad

Consultar existencias físicas, retenciones y cantidades disponibles por sucursal y variante.

- **RF09 - Explorar catálogo y asistencia de compra**

  Buscar prendas, aplicar filtros y utilizar opcionalmente el vestidor y las recomendaciones disponibles.

- **RF10 - Registrar movimientos de inventario**

  Registrar entradas, salidas, ajustes y transferencias autorizadas, conservando su trazabilidad.

- **RF11 - Realizar compra y pago en línea**

  Gestionar carrito y checkout hasta confirmar una compra mediante un pago verificable.

- **RF12 - Gestionar reservas de prendas**

  Registrar, consultar, preparar, entregar, cancelar y controlar el vencimiento de reservas.

- **RF13 - Registrar venta presencial**

  Consolidar ventas autorizadas con cálculo de importes, registro del cobro y actualización consistente de existencias.

- **RF14 - Obtener comprobantes de compra**

  Emitir o recuperar la representación de un comprobante, consultarla y descargarla con autorización.

- **RF15 - Gestionar cambios y reembolsos**

  Tramitar solicitudes, recepción de prendas, reemplazos, diferencias de precio, devoluciones y notificaciones.

- **RF16 - Registrar y consultar operaciones offline**

  Ofrecer una PWA instalable y almacenamiento persistente para consultar datos descargados y registrar operaciones locales según el rol.

- **RF17 - Sincronizar operaciones y resolver conflictos**

  Enviar pendientes, recuperar resultados, evitar duplicaciones y gestionar conflictos conservando su historial.

- **RF18 - Obtener reportes comerciales**

  Consultar reportes convencionales o generativos mediante texto y voz, mostrando resultados y exportaciones autorizadas.

- **RF19 - Preparar y consultar analítica inteligente**

  Preparar datos, entrenar y evaluar modelos supervisados y no supervisados, y consultar indicadores, predicciones y segmentos.

- **RF20 - Consultar auditoría y trazabilidad**

  Consultar eventos y relacionar responsables, acciones, operaciones y resultados dentro del ámbito autorizado.

Requisitos No Funcionales

- **RNF01 - Seguridad de acceso**
  Comprobar identidad, permisos y propiedad en operaciones privadas, rechazando sesiones revocadas.
- **RNF02 - Separación empresarial**
  Impedir accesos cruzados incluso al modificar manualmente identificadores y parámetros.
- **RNF03 - Persistencia local**
  Recuperar operaciones y pendientes después del cierre y reinicio normal de la aplicación.
- **RNF04 - Consistencia transaccional**
  Confirmar conjuntamente cabecera, detalles y movimientos; revertir el conjunto ante fallos intermedios.
- **RNF05 - Reenvío seguro**
  Repetir una operación confirmada sin generar registros ni efectos adicionales.
- **RNF06 - Recuperación ante fallos**
  Conservar información y permitir reintentar después de desconexiones, errores y respuestas perdidas.
- **RNF07 – Mantenibilidad**
  Mantener responsabilidades identificables de presentación, negocio, persistencia y sincronización.
- **RNF08 – Compatibilidad**
  Verificar la PWA en navegadores compatibles y la aplicación Flutter en Android.
- **RNF09 - Uso de recursos**
  Paginar consultas, limitar lotes y documentar tiempos, errores y consumo de memoria durante las pruebas.
- **RNF10 – Observabilidad**

Registrar estados, intentos y motivos de fallo sin exponer contraseñas, tokens ni credenciales.

- **RNF11 - Calidad verificable**
  Disponer de pruebas reproducibles de las reglas comerciales y técnicas críticas.
- **RNF12 - Correspondencia documental**
  Relacionar requisitos, modelos, componentes y evidencias con una versión identificada del software.

Reglas de Negocio

Las reglas establecen condiciones que deben cumplirse independientemente de la pantalla o dispositivo utilizado.

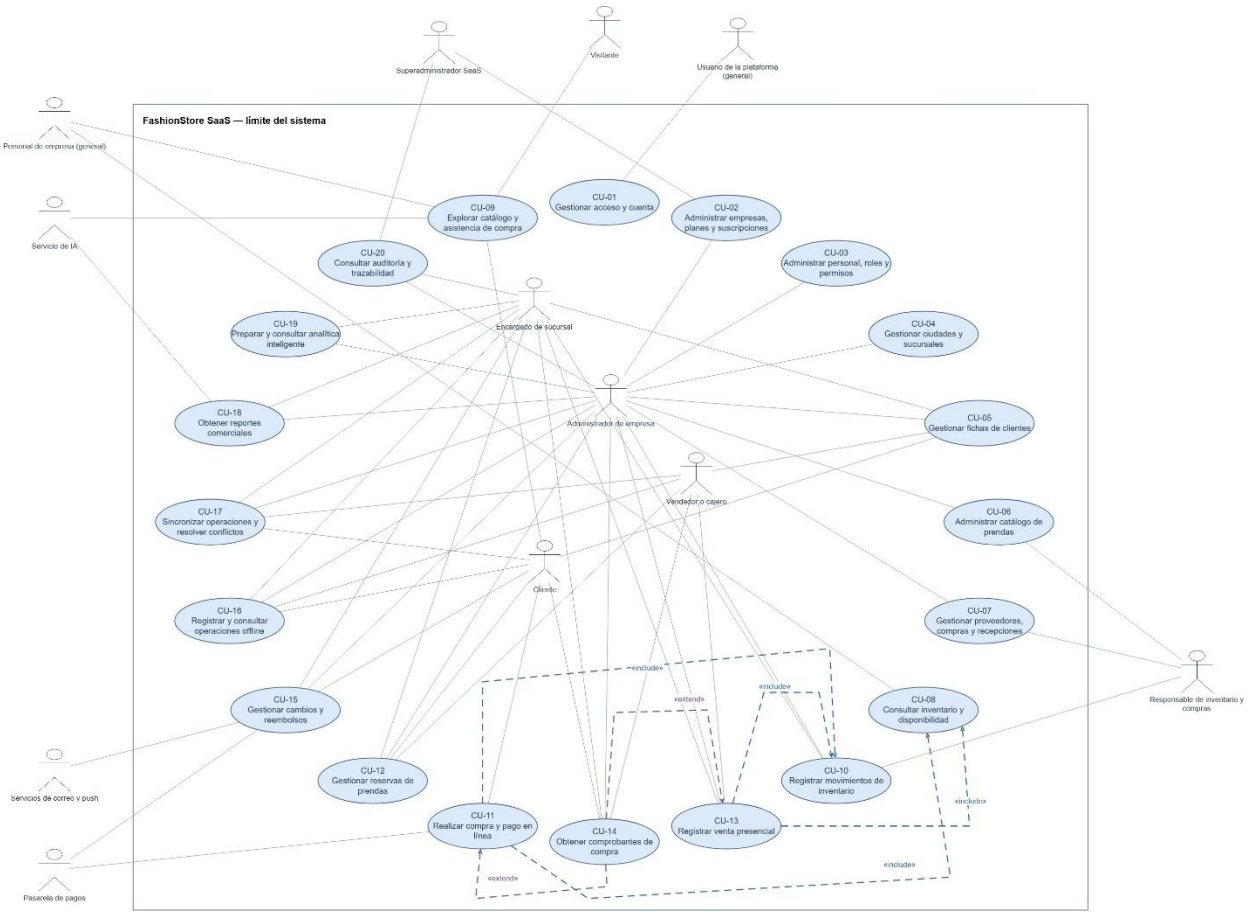
- **RN01** - La empresa constituye la unidad de aislamiento. Una sucursal pertenece a una sola empresa.
- **RN02** - El servidor obtiene el contexto empresarial de la sesión validada y verifica las referencias recibidas.
- **RN03** - Un correo identifica una cuenta global. El personal conserva su empresa; el cliente puede elegir distintas tiendas después de autenticarse.
- **RN04** - El administrador empresarial no modifica planes globales, contraseñas globales de clientes ni privilegios de plataforma.
- **RN05** - Básico admite inicialmente dos sucursales activas y Profesional diez. Los límites son configurables y se comprueban también al reactivar sucursales y ante solicitudes simultáneas.
- **RN06** - Los reportes convencionales están disponibles en ambos planes; los generativos requieren Profesional y autorización.
- **RN07** - El vencimiento de una suscripción conserva historial y pendientes, pero bloquea nuevas operaciones comerciales.
- **RN08** - El cambio a un plan menor se rechaza mientras la empresa exceda su límite de sucursales activas.
- **RN09** - Los importes se recalculan en el servidor. Se aplica el menor precio válido entre minorista, mayorista y oferta, sin acumular descuentos. El umbral mayorista inicial es de seis unidades de la misma prenda y es configurable.
- **RN10** - La disponibilidad central resulta de restar las retenciones vigentes a las existencias físicas. No se confirma una salida que produzca stock negativo.
- **RN11** - Una reserva central aceptada retiene disponibilidad durante 24 horas. Al vencer o cancelarse se libera; una reserva offline permanece provisional.
- **RN12** - Preparar una reserva, añadir productos al carrito o mantener un pago pendiente no constituye una venta confirmada.
- **RN13** - El pago electrónico se confirma mediante un resultado verificable del proveedor y por el importe autorizado. Los reenvíos no duplican su efecto.

- **RN14** - Cada movimiento modifica las existencias una sola vez mediante el mecanismo de inventario del sistema.
- **RN15** - El comprobante corresponde a una venta confirmada. Su consulta o descarga exige autorización y no crea otro documento comercial.
- **RN16** - La operación offline requiere autenticación previa y datos descargados. Su autorización local tendrá vigencia máxima de 24 horas, limitada por la suscripción conocida.
- **RN17** - Las ventas offline son de contado. Recibir efectivo localmente no garantiza aceptación central; un rechazo exige resolución o compensación documentada.
- **RN18** - Cada operación conserva UUID, autor, empresa, sucursal y contenido entre intentos. Reutilizar el UUID con contenido diferente se rechaza.
- **RN19** - Intentar enviar una operación o cerrar sesión no elimina pendientes. Cada operación del lote recibe su propio resultado.
- **RN20** - Stock insuficiente, cambio de precio o variante inactiva producen un resultado explícito de conflicto, sin aceptar parcialmente una venta. Los permisos revocados impiden confirmar.
- **RN21** - Corregir un conflicto crea una operación nueva vinculada con la original, conservando el historial.
- **RN22** - El cambio se solicita hasta siete días calendario desde la entrega, con compra comprobable, prenda elegible sin uso y una única sustitución por unidad.
- **RN23** - La recepción de la prenda y la entrega del reemplazo son eventos separados. El crédito utiliza el importe efectivamente pagado; las diferencias se cobran o devuelven íntegramente.
- **RN24** - Si no existe reemplazo, la solicitud permanece pendiente y puede cancelarse con devolución del crédito. El reembolso sigue el medio original, con un objetivo administrativo de cinco días hábiles.
- **RN25** - Los reportes consultan datos reales de la empresa autorizada. La IA interpreta parámetros; el backend controla las consultas y no ejecuta SQL libre proporcionado por el modelo.
- **RN26** - Las solicitudes por texto y voz admiten filtros variables. Se demostrarán al menos cuatro tipos de reportes; PDF y Excel corresponderán a los resultados autorizados mostrados.
- **RN27** - La analítica incluye modelos supervisados y no supervisados, evaluación e integración. Los datos sintéticos se identifican y las predicciones no se presentan como transacciones reales.
- **RN28** - El dashboard utiliza modelos previamente evaluados. El entrenamiento constituye un subflujo técnico separado y se ejecuta fuera del servidor de producción.

2.4. Diagrama UML de casos de uso

2.4.1. Modelos de casos de uso

- **CU01** - Gestionar acceso y cuenta
- **CU02** - Administrar empresas, planes y suscripciones
- **CU03** - Administrar personal, roles y permisos
- **CU04** - Gestionar ciudades y sucursales
- **CU05** - Gestionar fichas de clientes
- **CU06** - Administrar catálogo de prendas
- **CU07** - Gestionar proveedores, compras y recepciones
- **CU08** - Consultar inventario y disponibilidad
- **CU09** - Explorar catálogo y asistencia de compra
- **CU10** - Registrar movimientos de inventario
- **CU11** - Realizar compra y pago en línea
- **CU12** - Gestionar reservas de prendas
- **CU13** - Registrar venta presencial
- **CU14** - Obtener comprobantes de compra
- **CU15** - Gestionar cambios y reembolsos
- **CU16** - Registrar y consultar operaciones offline
- **CU17** - Sincronizar operaciones y resolver conflictos
- **CU18** - Obtener reportes comerciales
- **CU19** - Preparar y consultar analítica inteligente
- **CU20** - Consultar auditoría y trazabilidad



Especificación de casos de uso

ESPECIFICACIÓN DEL CASO DE USO CU01	
Caso de Uso	CU01 - Gestionar acceso y cuenta
Actor Principal	Usuario de la plataforma: SuperAdministrador SaaS, personal de empresa y Cliente
Plataforma	WEB/MÓVIL
Descripción	Registra clientes con identidad global, inicia y cierra sesión mediante correo y contraseña, permite recuperar el acceso y, en el caso del cliente, elegir la tienda después de autenticarse. El servidor devuelve el tipo de cuenta, el rol y el ámbito empresarial; el cierre de sesión revoca el acceso sin eliminar las operaciones pendientes almacenadas en el dispositivo.
Precondiciones	Conexión disponible para autenticarse o recuperar la contraseña. Cuenta activa y, en el personal, empresa activa con rol y sucursales asignados. El cliente requiere al menos una tienda habilitada para iniciar su operación.
Flujo normal	1. El actor abre la aplicación web o móvil y solicita iniciar sesión. 2. Ingresa correo y contraseña, sin indicar empresa. 3. El servidor valida la identidad global y devuelve el tipo de cuenta, el rol, la empresa del personal y las tiendas disponibles. 4. El sistema abre el espacio de plataforma, el panel de la empresa o las tiendas del cliente, según corresponda.

	5. El cliente elige la tienda en la que operará; el personal conserva la empresa asignada. 6. El actor cierra sesión y el servidor revoca el token sin eliminar las operaciones pendientes.
Flujos alternativos	1a. El interesado no tiene cuenta: se registra como cliente con correo y contraseña, y la cuenta queda habilitada para operar en las tiendas. 1b. El actor olvidó su contraseña: solicita la recuperación, recibe un enlace temporal de un solo uso, define la nueva contraseña y se invalidan las sesiones anteriores. 2a. Credenciales incorrectas o cuenta inexistente: se emite una respuesta uniforme, sin revelar información de la cuenta. 3a. Cuenta o empresa inactiva, o sesión revocada: se impide el acceso. 3b. El personal intenta operar en otra empresa: se deniega. 5a. El cliente no tiene tiendas habilitadas: se informa la situación y no se habilita la operación. 6a. Sin conexión: solo se permite continuar con el modo offline previamente habilitado (CU16).
Postcondiciones	Sesión activa con el contexto de plataforma, empresa o tienda. Al cerrar, el token queda revocado y las operaciones pendientes del dispositivo se conservan. Cuando se recuperó el acceso, la contraseña queda actualizada y las sesiones anteriores invalidadas.
Relaciones	
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU02	
Caso de Uso	CU02 - Administrar empresas, planes y suscripciones
Actor Principal	SuperAdministrador SaaS; Administrador de Empresa (consulta de su contrato)
Plataforma	WEB
Descripción	Aprovisiona de forma atómica la empresa y su administrador inicial, configura los planes y sus límites, y administra la asignación, renovación o cambio de suscripción comprobando los cupos contratados. El administrador empresarial consulta únicamente el estado y las condiciones del servicio de su propia empresa.
Precondiciones	Sesión con facultades de plataforma para el alta y la configuración; para la consulta, sesión de Administrador de Empresa. Planes registrados y vigentes. Para el cambio a un plan menor, sucursales activas dentro del límite del plan destino (RN08).
Flujo normal	1. El SuperAdministrador accede a la consola de plataforma. 2. Registra la empresa con nombre, identificador público y correo del administrador inicial. 3. Asigna el plan inicial y su vigencia. 4. Configura o actualiza los límites y las capacidades del plan. 5. Asigna, renueva o cambia la suscripción comprobando fechas y cupos. 6. Actualiza las capacidades efectivas y registra la auditoría del cambio. 7. El administrador empresarial consulta el estado y los límites de su contrato.

Flujos alternativos	1a. Un usuario de empresa intenta acceder a la consola: se deniega (RN04). 2a. El identificador público ya existe: se rechaza el alta. 2b. Falla la creación del administrador inicial: se revierte el alta completa. 3a. El plan no existe o está inactivo: se rechaza la asignación. 4a. El límite no es positivo o es menor al uso actual: se rechaza. 5a. El cambio a un plan menor deja más sucursales activas que las permitidas: se rechaza (RN08). 5b. La suscripción está vencida: se conserva el historial, se bloquean nuevas operaciones comerciales y se habilita la renovación (RN07). 7a. El administrador empresarial intenta modificar el plan: se deniega.
Postcondiciones	Empresa y administrador creados atómicamente. Plan y suscripción con vigencias definidas, capacidades efectivas actualizadas y cambios auditados.
Relaciones	Requiere CU01. Habilita CU03 y CU04 por los cupos y roles, y CU18 cuando el plan contratado es Profesional.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU03	
Caso de Uso	CU03 - Administrar personal, roles y permisos
Actor Principal	Administrador de Empresa
Plataforma	WEB
Descripción	Mantiene los empleados de la empresa con correo global único, asigna roles y sucursales, y activa, suspende o revoca sus autorizaciones. El sistema impide otorgar privilegios de plataforma y modificar la identidad global de los clientes, y verifica que los permisos efectivos correspondan a la asignación registrada.
Precondiciones	Sesión de Administrador de Empresa con la empresa activa. Roles y sucursales previamente registrados en el ámbito de la empresa. El correo del empleado no debe pertenecer a otra cuenta.
Flujo normal	1. El administrador consulta el personal y los roles de la empresa. 2. Registra el empleado con su correo global. 3. Asigna el rol y las sucursales permitidas. 4. Guarda y activa la autorización. 5. Verifica los permisos efectivos del usuario. 6. Suspende o revoca la autorización cuando corresponda.
Flujos alternativos	1a. Personal sin permiso de administración: se deniega. 2a. El correo ya pertenece a otra cuenta: se rechaza. 2b. El correo tiene formato inválido: se rechaza. 3a. El rol o la sucursal pertenecen a otra empresa: se rechaza. 3b. Se intenta asignar privilegios de plataforma o modificar la identidad global de un cliente: se rechaza (RN04). 5a. Los permisos efectivos no coinciden con la asignación: se corrige antes de continuar. 6a. El usuario revocado mantiene una sesión abierta: su siguiente operación se rechaza.

Postcondiciones	Empleado registrado con rol y sucursales; permisos efectivos comprobados; autorizaciones suspendidas o revocadas cuando corresponde.
Relaciones	Requiere CU01 y CU04. Condiciona las operaciones de CU10, CU12, CU13 y CU20 según el rol asignado.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU04	
Caso de Uso	CU04 - Gestionar ciudades y sucursales
Actor Principal	Administrador de Empresa
Plataforma	WEB
Descripción	Registra ciudades y sucursales con su dirección, responsables y estado, y permite consultarlas, actualizarlas, suspenderlas o reactivarlas. Cada alta o reactivación comprueba la pertenencia a la empresa y el cupo de sucursales activas de la suscripción contratada.
Precondiciones	Sesión de Administrador de Empresa con la empresa activa. Plan con cupo disponible para una nueva sucursal activa (RN05). Responsables con rol asignado dentro de la empresa.
Flujo normal	1. El administrador consulta las ciudades y sucursales de su empresa. 2. Registra la ciudad o selecciona una existente. 3. Ingresa dirección, encargado y estado de la sucursal. 4. Comprueba la pertenencia del encargado y el cupo del plan. 5. Guarda el alta, la actualización, la suspensión o la reactivación. 6. El sistema muestra la estructura operativa actualizada.
Flujos alternativos	1a. La sucursal pertenece a otra empresa: no se encuentra. 3a. El encargado es de otra empresa o no tiene rol: se rechaza. 4a. El cupo de sucursales activas está agotado: se rechaza (RN05). 4b. Dos reactivaciones simultáneas superan el límite: la segunda se rechaza. 5a. La sucursal tiene operaciones abiertas: se impide eliminarla y se ofrece suspenderla. 6a. Sin conexión: se conserva la última estructura conocida.
Postcondiciones	Sucursal registrada o actualizada; cupo consumido o liberado; estructura operativa vigente para las demás operaciones.
Relaciones	Requiere CU01. Consume el cupo definido en CU02 y delimita CU05, CU08, CU10, CU12 y CU13.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU05	
Caso de Uso	CU05 - Gestionar fichas de clientes
Actor Principal	Administrador de Empresa, Encargado de sucursal y Vendedor o cajero; Cliente (consulta de su propia ficha)
Plataforma	WEB
Descripción	Crea y mantiene la ficha comercial del cliente dentro de la empresa, con sus datos de contacto e historial de operaciones, evitando duplicados. El cliente consulta su

	propia información e historial entre tiendas, mientras la identidad global permanece separada del historial comercial de cada empresa.
Precondiciones	Sesión del personal autorizado o del cliente autenticado para su propia ficha. Empresa activa y cuenta global del cliente registrada. El acceso a fichas ajenas exige pertenencia a la empresa y el permiso correspondiente.
Flujo normal	1. El actor busca la ficha por identificación o nombre. 2. Crea o actualiza los datos de contacto permitidos. 3. El sistema valida la identificación única dentro de la empresa. 4. Consulta el historial de operaciones según el rol y la propiedad. 5. Registra la auditoría de la modificación.
Flujos alternativos	1a. La ficha pertenece a otra empresa: no se encuentra. 2a. La identificación ya existe en la empresa: se rechaza el duplicado. 2b. Los datos de contacto están incompletos: se rechaza. 4a. El cliente autenticado intenta consultar otra ficha: se deniega. 4b. El personal carece de permiso: se deniega.
Postcondiciones	Ficha comercial creada o actualizada e historial consultado. La identidad global permanece separada del historial comercial de la empresa.
Relaciones	Requiere CU01. Toma información de CU11, CU12, CU13 y CU15 para componer el historial.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU06	
Caso de Uso	CU06 - Administrar catálogo de prendas
Actor Principal	Administrador de Empresa y Responsable de inventario y compras
Plataforma	WEB
Descripción	Organiza categorías, temporadas y colecciones; registra prendas con sus tallas, combinaciones de colores, material y medidas; y configura precios minoristas, mayoristas, umbral de mayoreo y ofertas vigentes. Los elementos pueden publicarse o desactivarse conservando sus relaciones comerciales e históricas.
Precondiciones	Sesión con permiso de catálogo en la empresa activa. Para registrar prendas y variantes, categorías y temporadas previamente creadas en la misma empresa. Precios definidos conforme a la regla de cálculo del servidor (RN09).
Flujo normal	1. El usuario consulta categorías, temporadas y colecciones. 2. Crea o edita la estructura del catálogo. 3. Registra la prenda con atributos, tallas, colores, material y medidas. 4. Define variantes, combinaciones y SKU. 5. Configura precios minorista y mayorista, umbral de mayoreo y ofertas. 6. Publica o desactiva los elementos conservando sus relaciones comerciales.
Flujos alternativos	1a. La categoría padre es de otra empresa o genera un ciclo: se rechaza. 2a. Las fechas de la temporada están invertidas: se rechaza. 3a. El SKU ya existe en la empresa: se rechaza. 3b. Faltan atributos o medidas obligatorias: se rechaza. 5a. El precio es negativo o el umbral es inválido: se rechaza.

	5b. La oferta está vencida: no se aplica (RN09). 6a. La prenda tiene ventas o reservas: se desactiva, no se elimina.
Postcondiciones	Catálogo actualizado con precios y ofertas vigentes; prendas publicadas o desactivadas sin perder su historial comercial.
Relaciones	Requiere CU01. Alimenta CU08, CU09, CU11, CU12 y CU18.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU07	
Caso de Uso	CU07 - Gestionar proveedores, compras y recepciones
Actor Principal	Administrador de Empresa y Responsable de inventario y compras
Plataforma	WEB/MÓVIL
Descripción	Mantiene la ficha de los proveedores, prepara compras con variantes, cantidades y costos, y confirma la recepción física con sus detalles y entradas de inventario en una sola transacción. La aplicación móvil permite consultar el directorio de proveedores, las compras registradas y el detalle de cada recepción.
Precondiciones	Sesión con permiso de compras e inventario en la empresa activa. Proveedor registrado, sucursal de recepción autorizada y variantes existentes en el catálogo. Conexión al servidor para confirmar la recepción.
Flujo normal	1. El usuario registra o selecciona el proveedor. 2. Prepara la compra con variantes, cantidades y costos. 3. Selecciona la sucursal de recepción. 4. Confirma la recepción física. 5. El sistema registra los detalles, las entradas de inventario y la auditoría en una transacción. 6. Consulta el documento y su detalle.
Flujos alternativos	1a. El NIT está repetido o el proveedor es de otra empresa: se rechaza. 2a. La variante pertenece a otra empresa: se rechaza. 2b. La cantidad o el costo no son positivos: se rechaza. 3a. La sucursal no está autorizada: se deniega. 4a. Falla un detalle de la recepción: se revierte la operación completa. 4b. Se reenvía una recepción ya confirmada: no se duplica. 6a. Se repite la consulta del documento: no genera movimientos.
Postcondiciones	Compra y recepción registradas; existencias incrementadas una sola vez; auditoría y documento disponibles para consulta.
Relaciones	Requiere CU01. Produce entradas de inventario propias de CU10 dentro de la misma transacción.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU08	
Caso de Uso	CU08 - Consultar inventario y disponibilidad
Actor Principal	Personal de empresa: Administrador de Empresa, Encargado de sucursal, Responsable de inventario y compras y Vendedor o cajero
Plataforma	WEB/MÓVIL

Descripción	Obtiene las existencias físicas, las unidades retenidas por reservas vigentes y la disponibilidad resultante por sucursal y variante. El mismo cálculo se reutiliza al confirmar una compra o una venta, y cuando la consulta proviene de datos descargados se identifica la antigüedad de la información.
Precondiciones	Sesión con sucursal o sucursales asignadas dentro de la empresa activa. Inventario registrado para las variantes consultadas. Para información vigente, conexión con el servidor; en modo local, catálogo descargado y vigencia offline habilitada.
Flujo normal	1. El usuario selecciona la empresa y la sucursal autorizada. 2. Consulta las existencias físicas. 3. El sistema obtiene las retenciones vigentes de las reservas. 4. Calcula la disponibilidad de las variantes solicitadas. 5. Presenta las cantidades y la fecha de actualización de la información.
Flujos alternativos	1a. La sucursal no está autorizada para el usuario: se deniega. 2a. No existen registros para la variante: se muestra un resultado vacío. 5a. No hay conexión: se muestra la copia local con su antigüedad. 5b. La copia local supera las veinticuatro horas: se advierte y se limita la operación.
Postcondiciones	Disponibilidad presentada con su fecha de actualización. No se modifican existencias ni retenciones.
Relaciones	La ejecutan de forma obligatoria antes de confirmar CU11 y CU13. Es utilizada por CU09, CU12 y CU17.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU09	
Caso de Uso	CU09 - Explorar catálogo y asistencia de compra
Actor Principal	Visitante y Cliente; Servicio de inteligencia artificial (vestidor y recomendaciones)
Plataforma	WEB/MÓVIL
Descripción	Permite buscar prendas por categoría, temporada, talla, color y precio, consultar sus variantes y disponibilidad, y utilizar opcionalmente el vestidor virtual y las recomendaciones sobre prendas de la tienda elegida. Las prendas seleccionadas pueden derivar en una compra o en una reserva independiente.
Precondiciones	Tienda pública seleccionada o contexto empresarial autorizado. Catálogo publicado con al menos una prenda activa. Para el vestidor y las recomendaciones, servicio de inteligencia artificial disponible; para operaciones privadas, sesión de cliente.
Flujo normal	1. El actor elige la tienda o el contexto empresarial autorizado. 2. Busca prendas y aplica filtros. 3. Consulta variantes y precios aplicables. 4. Consulta la disponibilidad (CU08). 5. Utiliza opcionalmente el vestidor o las recomendaciones. 6. Selecciona prendas para una compra o una reserva.
Flujos alternativos	1a. La empresa no existe o está inactiva: no se encuentra. 2a. No hay coincidencias con los filtros: se muestra un resultado vacío. 4a. No hay conexión: se utiliza el catálogo descargado previamente (CU16).

	5a. El permiso de cámara se rechaza: se informa y se continúa sin vestidor. 5b. El servicio de inteligencia artificial no responde: se continúa sin recomendaciones. 6a. El visitante intenta una operación privada: se solicita crear cuenta.
Postcondiciones	Prendas seleccionadas por el actor. No se confirma ninguna transacción comercial.
Relaciones	
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU10	
Caso de Uso	CU10 - Registrar movimientos de inventario
Actor Principal	Administrador de Empresa, Responsable de inventario y compras y Encargado de sucursal
Plataforma	WEB/MÓVIL
Descripción	Registra entradas, salidas, ajustes y transferencias autorizadas con su motivo, cantidades y responsables, aplicando un único efecto sobre las existencias mediante el mecanismo de movimientos del sistema. La confirmación conserva la trazabilidad y revierte el conjunto cuando falla un paso intermedio.
Precondiciones	Sesión con permiso de inventario en la empresa activa. Sucursales y variantes existentes y autorizadas para el usuario. Existencias suficientes para una salida o transferencia (RN10). Conexión al servidor.
Flujo normal	1. El usuario selecciona el tipo de movimiento y la sucursal. 2. Elige las variantes y las cantidades. 3. El sistema valida cantidades, pertenencia y autorización. 4. Controla o bloquea las existencias afectadas. 5. Registra el movimiento y su efecto único sobre el stock. 6. Confirma la trazabilidad y el responsable.
Flujos alternativos	1a. La sucursal es ajena a la empresa: se deniega. 3a. La cantidad no es positiva: se rechaza. 3b. La variante pertenece a otra empresa: se rechaza. 4a. El stock es insuficiente: se rechaza sin producir efecto (RN10). 5a. Falla un paso intermedio: se revierte el movimiento. 5b. Se reintenta con el mismo UUID: no se duplica (RN14 y RN18).
Postcondiciones	Movimiento registrado una sola vez, existencias actualizadas y auditoría con responsable y motivo.
Relaciones	Recibe las entradas de CU07 y las reposiciones de CU15.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU11	
Caso de Uso	CU11 - Realizar compra y pago en línea
Actor Principal	Cliente y Pasarela de pagos
Plataforma	WEB/MÓVIL
Descripción	Administra el carrito y el checkout con sucursal y forma de entrega, recalcula los importes en el servidor y consulta la disponibilidad antes de confirmar. El pago

	electrónico se valida con el resultado verificable de la pasarela y, solo entonces, se confirma la venta con su salida única de inventario. Un pago pendiente o abandonado no constituye una venta.
Precondiciones	Cliente autenticado con tienda seleccionada y suscripción vigente de la empresa. Carrito con al menos un ítem y disponibilidad suficiente (RN10). Pasarela de pagos configurada y conexión al servidor.
Flujo normal	1. El cliente prepara el carrito con variantes y cantidades. 2. El servidor recalcula precios y descuentos. 3. El sistema consulta la disponibilidad (CU08). 4. El cliente selecciona sucursal y forma de entrega. 5. Inicia el pago electrónico. 6. El sistema valida el resultado firmado de la pasarela. 7. Confirma la venta y la salida de inventario (CU10) una sola vez. 8. Presenta el resultado y ofrece el comprobante.
Flujos alternativos	2a. El precio o el descuento fue alterado: se recalcula (RN09). 3a. El stock es insuficiente: se informa y no se confirma. 4a. El carrito está vacío: no se inicia el checkout. 5a. La pasarela no está disponible: se conserva el pago como pendiente. 6a. La firma es inválida o el importe no coincide: se rechaza (RN13). 6b. El pago es rechazado o expira: se conserva el estado fallido sin confirmar la venta. 6c. El evento se repite: se devuelve la misma confirmación. 7a. Falla un paso intermedio: se revierten la venta y la salida de inventario.
Postcondiciones	Venta confirmada y pagada; inventario descontado una sola vez; comprobante disponible; carrito cerrado. Si el pago no se confirmó, la venta no existe y el carrito permanece.
Relaciones	CU14 extiende a este caso en el punto «compra confirmada».
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU12	
Caso de Uso	CU12 - Gestionar reservas de prendas
Actor Principal	Cliente; Encargado de sucursal, Vendedor o cajero y Administrador de Empresa
Plataforma	WEB/MÓVIL
Descripción	Registra reservas con cliente, sucursal y detalles, retiene la disponibilidad durante veinticuatro horas y permite preparar las prendas, confirmar el pago cuando corresponda, entregar o cancelar. El vencimiento libera la retención sin generar una venta ficticia, y el cliente opera únicamente sobre sus propias reservas.
Precondiciones	Cliente autenticado o personal con permiso de atención de reservas. Sucursal con existencias disponibles para las variantes solicitadas (RN10 y RN11). Conexión para registrar la reserva central.
Flujo normal	1. El actor selecciona cliente, sucursal y prendas. 2. El sistema consulta la disponibilidad (CU08). 3. Registra la reserva con retención de veinticuatro horas. 4. El personal prepara las prendas y confirma el pago cuando corresponde.

	5. Se entrega la reserva y se registra la venta asociada. 6. Se cancela o se deja vencer, liberando la retención.
Flujos alternativos	2a. El disponible es insuficiente: se rechaza. 3a. Ya existe una reserva para la misma unidad: se rechaza el duplicado. 4a. El pago continúa pendiente: no se entrega. 5a. La reserva ya fue atendida: no se duplica la venta. 5b. El cliente no es el titular: se deniega. 6a. La reserva vence: se libera la retención sin generar venta (RN11).
Postcondiciones	Reserva registrada, entregada o cancelada; retención liberada cuando corresponde; historial conservado.
Relaciones	Se vincula con CU13 para la entrega en tienda y con CU14 para el comprobante.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU13	
Caso de Uso	CU13 - Registrar venta presencial
Actor Principal	Vendedor o cajero, Encargado de sucursal y Administrador de Empresa
Plataforma	WEB
Descripción	Consolida la venta en la terminal de caja con los productos del ticket, la disponibilidad consultada y los importes calculados por el servidor, registrando el efectivo recibido y el vuelto. La confirmación genera el registro comercial, la salida única de inventario y la auditoría correspondiente, y deja disponible el comprobante como acción adicional.
Precondiciones	Sesión de cajero o encargado con sucursal y caja habilitadas. Productos existentes con disponibilidad suficiente (RN10). Conexión con el servidor y suscripción vigente de la empresa.
Flujo normal	1. El cajero selecciona la sucursal y la caja. 2. Agrega productos por SKU o carga una reserva de probador. 3. El sistema consulta la disponibilidad (CU08). 4. Calcula importes y descuentos en el servidor. 5. Registra el medio de pago, el efectivo recibido y el vuelto. 6. Confirma la venta y los movimientos (CU10) en una transacción. 7. Emite el comprobante y ofrece su descarga.
Flujos alternativos	2a. El SKU no existe o está inactivo: se rechaza. 3a. El stock es insuficiente: se rechaza. 4a. El precio fue manipulado: se recalcula o se rechaza (RN09). 5a. El efectivo recibido es insuficiente: no se confirma. 6a. Falla un paso intermedio: se revierte la venta completa. 6b. Se reenvía la confirmación: no se duplica. 7a. La generación del comprobante falla: la venta permanece confirmada y el documento puede recuperarse (RN15).
Postcondiciones	Venta presencial confirmada, stock descontado una sola vez, auditoría registrada y comprobante disponible.
Relaciones	CU14 extiende en el punto «venta registrada». Puede atender reservas de CU12.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU14	
Caso de Uso	CU14 - Obtener comprobantes de compra
Actor Principal	Cliente, Vendedor o cajero, Encargado de sucursal y Administrador de Empresa
Plataforma	WEB/MÓVIL
Descripción	Recupera o genera la representación del comprobante de una venta confirmada, verifica la empresa, el rol y la propiedad de la compra, y presenta los datos de la operación con su descarga en PDF. La consulta o descarga no crea un nuevo documento comercial ni altera la venta registrada.
Precondiciones	Venta confirmada en la empresa correspondiente. Sesión vigente con autorización o propiedad sobre la compra (RN15). Representación del comprobante disponible y conexión al servidor.
Flujo normal	1. El actor solicita el comprobante de una venta confirmada. 2. El sistema verifica empresa, rol y propiedad. 3. Recupera el documento existente o genera su representación. 4. Presenta los datos de la venta y los elementos de verificación. 5. Descarga el documento en PDF. 6. La venta conserva su registro comercial sin cambios.
Flujos alternativos	1a. La venta no existe o pertenece a otra empresa: no se encuentra. 1b. El pago no está confirmado: se impide el comprobante definitivo. 2a. El actor no tiene autorización: se deniega. 3a. La generación falla: se reintenta sin duplicar el documento. 5a. Se accede por una ruta privada directa: se deniega (RN15).
Postcondiciones	Comprobante consultado o descargado; sin nuevo documento comercial; venta sin modificaciones.
Relaciones	
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU15	
Caso de Uso	CU15 - Gestionar cambios y reembolsos
Actor Principal	Cliente; Encargado de sucursal y Administrador de Empresa. Participan la Pasarela de pagos y los Servicios de correo y notificaciones push
Plataforma	WEB/MÓVIL
Descripción	Registra y atiende solicitudes de cambio dentro de los siete días calendario posteriores a la entrega, verificando la compra, el estado y la elegibilidad de la prenda. La recepción física, el reemplazo, las diferencias de precio y el reembolso por el medio original se registran como eventos separados, con notificación y trazabilidad del resultado.
Precondiciones	Venta confirmada y comprobable con la prenda elegible, sin uso y dentro del plazo (RN22). Sesión del cliente o del personal autorizado. Pasarela disponible para cobros o devoluciones y suscripción vigente de la empresa.
Flujo normal	1. El cliente consulta su compra y las unidades adquiridas. 2. El sistema verifica plazo, estado y elegibilidad.

	3. Se registra la solicitud de cambio. 4. El personal recibe e inspecciona la prenda. 5. Se asigna el reemplazo o se determina la devolución. 6. Se cobra o se devuelve la diferencia y se notifica al cliente. 7. Se registra la entrega o el reembolso.
Flujos alternativos	2a. La solicitud está fuera de los siete días: se rechaza (RN22). 2b. La unidad ya fue cambiada: se rechaza. 2c. La prenda no es elegible: se informa el motivo. 4a. La prenda no es conforme en la recepción: se rechaza el cambio. 5a. No existe reemplazo: la solicitud queda pendiente y puede cancelarse con devolución del crédito (RN24). 6a. La diferencia no es aceptada: se cancela el reemplazo y se compensa. 6b. La notificación falla: se conserva en la bandeja del servicio. 7a. El reembolso no sigue el medio original: se rechaza.
Postcondiciones	Solicitud atendida, pendiente o cancelada; stock repuesto; diferencia cobrada o devuelta; notificación registrada y trazabilidad conservada.
Relaciones	Requiere CU14 para la compra comprobable y CU10 para la reposición. Participan la pasarela y el servicio de notificaciones.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU16	
Caso de Uso	CU16 - Registrar y consultar operaciones offline
Actor Principal	Cliente, Vendedor o cajero, Encargado de sucursal y Administrador de Empresa
Plataforma	WEB/MÓVIL
Descripción	Descarga el catálogo y la disponibilidad conocida para consultarlos sin conexión y registrar reservas o ventas de contado según el rol del usuario. Cada operación conserva su identificador, autor, empresa, sucursal y estado provisional, y permanece disponible después de cerrar o reiniciar la aplicación.
Precondiciones	Autenticación previa con conexión y datos descargados en el dispositivo. Autorización local con vigencia máxima de veinticuatro horas, limitada por la suscripción conocida (RN16). Almacenamiento local disponible: IndexedDB en web y SQLite en móvil.
Flujo normal	1. El usuario se autentica con conexión y descarga el catálogo y la disponibilidad. 2. El sistema guarda el snapshot y la vigencia offline. 3. El actor consulta los datos locales. 4. Registra una reserva o una venta de contado con un UUID. 5. El sistema guarda cabecera, detalles y pendiente de forma atómica. 6. El actor consulta las operaciones después de reiniciar la aplicación. 7. El sistema emite la constancia provisional.
Flujos alternativos	1a. La descarga se interrumpe: se conserva el snapshot anterior. 2a. No hay espacio de almacenamiento: se informa sin borrar pendientes. 3a. La sesión offline está vencida: se bloquea el registro (RN16). 3b. Se inicia con otra cuenta: no se muestran datos de la sesión anterior. 4a. El stock local es insuficiente: se advierte o se impide la cantidad.

	4b. El rol no permite operar offline: se impide la operación. 5a. Falla el guardado local: se revierte y no se declara éxito.
Postcondiciones	Operación provisional persistida con UUID y estado; pendiente de sincronizar; constancia disponible tras cerrar o reiniciar la aplicación.
Relaciones	Requiere CU01. Precede a CU17 y utiliza el catálogo de CU09 y la disponibilidad de CU08.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU17	
Caso de Uso	CU17 - Sincronizar operaciones y resolver conflictos
Actor Principal	Cliente, Vendedor o cajero, Encargado de sucursal y Administrador de Empresa
Plataforma	WEB/MÓVIL
Descripción	Envía las operaciones pendientes de forma manual o automática, conservando el UUID y el contenido entre intentos, y recupera un resultado por operación: confirmación, rechazo o conflicto. Los errores temporales mantienen el pendiente, y la corrección de un conflicto genera una operación nueva enlazada o una compensación documentada sin ocultar el historial.
Precondiciones	Operaciones pendientes registradas localmente. Conectividad y sesión vigente con los permisos necesarios. Suscripción conocida por el servidor y contenido de la operación sin modificaciones respecto del original (RN18 y RN19).
Flujo normal	1. El sistema detecta conectividad o recibe la solicitud manual de envío. 2. Envía el lote de pendientes sin cambiar UUID ni contenido. 3. Valida sesión, suscripción y reglas por operación. 4. Confirma la transacción de forma idempotente. 5. Guarda el resultado de cada operación en el dispositivo. 6. Consulta y resuelve los conflictos detectados.
Flujos alternativos	2a. Un UUID aparece con contenido distinto: se rechaza (RN18). 3a. La suscripción está vencida: se rechazan las operaciones comerciales nuevas (RN07). 3b. Los permisos fueron revocados: no se confirma la operación (RN20). 4a. Se pierde la respuesta después del commit: se consulta y se reenvía sin duplicar. 4b. La sesión expira (error 401): se espera la reautenticación sin perder pendientes. 5a. Ocurre un error temporal: el pendiente se conserva. 6a. El usuario no tiene permiso para resolver: se deniega. 6b. Se acepta la corrección: se crea una operación nueva enlazada (RN21). 6c. El precio no es aceptado: se registra una compensación documentada (RN17).
Postcondiciones	Cada pendiente con resultado confirmado, rechazado o en conflicto; UUID e historial conservados; conflictos resueltos o pendientes de decisión.
Relaciones	Requiere CU16. Valida con CU08 y CU10, y registra sus eventos en CU20.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU18

Caso de Uso	CU18 - Obtener reportes comerciales
Actor Principal	Administrador de Empresa y Encargado de sucursal; Servicio de inteligencia artificial
Plataforma	WEB/MÓVIL
Descripción	Genera reportes convencionales con filtros de fecha, sucursal, categoría y producto, y atiende solicitudes escritas o dictadas por voz que la inteligencia artificial interpreta en parámetros autorizados. Los resultados se presentan en tabla o gráfico y pueden exportarse a PDF y Excel conservando los datos y filtros mostrados.
Precondiciones	Sesión con permiso de reportes en la empresa activa. Datos comerciales registrados en el periodo consultado. Para reportes generativos, plan Profesional vigente y autorización del usuario (RN06). Conexión al servidor.
Flujo normal	1. El usuario selecciona un reporte convencional o escribe o dicta una solicitud. 2. El sistema valida permisos y plan. 3. Verifica o interpreta los filtros de la solicitud. 4. Ejecuta la consulta parametrizada sobre datos autorizados. 5. Presenta la tabla o el gráfico. 6. Exporta a PDF o Excel los mismos resultados.
Flujos alternativos	1a. El dictado no está disponible: se habilita la escritura de la solicitud. 2a. El plan Básico solicita un reporte generativo: se bloquea la función (RN06). 2b. Se recibe un filtro de otra empresa: se rechaza. 3a. La solicitud es ambigua: se pide precisión antes de consultar. 4a. El servicio de inteligencia artificial no responde: se informa y se ofrece el reporte convencional. 4b. Se solicita SQL libre o una empresa arbitraria: se rechaza (RN25). 4c. La consulta excede el alcance permitido: se limita (RNF09). 5a. No hay datos: se muestra el resultado vacío. 6a. La exportación falla: se reintenta sin alterar los datos.
Postcondiciones	Reporte generado y, si se solicitó, exportado. Los datos de la empresa no se modifican.
Relaciones	Requiere CU01. Los reportes generativos dependen de CU19 y del plan Profesional.
Prototipo	

ESPECIFICACIÓN DEL CASO DE USO CU19	
Caso de Uso	CU19 - Preparar y consultar analítica inteligente
Actor Principal	Administrador de Empresa y Encargado de sucursal
Plataforma	WEB
Descripción	Comprende el subflujo técnico de preparación de datos sintéticos, entrenamiento, evaluación y versionado de modelos supervisados y no supervisados, y el subflujo empresarial de consulta de indicadores, predicciones y segmentos en el dashboard. Los resultados se presentan con sus métricas y limitaciones, identificando el origen sintético de los datos de entrenamiento.

Precondiciones	Modelos previamente evaluados y versionados; abrir el dashboard no ejecuta entrenamiento (RN28). Sesión con permiso de analítica en la empresa autorizada. Para el entrenamiento, herramienta técnica y datos preparados fuera del servidor de producción.
Flujo normal	1. El responsable técnico prepara datos sintéticos reproducibles y variables. 2. Entrena los modelos supervisado y no supervisado. 3. Evalúa y registra las métricas obtenidas. 4. Versiona los artefactos y su procedencia. 5. El usuario empresarial consulta los KPI, predicciones y segmentos de su empresa. 6. El sistema presenta los resultados con sus métricas y limitaciones.
Flujos alternativos	1a. Los datos son insuficientes: se impide el entrenamiento. 2a. Se detecta fuga de variables o datos inválidos: se rechaza el conjunto. 4a. El artefacto tiene una versión incompatible: se rechaza su carga. 5a. No existe modelo disponible: se mantienen los KPI y se advierte. 5b. Se consulta una empresa ajena: se deniega. 6a. No hay historial suficiente: se informa la limitación.
Postcondiciones	Modelos versionados con sus métricas y dashboard con resultados del ámbito autorizado. Abrir el dashboard no ejecuta entrenamiento ni modifica los artefactos.
Relaciones	Requiere CU01. Alimenta CU18. El entrenamiento es un subflujo técnico ejecutado fuera de producción (RN28).
Prototipo	

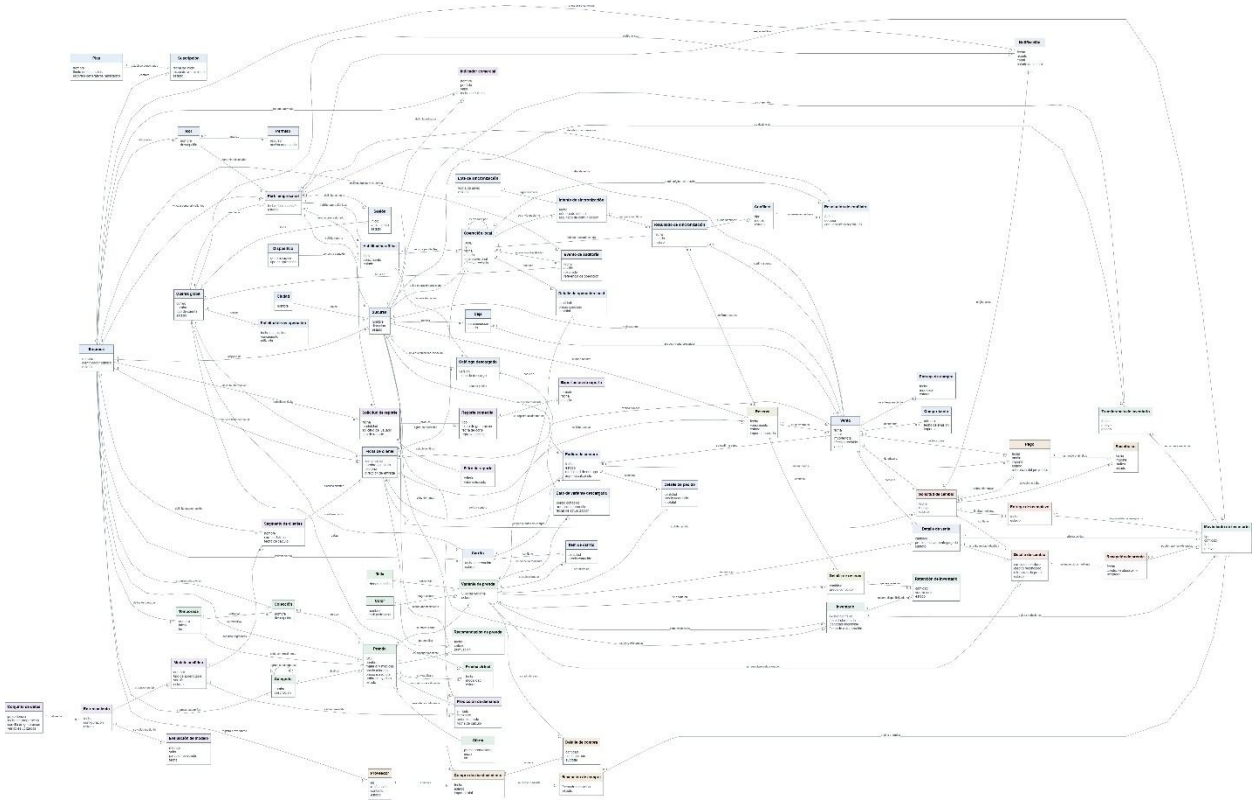
ESPECIFICACIÓN DEL CASO DE USO CU20	
Caso de Uso	CU20 - Consultar auditoría y trazabilidad
Actor Principal	SuperAdministrador SaaS, Administrador de Empresa y Encargado de sucursal
Plataforma	WEB
Descripción	Consulta los eventos autorizados de acceso, operaciones, pagos, cambios y sincronización, con filtros por fecha, usuario, acción o UUID, y relaciona responsable, acción y resultado. La presentación conserva la evidencia del evento sin exponer contraseñas, tokens ni credenciales de servicios externos.
Precondiciones	Sesión con permiso de auditoría y ámbito definido por rol y empresa. Eventos registrados en el periodo consultado. Conexión al servidor.
Flujo normal	1. El actor se autentica con permiso de auditoría. 2. Aplica filtros de fecha, usuario, acción o UUID. 3. Consulta los eventos paginados. 4. Relaciona responsable, acción y resultado. 5. Presenta la evidencia sin datos sensibles.
Flujos alternativos	1a. El usuario no tiene permiso de auditoría: se deniega. 2a. Se consulta una empresa ajena: no se encuentra. 3a. Los filtros son demasiado amplios: se pagina y se limita el resultado (RNF09). 4a. El evento no tiene correlación completa: se muestran los datos disponibles.
Postcondiciones	Eventos consultados sin modificar el registro; evidencia disponible sin contraseñas, tokens ni credenciales.

Relaciones	Recibe los eventos de CU01 a CU19.
Prototipo	

3. CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX

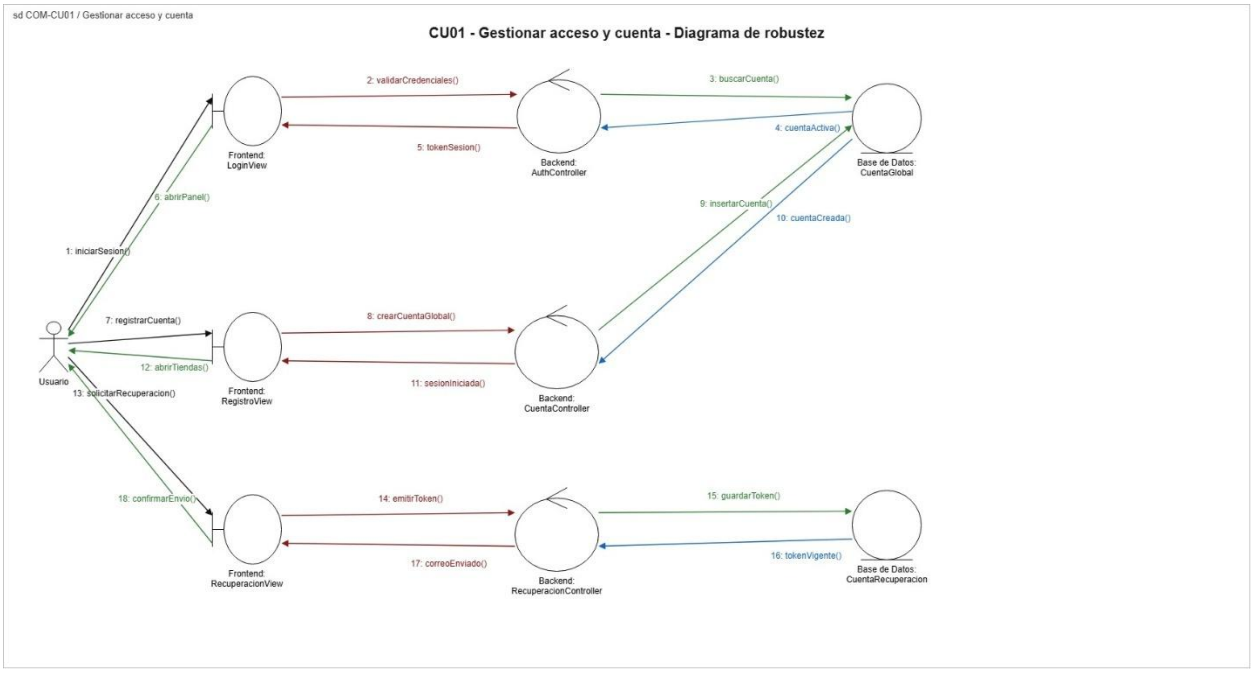
3.1. Modelo de dominio

El modelo de dominio de FashionStore representa los principales conceptos del negocio y las relaciones necesarias para comprender el funcionamiento de la plataforma. Su alcance comprende la administración del servicio SaaS, la organización de las empresas y sucursales, la identidad de los usuarios, el catálogo de prendas, el abastecimiento, los inventarios y las operaciones comerciales. Asimismo, incorpora los conceptos relacionados con la operación sin conexión, la sincronización, los reportes y la analítica inteligente.

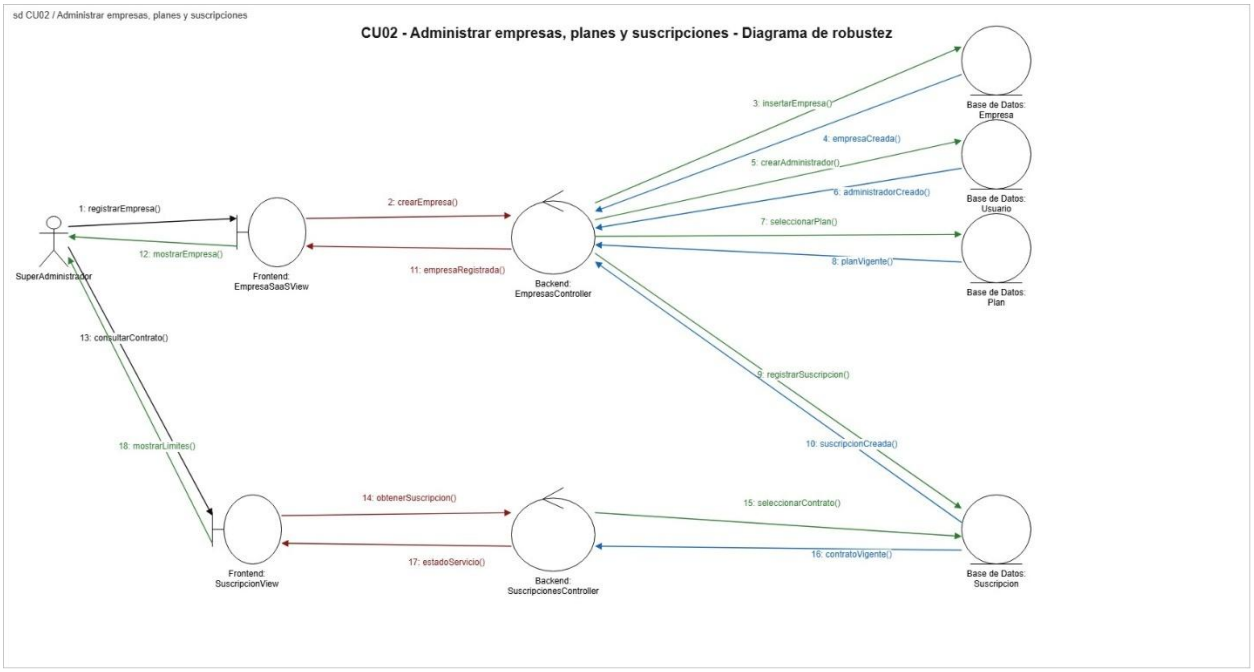


3.2. Diagramas de robustez

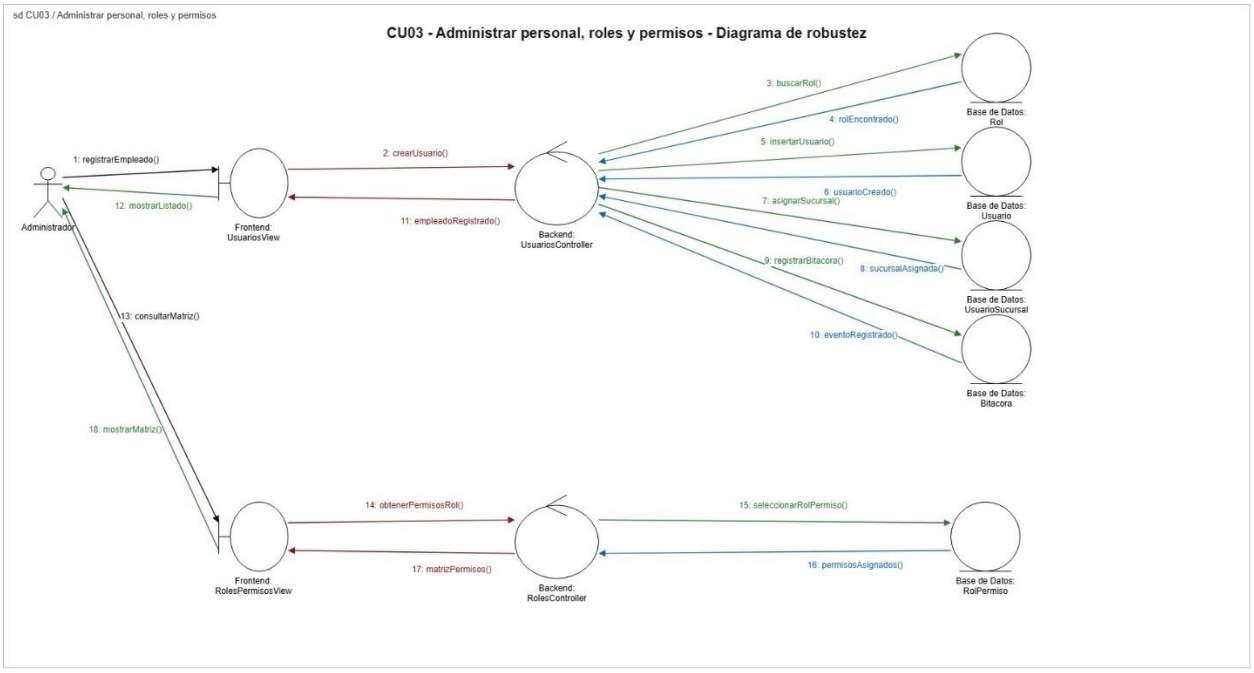
CU01 - Gestionar acceso y cuenta



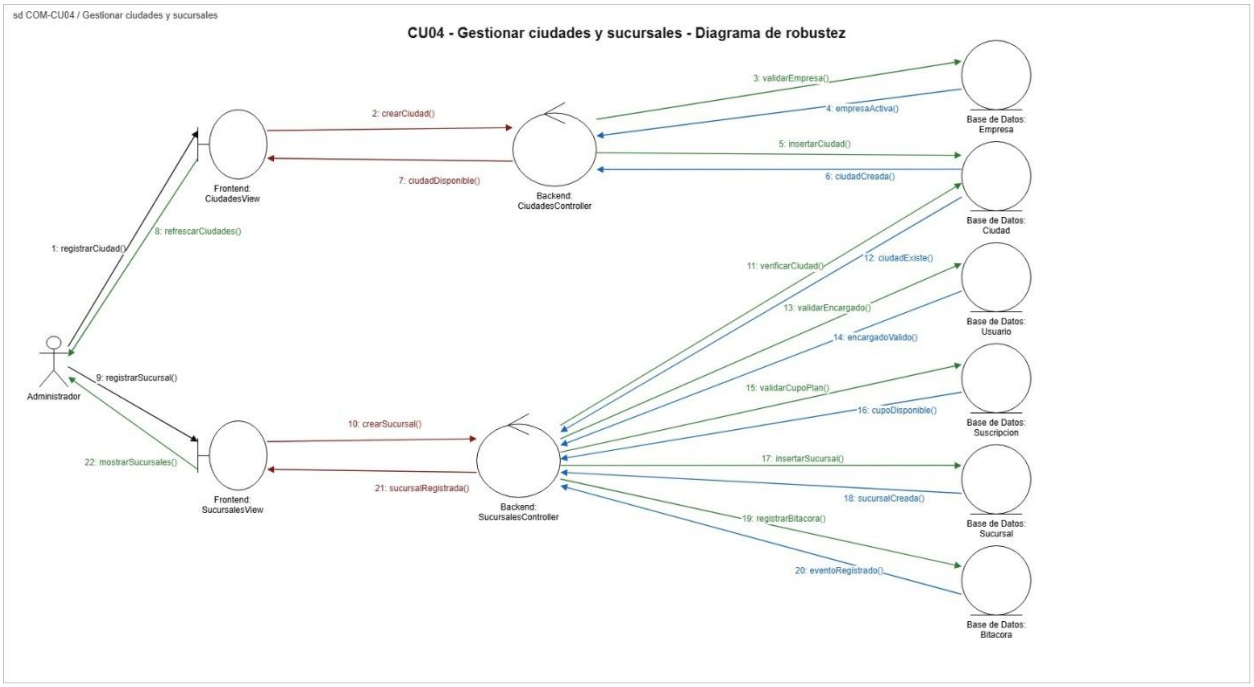
CU02 - Administrar empresas, planes y suscripciones



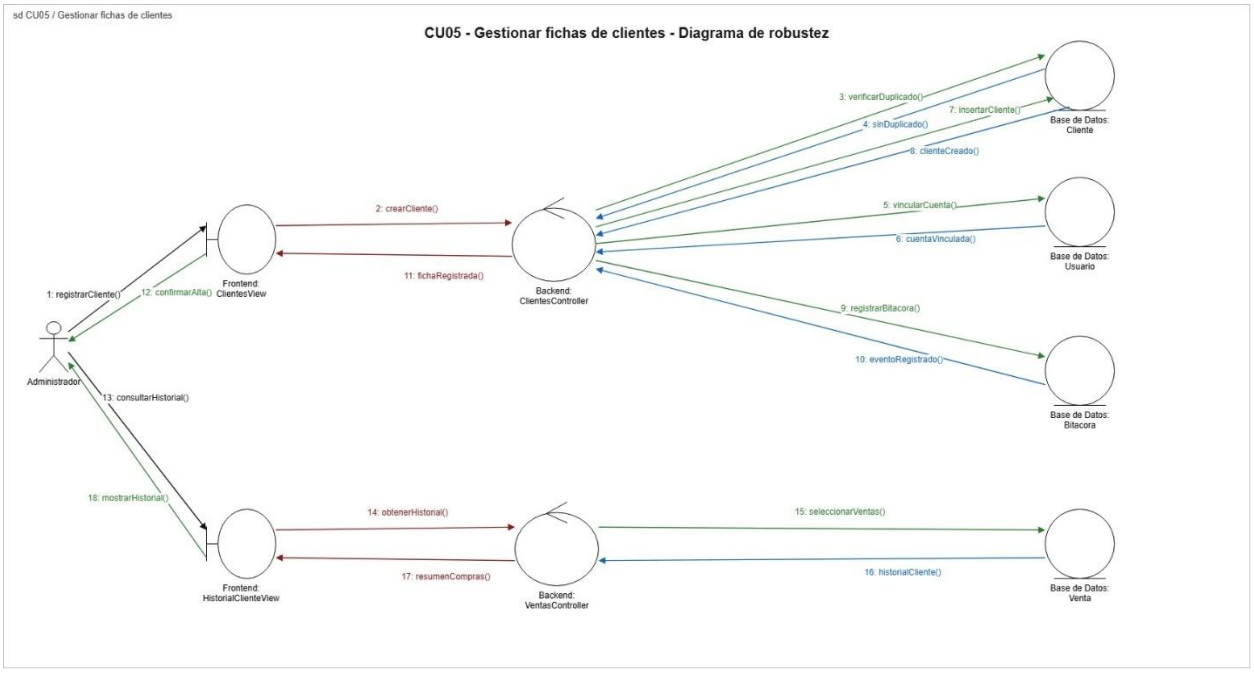
CU03 - Administrar personal, roles y permisos



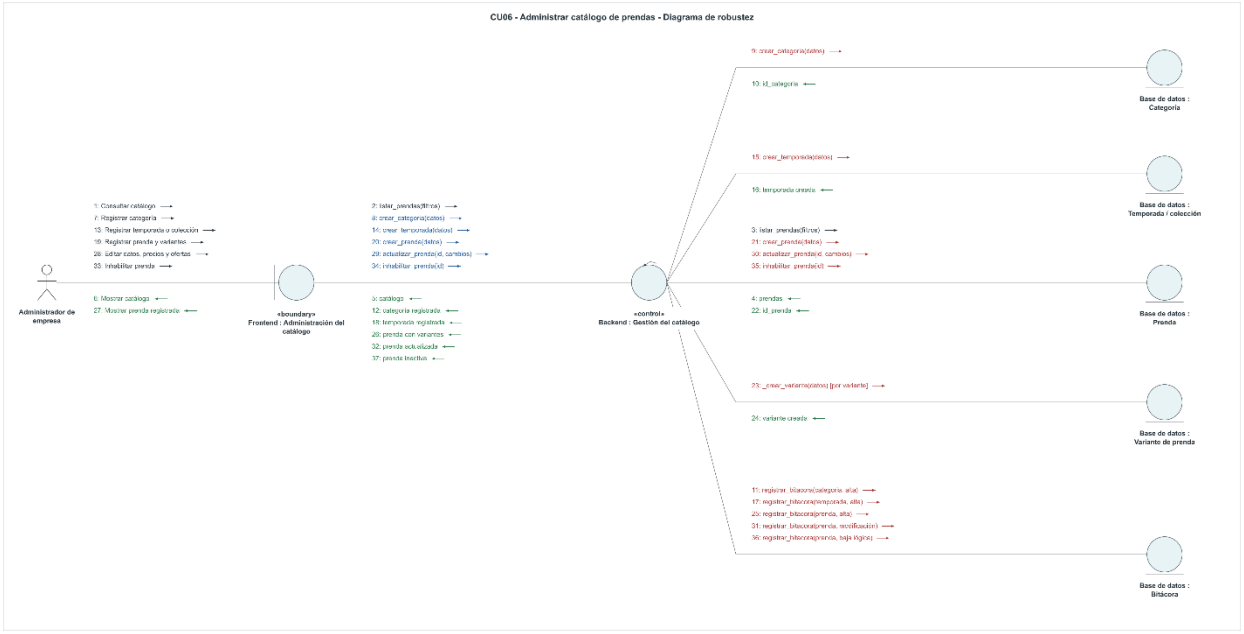
CU04 - Gestionar ciudades y sucursales



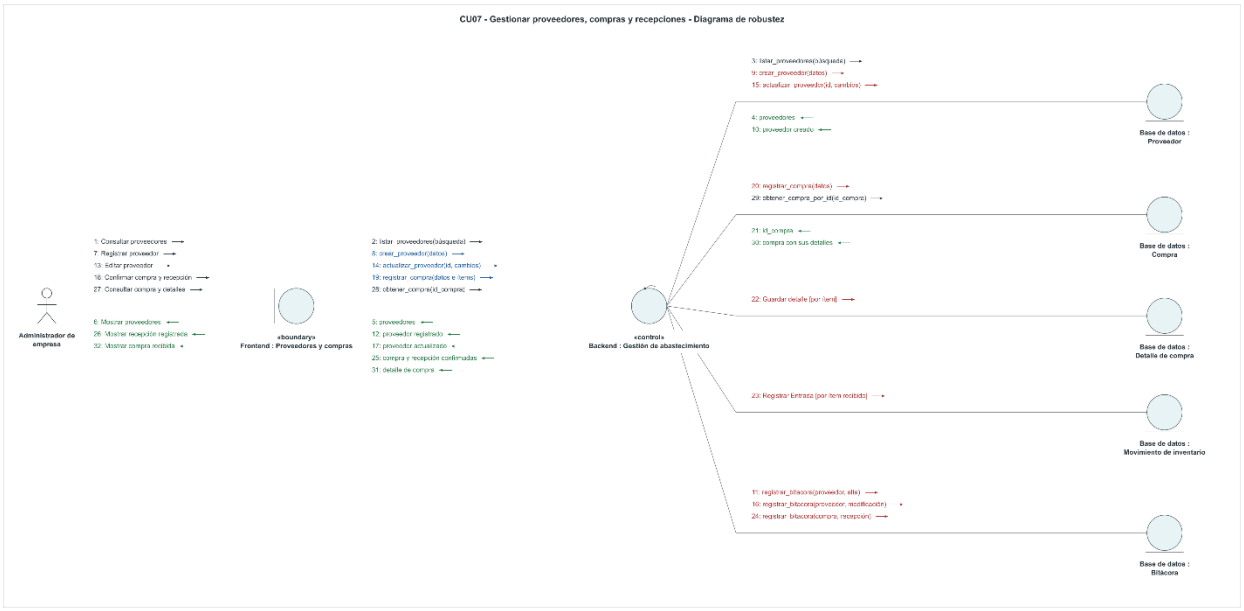
CU05 - Gestionar fichas de clientes



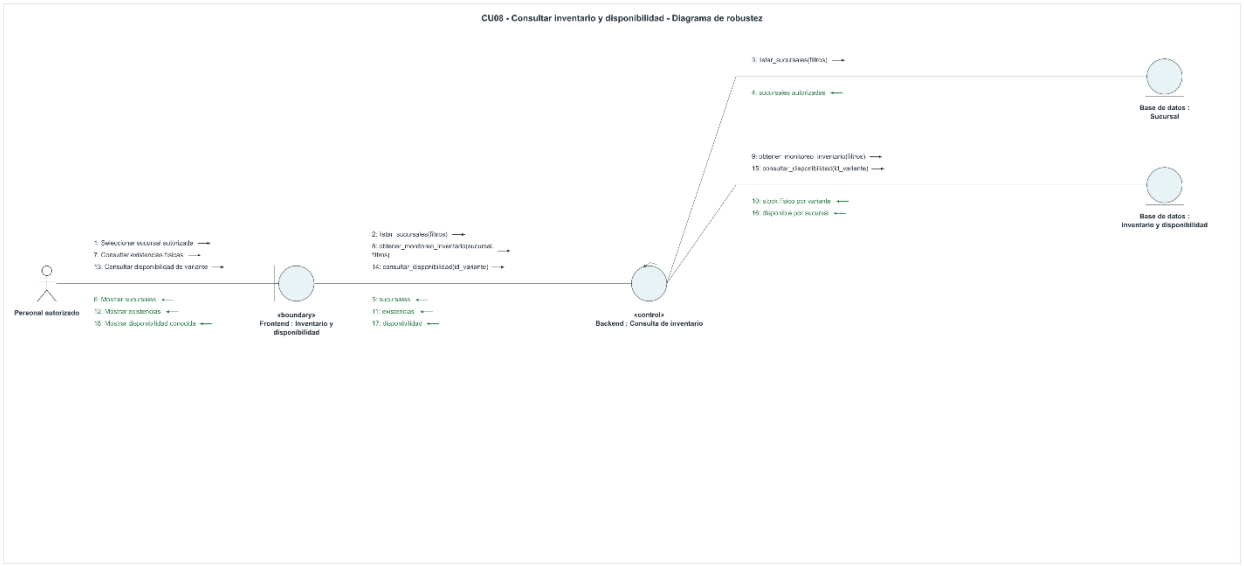
CU06 - Administrar catálogo de prendas



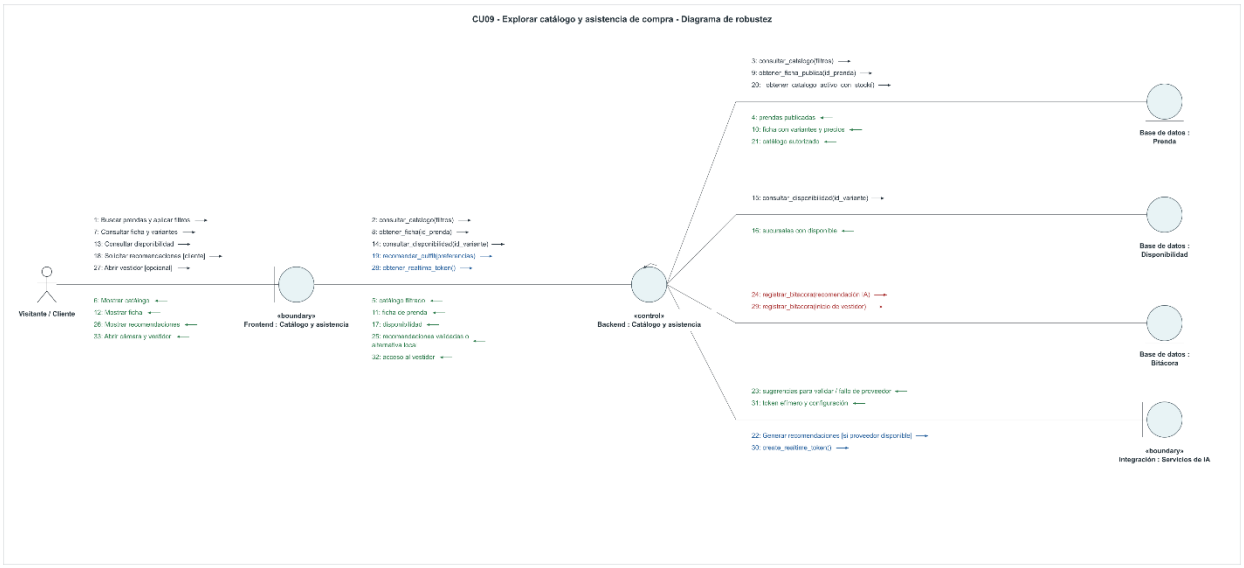
CU07 - Gestionar proveedores, compras y recepciones



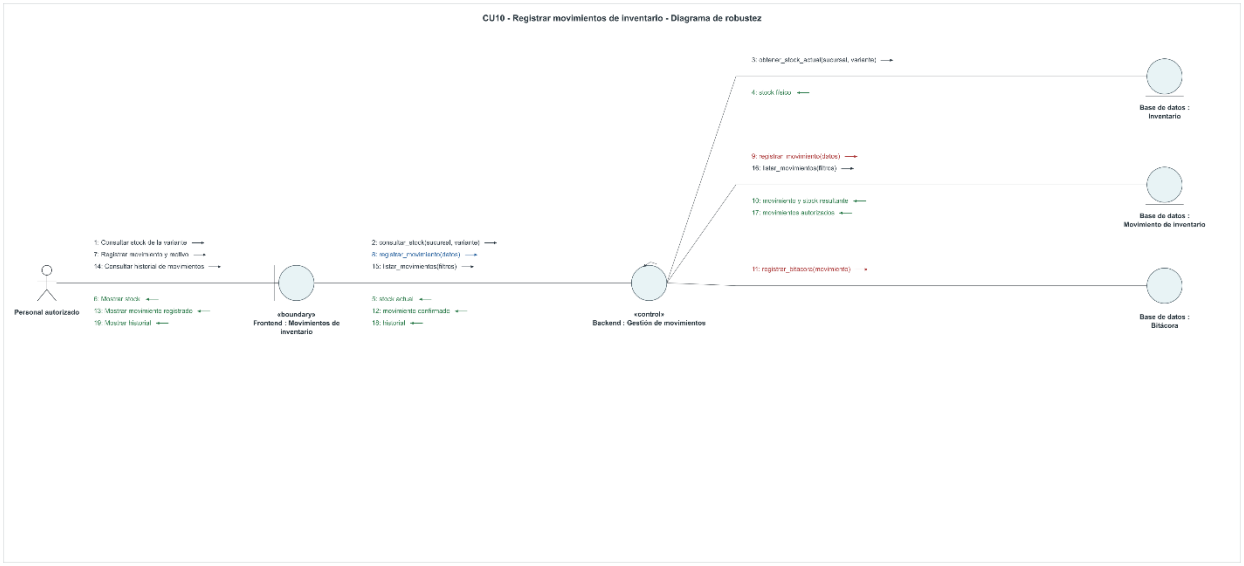
CU08 - Consultar inventario y disponibilidad



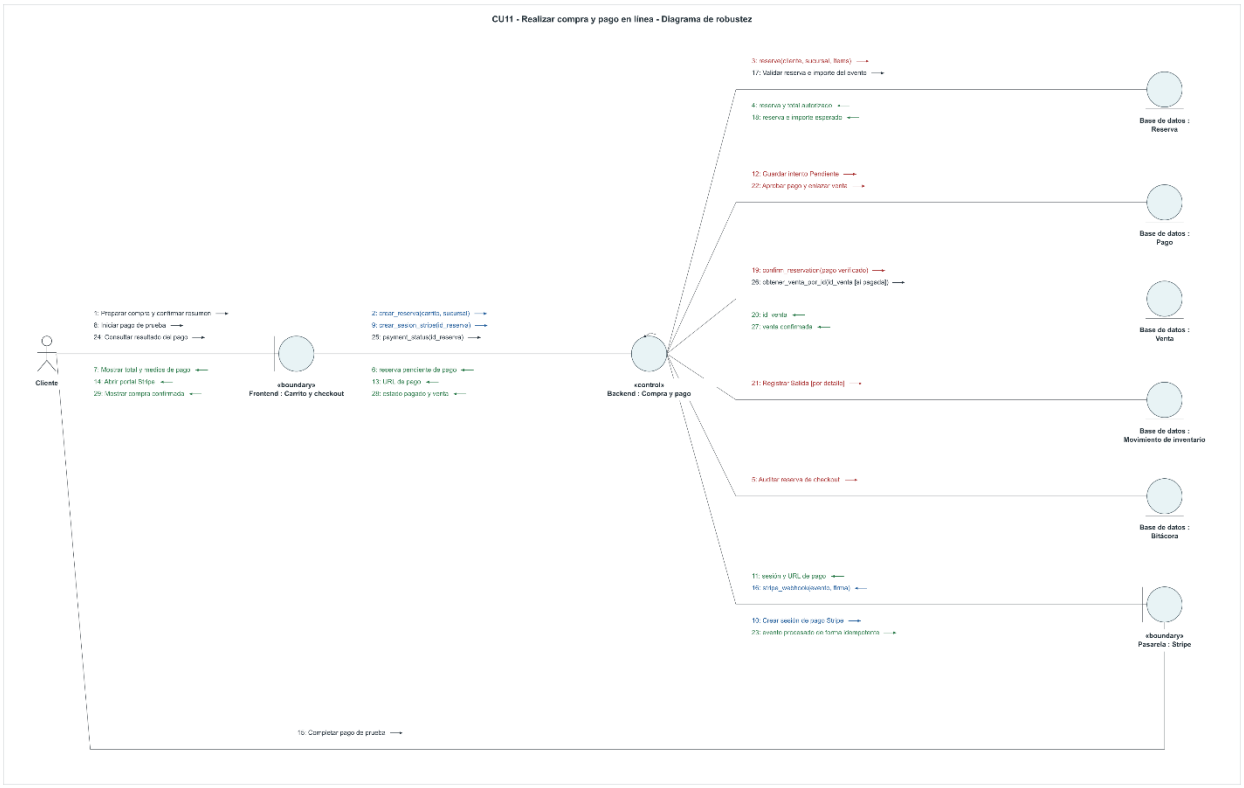
CU09 - Explorar catálogo y asistencia de compra



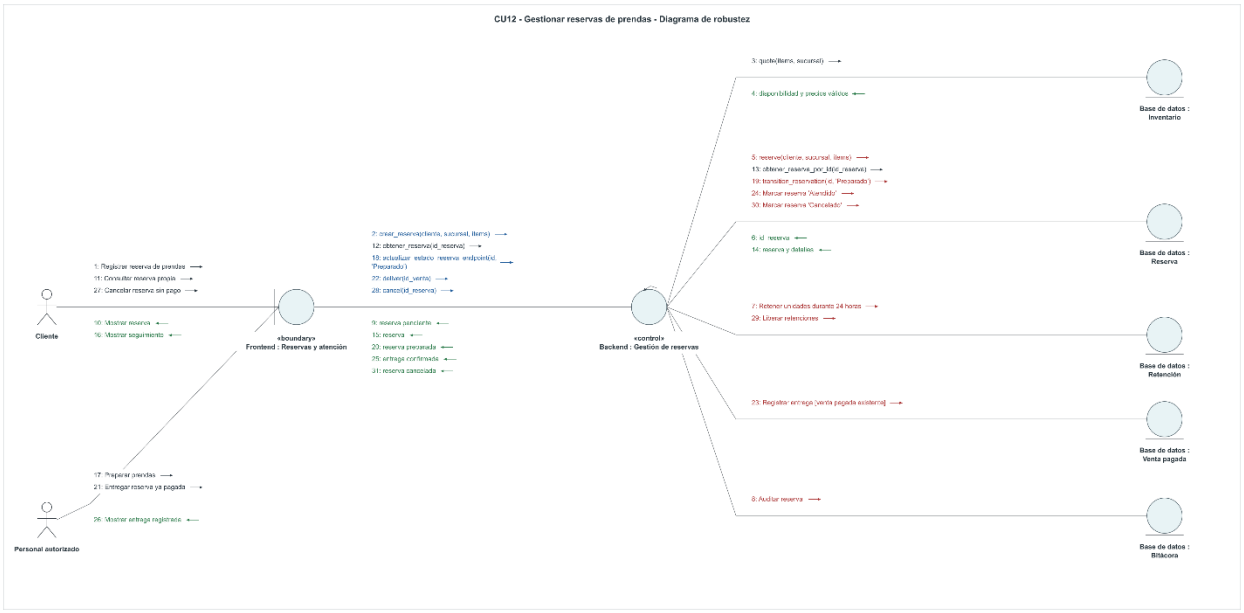
CU10 - Registrar movimientos de inventario



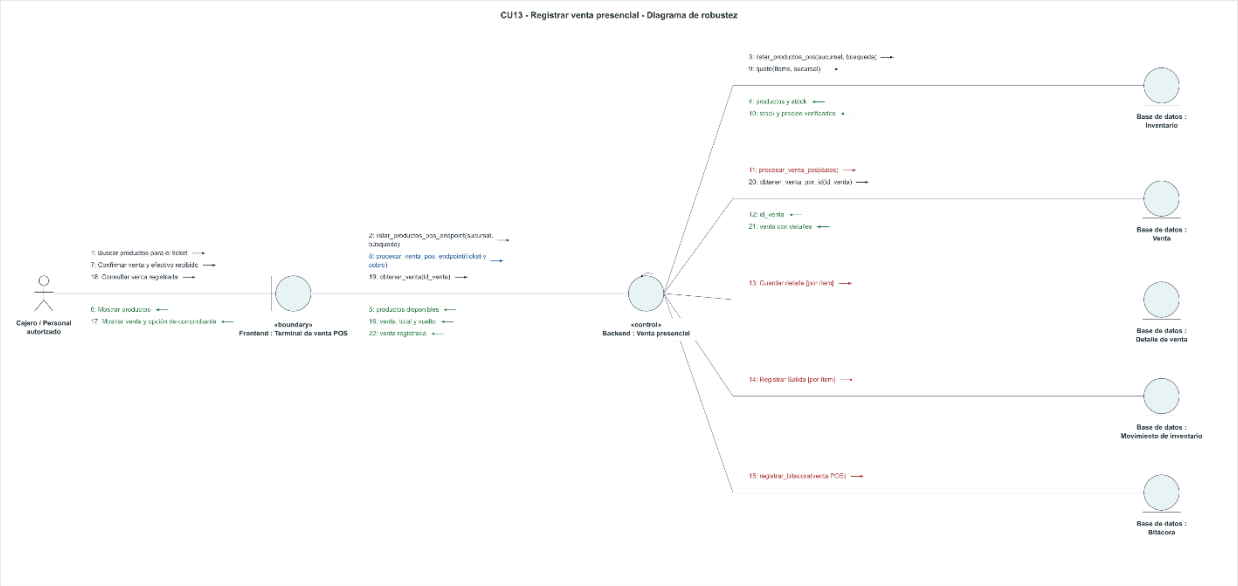
CU11 - Realizar compra y pago en línea



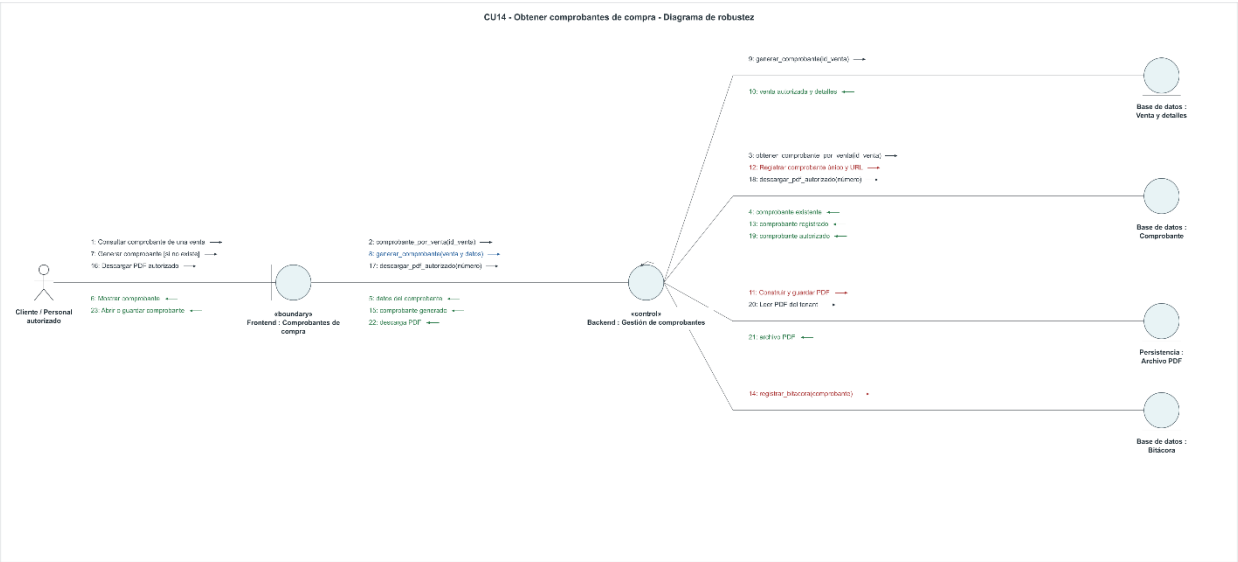
CU12 - Gestionar reservas de prendas



CU13 - Registrar venta presencial

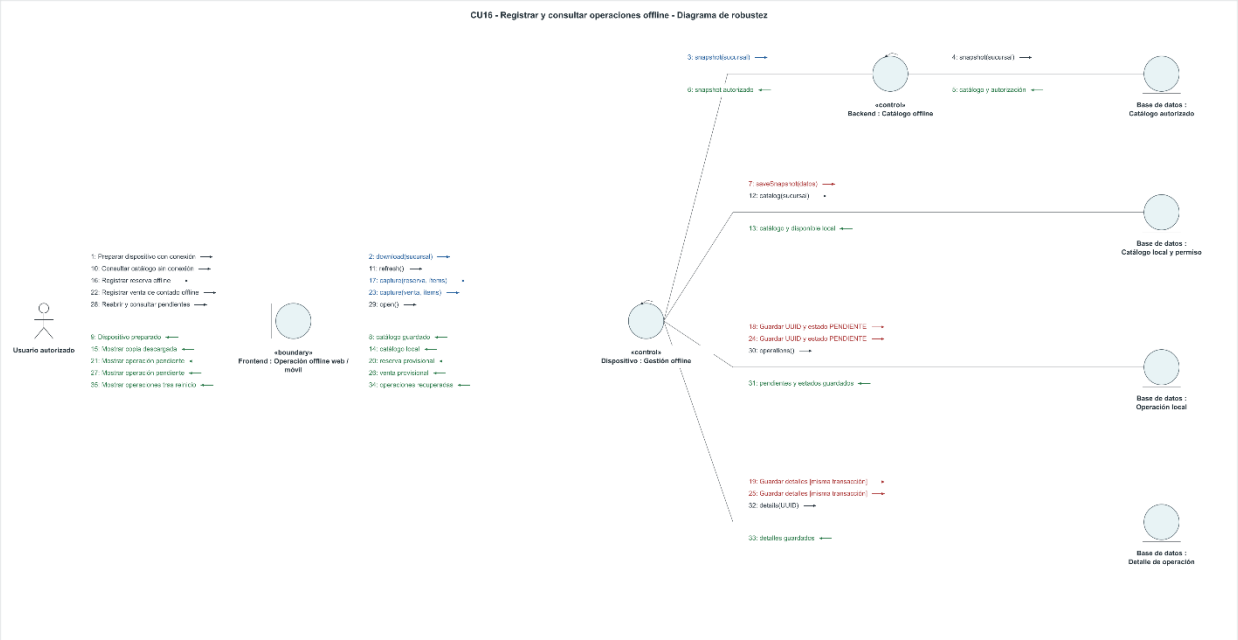


CU14 - Obtener comprobantes de compra



CU15 - Gestionar cambios y reembolsos

CU16 - Registrar y consultar operaciones offline



CU17 - Sincronizar operaciones y resolver conflictos

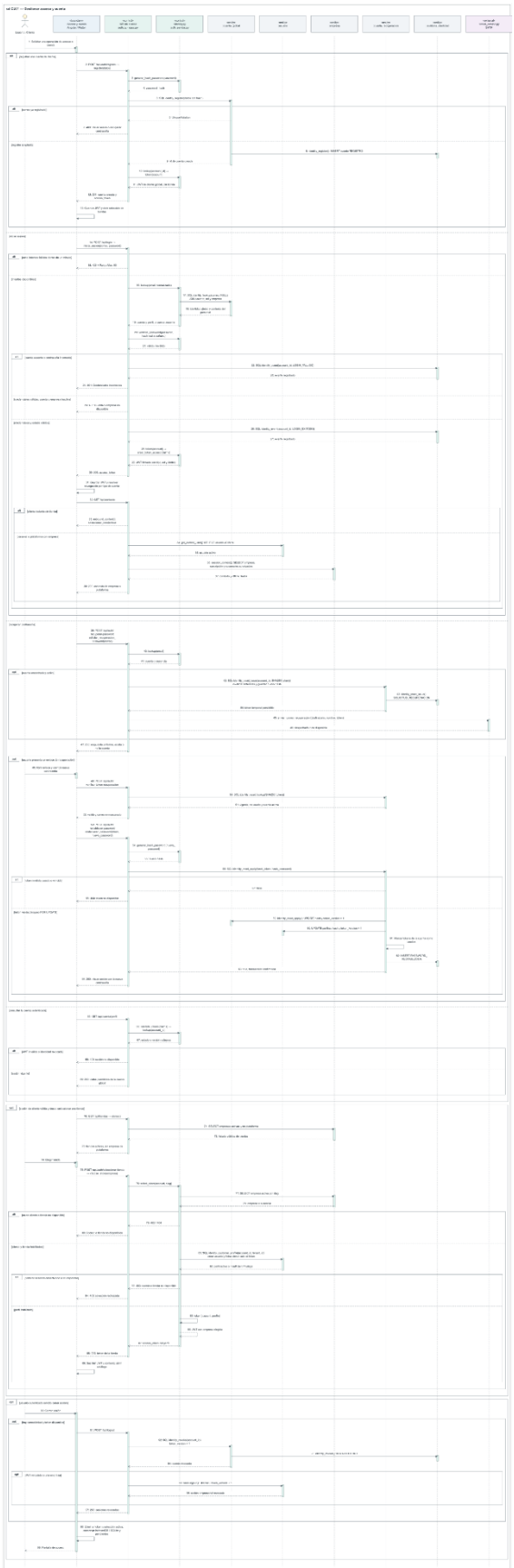
CU18 - Obtener reportes comerciales

CU19 - Preparar y consultar analítica inteligente

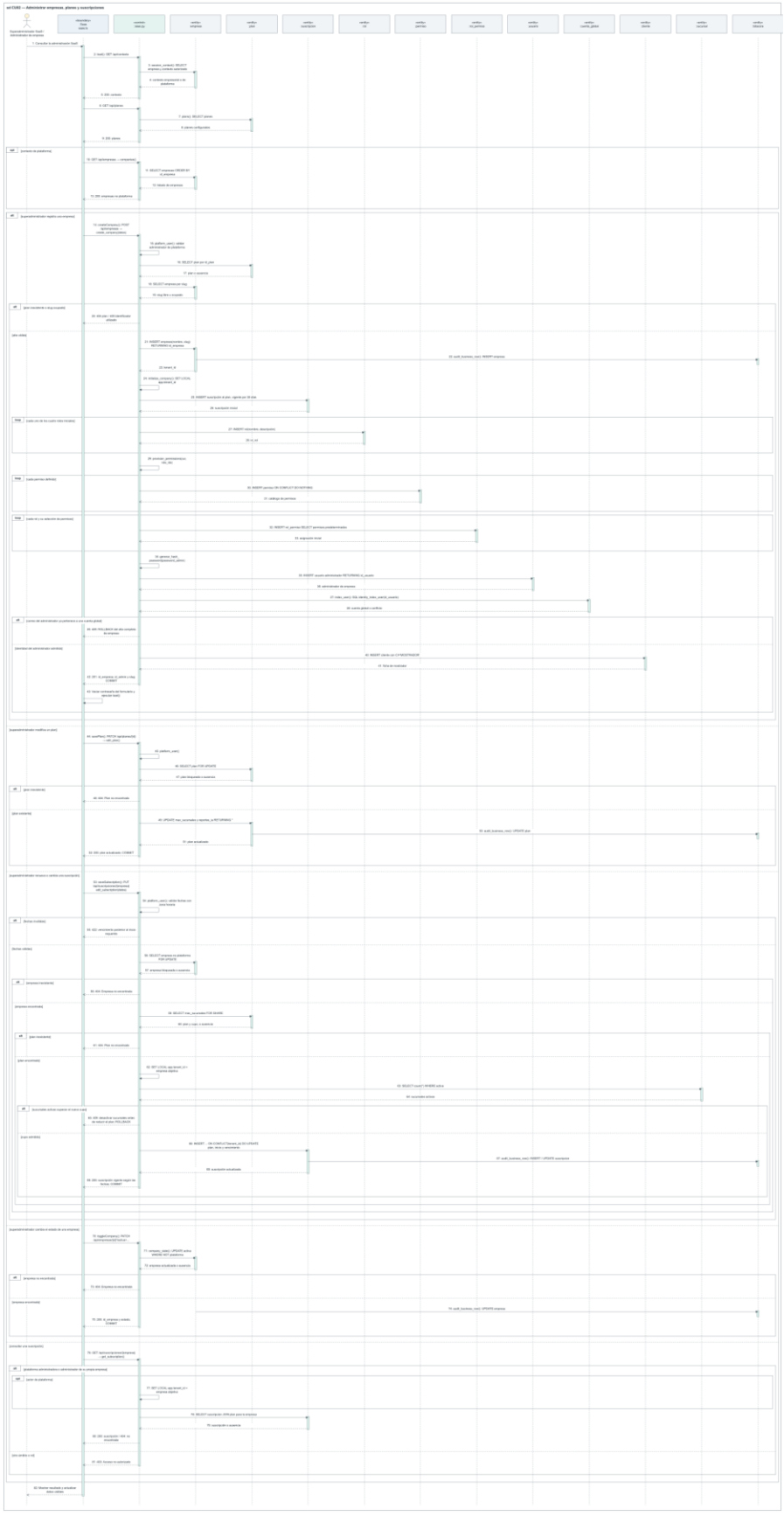
CU20 - Consultar auditoría y trazabilidad

3.3. Diagramas de secuencia UML

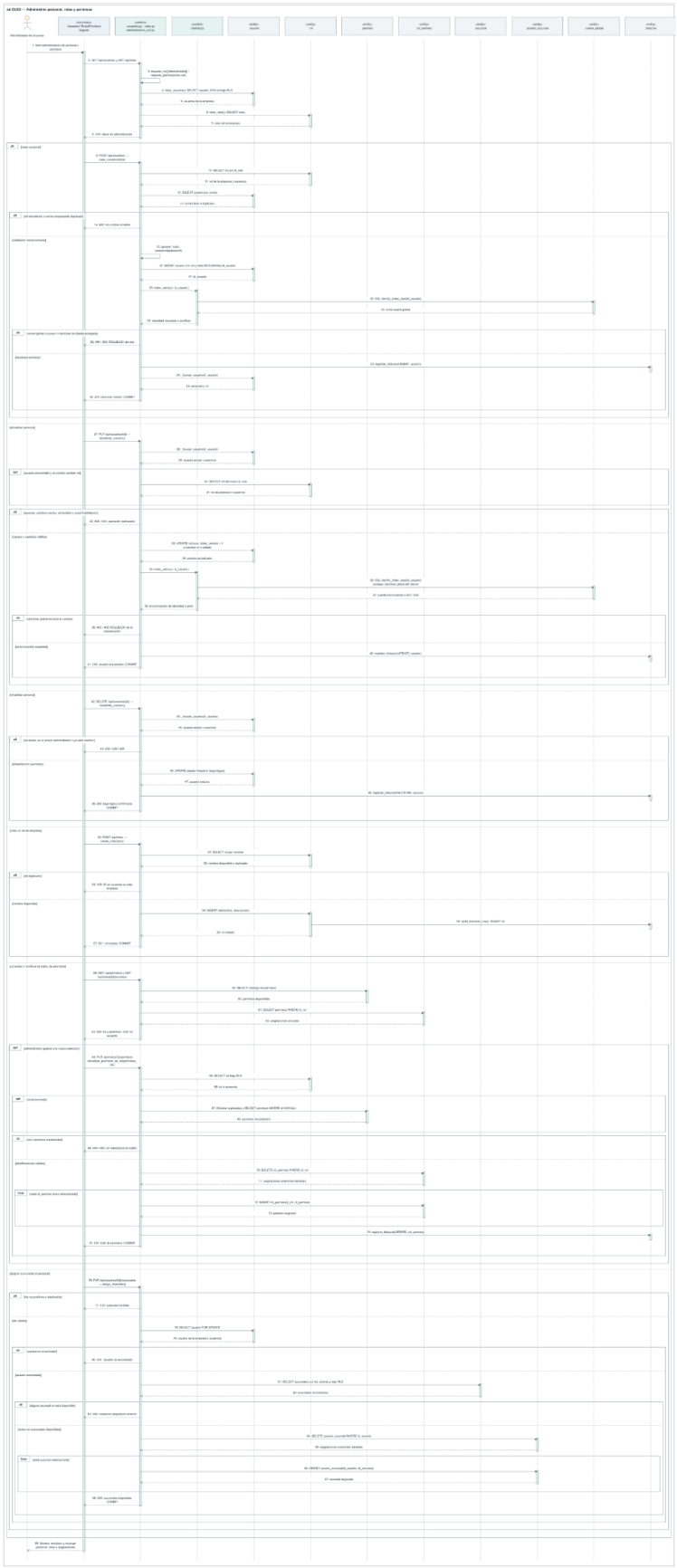
CU01 - Gestionar acceso y cuenta



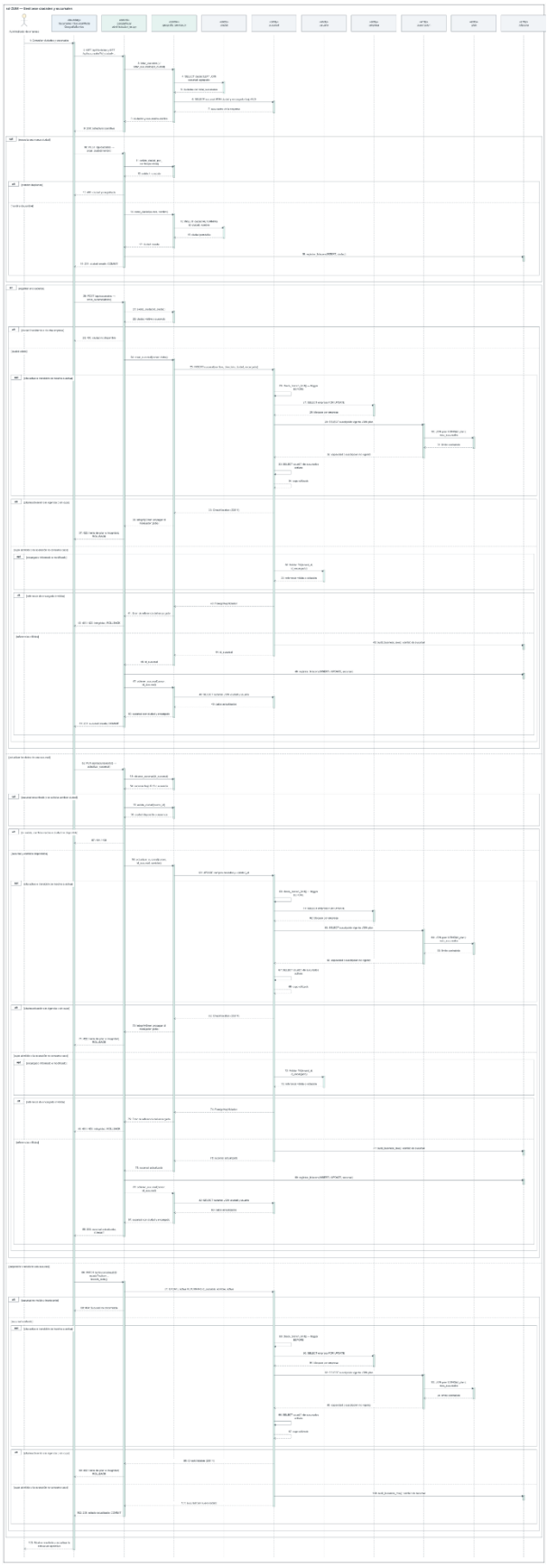
CU02 - Administrar empresas, planes y suscripciones


uc1 CU02 — Administrar empresas, planes y suscripciones
Superadministrador Saas / Administrador de empresa
«frontend.py» Clase cliente
«servicio» saas.py
«servicio» empresa
«servicio» plan
«servicio» suscripcion
«servicio» rol
«servicio» permiso
«servicio» rol_permiso
«servicio» usuario
«servicio» cuenta_global
«servicio» cliente
«servicio» sucursal
«servicio» billetera
1: Consultar la administración SaaS
2: host(): GET /admin/tenants
3: session_context(): SELECT empresa y contexto autorizado
4: contexto empresarial o de plataforma
5: 200: contexto
6: GET /api/planes
7: plans(): SELECT planes
8: planes configurables
9: 200: planes
opt [contexto de plataforma]
10: GET /api/empresas — compañías/
11: SELECT empresas ORDER BY id_empresa
12: listado de empresas
13: 200: empresas no plataforma
alt [superadministrador registra una empresa]
14: createCompany(): POST /api/empresas — create_company(datos)
15: platform_user(): validar administrador de plataforma
16: SELECT plan por id_plan
17: plan o ausencia
18: SELECT empresa.ruc= ruc
19: ruc libre o ocupado
alt [ruc inexistente o ruc ocupado]
20: 409 plan / ABC Identificador utilizado
[alta válida]
21: INSERT empresas(nombre, ruc) RETURNING id_empresa
22: tenant_id
23: SCHEMA_CREATE(): SET LOCAL search_path id
24: INSERT suscripción al plan, vigente por 30 días
25: suscripción inicial
loop [cada uno de los cuatro roles iniciales]
27: INSERT roles(rol, descripción)
28: id_rol
29: provisión_permisos(empresa, rol_id)
loop [cada permiso definido]
30: INSERT permiso ON CONFLICT DO NOTHING
31: catálogo de permisos
loop [cada rol y su selección de permisos]
32: INSERT rol_permiso SELECT permisos predeterminados
33: asignación inicial
34: genesis_hash, password(password_admin)
35: INSERT usuario administrador RETURNING id_usuario
36: administrador de empresa
37: hide(), view(): SSL Identity, status_code(id_usuario)
38: cuenta global o conflicto
alt [correo del administrador ya pertenece a una cuenta global]
39: 409 RESOLÁNCE tell who complete the empresa
[semilla del administrador admitida]
40: INSERT clients con CI Y MODIFINADOR
41: Tarta de modelado
42: OK si empresa, id_admin y ruta CONNECT
43: Cerrar conchaarta del formulario y ejecutar back()
[superadministrador modifica un plan]
44: savePlan(): PATCH /api/planes/{id}/ edit_plan()
45: platform_user()
46: SELECT plan FOR UPDATE
47: plan bloqueado o ausencia
alt [plan inexistente]
48: 404: Plan no encontrado
[plan existente]
49: UPDATE max_sucursales y reportes_ia RETURNING *
50: audit_business_row(): UPDATE plan
51: plan actualizado
52: 200: plan actualizado / CONNECT
[superadministrador renueva o cambia una suscripción]
53: renewSubscription(): PUT /api/empresas/{id}/suscripcion/ edit_subscription(datos)
54: platform_user(): validar fechas con nueva fecha
alt [fechas inválidas]
55: 422: vencimiento posterior al inicio requerido
[fechas válidas]
56: SELECT empresa no plataforma FOR UPDATE
57: empresa bloqueada o ausencia
alt [empresa inexistente]
58: 404: Empresa no encontrada
[empresa encontrada]
59: SELECT max_sucursales FOR SHARE
60: plan y cupo o ausencia
alt [plan inexistente]
61: 404: Plan no encontrado
[plan encontrado]
62: GET /2026: app.tenant_cr= empresa objetos
63: SELECT count(*) WHERE activa
64: sucursales activas
alt [sucursales activas superan el nuevo cupo]
65: 409: desactivar sucursales antes de reducir el plan / RESILIENCE
[cupo admitido]
66: INSERT ... ON CONFLICT(tenant_id) DO UPDATE plan, vence o renovamos
67: audit_business_row(): INSERT / UPDATE suscripcion
68: suscripción actualizada
69: 200: suscripción vigente según las fechas / CONNECT
[superadministrador cambia el estado de una empresa]
70: toggleCompany(): PATCH /api/empresas/{id}/ set_active(datos)
71: verificar_plataforma(): UPDATE activa WHERE id= plataforma
72: empresa actualizada o ausencia
alt [empresa no encontrada]
73: 404: Empresa no encontrada
[empresa encontrada]
74: audit_business_row(): UPDATE empresa
75: 200: id_empresa o estado, CONNECT
[consultar una suscripción]
76: GET /api/suscripciones/empresa/{id_subscription()}
alt [plataforma administrador o usuario/rol de la empresa empresa]
loop [cada plataforma]
77: GET /2026: app.tenant_cr= empresa objetos
78: SELECT suscripción JOIN plan para la empresa
79: suscripción o ausencia
80: 200: suscripción + 404: no encontrada
[otro emisor o rol]
81: 403: Acceso no autorizado
82: Mostrar resultado o actualizar datos visibles

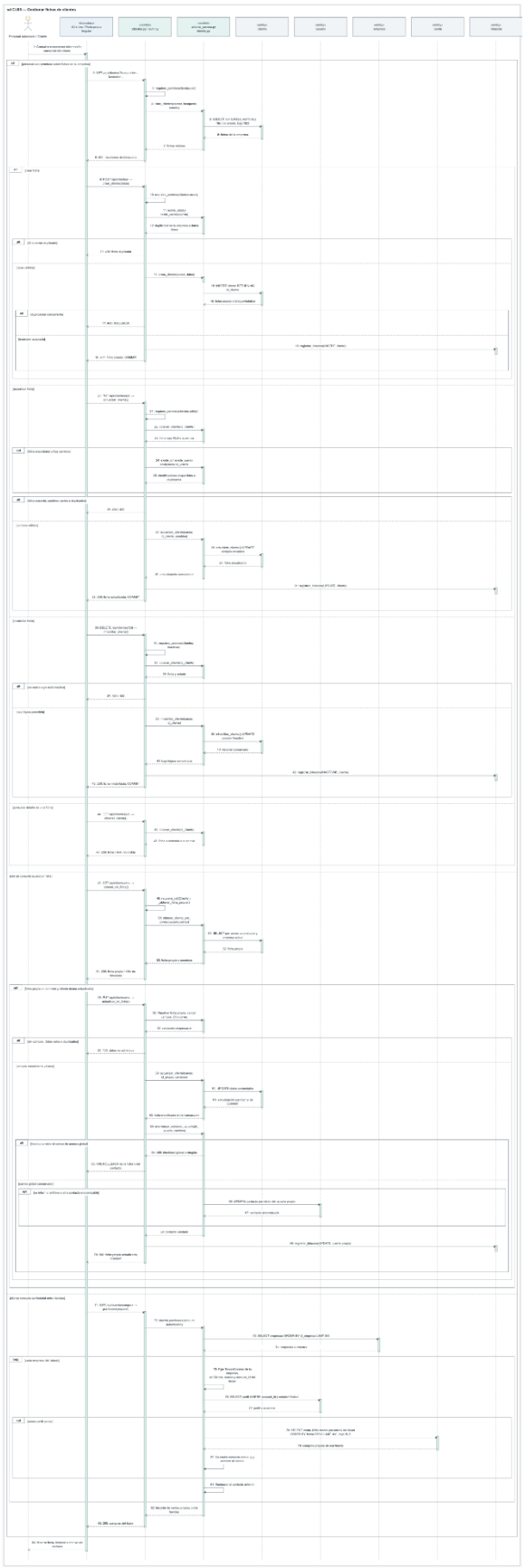
CU03 - Administrar personal, roles y permisos


sd CU03 — Administrar personal, roles y permisos
Administrador de empresa
vista.empresa / vistaadmin.PanelAdministracionEmpresa
usuarios / usuarios.php — lógica de administración_empresa
clases/clases.php/app.js
config/db.php
config/roles
config/permisos
config/rol_permisos
config/sucursal
config/usuario_sucursal
config/cuenta_global
config/clases
1. Usuario administrador de empresa accede a la sección de personal y permisos
2. GET /usuarios.php?accion=listar
3. require_once('../config/db.php')
require_once('../clases/clases.php')
4. new _Usuarios() / $PDO
5. $_SESSION['id_empresa']
6. new _roles() SELECT roles
7. roles de la empresa
8. HTML con tablas de administración
alt [owner con acceso]
9. POST /gestionar_usuario.php?accion=crear_usuario
10. $ROL[$rol] en rol_id_rol
11. $_SESSION['id_empresa'] y $usuarios
12. SELECT id FROM usuarios WHERE correo = ?
13. correo ya registrado
14. correo ya registrado
alt [rol inexistente o correo empresarial duplicado]
15. HTTP 400 rol o correo inválido
[validación exitosa/correcta]
16. generar_hash($password)
17. INSERT INTO usuarios (nombre, correo, password, id_rol, id_empresa, activo) VALUES (?, ?, ?, ?, ?, 1)
18. $id_usuario
19. $_SESSION['usuarios'][] = [id, nombre, correo, rol, activo]
20. $_SESSION['mensaje'] = "Usuario creado correctamente"
21. header("Location: usuarios.php")
22. Redirección a usuarios.php
alt [correo global (cruza con clientes de cuenta principal)]
23. SELECT id FROM usuarios WHERE correo = ? AND id_empresa = ?
24. $_SESSION['error'] = "Ese correo ya pertenece a un usuario de esta empresa"
25. header("Location: usuarios.php")
26. Redirección a usuarios.php
[duplicado activo/pausado]
27. SELECT id, activo FROM usuarios WHERE correo = ? AND id_empresa = ?
28. $_SESSION['error'] = "Ese correo ya está registrado en esta empresa"
29. header("Location: usuarios.php")
30. Redirección a usuarios.php
[administrar personal]
31. $_POST['editar_usuario'] = id_usuario
32. $_SESSION['usuarios'][$id_usuario]
33. $_SESSION['usuarios'][$id_usuario]['activo'] = !$_SESSION['usuarios'][$id_usuario]['activo']
34. header("Location: usuarios.php")
35. Redirección a usuarios.php
opt [usuario encontrado y se solicita cambiar rol]
36. SELECT id, nombre FROM roles WHERE id = ? AND id_empresa = ?
37. $_SESSION['usuarios'][$id_usuario]['id_rol'] = $nuevo_rol
38. $_SESSION['mensaje'] = "Rol actualizado correctamente"
39. header("Location: usuarios.php")
40. Redirección a usuarios.php
alt [cuenta, cambios activos, rol/nombre/correo duplicado]
41. $_SESSION['error'] = "Ya existe un usuario con ese correo en esta empresa"
42. header("Location: usuarios.php")
43. Redirección a usuarios.php
[usuario y cambios válidos]
44. $_SESSION['usuarios'][$id_usuario]['nombre'] = $nuevo_nombre
45. $_SESSION['usuarios'][$id_usuario]['correo'] = $nuevo_correo
46. $_SESSION['usuarios'][$id_usuario]['id_rol'] = $nuevo_rol
47. $_SESSION['usuarios'][$id_usuario]['activo'] = $nuevo_activo
48. header("Location: usuarios.php")
49. Redirección a usuarios.php
alt [error de conexión durante el cambio]
50. $_SESSION['error'] = "Error de conexión. Intentá de nuevo."
51. header("Location: usuarios.php")
52. Redirección a usuarios.php
[eliminar personal]
53. $id_usuario = $_POST['eliminar_usuario']
54. $_SESSION['usuarios'][$id_usuario]
55. $_SESSION['usuarios'][$id_usuario]['activo'] = 0
56. $_SESSION['mensaje'] = "Usuario desactivado correctamente"
57. header("Location: usuarios.php")
58. Redirección a usuarios.php
alt [no existe, es el propio administrador o ya está inactivo]
59. $_SESSION['error'] = "No se puede desactivar a este usuario"
60. header("Location: usuarios.php")
61. Redirección a usuarios.php
[reactivar personal]
62. $_SESSION['usuarios'][$id_usuario]['activo'] = 1
63. $_SESSION['mensaje'] = "Usuario reactivado correctamente"
64. header("Location: usuarios.php")
65. Redirección a usuarios.php
[crear sin rol de empresa]
66. POST /gestionar_usuario.php?accion=crear_usuario
67. SELECT id FROM roles WHERE nombre = 'Sin rol' AND id_empresa = ?
68. $id_rol_sin_rol = $_SESSION['id_rol']
69. $_SESSION['usuarios'][] = [id, nombre, correo, rol, activo]
70. $_SESSION['mensaje'] = "Usuario creado sin rol asignado"
71. header("Location: usuarios.php")
72. Redirección a usuarios.php
[sin nombre]
73. $_SESSION['error'] = "El nombre es obligatorio"
74. header("Location: usuarios.php")
75. Redirección a usuarios.php
[nombre disponible]
76. INSERT INTO usuarios (nombre, correo, password, id_rol, id_empresa, activo) VALUES (?, ?, ?, ?, ?, 1)
77. $id_usuario
78. $_SESSION['usuarios'][] = [id, nombre, correo, rol, activo]
79. $_SESSION['mensaje'] = "Usuario creado correctamente"
80. header("Location: usuarios.php")
81. Redirección a usuarios.php
CU asociado: CU01 — Iniciar sesión
CU asociado: CU02 — Registrar empresa
CU asociado: CU04 — Gestionar sucursales
CU asociado: CU05 — Gestionar roles y permisos
CU asociado: CU06 — Gestionar clientes
CU asociado: CU07 — Gestionar productos
CU asociado: CU08 — Gestionar stock
CU asociado: CU09 — Registrar venta
CU asociado: CU10 — Gestionar cuenta corriente
CU asociado: CU11 — Gestionar caja
CU asociado: CU12 — Ver reportes
CU asociado: CU13 — Configurar empresa
CU asociado: CU14 — Gestionar cuenta global
CU asociado: CU15 — Cerrar sesión

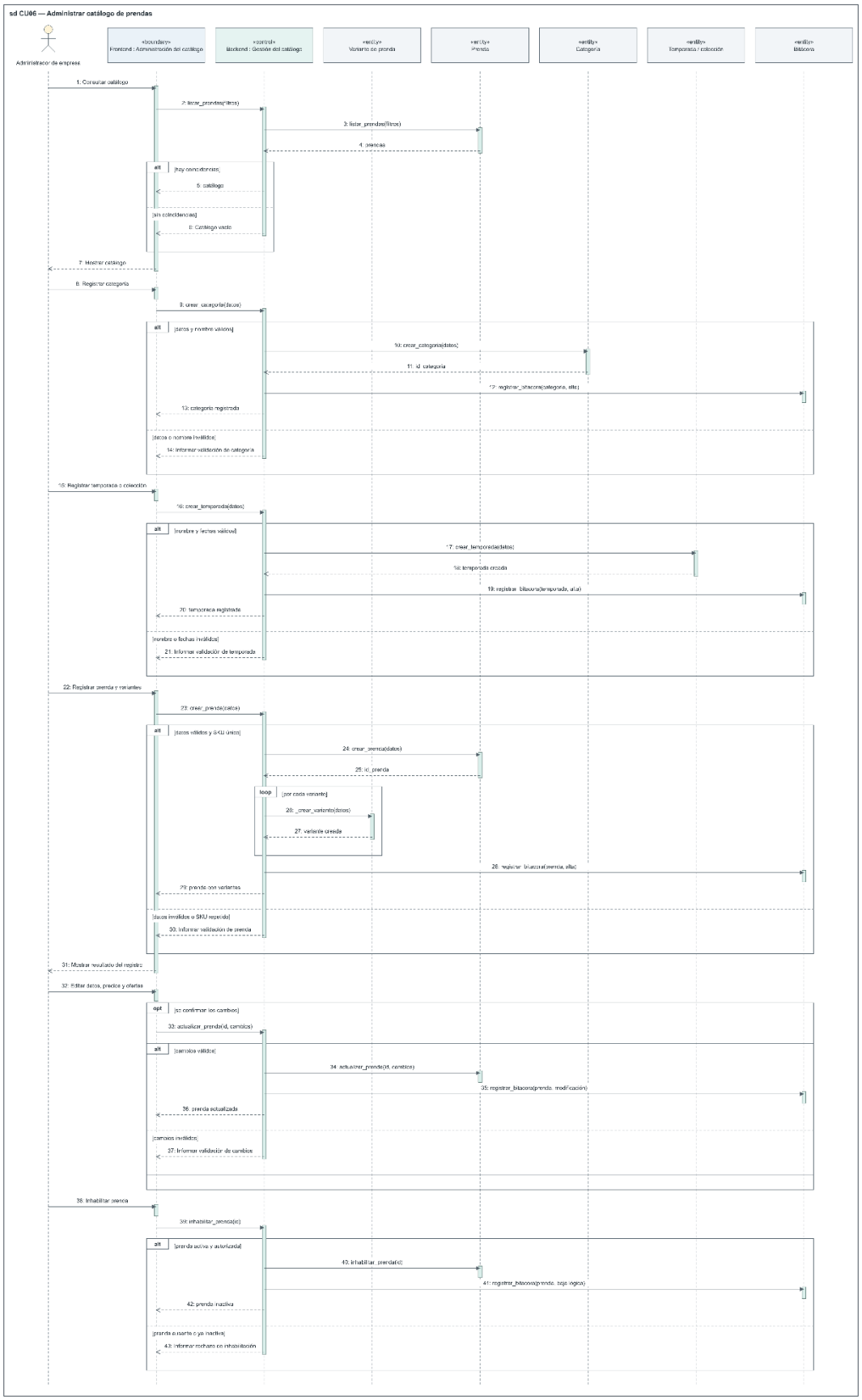
CU04 - Gestionar ciudades y sucursales



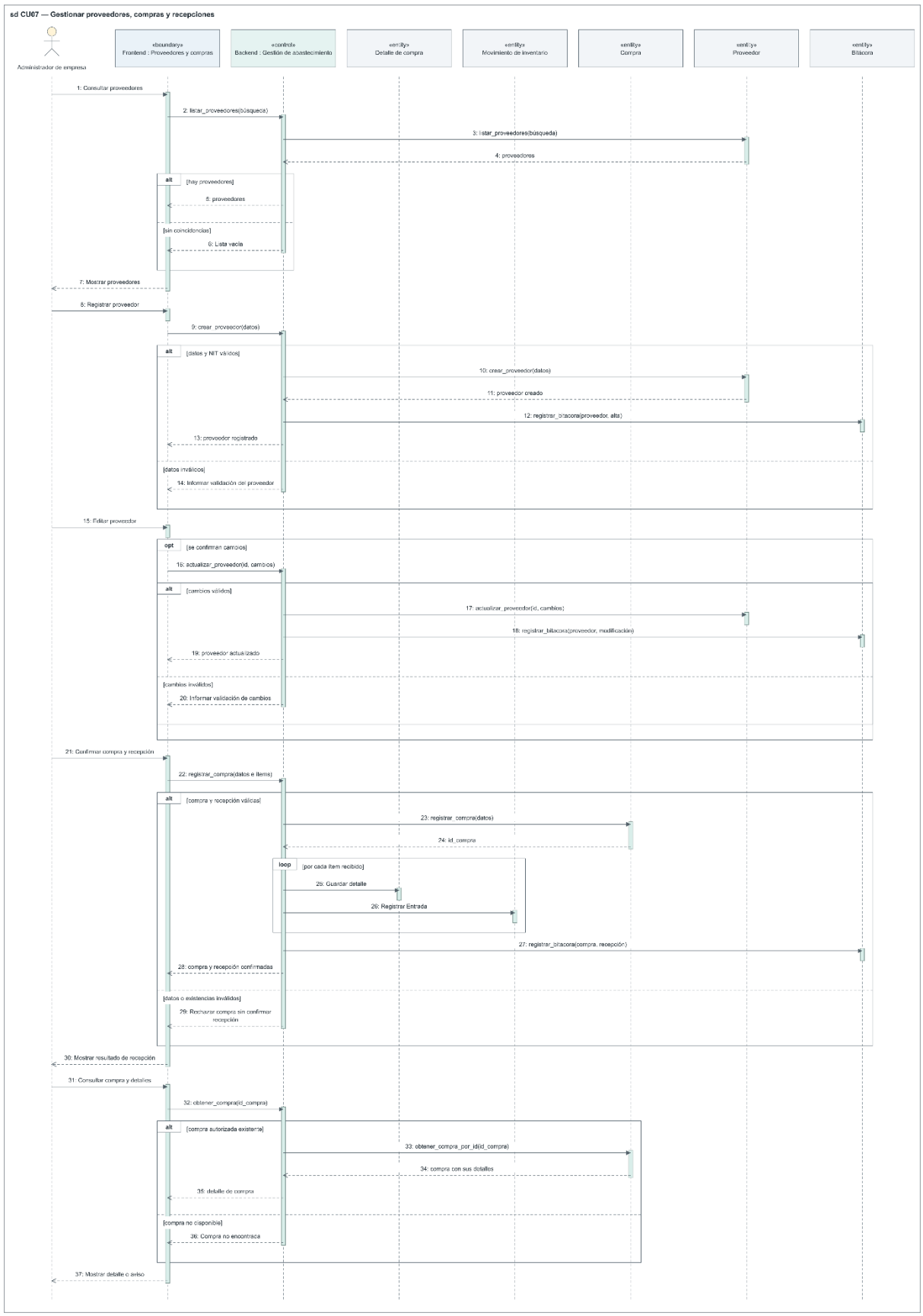
CU05 - Gestionar fichas de clientes



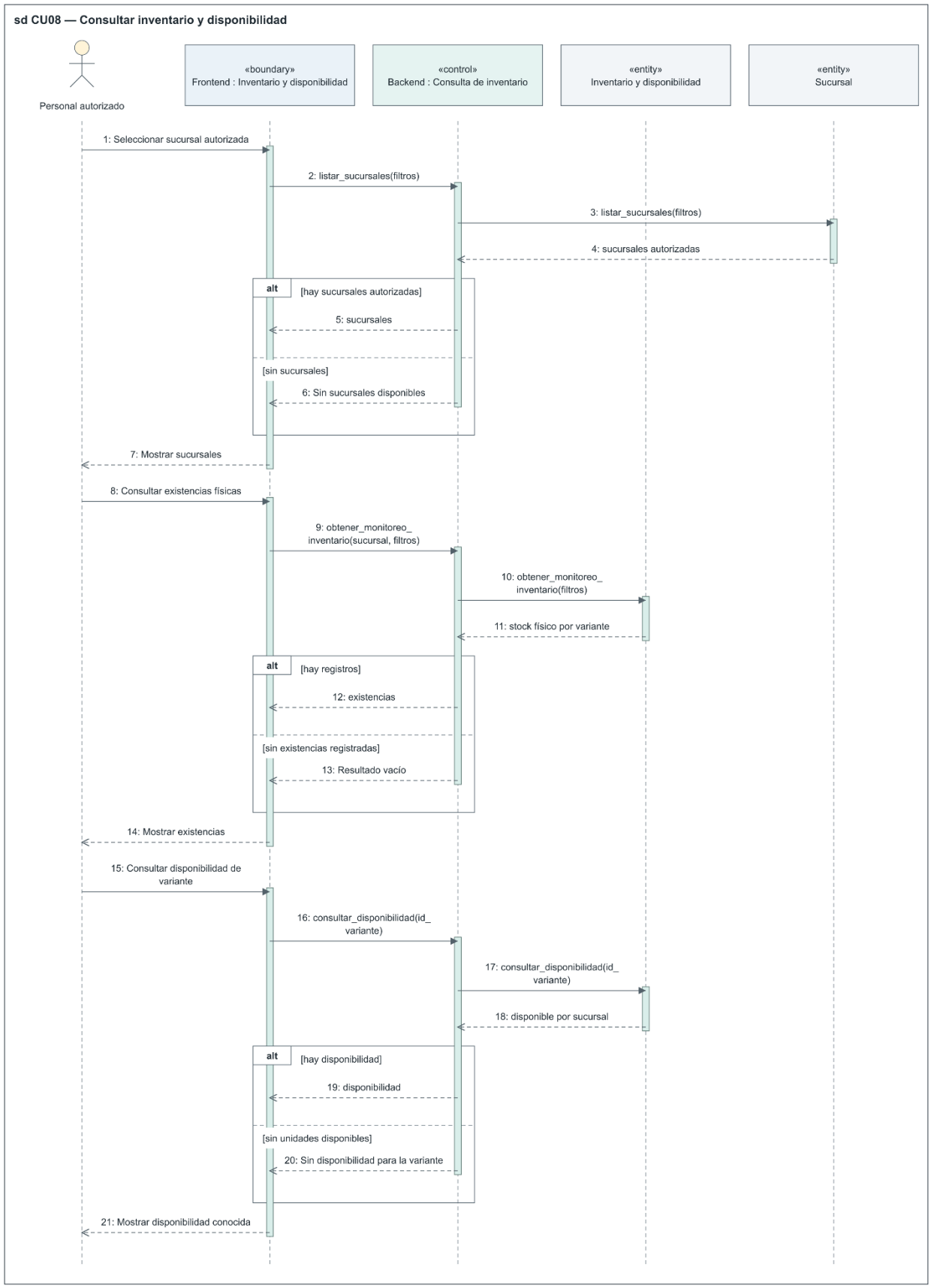
CU06 - Administrar catálogo de prendas


sd CU06 — Administrar catálogo de prendas
Administrador de empresa
«boundary»
Frontend : Administración del catálogo
«control»
Backend : Gestión del catálogo
«entity»
Variante de prenda
«entity»
Prenda
«entity»
Categoría
«entity»
Temporada / colección
«entity»
Bitácora
1: Consultar catálogo
2: listar_prendas(filtros)
3: listar_prendas(filtros)
4: piezas
alt
[hay coincidencias]
5: catálogo
[sin coincidencias]
6: Catálogo vacío
7: Mostrar catálogo
8: Registrar categoría
9: crear_categoria(datos)
alt
[datos y nombre válidos]
10: crear_categoria(datos)
11: id_categoria
12: registrar_bitacora(categoria, alta)
13: categoría registrada
[datos o nombre inválidos]
14: informar validación de categoría
15: Registrar temporada o colección
16: crear_temporada(datos)
alt
[nombre y fechas válidos]
17: crear_temporada(datos)
18: temporada creada
19: registrar_bitacora(temporada, alta)
20: temporada registrada
[nombre o fechas inválidos]
21: informar validación de temporada
22: Registrar prenda y variantes
23: crear_prenda(datos)
alt
[datos válidos y SKU únicos]
24: crear_prenda(datos)
25: id_prenda
loop
[por cada variante]
26: crear_variante(datos)
27: variante creada
28: registrar_bitacora(prenda, alta)
29: prenda con variantes
[datos inválidos o SKU repetido]
30: informar validación de prenda
31: Mostrar resultado del registro
32: Editar datos, precios y ofertas
opt
[si confirman los cambios]
33: actualizar_prenda(id, cambios)
alt
[cambios válidos]
34: actualizar_prenda(id, cambios)
35: registrar_bitacora(prenda, modificación)
36: prenda actualizada
[cambios inválidos]
37: Informar validación de cambios
38: Inhabilitar prenda
39: inhabilitar_prenda(id)
alt
[prenda activa y autorizada]
40: inhabilitar_prenda(id)
41: registrar_bitacora(prenda, baja lógica)
42: prenda inactiva
[prenda ausente o ya inactiva]
43: informar rechazo de inhabilitación

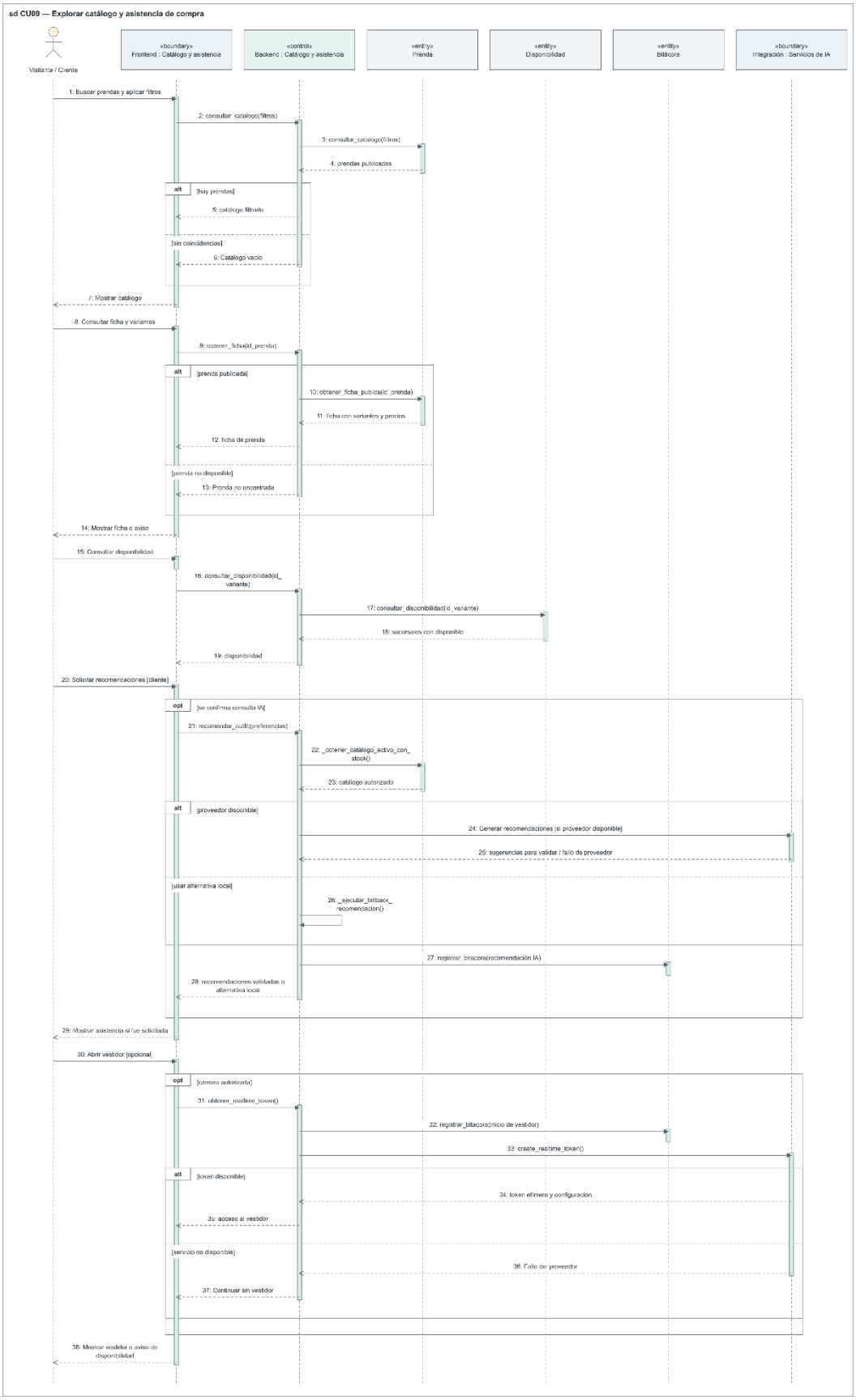
CU07 - Gestionar proveedores, compras y recepciones


sd CU07 — Gestionar proveedores, compras y recepciones
Administrador de empresa
«boundary»
Frontend : Proveedores y compras
«control»
Backend : Gestión de abastecimiento
«entity»
Detalle de compra
«entity»
Movimiento de inventario
«entity»
Compra
«entity»
Proveedor
«entity»
Bitácora
1: Consultar proveedores
2: listar_proveedores(búsqueda)
3: listar_proveedores(búsqueda)
4: proveedores
alt
[hay proveedores]
5: proveedores
[sin coincidencias]
6: Lista vacía
7: Mostrar proveedores
8: Registrar proveedor
9: crear_proveedor(datos)
alt
[datos y NIT válidos]
10: crear_proveedor(datos)
11: proveedor creado
12: registrar_bitacora(proveedor, alta)
13: proveedor registrado
[datos inválidos]
14: Informar validación del proveedor
15: Editar proveedor
opt
[se confirman cambios]
16: actualizar_proveedor(id, cambios)
alt
[cambios válidos]
17: actualizar_proveedor(id, cambios)
18: registrar_bitacora(proveedor, modificación)
19: proveedor actualizado
[cambios inválidos]
20: Informar validación de cambios
21: Confirmar compra y recepción
22: registrar_compra(datos e ítems)
alt
[compra y recepción válidas]
23: registrar_compra(datos)
24: id_compra
loop
[por cada ítem recibido]
25: Guardar detalle
26: Registrar Entrada
27: registrar_bitacora(compra, recepción)
28: compra y recepción confirmadas
[datos o existencias inválidas]
29: Rechazar compra sin confirmar recepción
30: Mostrar resultado de recepción
31: Consultar compra y detalles
32: obtener_compra(id_compra)
alt
[compra autorizada existente]
33: obtener_compra_por_id(id_compra)
34: compra con sus detalles
35: detalle de compra
[compra no disponible]
36: Compra no encontrada
37: Mostrar detalle o aviso

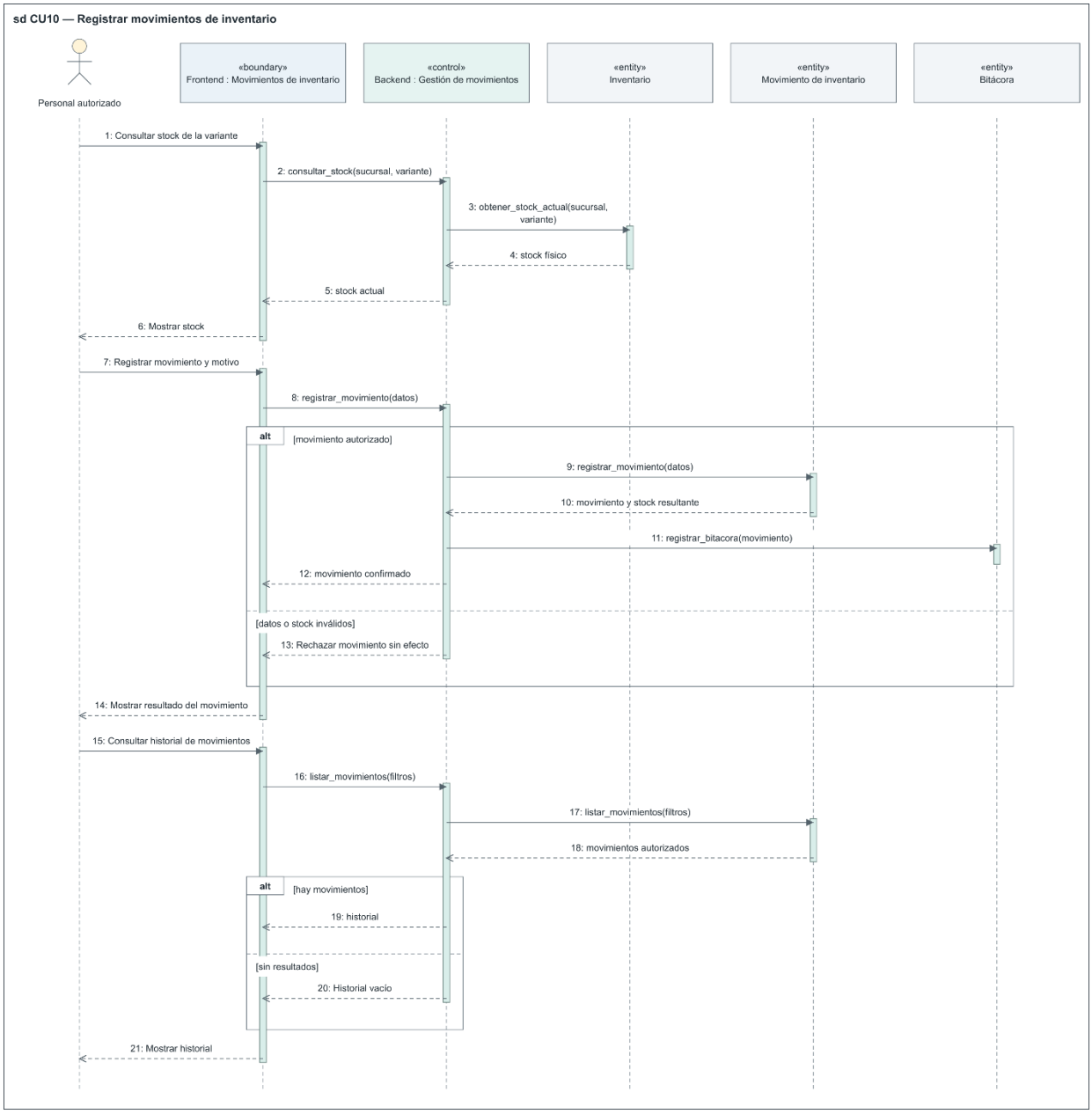
CU08 - Consultar inventario y disponibilidad


sd CU08 — Consultar inventario y disponibilidad
Personal autorizado
«boundary»
Frontend : Inventario y disponibilidad
«control»
Backend : Consulta de inventario
«entity»
Inventario y disponibilidad
«entity»
Sucursal
1: Seleccionar sucursal autorizada
2: listar_sucursales(filtros)
3: listar_sucursales(filtros)
4: sucursales autorizadas
alt
[hay sucursales autorizadas]
5: sucursales
[sin sucursales]
6: Sin sucursales disponibles
7: Mostrar sucursales
8: Consultar existencias físicas
9: obtener_monitoreo_ inventario(sucursal, filtros)
10: obtener_monitoreo_ inventario(filtros)
11: stock físico por variante
alt
[hay registros]
12: existencias
[sin existencias registradas]
13: Resultado vacío
14: Mostrar existencias
15: Consultar disponibilidad de variante
16: consultar_disponibilidad(id_ variante)
17: consultar_disponibilidad(id_ variante)
18: disponible por sucursal
alt
[hay disponibilidad]
19: disponibilidad
[sin unidades disponibles]
20: Sin disponibilidad para la variante
21: Mostrar disponibilidad conocida

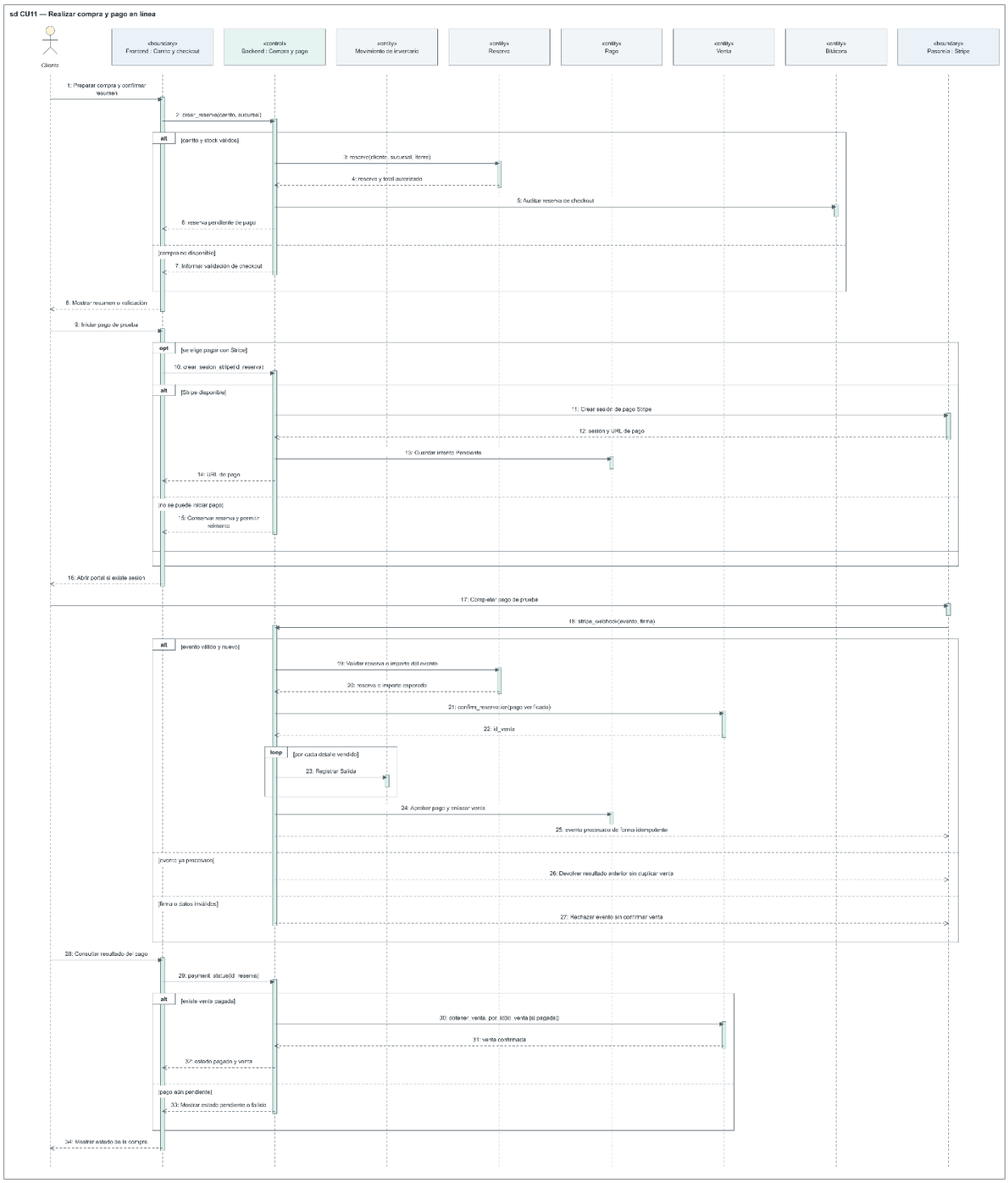
CU09 - Explorar catálogo y asistencia de compra


sd CU09 — Explorar catálogo y asistencia de compra
Visitante / Cliente
«boundary»
Frontend : Catálogo y asistencia
«control»
Backend : Catálogo y asistencia
«entity»
Prenda
«entity»
Disponibilidad
«entity»
Bitácora
«boundary»
Integración : Servicios de IA
1: Buscar prendas y aplicar filtros
2: consultar_catalogo(filtros)
3: consultar_catalogo(filtros)
4: prendas publicadas
alt
[hay prendas]
5: catálogo filtrado
[sin coincidencias]
6: Catálogo vacío
7: Mostrar catálogo
8: Consultar ficha y variantes
9: obtener_ficha(id_prenda)
alt
[prenda publicada]
10: obtener_ficha_publica(id_prenda)
11: ficha con variantes y precios
12: ficha de prenda
[prenda no disponible]
13: Prenda no encontrada
14: Mostrar ficha o aviso
15: Consultar disponibilidad
16: consultar_disponibilidad(id_variante)
17: consultar_disponibilidad(id_variante)
18: sucursales con disponible
19: disponibilidad
20: Solicitar recomendaciones [cliente]
opt
[se confirma consulta IA]
21: recomendar_outfit(preferencias)
22: _obtener_catalogo_activo_con_stock()
23: catálogo autorizado
alt
[proveedor disponible]
24: Generar recomendaciones [si proveedor disponible]
25: sugerencias para validar / fallo de proveedor
[usar alternativa local]
26: _ejecutar_fallback_recomendacion()
27: registrar_bitacora(recomendación IA)
28: recomendaciones validadas o alternativa local
29: Mostrar asistencia si fue solicitada
30: Abrir vestidor [opcional]
opt
[cámara autorizada]
31: obtener_realtime_token()
32: registrar_bitacora(inicio de vestidor)
33: create_realtime_token()
alt
[token disponible]
34: token efímero y configuración
35: acceso al vestidor
[servicio no disponible]
36: Fallo del proveedor
37: Continuar sin vestidor
38: Mostrar vestidor o aviso de disponibilidad

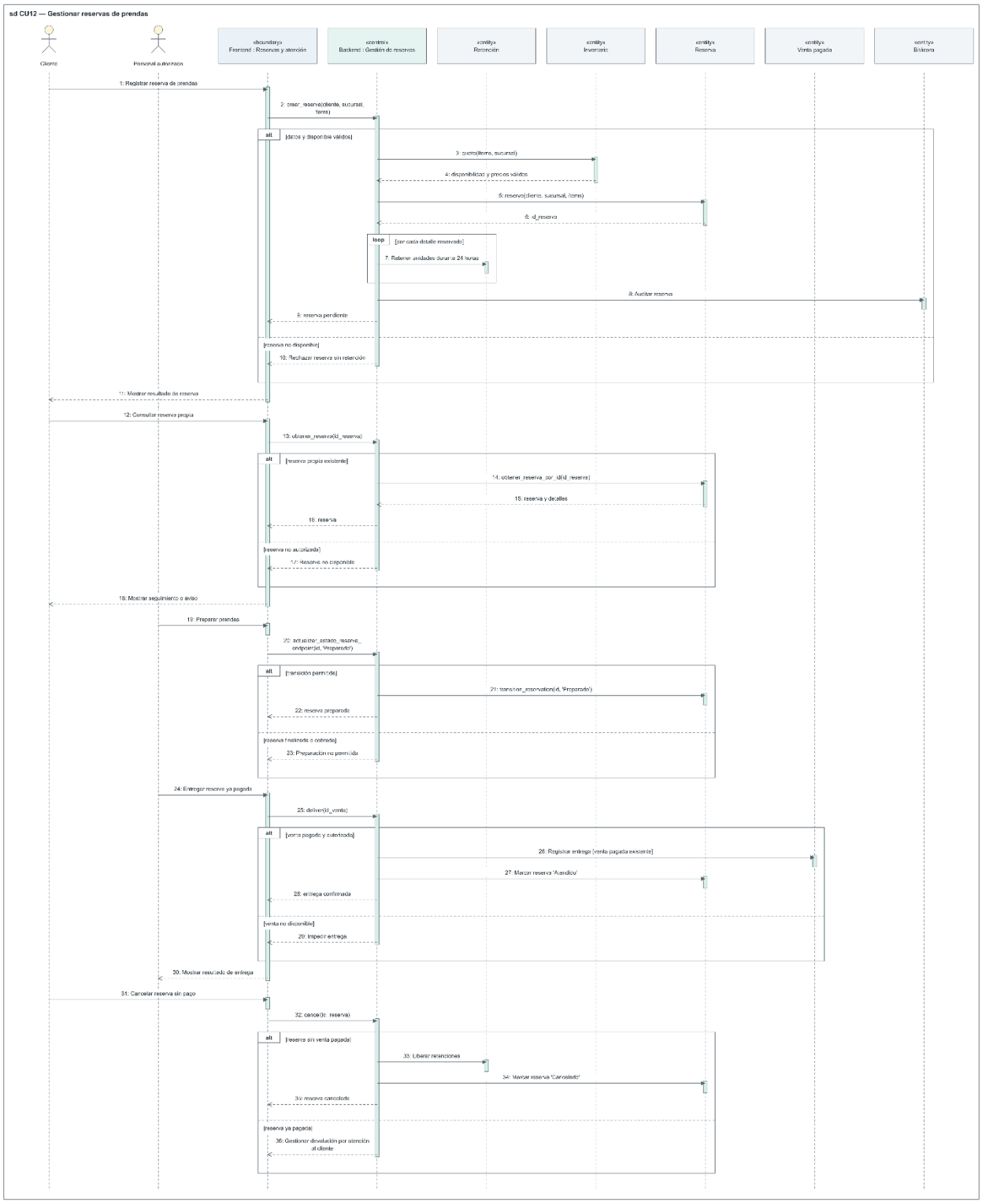
CU10 - Registrar movimientos de inventario



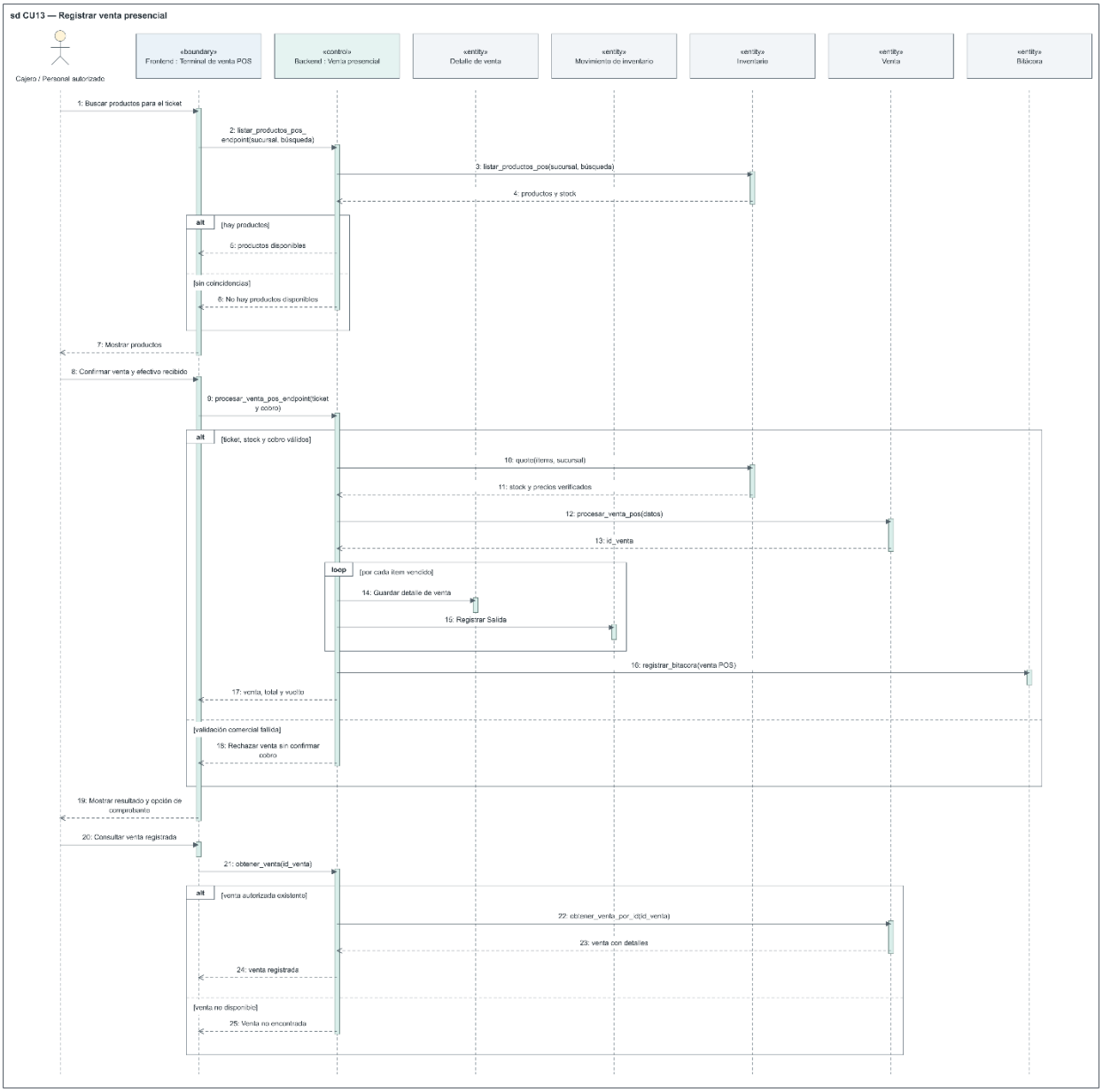
CU11 - Realizar compra y pago en línea



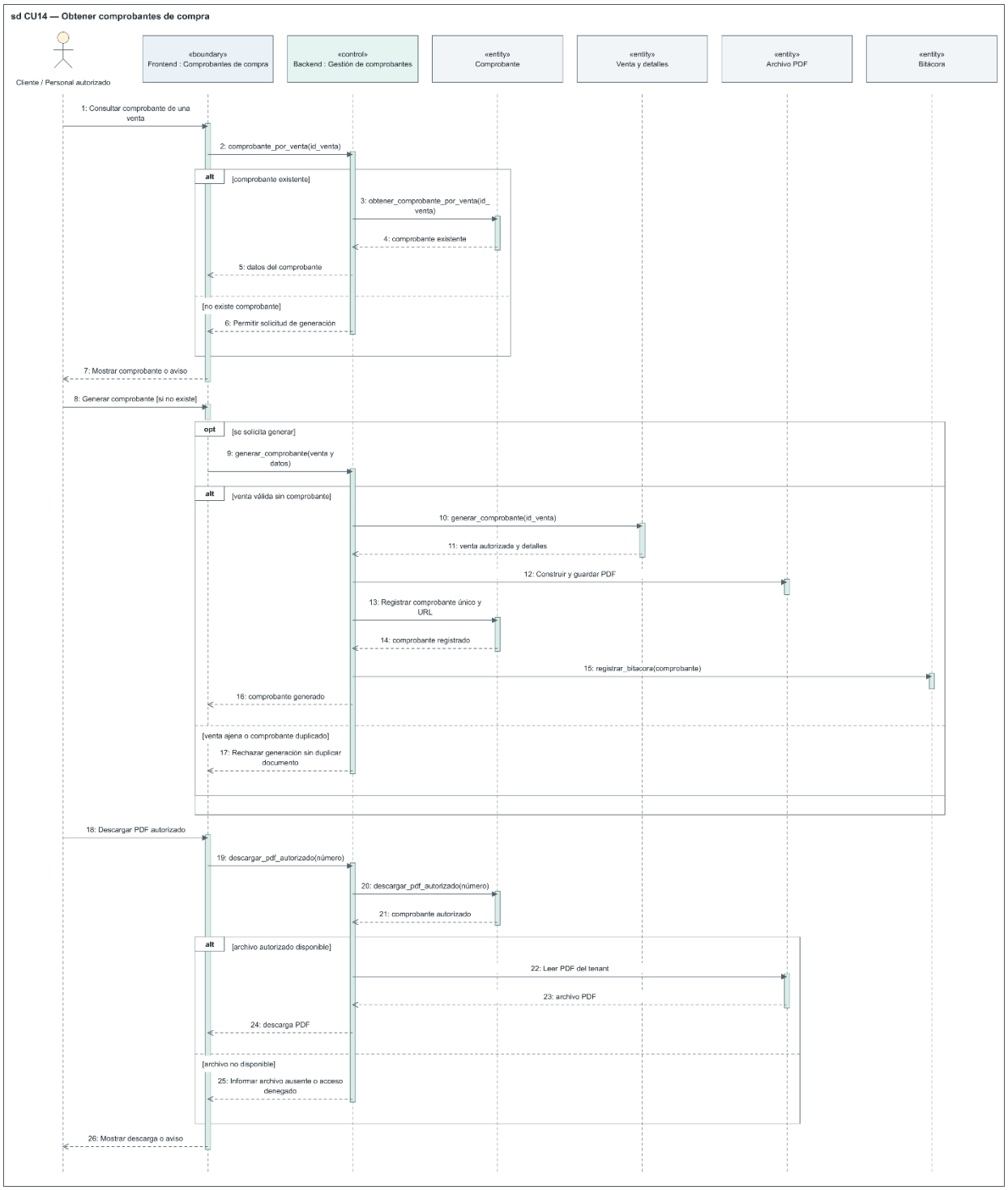
CU12 - Gestionar reservas de prendas



CU13 - Registrar venta presencial

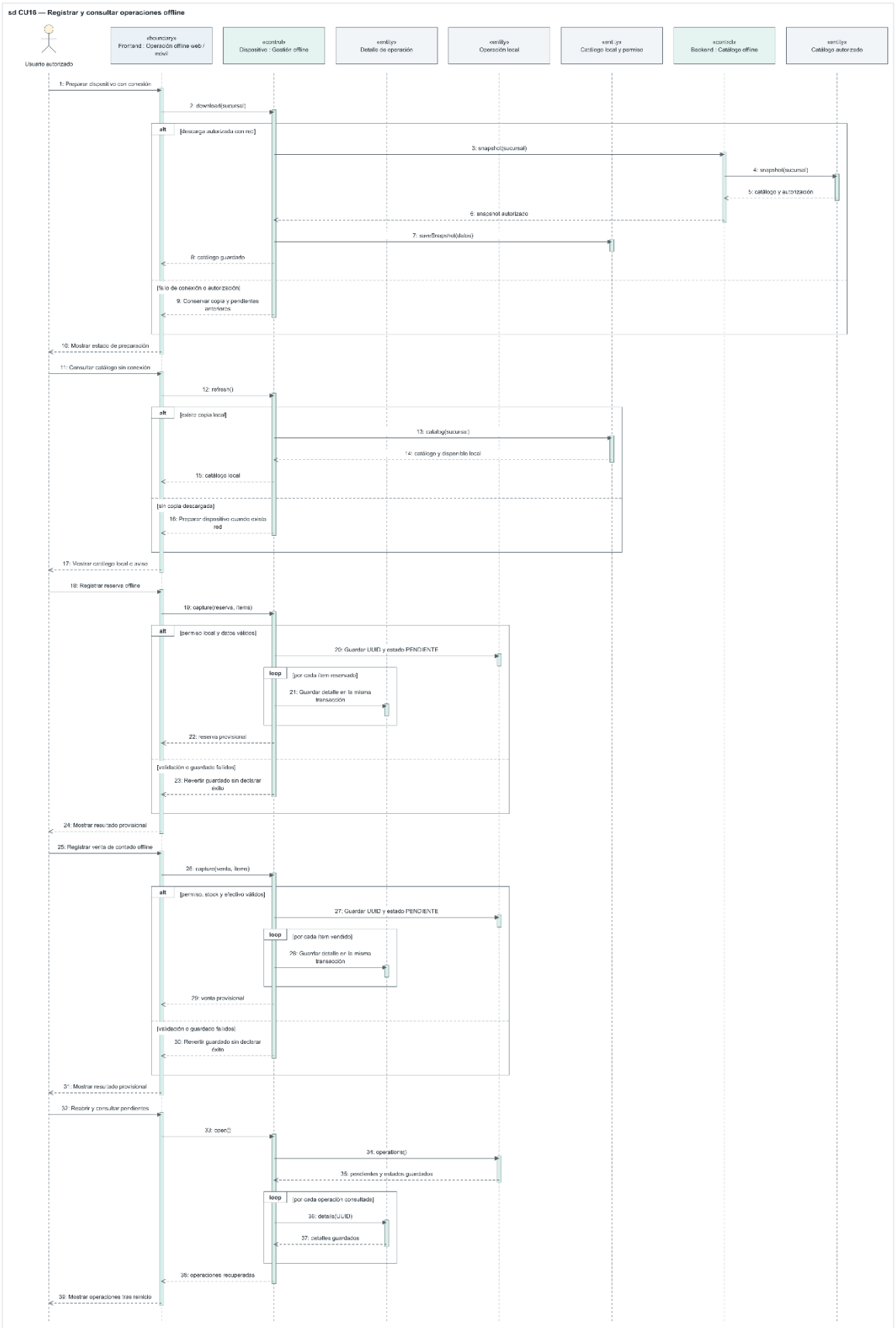


CU14 - Obtener comprobantes de compra



CU15 - Gestionar cambios y reembolsos

CU16 - Registrar y consultar operaciones offline


sd CU16 — Registrar y consultar operaciones offline
Usuario autorizado
«boundary»
Frontend : Operación offline web / movil
«control»
Dispositivo : Gestión offline
«entity»
Detalle de operación
«entity»
Operación local
«entity»
Catálogo local y permiso
«control»
Backend : Catálogo offline
«entity»
Catálogo autorizado
1: Preparar dispositivo con conexión
2: download(sucursal)
alt
[descarga autorizada con red]
3: snapshot(sucursal)
4: snapshot(sucursal)
5: catálogo y autorización
6: snapshot autorizado
7: saveSnapshot(datos)
8: catálogo guardado
[fallo de conexión o autorización]
9: Conservar copia y pendientes anteriores
10: Mostrar estado de preparación
11: Consultar catálogo sin conexión
12: refresh()
alt
[existe copia local]
13: catalog(sucursal)
14: catálogo y disponible local
15: catálogo local
[sin copia descargada]
16: Preparar dispositivo cuando exista red
17: Mostrar catálogo local o aviso
18: Registrar reserva offline
19: capture(reserva, items)
alt
[permiso local y datos válidos]
20: Guardar UUID y estado PENDIENTE
loop
[por cada ítem reservado]
21: Guardar detalle en la misma transacción
22: reserva provisional
[validación o guardado fallidos]
23: Revertir guardado sin declarar éxito
24: Mostrar resultado provisional
25: Registrar venta de contado offline
26: capture(venta, items)
alt
[permiso, stock y efectivo válidos]
27: Guardar UUID y estado PENDIENTE
loop
[por cada ítem vendido]
28: Guardar detalle en la misma transacción
29: venta provisional
[validación o guardado fallidos]
30: Revertir guardado sin declarar éxito
31: Mostrar resultado provisional
32: Reabrir y consultar pendientes
33: open()
34: operations()
35: pendientes y estados guardados
loop
[por cada operación consultada]
36: details(UUID)
37: detalles guardados
38: operaciones recuperadas
39: Mostrar operaciones tras reinicio

CU17 - Sincronizar operaciones y resolver conflictos

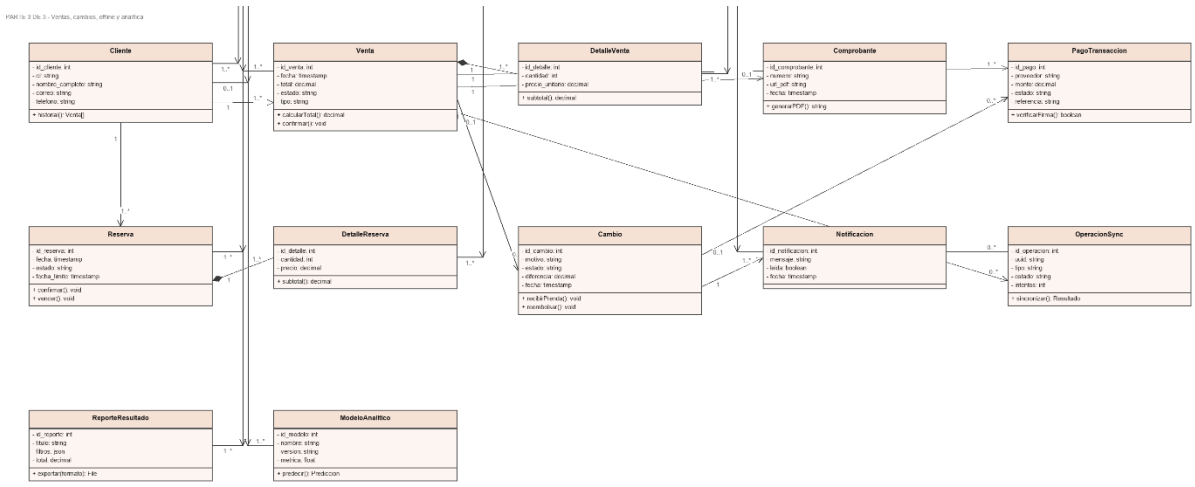
CU18 - Obtener reportes comerciales

CU19 - Preparar y consultar analítica inteligente

CU20 - Consultar auditoría y trazabilidad

3.4. Diagrama de clases UML


Empresa
Sucursal
Ciudad
Usuario
Rol
Plan
Suscripcion
UsuarioSucursal
Permiso
Bitacora
CuentaGlobal
CuentaRecuperacion
Categoria
Temporada
Prenda
VariantePrenda
Proveedor
Inventario
MovimientoInventario
DetalleCompra
Compra



3.5. Diseño de la base de datos

La base de datos de FashionStore se organiza mediante un modelo relacional que representa la estructura empresarial, el catálogo de prendas y las operaciones comerciales de la plataforma. Su diseño transforma los conceptos identificados en el modelo de dominio en tablas, atributos y relaciones, considerando las reglas de negocio y las necesidades de persistencia de los casos de uso.

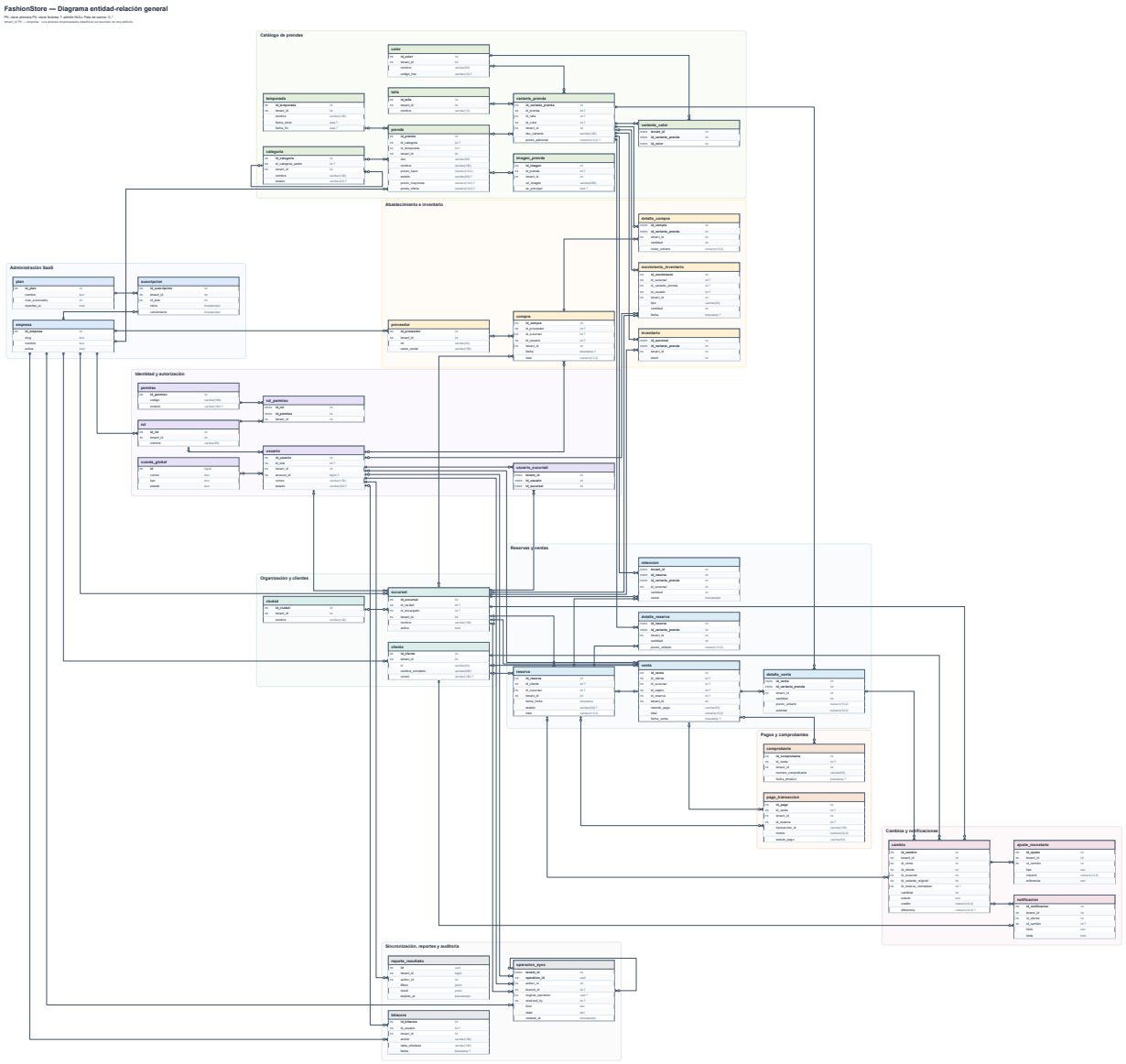
La persistencia central utiliza PostgreSQL y comprende 45 tablas relacionadas mediante claves primarias y foráneas. El diseño incorpora la administración SaaS, la identidad de las cuentas, la organización de las empresas y sucursales, el abastecimiento, el inventario, las ventas y los servicios complementarios.

Organización de la información

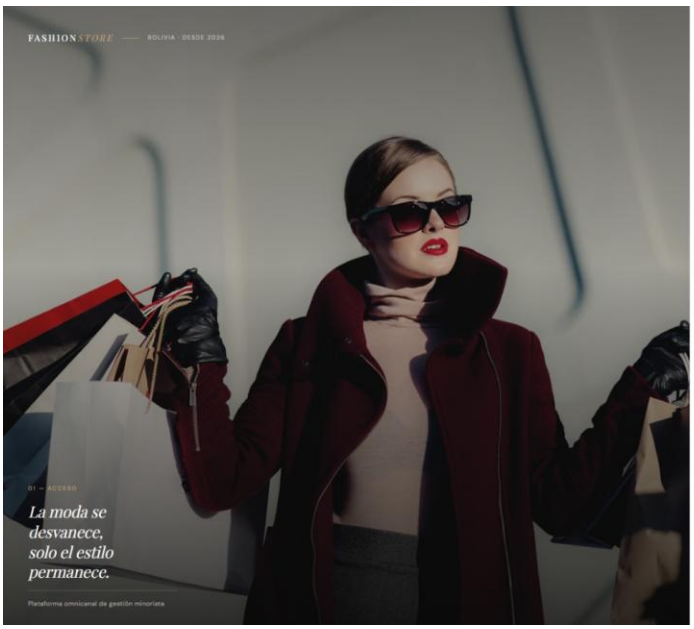
Las tablas se agrupan por su responsabilidad dentro del sistema. Esta agrupación facilita comprender el modelo general y reconocer las relaciones entre sus diferentes áreas.

Área	Tablas representativas	Finalidad
Administración SaaS	empresa, plan, suscripcion	Representar las empresas usuarias, las condiciones de los planes y la vigencia de sus suscripciones.

Identidad y autorización	cuenta_global, usuario, rol, permiso, rol_permiso, usuario_sucursal	Separar la identidad de acceso de las autorizaciones empresariales y las sucursales asignadas.
Organización y clientes	ciudad, sucursal, cliente	Registrar la estructura operativa y las fichas comerciales de los clientes.
Catálogo	categoria, temporada, prenda, variante_prenda, talla, color, variante_color, imagen_prenda	Describir las prendas, sus variantes, combinaciones de colores y recursos asociados.
Abastecimiento e inventario	proveedor, compra, detalle_compra, inventario, movimiento_inventario	Registrar compras, recepciones, existencias y movimientos por sucursal y variante.
Reservas y ventas	reserva, detalle_reserva, retencion, venta, detalle_venta	Conservar las operaciones y sus productos, diferenciando las reservas, las retenciones y las ventas confirmadas.
Pagos y comprobantes	pago_transaccion, stripe_event, comprobante	Mantener el seguimiento de los pagos, sus eventos y los documentos vinculados con las ventas.
Cambios y ajustes	cambio, ajuste_monetario	Registrar las solicitudes de cambio y los ajustes económicos relacionados.
Sincronización y servicios complementarios	operacion_sync, notificacion, dispositivo_push, push_entrega, reporte_resultado, bitacora, auditoria_identidad	Conservar resultados de sincronización, notificaciones, reportes y trazabilidad de acciones.



3.6. Diseño de interfaces de usuario





4. CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN

4.1. Herramientas y tecnologías utilizadas

El desarrollo de FashionStore combina tecnologías para la aplicación web, la aplicación móvil, los servicios del servidor y la persistencia de información. La selección permite compartir las reglas del negocio mediante una API, mantener los datos comerciales separados por empresa e incorporar operación sin conexión, reportes generativos y analítica con aprendizaje automático. También se emplean herramientas para realizar pruebas, documentar la solución y administrar su despliegue.

Desarrollo del backend. Se utiliza Python con FastAPI para implementar la API REST que atiende a los clientes web y móvil. El backend concentra la autenticación, la autorización, las reglas comerciales y la integración con servicios externos. Pydantic define los esquemas de entrada y salida y valida la estructura de los datos recibidos. Uvicorn ejecuta la aplicación como servidor ASGI. La organización por rutas, servicios y esquemas facilita la separación de responsabilidades y el mantenimiento de los módulos. FastAPI proporciona, además, la descripción OpenAPI de los servicios y una interfaz de documentación interactiva.

Persistencia central y aislamiento empresarial. PostgreSQL almacena empresas, usuarios, productos, inventarios, compras, ventas, reservas y demás registros comerciales. La conexión desde Python se realiza mediante psycopg2, utilizando consultas parametrizadas y transacciones. El esquema se administra con migraciones SQL versionadas. El modelo multitenant incorpora un

identificador de empresa y políticas de seguridad por fila, denominadas Row-Level Security, junto con las validaciones del backend. La configuración local principal utiliza PostgreSQL 16, mientras que el entorno nativo de producción y el ambiente específico de pruebas de compatibilidad utilizan PostgreSQL 17.

Aplicación web. La interfaz se desarrolla con Angular 20 y TypeScript. HTML y SCSS permiten estructurar las pantallas y definir su presentación. Angular Router organiza la navegación; los formularios permiten capturar y validar información, y HttpClient comunica la aplicación con el backend. RxJS apoya el tratamiento de operaciones asíncronas. Node.js, npm y Angular CLI se utilizan para administrar dependencias, ejecutar el entorno de desarrollo y compilar los archivos que se publican en el servidor.

Aplicación web progresiva y almacenamiento offline. La versión web incorpora un manifiesto de instalación y el Service Worker de Angular para conservar los recursos necesarios para cargar la aplicación. La información comercial descargada y las operaciones pendientes se almacenan en IndexedDB mediante Dexie. Esta separación permite distinguir la caché de la aplicación de los registros que deben sincronizarse. La cola local conserva identificadores UUID, detalles, estados e intentos de envío para recuperar las operaciones después de una interrupción y procesarlas mediante el contrato de sincronización del backend.

Aplicación móvil. Se emplean Flutter y Dart para construir la aplicación destinada a Android. La comunicación con la API se realiza mediante el paquete http. SQLite, integrado a través de sqflite, permite conservar el catálogo descargado y las operaciones locales. Flutter Secure Storage almacena información sensible de sesión, mientras connectivity_plus permite observar cambios de conectividad. También se utilizan paquetes para generar UUID, seleccionar imágenes, solicitar permisos y abrir archivos o contenido web. La generación del APK se realiza localmente con las herramientas de Flutter y el entorno de compilación de Android.

Autenticación y control de acceso. La identificación de los usuarios se basa en correo electrónico y contraseña. El backend utiliza JSON Web Tokens, gestionados mediante python-jose, para validar las solicitudes autenticadas. Las contraseñas se almacenan como hashes bcrypt mediante Passlib. Los roles, permisos y contextos empresariales se comprueban en el servidor, complementando las restricciones de navegación de las interfaces. Las claves de servicios y la configuración privada se mantienen fuera del código publicado mediante variables de entorno y archivos de configuración excluidos del repositorio.

Inteligencia artificial generativa y entrada por voz. La integración con Google Gemini se realiza mediante el SDK google-genai. En los reportes generativos, el

modelo interpreta la solicitud del usuario y devuelve una intención estructurada que el backend valida antes de ejecutar consultas permitidas. No se delega al modelo la ejecución de SQL libre. La entrada por voz utiliza las interfaces SpeechRecognition disponibles en navegadores compatibles y el paquete speech_to_text en Flutter. La transcripción se muestra como texto editable y puede sustituirse por escritura manual.

Aprendizaje automático y analítica. Se utiliza scikit-learn para implementar un modelo supervisado de estimación de demanda con HistGradientBoostingRegressor y un modelo no supervisado de segmentación de clientes con K-Means. NumPy permite preparar las variables numéricas; StandardScaler participa en el procesamiento de las variables de segmentación y joblib conserva los modelos entrenados. La evaluación incluye MAE y RMSE para la demanda y el coeficiente de silueta para los agrupamientos. El entrenamiento inicial emplea conjuntos sintéticos reproducibles y se ejecuta localmente. En producción se cargan los artefactos para inferencia, verificando su versión e integridad.

Pagos, documentos y notificaciones. Stripe se integra en modo de prueba para gestionar pagos electrónicos y recibir eventos mediante webhooks. Los documentos PDF se generan con FPDF y ReportLab, mientras openpyxl permite exportar reportes a XLSX. La integración de notificaciones push utiliza Firebase Cloud Messaging, Firebase Admin en el backend y los paquetes Firebase correspondientes en Flutter. Su habilitación requiere la configuración del proyecto Firebase y de sus credenciales. La bandeja persistente de notificaciones conserva los avisos comerciales independientemente de la recepción del push. La recuperación de contraseña incorpora envío de correo mediante SMTP y Mailpit se utiliza como servicio de correo de desarrollo.

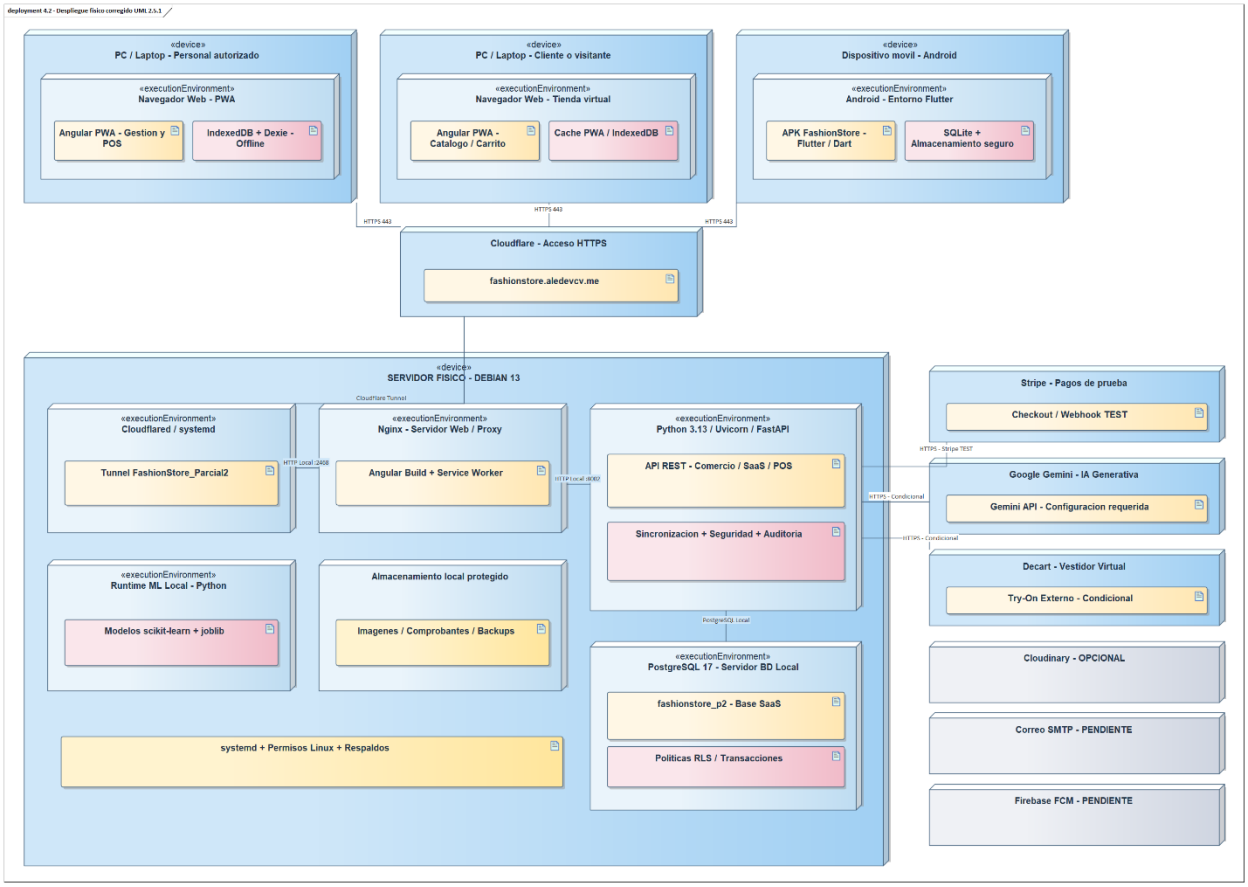
Imágenes y vestidor virtual. Pillow y OpenCV apoyan la validación y el procesamiento de imágenes. El proyecto incorpora almacenamiento de fotografías mediante Cloudinary y recursos de catálogo administrados por la aplicación. El vestidor heredado integra Decart mediante su SDK web y su cliente Python para la modalidad de prueba virtual en tiempo real. También conserva alternativas configurables para la generación de imágenes. La disponibilidad de estas modalidades depende de los permisos de cámara, las credenciales y las condiciones del proveedor utilizado.

Entorno local y despliegue. Docker y Docker Compose organizan los servicios necesarios para el desarrollo y las pruebas locales, incluyendo ambientes de base de datos separados. El servidor de producción utiliza Debian 13 y ejecuta la aplicación de manera nativa, sin Docker. La API funciona en un entorno virtual Python 3.13 administrado por systemd. Nginx sirve la compilación de Angular y dirige las solicitudes de la API hacia Uvicorn. Cloudflared conecta el servicio con

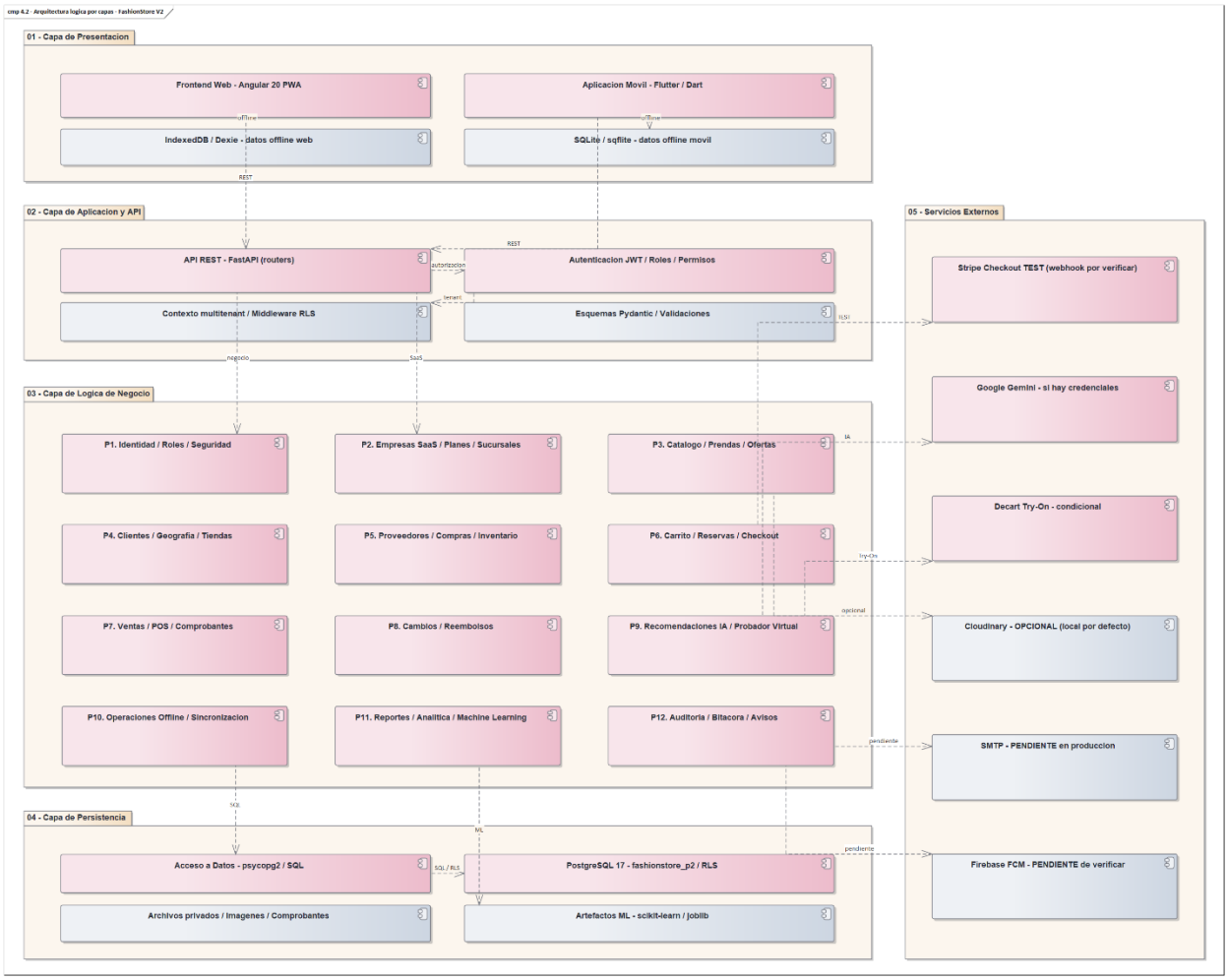
Cloudflare Tunnel para publicarlo mediante HTTPS en fashionstore.aledevcv.me. Las versiones se transfieren mediante SSH y SCP; los procedimientos incluyen respaldo y activación de la versión preparada.

4.2. Diseño de la arquitectura del sistema

4.2.1. Diseño físico (despliegue)



4.2.2. Diseño lógico (capas)



4.3. Implementación de la base de datos

4.3.1. Script Base de Datos

```
BEGIN;
-- Roles de propiedad y autorización; sin contraseñas. No altera roles existentes.
DO $roles$ BEGIN IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE
rolname='fashionstore_p2_app') THEN CREATE ROLE "fashionstore_p2_app" NOLOGIN
NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION NOBYPASSRLS;
END IF; END $roles$;
DO $roles$ BEGIN IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE
rolname='fashionstore_p2_owner') THEN CREATE ROLE "fashionstore_p2_owner"
NOLOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION
NOBYPASSRLS; END IF; END $roles$;
--
-- PostgreSQL database dump
--
```

```
-- Dumped from database version 17.11 (Debian 17.11-0+deb13u1)
-- Dumped by pg_dump version 17.11 (Debian 17.11-0+deb13u1)
SET statement_timeout = 0;
SET lock_timeout = 0;
SET idle_in_transaction_session_timeout = 0;
SET transaction_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SELECT pg_catalog.set_config('search_path', '', false);
SET check_function_bodies = false;
SET xmloption = content;
SET client_min_messages = warning;
SET row_security = off;
--
-- Name: uuid-ossp; Type: EXTENSION; Schema: -; Owner: -
--
CREATE EXTENSION IF NOT EXISTS "uuid-ossp" WITH SCHEMA public;
--
-- Name: EXTENSION "uuid-ossp"; Type: COMMENT; Schema: -; Owner:
--
COMMENT ON EXTENSION "uuid-ossp" IS 'generate universally unique identifiers (UUIDs)';
--
-- Name: actualizar_stock_por_movimiento(); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.actualizar_stock_por_movimiento() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
BEGIN
 INSERT INTO inventario(tenant_id,id_sucursal,id_variante_prenda,stock)
 VALUES(NEW.tenant_id,NEW.id_sucursal,NEW.id_variante_prenda,0)
 ON CONFLICT(id_sucursal,id_variante_prenda) DO NOTHING;
 IF NEW.tipo='Entrada' THEN
  UPDATE inventario SET stock=stock+NEW.cantidad WHERE tenant_id=NEW.tenant_id
AND id_sucursal=NEW.id_sucursal AND id_variante_prenda=NEW.id_variante_prenda;
 ELSIF NEW.tipo='Salida' THEN
  UPDATE inventario SET stock=stock-NEW.cantidad WHERE tenant_id=NEW.tenant_id
AND id_sucursal=NEW.id_sucursal AND id_variante_prenda=NEW.id_variante_prenda AND
stock>=NEW.cantidad;
```

```
  IF NOT FOUND THEN RAISE EXCEPTION 'Stock insuficiente' USING ERRCODE='23514';
END IF;
 END IF;
 RETURN NEW;
END $$;
ALTER FUNCTION public.actualizar_stock_por_movimiento() OWNER TO
fashionstore_p2_owner;
--
-- Name: audit_business_row(); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.audit_business_row() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
DECLARE data JSONB; tid INTEGER; actor INTEGER; record_id INTEGER;
BEGIN
 IF TG_OP='DELETE' THEN data:=to_jsonb(OLD); ELSE data:=to_jsonb(NEW); END IF;
 tid:=COALESCE((data->>'tenant_id')::integer,(SELECT id_empresa FROM empresa WHERE
plataforma LIMIT 1));
 actor:=NULLIF(current_setting('app.user_id',true),'')::integer;
 IF NOT EXISTS(SELECT 1 FROM usuario WHERE tenant_id=tid AND id_usuario=actor)
THEN actor:=NULL; END IF;
 record_id:=COALESCE((data->>'id_cambio')::integer,(data->>'id_venta')::integer,(data-
>>'id_usuario')::integer,
  (data->>'id_movimiento')::integer,(data->>'id_cliente')::integer,(data-
>>'id_sucursal')::integer,(data->>'id_prenda')::integer,(data->>'id_rol')::integer,(data-
>>'id_suscripcion')::integer,(data->>'id_pago')::integer,(data->>'id_empresa')::integer,(data-
>>'id_plan')::integer);
 INSERT INTO bitacora(tenant_id,id_usuario,accion,tabla_afectada,registro_id,detalle)
 VALUES(tid,actor,TG_OP,TG_TABLE_NAME,record_id,

jsonb_build_object('operacion',TG_OP,'actor_sesion',NULLIF(current_setting('app.user_id',true)
,'')::text);
 IF TG_OP='DELETE' THEN RETURN OLD; ELSE RETURN NEW; END IF;
END $$;
ALTER FUNCTION public.audit_business_row() OWNER TO fashionstore_p2_owner;
--
-- Name: bind_push_device(text); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE FUNCTION public.bind_push_device(value text) RETURNS void
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'public', 'pg_temp'
    AS $$
DECLARE tid INTEGER:=current_tenant(); uid
INTEGER:=nullif(current_setting('app.user_id',true),'')::integer;
BEGIN
 IF tid IS NULL OR uid IS NULL OR length(value)<20 OR length(value)>4096 OR NOT
EXISTS(SELECT 1 FROM usuario WHERE tenant_id=tid AND id_usuario=uid AND
estado='Activo') THEN
  RAISE EXCEPTION 'Sesión o dispositivo no válido';
 END IF;
 PERFORM pg_advisory_xact_lock(hashtextextended(value,262026));
 DELETE FROM dispositivo_push WHERE token=value;
 INSERT INTO dispositivo_push(tenant_id,id_usuario,token) VALUES(tid,uid,value);
END $$;
ALTER FUNCTION public.bind_push_device(value text) OWNER TO fashionstore_p2_owner;
--
-- Name: branch_visible(integer); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.branch_visible(value integer) RETURNS boolean
    LANGUAGE sql STABLE
    AS $$
 SELECT COALESCE(current_setting('app.role',true),'') IN ('','Administrador','Cliente','Servicio
push')
 OR EXISTS(SELECT 1 FROM usuario_sucursal WHERE
id_usuario=nullif(current_setting('app.user_id',true),'')::integer AND id_sucursal=value AND
tenant_id=current_tenant())
$$;
ALTER FUNCTION public.branch_visible(value integer) OWNER TO fashionstore_p2_owner;
--
-- Name: canonical_variant_colors(); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.canonical_variant_colors() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
BEGIN
```

```
 NEW.color_ids:=ARRAY(SELECT DISTINCT n FROM
unnest(array_append(COALESCE(NEW.color_ids,'{}'::integer[]),NEW.id_color)) n ORDER
BY n);
 IF cardinality(NEW.color_ids)>10 OR EXISTS(SELECT 1 FROM unnest(NEW.color_ids) n
WHERE NOT EXISTS(SELECT 1 FROM color c WHERE c.tenant_id=NEW.tenant_id AND
c.id_color=n)) THEN RAISE EXCEPTION 'Colores de variante no válidos para esta empresa'
USING ERRCODE='23503'; END IF;
 RETURN NEW;
END $$;
ALTER FUNCTION public.canonical_variant_colors() OWNER TO fashionstore_p2_owner;
--
-- Name: check_branch_limit(); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.check_branch_limit() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
DECLARE cap INTEGER; used INTEGER;
BEGIN
 IF NEW.activa AND (TG_OP='INSERT' OR NOT OLD.activa) THEN
  PERFORM 1 FROM empresa WHERE id_empresa=NEW.tenant_id FOR UPDATE;
  SELECT p.max_sucursales INTO cap FROM suscripcion s JOIN plan p USING(id_plan)
   WHERE s.tenant_id=NEW.tenant_id AND now() BETWEEN s.inicio AND s.vencimiento;
  IF cap IS NULL THEN RAISE EXCEPTION 'Suscripción no vigente' USING
ERRCODE='23514'; END IF;
  SELECT count(*) INTO used FROM sucursal WHERE tenant_id=NEW.tenant_id AND
activa;
  IF used>=cap THEN RAISE EXCEPTION 'Límite de sucursales del plan alcanzado' USING
ERRCODE='23514'; END IF;
 END IF;
 RETURN NEW;
END $$;
ALTER FUNCTION public.check_branch_limit() OWNER TO fashionstore_p2_owner;
--
-- Name: current_tenant(); Type: FUNCTION; Schema: public; Owner: fashionstore_p2_owner
--
CREATE FUNCTION public.current_tenant() RETURNS integer
    LANGUAGE sql STABLE
    AS $$
 SELECT NULLIF(current_setting('app.tenant_id',true),'')::integer
```

```
$$;
ALTER FUNCTION public.current_tenant() OWNER TO fashionstore_p2_owner;
--
-- Name: identity_customer_profile(bigint, integer); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_customer_profile(p_account bigint, p_tenant integer)
RETURNS integer
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
DECLARE a public.cuenta_global; uid INTEGER; rid INTEGER; existing_state TEXT; BEGIN
 IF NULLIF(current_setting('app.account_id',true),'')::BIGINT IS DISTINCT FROM p_account
THEN RAISE EXCEPTION 'Identidad no autorizada' USING ERRCODE='42501'; END IF;
 SELECT * INTO a FROM public.cuenta_global WHERE id=p_account AND tipo='CLIENTE'
AND estado='Activo' FOR UPDATE;
 IF NOT FOUND THEN RAISE EXCEPTION 'Cuenta no disponible' USING
ERRCODE='42501'; END IF;
 IF NOT EXISTS(SELECT 1 FROM public.empresa WHERE id_empresa=p_tenant AND activa
AND NOT plataforma) THEN RAISE EXCEPTION 'Tienda no disponible' USING
ERRCODE='42501'; END IF;
 SELECT id_usuario,estado INTO uid,existing_state FROM public.usuario WHERE
tenant_id=p_tenant AND account_id=p_account;
 IF uid IS NOT NULL THEN
  IF existing_state<>'Activo' THEN RAISE EXCEPTION 'Acceso desactivado en esta tienda'
USING ERRCODE='42501'; END IF;
  RETURN uid;
 END IF;
 SELECT id_rol INTO rid FROM public.rol WHERE tenant_id=p_tenant AND
nombre='Cliente';
 IF rid IS NULL THEN RAISE EXCEPTION 'Tienda sin rol de cliente' USING
ERRCODE='42501'; END IF;
 INSERT INTO
public.usuario(tenant_id,account_id,nombre,apellido,correo,password_hash,telefono,id_role)
VALUES(p_tenant,a.id,a.nombre,a.apellido,a.correo,a.password_hash,a.telefono,rid)
RETURNING id_usuario INTO uid;
 INSERT INTO public.cliente(tenant_id,ci,nombre_completo,telefono,correo,direccion_envio)
 VALUES(p_tenant,'WEB-'||a.id,a.nombre||' '||a.apellido,a.telefono,a.correo,a.direccion_envio);
 RETURN uid;
END $$;
```

```
ALTER FUNCTION public.identity_customer_profile(p_account bigint, p_tenant integer)
OWNER TO fashionstore_p2_owner;
--
-- Name: identity_event(bigint, text); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_event(p_id bigint, p_action text) RETURNS void
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
BEGIN
 IF p_action NOT IN ('LOGIN_EXITOSO','LOGIN_FALLIDO') THEN RAISE EXCEPTION
'Evento no permitido' USING ERRCODE='42501'; END IF;
 INSERT INTO public.auditoria_identidad(account_id,accion) VALUES(p_id,p_action);
END $$;
ALTER FUNCTION public.identity_event(p_id bigint, p_action text) OWNER TO
fashionstore_p2_owner;
--
-- Name: identity_index_user(integer); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_index_user(p_user integer) RETURNS bigint
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
DECLARE u RECORD; a public.cuenta_global; aid BIGINT; kind TEXT; BEGIN
 SELECT v.*,r.nombre AS rol,e.plataforma INTO u FROM public.usuario v JOIN public.rol r
ON r.id_rol=v.id_role JOIN public.empresa e ON e.id_empresa=v.tenant_id WHERE
v.id_usuario=p_user AND v.tenant_id=public.current_tenant();
 IF NOT FOUND THEN RAISE EXCEPTION 'Usuario fuera de la empresa' USING
ERRCODE='42501'; END IF;
 kind:=CASE WHEN u.rol='Cliente' THEN 'CLIENTE' WHEN u.plataforma THEN
'PLATAFORMA' ELSE 'PERSONAL' END;
 IF u.account_id IS NULL THEN
  INSERT INTO
public.cuenta_global(correo,password_hash,nombre,apellido,telefono,tipo,estado)
VALUES(lower(trim(u.correo)),u.password_hash,u.nombre,u.apellido,u.telefono,kind,u.estado)
RETURNING id INTO aid;
  UPDATE public.usuario SET account_id=aid,correo=lower(trim(correo)) WHERE
id_usuario=p_user;
```

```
 ELSE
  SELECT * INTO a FROM public.cuenta_global WHERE id=u.account_id FOR UPDATE;
  aid:=a.id;
  IF a.tipo='CLIENTE' THEN
   IF kind<>'CLIENTE' OR lower(trim(u.correo))<>a.correo OR
u.password_hash<>a.password_hash THEN
    RAISE EXCEPTION 'La identidad del cliente se administra desde su propia cuenta global'
USING ERRCODE='42501';
   END IF;
  ELSE
   UPDATE public.cuenta_global SET
correo=lower(trim(u.correo)),password_hash=u.password_hash,nombre=u.nombre,apellido=u.ap
ellido,telefono=u.telefono,tipo=kind,estado=u.estado,
    token_version=token_version+CASE WHEN correo<>lower(trim(u.correo)) OR
password_hash<>u.password_hash OR tipo<>kind OR estado<>u.estado THEN 1 ELSE 0 END
WHERE id=aid;
  END IF;
 END IF;
 RETURN aid;
END $$;
ALTER FUNCTION public.identity_index_user(p_user integer) OWNER TO
fashionstore_p2_owner;
--
-- Name: identity_lookup(text, bigint); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_lookup(p_correo text DEFAULT NULL::text, p_id bigint
DEFAULT NULL::bigint) RETURNS jsonb
    LANGUAGE sql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
 SELECT
to_jsonb(a)||jsonb_build_object('tenant_id',u.tenant_id,'id_usuario',u.id_usuario,'rol',r.nombre,'sl
ug',e.slug,'empresa_activa',e.activa,'local_estado',u.estado,'local_version',u.token_version)
 FROM public.cuenta_global a LEFT JOIN public.usuario u ON u.account_id=a.id AND
a.tipo<>'CLIENTE'
 LEFT JOIN public.rol r ON r.id_rol=u.id_role LEFT JOIN public.empresa e ON
e.id_empresa=u.tenant_id
 WHERE (p_id IS NOT NULL AND a.id=p_id) OR (p_id IS NULL AND
a.correo=lower(trim(p_correo))) LIMIT 1
```

```
$$;
ALTER FUNCTION public.identity_lookup(p_correo text, p_id bigint) OWNER TO
fashionstore_p2_owner;
--
-- Name: identity_register(jsonb); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_register(p_data jsonb) RETURNS bigint
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
DECLARE aid BIGINT; BEGIN
 INSERT INTO
public.cuenta_global(correo,password_hash,nombre,apellido,telefono,ci,direccion_envio,correo_
recuperacion,tipo)
 VALUES(lower(trim(p_data->>'correo')),p_data->>'password_hash',p_data->>'nombre',p_data-
>>'apellido',p_data->>'telefono',p_data->>'ci',p_data->>'direccion_envio',p_data-
>>'correo_recuperacion','CLIENTE') RETURNING id INTO aid;
 INSERT INTO public.auditoria_identidad(account_id,accion) VALUES(aid,'REGISTRO');
RETURN aid;
END $$;
ALTER FUNCTION public.identity_register(p_data jsonb) OWNER TO
fashionstore_p2_owner;
--
-- Name: identity_reset_apply(text, text); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_reset_apply(p_hash text, p_password_hash text)
RETURNS boolean
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
DECLARE aid BIGINT; BEGIN
 SELECT t.account_id INTO aid FROM public.cuenta_recuperacion t JOIN
public.cuenta_global a ON a.id=t.account_id WHERE t.token_hash=p_hash AND NOT t.usado
AND t.expiracion>now() AND a.estado='Activo' FOR UPDATE OF t;
 IF aid IS NULL THEN RETURN false; END IF;
 UPDATE public.cuenta_global SET
password_hash=p_password_hash,token_version=token_version+1 WHERE id=aid;
```

```
 UPDATE public.usuario SET
password_hash=p_password_hash,token_version=token_version+1 WHERE account_id=aid;
 UPDATE public.cuenta_recuperacion SET usado=true WHERE account_id=aid;
 INSERT INTO public.auditoria_identidad(account_id,accion)
VALUES(aid,'PASSWORD_RESTABLECIDA'); RETURN true;
END $$;
ALTER FUNCTION public.identity_reset_apply(p_hash text, p_password_hash text) OWNER
TO fashionstore_p2_owner;
--
-- Name: identity_reset_issue(bigint, text); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_reset_issue(p_id bigint, p_hash text) RETURNS void
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
BEGIN
 UPDATE public.cuenta_recuperacion SET usado=true WHERE account_id=p_id AND NOT
usado;
 INSERT INTO public.cuenta_recuperacion(token_hash,account_id) VALUES(p_hash,p_id);
 INSERT INTO public.auditoria_identidad(account_id,accion)
VALUES(p_id,'SOLICITUD_RECUPERACION');
END $$;
ALTER FUNCTION public.identity_reset_issue(p_id bigint, p_hash text) OWNER TO
fashionstore_p2_owner;
--
-- Name: identity_reset_lookup(text); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_reset_lookup(p_hash text) RETURNS jsonb
    LANGUAGE sql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
 SELECT jsonb_build_object('id',a.id,'correo',a.correo,'valido',NOT t.usado AND
t.expiracion>now() AND a.estado='Activo') FROM public.cuenta_recuperacion t JOIN
public.cuenta_global a ON a.id=t.account_id WHERE token_hash=p_hash
$$;
ALTER FUNCTION public.identity_reset_lookup(p_hash text) OWNER TO
fashionstore_p2_owner;
--
```

```
-- Name: identity_revoke(bigint); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.identity_revoke(p_id bigint) RETURNS void
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'public'
    AS $$
BEGIN
 IF NULLIF(current_setting('app.account_id',true),'')::BIGINT IS DISTINCT FROM p_id
THEN RAISE EXCEPTION 'Identidad no autorizada' USING ERRCODE='42501'; END IF;
 UPDATE public.cuenta_global SET token_version=token_version+1 WHERE id=p_id;
 INSERT INTO public.auditoria_identidad(account_id,accion) VALUES(p_id,'LOGOUT');
END $$;
ALTER FUNCTION public.identity_revoke(p_id bigint) OWNER TO fashionstore_p2_owner;
--
-- Name: own_customer(); Type: FUNCTION; Schema: public; Owner: fashionstore_p2_owner
--
CREATE FUNCTION public.own_customer() RETURNS integer
    LANGUAGE sql STABLE
    AS $$
 SELECT id_cliente FROM cliente WHERE tenant_id=current_tenant()
 AND correo=NULLIF(current_setting('app.email',true),'') LIMIT 1
$$;
ALTER FUNCTION public.own_customer() OWNER TO fashionstore_p2_owner;
--
-- Name: sync_variant_colors(); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.sync_variant_colors() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
BEGIN
 DELETE FROM variante_color WHERE tenant_id=NEW.tenant_id AND
id_variante_prenda=NEW.id_variante_prenda;
 INSERT INTO variante_color(tenant_id,id_variante_prenda,id_color) SELECT
NEW.tenant_id,NEW.id_variante_prenda,unnest(NEW.color_ids);
 RETURN NEW;
END $$;
ALTER FUNCTION public.sync_variant_colors() OWNER TO fashionstore_p2_owner;
--
```

```
-- Name: variant_color_label(integer); Type: FUNCTION; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE FUNCTION public.variant_color_label(variant integer) RETURNS text
    LANGUAGE sql STABLE
    AS $$
 SELECT string_agg(c.nombre,' / ' ORDER BY c.nombre) FROM variante_color vc JOIN color
c USING(tenant_id,id_color) WHERE vc.id_variante_prenda=variant
$$;
ALTER FUNCTION public.variant_color_label(variant integer) OWNER TO
fashionstore_p2_owner;
SET default_tablespace = '';
SET default_table_access_method = heap;
--
-- Name: ajuste_monetario; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.ajuste_monetario (
    id_ajuste integer NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_cambio integer NOT NULL,
    tipo text NOT NULL,
    importe numeric(12,2) NOT NULL,
    origen text NOT NULL,
    referencia text NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT ajuste_monetario_importe_check CHECK ((importe > (0)::numeric)),
    CONSTRAINT ajuste_monetario_tipo_check CHECK ((tipo = ANY (ARRAY['Cobro'::text,
'Reembolso'::text])))
);
ALTER TABLE public.ajuste_monetario OWNER TO fashionstore_p2_owner;
--
-- Name: ajuste_monetario_id_ajuste_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.ajuste_monetario_id_ajuste_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
```

```
    CACHE 1;
ALTER SEQUENCE public.ajuste_monetario_id_ajuste_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: ajuste_monetario_id_ajuste_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.ajuste_monetario_id_ajuste_seq OWNED BY
public.ajuste_monetario.id_ajuste;
--
-- Name: auditoria_identidad; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.auditoria_identidad (
    id bigint NOT NULL,
    account_id bigint,
    accion text NOT NULL,
    fecha timestamp with time zone DEFAULT now() NOT NULL
);
ALTER TABLE public.auditoria_identidad OWNER TO fashionstore_p2_owner;
--
-- Name: auditoria_identidad_id_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.auditoria_identidad_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.auditoria_identidad_id_seq OWNER TO fashionstore_p2_owner;
--
-- Name: auditoria_identidad_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.auditoria_identidad_id_seq OWNED BY
public.auditoria_identidad.id;
--
-- Name: bitacora; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.bitacora (
```

```
    id_bitacora integer NOT NULL,
    id_usuario integer,
    accion character varying(100) NOT NULL,
    tabla_afectada character varying(100) NOT NULL,
    registro_id integer,
    detalle text,
    ip_address character varying(45),
    fecha timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.bitacora OWNER TO fashionstore_p2_owner;
--
-- Name: bitacora_id_bitacora_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.bitacora_id_bitacora_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.bitacora_id_bitacora_seq OWNER TO fashionstore_p2_owner;
--
-- Name: bitacora_id_bitacora_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.bitacora_id_bitacora_seq OWNED BY public.bitacora.id_bitacora;
--
-- Name: cambio; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.cambio (
    id_cambio integer NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_venta integer NOT NULL,
    id_cliente integer NOT NULL,
    id_sucursal integer NOT NULL,
    id_variante_original integer NOT NULL,
    cantidad integer NOT NULL,
    motivo text NOT NULL,
```

```
    estado text DEFAULT 'Solicitado'::text NOT NULL,
    credito numeric(12,2) NOT NULL,
    diferencia numeric(12,2),
    id_reserva_reemplazo integer,
    recibido_at timestamp with time zone,
    entregado_at timestamp with time zone,
    cancelar boolean DEFAULT false NOT NULL,
    pago_diferencia jsonb,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT cambio_cantidad_check CHECK ((cantidad > 0)),
    CONSTRAINT cambio_credito_check CHECK ((credito >= (0)::numeric)),
    CONSTRAINT cambio_estado_check CHECK ((estado = ANY (ARRAY['Solicitado'::text,
'Recibido'::text, 'PagoPendiente'::text, 'ReembolsoPendiente'::text, 'Listo'::text, 'Entregado'::text,
'Reembolsado'::text, 'Cancelado'::text, 'Rechazado'::text])))
);
ALTER TABLE public.cambio OWNER TO fashionstore_p2_owner;
--
-- Name: cambio_id_cambio_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.cambio_id_cambio_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.cambio_id_cambio_seq OWNER TO fashionstore_p2_owner;
--
-- Name: cambio_id_cambio_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.cambio_id_cambio_seq OWNED BY public.cambio.id_cambio;
--
-- Name: categoria; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.categoria (
    id_categoria integer NOT NULL,
    nombre character varying(100) NOT NULL,
    descripcion character varying(255),
```

```
    id_categoria_padre integer,
    estado character varying(20) DEFAULT 'Activo'::character varying,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    CONSTRAINT categoria_estado_check CHECK (((estado)::text = ANY
((ARRAY['Activo'::character varying, 'Inactivo'::character varying])::text[])))
);
ALTER TABLE public.categoria OWNER TO fashionstore_p2_owner;
--
-- Name: categoria_id_categoria_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.categoria_id_categoria_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.categoria_id_categoria_seq OWNER TO fashionstore_p2_owner;
--
-- Name: categoria_id_categoria_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.categoria_id_categoria_seq OWNED BY
public.categoria.id_categoria;
--
-- Name: ciudad; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.ciudad (
    id_ciudad integer NOT NULL,
    nombre character varying(100) NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.ciudad OWNER TO fashionstore_p2_owner;
--
-- Name: ciudad_id_ciudad_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.ciudad_id_ciudad_seq
    AS integer
```

```
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.ciudad_id_ciudad_seq OWNER TO fashionstore_p2_owner;
--
-- Name: ciudad_id_ciudad_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.ciudad_id_ciudad_seq OWNED BY public.ciudad.id_ciudad;
--
-- Name: cliente; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.cliente (
    id_cliente integer NOT NULL,
    ci character varying(20) NOT NULL,
    nombre_completo character varying(200) NOT NULL,
    telefono character varying(20),
    correo character varying(150),
    direccion_envio character varying(255),
    estado character varying(20) DEFAULT 'Activo'::character varying,
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    CONSTRAINT cliente_estado_check CHECK (((estado)::text = ANY
((ARRAY['Activo'::character varying, 'Inactivo'::character varying])::text[])))
);
ALTER TABLE public.cliente OWNER TO fashionstore_p2_owner;
--
-- Name: cliente_id_cliente_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.cliente_id_cliente_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
```

```
ALTER SEQUENCE public.cliente_id_cliente_seq OWNER TO fashionstore_p2_owner;
--
-- Name: cliente_id_cliente_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.cliente_id_cliente_seq OWNED BY public.cliente.id_cliente;
--
-- Name: color; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.color (
    id_color integer NOT NULL,
    nombre character varying(50) NOT NULL,
    codigo_hex character varying(10),
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.color OWNER TO fashionstore_p2_owner;
--
-- Name: color_id_color_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.color_id_color_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.color_id_color_seq OWNER TO fashionstore_p2_owner;
--
-- Name: color_id_color_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.color_id_color_seq OWNED BY public.color.id_color;
--
-- Name: compra; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.compra (
    id_compra integer NOT NULL,
    id_proveedor integer,
    id_sucursal integer,
```

```
    fecha timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    total numeric(12,2) DEFAULT 0.00 NOT NULL,
    id_usuario integer,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.compra OWNER TO fashionstore_p2_owner;
--
-- Name: compra_id_compra_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.compra_id_compra_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.compra_id_compra_seq OWNER TO fashionstore_p2_owner;
--
-- Name: compra_id_compra_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.compra_id_compra_seq OWNED BY public.compra.id_compra;
--
-- Name: comprobante; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.comprobante (
    id_comprobante integer NOT NULL,
    id_venta integer,
    numero_comprobante character varying(50) NOT NULL,
    nit_ci character varying(20) NOT NULL,
    razon_social character varying(150) NOT NULL,
    fecha_emision timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    url_pdf character varying(255),
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.comprobante OWNER TO fashionstore_p2_owner;
--
-- Name: comprobante_id_comprobante_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
```

```
--
CREATE SEQUENCE public.comprobante_id_comprobante_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.comprobante_id_comprobante_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: comprobante_id_comprobante_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.comprobante_id_comprobante_seq OWNED BY
public.comprobante.id_comprobante;
--
-- Name: cuenta_global; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.cuenta_global (
    id bigint NOT NULL,
    correo text NOT NULL,
    password_hash text NOT NULL,
    nombre text NOT NULL,
    apellido text DEFAULT ''::text NOT NULL,
    telefono text,
    ci text,
    direccion_envio text,
    correo_recuperacion text,
    tipo text NOT NULL,
    estado text DEFAULT 'Activo'::text NOT NULL,
    token_version integer DEFAULT 0 NOT NULL,
    creada timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT cuenta_global_correo_check CHECK ((correo = lower(TRIM(BOTH FROM
correo)))),
    CONSTRAINT cuenta_global_estado_check CHECK ((estado = ANY
(ARRAY['Activo'::text, 'Inactivo'::text]))),
    CONSTRAINT cuenta_global_tipo_check CHECK ((tipo = ANY (ARRAY['CLIENTE'::text,
'PERSONAL'::text, 'PLATAFORMA'::text])))
);
```

```
ALTER TABLE public.cuenta_global OWNER TO fashionstore_p2_owner;
--
-- Name: cuenta_global_id_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.cuenta_global_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.cuenta_global_id_seq OWNER TO fashionstore_p2_owner;
--
-- Name: cuenta_global_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.cuenta_global_id_seq OWNED BY public.cuenta_global.id;
--
-- Name: cuenta_recuperacion; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.cuenta_recuperacion (
    token_hash text NOT NULL,
    account_id bigint NOT NULL,
    expiracion timestamp with time zone DEFAULT (now() + '00:30:00'::interval) NOT NULL,
    usado boolean DEFAULT false NOT NULL
);
ALTER TABLE public.cuenta_recuperacion OWNER TO fashionstore_p2_owner;
--
-- Name: detalle_compra; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.detalle_compra (
    id_compra integer NOT NULL,
    id_variante_prenda integer NOT NULL,
    cantidad integer NOT NULL,
    costo_unitario numeric(10,2) NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    CONSTRAINT detalle_compra_cantidad_check CHECK ((cantidad > 0)),
    CONSTRAINT detalle_compra_costo_unitario_check CHECK ((costo_unitario >=
(0)::numeric))
);
```

```
ALTER TABLE public.detalle_compra OWNER TO fashionstore_p2_owner;
--
-- Name: detalle_reserva; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.detalle_reserva (
    id_reserva integer NOT NULL,
    id_variante_prenda integer NOT NULL,
    cantidad integer DEFAULT 1 NOT NULL,
    precio_unitario numeric(10,2) NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    CONSTRAINT detalle_reserva_cantidad_check CHECK ((cantidad > 0))
);
ALTER TABLE public.detalle_reserva OWNER TO fashionstore_p2_owner;
--
-- Name: detalle_venta; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.detalle_venta (
    id_venta integer NOT NULL,
    id_variante_prenda integer NOT NULL,
    cantidad integer NOT NULL,
    precio_unitario numeric(10,2) NOT NULL,
    subtotal numeric(10,2) NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    CONSTRAINT detalle_venta_cantidad_check CHECK ((cantidad > 0))
);
ALTER TABLE public.detalle_venta OWNER TO fashionstore_p2_owner;
--
-- Name: dispositivo_push; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.dispositivo_push (
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_usuario integer NOT NULL,
    token text NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL
);
ALTER TABLE public.dispositivo_push OWNER TO fashionstore_p2_owner;
--
-- Name: empresa; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.empresa (
```

```
    id_empresa integer NOT NULL,
    slug text NOT NULL,
    nombre text NOT NULL,
    activa boolean DEFAULT true NOT NULL,
    plataforma boolean DEFAULT false NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT empresa_slug_check CHECK ((slug ~ '^[a-z0-9][a-z0-9-]{1,60}$'::text))
);
ALTER TABLE public.empresa OWNER TO fashionstore_p2_owner;
--
-- Name: empresa_id_empresa_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.empresa_id_empresa_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.empresa_id_empresa_seq OWNER TO fashionstore_p2_owner;
--
-- Name: empresa_id_empresa_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.empresa_id_empresa_seq OWNED BY
public.empresa.id_empresa;
--
-- Name: imagen_prenda; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.imagen_prenda (
    id_imagen integer NOT NULL,
    id_prenda integer,
    url_imagen character varying(255) NOT NULL,
    es_principal boolean DEFAULT false,
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.imagen_prenda OWNER TO fashionstore_p2_owner;
--
```

```
-- Name: imagen_prenda_id_imagen_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.imagen_prenda_id_imagen_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.imagen_prenda_id_imagen_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: imagen_prenda_id_imagen_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.imagen_prenda_id_imagen_seq OWNED BY
public.imagen_prenda.id_imagen;
--
-- Name: inventario; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.inventario (
    id_sucursal integer NOT NULL,
    id_variante_prenda integer NOT NULL,
    stock integer DEFAULT 0 NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    CONSTRAINT inventario_stock_check CHECK ((stock >= 0))
);
ALTER TABLE public.inventario OWNER TO fashionstore_p2_owner;
--
-- Name: retencion; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.retencion (
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_reserva integer NOT NULL,
    id_sucursal integer NOT NULL,
    id_variante_prenda integer NOT NULL,
    cantidad integer NOT NULL,
    vence timestamp with time zone NOT NULL,
    CONSTRAINT retencion_cantidad_check CHECK ((cantidad > 0))
```

```
);
ALTER TABLE public.retencion OWNER TO fashionstore_p2_owner;
--
-- Name: inventario_catalogo; Type: VIEW; Schema: public; Owner: fashionstore_p2_owner
--
CREATE VIEW public.inventario_catalogo WITH (security_invoker='true') AS
 SELECT tenant_id,
    id_sucursal,
    id_variante_prenda,
    (GREATEST((stock - COALESCE(( SELECT sum(r.cantidad) AS sum
           FROM public.retencion r
          WHERE ((r.tenant_id = i.tenant_id) AND (r.id_sucursal = i.id_sucursal) AND
(r.id_variante_prenda = i.id_variante_prenda) AND (r.vence > now()))), (0)::bigint)),
(0)::bigint))::integer AS stock
   FROM public.inventario i;
ALTER VIEW public.inventario_catalogo OWNER TO fashionstore_p2_owner;
--
-- Name: movimiento_inventario; Type: TABLE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TABLE public.movimiento_inventario (
    id_movimiento integer NOT NULL,
    id_sucursal integer,
    id_variante_prenda integer,
    tipo character varying(20) NOT NULL,
    cantidad integer NOT NULL,
    motivo character varying(255) NOT NULL,
    id_usuario integer,
    fecha timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    CONSTRAINT movimiento_inventario_cantidad_check CHECK ((cantidad > 0)),
    CONSTRAINT movimiento_inventario_tipo_check CHECK (((tipo)::text = ANY
((ARRAY['Entrada'::character varying, 'Salida'::character varying, 'Traspaso'::character
varying])::text[])))
);
ALTER TABLE public.movimiento_inventario OWNER TO fashionstore_p2_owner;
--
-- Name: movimiento_inventario_id_movimiento_seq; Type: SEQUENCE; Schema: public;
Owner: fashionstore_p2_owner
--
```

```
CREATE SEQUENCE public.movimiento_inventario_id_movimiento_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.movimiento_inventario_id_movimiento_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: movimiento_inventario_id_movimiento_seq; Type: SEQUENCE OWNED BY;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.movimiento_inventario_id_movimiento_seq OWNED BY
public.movimiento_inventario.id_movimiento;
--
-- Name: notificacion; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.notificacion (
    id_notificacion integer NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_cliente integer NOT NULL,
    titulo text NOT NULL,
    mensaje text NOT NULL,
    id_cambio integer,
    leida boolean DEFAULT false NOT NULL,
    push_enviado boolean DEFAULT false NOT NULL,
    intentos integer DEFAULT 0 NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL
);
ALTER TABLE public.notificacion OWNER TO fashionstore_p2_owner;
--
-- Name: notificacion_id_notificacion_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.notificacion_id_notificacion_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
```

```
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.notificacion_id_notificacion_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: notificacion_id_notificacion_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.notificacion_id_notificacion_seq OWNED BY
public.notificacion.id_notificacion;
--
-- Name: operacion_sync; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.operacion_sync (
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    operation_id uuid NOT NULL,
    author_id integer NOT NULL,
    branch_id integer,
    device_id uuid NOT NULL,
    kind text NOT NULL,
    payload_hash text NOT NULL,
    envelope jsonb NOT NULL,
    state text NOT NULL,
    result jsonb NOT NULL,
    original_operation uuid,
    resolved_by integer,
    resolution text,
    compensation numeric(12,2),
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    resolved_at timestamp with time zone,
    CONSTRAINT operacion_sync_check CHECK (((branch_id IS NOT NULL) OR (state =
'RECHAZADA'::text))),
    CONSTRAINT operacion_sync_kind_check CHECK ((kind = ANY (ARRAY['venta'::text,
'reserva'::text]))),
    CONSTRAINT operacion_sync_state_check CHECK ((state = ANY
(ARRAY['CONFIRMADA'::text, 'RECHAZADA'::text, 'EN_CONFLICTO'::text])))
);
ALTER TABLE public.operacion_sync OWNER TO fashionstore_p2_owner;
--
-- Name: pago_transaccion; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
```

```
--
CREATE TABLE public.pago_transaccion (
    id_pago integer NOT NULL,
    id_venta integer,
    pasarela character varying(50) NOT NULL,
    transaccion_id character varying(100) NOT NULL,
    monto numeric(10,2) NOT NULL,
    estado_pago character varying(50) NOT NULL,
    payload_respuesta jsonb,
    fecha timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_reserva integer,
    url_pago text,
    vence timestamp with time zone
);
ALTER TABLE public.pago_transaccion OWNER TO fashionstore_p2_owner;
--
-- Name: pago_transaccion_id_pago_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.pago_transaccion_id_pago_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.pago_transaccion_id_pago_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: pago_transaccion_id_pago_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.pago_transaccion_id_pago_seq OWNED BY
public.pago_transaccion.id_pago;
--
-- Name: permiso; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.permiso (
    id_permiso integer NOT NULL,
```

```
    nombre character varying(100) NOT NULL,
    codigo character varying(100) NOT NULL,
    modulo character varying(100),
    descripcion character varying(255)
);
ALTER TABLE public.permiso OWNER TO fashionstore_p2_owner;
--
-- Name: permiso_id_permiso_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.permiso_id_permiso_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.permiso_id_permiso_seq OWNER TO fashionstore_p2_owner;
--
-- Name: permiso_id_permiso_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.permiso_id_permiso_seq OWNED BY public.permiso.id_permiso;
--
-- Name: plan; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.plan (
    id_plan integer NOT NULL,
    nombre text NOT NULL,
    max_sucursales integer NOT NULL,
    reportes_ia boolean DEFAULT false NOT NULL,
    CONSTRAINT plan_max_sucursales_check CHECK ((max_sucursales > 0))
);
ALTER TABLE public.plan OWNER TO fashionstore_p2_owner;
--
-- Name: plan_id_plan_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.plan_id_plan_seq
    AS integer
```

```
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.plan_id_plan_seq OWNER TO fashionstore_p2_owner;
--
-- Name: plan_id_plan_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.plan_id_plan_seq OWNED BY public.plan.id_plan;
--
-- Name: prenda; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.prenda (
    id_prenda integer NOT NULL,
    sku character varying(50) NOT NULL,
    nombre character varying(150) NOT NULL,
    descripcion text,
    marca character varying(100) DEFAULT 'FashionStore'::character varying,
    genero character varying(20) DEFAULT 'Unisex'::character varying,
    precio_base numeric(10,2) NOT NULL,
    id_categoria integer,
    id_temporada integer,
    estado character varying(20) DEFAULT 'Activo'::character varying,
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    material text,
    medidas text,
    precio_mayorista numeric(10,2),
    umbral_mayorista integer DEFAULT 6 NOT NULL,
    precio_oferta numeric(10,2),
    oferta_inicio date,
    oferta_fin date,
    admite_cambio boolean DEFAULT true NOT NULL,
    CONSTRAINT offer_dates CHECK (((oferta_fin IS NULL) OR (oferta_inicio IS NULL) OR
(oferta_fin >= oferta_inicio))),
```

```
    CONSTRAINT prenda_estado_check CHECK (((estado)::text = ANY
((ARRAY['Activo'::character varying, 'Inactivo'::character varying, 'Borrador'::character
varying])::text[]))),
    CONSTRAINT prenda_genero_check CHECK (((genero)::text = ANY
((ARRAY['Dama'::character varying, 'Caballero'::character varying, 'Unisex'::character varying,
'Nino'::character varying])::text[]))),
    CONSTRAINT prenda_precio_base_check CHECK ((precio_base >= (0)::numeric)),
    CONSTRAINT prenda_precio_mayorista_check CHECK ((precio_mayorista >=
(0)::numeric)),
    CONSTRAINT prenda_precio_oferta_check CHECK ((precio_oferta >= (0)::numeric)),
    CONSTRAINT prenda_umbral_mayorista_check CHECK ((umbral_mayorista > 0))
);
ALTER TABLE public.prenda OWNER TO fashionstore_p2_owner;
--
-- Name: prenda_id_prenda_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.prenda_id_prenda_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.prenda_id_prenda_seq OWNER TO fashionstore_p2_owner;
--
-- Name: prenda_id_prenda_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.prenda_id_prenda_seq OWNED BY public.prenda.id_prenda;
--
-- Name: proveedor; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.proveedor (
    id_proveedor integer NOT NULL,
    nit character varying(30) NOT NULL,
    razon_social character varying(150) NOT NULL,
    contacto character varying(100),
    telefono character varying(20),
    correo character varying(150),
```

```
    direccion character varying(255),
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.proveedor OWNER TO fashionstore_p2_owner;
--
-- Name: proveedor_id_proveedor_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.proveedor_id_proveedor_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.proveedor_id_proveedor_seq OWNER TO fashionstore_p2_owner;
--
-- Name: proveedor_id_proveedor_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.proveedor_id_proveedor_seq OWNED BY
public.proveedor.id_proveedor;
--
-- Name: push_entrega; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.push_entrega (
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_notificacion integer NOT NULL,
    token_hash text NOT NULL,
    enviado boolean DEFAULT false NOT NULL,
    intentos integer DEFAULT 0 NOT NULL,
    proximo_intento timestamp with time zone DEFAULT now() NOT NULL,
    ultimo_estado text
);
ALTER TABLE public.push_entrega OWNER TO fashionstore_p2_owner;
--
-- Name: reporte_resultado; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.reporte_resultado (
```

```
    id uuid NOT NULL,
    tenant_id bigint DEFAULT (NULLIF(current_setting('app.tenant_id'::text, true),
''::text))::bigint NOT NULL,
    author_id integer NOT NULL,
    filters jsonb NOT NULL,
    result jsonb NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL,
    expires_at timestamp with time zone DEFAULT (now() + '24:00:00'::interval) NOT NULL
);
ALTER TABLE ONLY public.reporte_resultado FORCE ROW LEVEL SECURITY;
ALTER TABLE public.reporte_resultado OWNER TO fashionstore_p2_owner;
--
-- Name: reserva; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.reserva (
    id_reserva integer NOT NULL,
    id_cliente integer,
    id_sucursal integer,
    fecha_reserva timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    fecha_limite timestamp without time zone NOT NULL,
    estado character varying(30) DEFAULT 'Pendiente'::character varying,
    total numeric(10,2) DEFAULT 0.00 NOT NULL,
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    checkout_generation integer DEFAULT 1 NOT NULL,
    CONSTRAINT reserva_estado_check CHECK (((estado)::text = ANY
((ARRAY['Pendiente'::character varying, 'Preparado'::character varying, 'Atendido'::character
varying, 'Cancelado'::character varying])::text[])))
);
ALTER TABLE public.reserva OWNER TO fashionstore_p2_owner;
--
-- Name: reserva_id_reserva_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.reserva_id_reserva_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
```

```
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.reserva_id_reserva_seq OWNER TO fashionstore_p2_owner;
--
-- Name: reserva_id_reserva_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.reserva_id_reserva_seq OWNED BY public.reserva.id_reserva;
--
-- Name: rol; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.rol (
    id_rol integer NOT NULL,
    nombre character varying(50) NOT NULL,
    descripcion character varying(255),
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.rol OWNER TO fashionstore_p2_owner;
--
-- Name: rol_id_rol_seq; Type: SEQUENCE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE SEQUENCE public.rol_id_rol_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.rol_id_rol_seq OWNER TO fashionstore_p2_owner;
--
-- Name: rol_id_rol_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.rol_id_rol_seq OWNED BY public.rol.id_rol;
--
-- Name: rol_permiso; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.rol_permiso (
```

```
    id_rol integer NOT NULL,
    id_permiso integer NOT NULL,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.rol_permiso OWNER TO fashionstore_p2_owner;
--
-- Name: schema_migrations; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.schema_migrations (
    version text NOT NULL,
    checksum text NOT NULL,
    applied_at timestamp with time zone DEFAULT now() NOT NULL
);
ALTER TABLE public.schema_migrations OWNER TO fashionstore_p2_owner;
--
-- Name: stripe_event; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.stripe_event (
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    event_id text NOT NULL,
    event_type text NOT NULL,
    result jsonb NOT NULL,
    created_at timestamp with time zone DEFAULT now() NOT NULL
);
ALTER TABLE public.stripe_event OWNER TO fashionstore_p2_owner;
--
-- Name: sucursal; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.sucursal (
    id_sucursal integer NOT NULL,
    nombre character varying(100) NOT NULL,
    direccion character varying(255) NOT NULL,
    telefono character varying(20),
    id_ciudad integer,
    id_encargado integer,
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    activa boolean DEFAULT true NOT NULL
);
```

```
ALTER TABLE public.sucursal OWNER TO fashionstore_p2_owner;
--
-- Name: sucursal_id_sucursal_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.sucursal_id_sucursal_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.sucursal_id_sucursal_seq OWNER TO fashionstore_p2_owner;
--
-- Name: sucursal_id_sucursal_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.sucursal_id_sucursal_seq OWNED BY public.sucursal.id_sucursal;
--
-- Name: suscripcion; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.suscripcion (
    id_suscripcion integer NOT NULL,
    tenant_id integer NOT NULL,
    id_plan integer NOT NULL,
    inicio timestamp with time zone NOT NULL,
    vencimiento timestamp with time zone NOT NULL,
    CONSTRAINT suscripcion_check CHECK ((vencimiento > inicio))
);
ALTER TABLE public.suscripcion OWNER TO fashionstore_p2_owner;
--
-- Name: suscripcion_id_suscripcion_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.suscripcion_id_suscripcion_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
```

```
    CACHE 1;
ALTER SEQUENCE public.suscripcion_id_suscripcion_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: suscripcion_id_suscripcion_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.suscripcion_id_suscripcion_seq OWNED BY
public.suscripcion.id_suscripcion;
--
-- Name: talla; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.talla (
    id_talla integer NOT NULL,
    nombre character varying(10) NOT NULL,
    descripcion character varying(50),
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.talla OWNER TO fashionstore_p2_owner;
--
-- Name: talla_id_talla_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.talla_id_talla_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.talla_id_talla_seq OWNER TO fashionstore_p2_owner;
--
-- Name: talla_id_talla_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.talla_id_talla_seq OWNED BY public.talla.id_talla;
--
-- Name: temporada; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.temporada (
```

```
    id_temporada integer NOT NULL,
    nombre character varying(100) NOT NULL,
    descripcion character varying(255),
    fecha_inicio date,
    fecha_fin date,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    estado boolean DEFAULT true NOT NULL,
    created_at timestamp without time zone DEFAULT now()
);
ALTER TABLE public.temporada OWNER TO fashionstore_p2_owner;
--
-- Name: temporada_id_temporada_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.temporada_id_temporada_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.temporada_id_temporada_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: temporada_id_temporada_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.temporada_id_temporada_seq OWNED BY
public.temporada.id_temporada;
--
-- Name: usuario; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.usuario (
    id_usuario integer NOT NULL,
    nombre character varying(100) NOT NULL,
    apellido character varying(100) NOT NULL,
    correo character varying(150) NOT NULL,
    password_hash character varying(255) NOT NULL,
    telefono character varying(20),
    estado character varying(20) DEFAULT 'Activo'::character varying,
```

```
    id_role integer,
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    token_version integer DEFAULT 0 NOT NULL,
    account_id bigint,
    CONSTRAINT usuario_estado_check CHECK (((estado)::text = ANY
((ARRAY['Activo'::character varying, 'Inactivo'::character varying])::text[])))
);
ALTER TABLE public.usuario OWNER TO fashionstore_p2_owner;
--
-- Name: usuario_id_usuario_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.usuario_id_usuario_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.usuario_id_usuario_seq OWNER TO fashionstore_p2_owner;
--
-- Name: usuario_id_usuario_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.usuario_id_usuario_seq OWNED BY public.usuario.id_usuario;
--
-- Name: usuario_sucursal; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.usuario_sucursal (
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_usuario integer NOT NULL,
    id_sucursal integer NOT NULL
);
ALTER TABLE public.usuario_sucursal OWNER TO fashionstore_p2_owner;
--
-- Name: usuario_token; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.usuario_token (
```

```
    id_token integer NOT NULL,
    id_usuario integer,
    token_recuperacion character varying(255),
    expiracion timestamp without time zone,
    usado boolean DEFAULT false,
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL
);
ALTER TABLE public.usuario_token OWNER TO fashionstore_p2_owner;
--
-- Name: usuario_token_id_token_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.usuario_token_id_token_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.usuario_token_id_token_seq OWNER TO fashionstore_p2_owner;
--
-- Name: usuario_token_id_token_seq; Type: SEQUENCE OWNED BY; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.usuario_token_id_token_seq OWNED BY
public.usuario_token.id_token;
--
-- Name: variante_color; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.variante_color (
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    id_variante_prenda integer NOT NULL,
    id_color integer NOT NULL
);
ALTER TABLE public.variante_color OWNER TO fashionstore_p2_owner;
--
-- Name: variante_prenda; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.variante_prenda (
```

```
    id_variante_prenda integer NOT NULL,
    id_prenda integer,
    id_talla integer,
    id_color integer,
    sku_variante character varying(100) NOT NULL,
    precio_adicional numeric(10,2) DEFAULT 0.00,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    color_ids integer[] DEFAULT '{}'::integer[] NOT NULL
);
ALTER TABLE public.variante_prenda OWNER TO fashionstore_p2_owner;
--
-- Name: variante_prenda_id_variante_prenda_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.variante_prenda_id_variante_prenda_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.variante_prenda_id_variante_prenda_seq OWNER TO
fashionstore_p2_owner;
--
-- Name: variante_prenda_id_variante_prenda_seq; Type: SEQUENCE OWNED BY; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER SEQUENCE public.variante_prenda_id_variante_prenda_seq OWNED BY
public.variante_prenda.id_variante_prenda;
--
-- Name: venta; Type: TABLE; Schema: public; Owner: fashionstore_p2_owner
--
CREATE TABLE public.venta (
    id_venta integer NOT NULL,
    id_cliente integer,
    id_sucursal integer,
    id_cajero integer,
    id_reserva integer,
    tipo_venta character varying(20) NOT NULL,
    metodo_pago character varying(30) NOT NULL,
```

```
    subtotal numeric(10,2) NOT NULL,
    descuento numeric(10,2) DEFAULT 0.00,
    total numeric(10,2) NOT NULL,
    fecha_venta timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    tenant_id integer DEFAULT public.current_tenant() NOT NULL,
    fecha_entrega timestamp with time zone,
    origen_datos text DEFAULT 'operativo'::text NOT NULL,
    referencia_demo text,
    CONSTRAINT venta_metodo_pago_check CHECK (((metodo_pago)::text = ANY
((ARRAY['Efectivo'::character varying, 'Tarjeta'::character varying, 'QR'::character varying,
'Transferencia'::character varying])::text[]))),
    CONSTRAINT venta_origen_datos_check CHECK ((origen_datos = ANY
(ARRAY['operativo'::text, 'demostracion'::text]))),
    CONSTRAINT venta_tipo_venta_check CHECK (((tipo_venta)::text = ANY
((ARRAY['Presencial'::character varying, 'Online'::character varying])::text[])))
);
ALTER TABLE public.venta OWNER TO fashionstore_p2_owner;
--
-- Name: venta_id_venta_seq; Type: SEQUENCE; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE SEQUENCE public.venta_id_venta_seq
    AS integer
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
ALTER SEQUENCE public.venta_id_venta_seq OWNER TO fashionstore_p2_owner;
--
-- Name: venta_id_venta_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER SEQUENCE public.venta_id_venta_seq OWNED BY public.venta.id_venta;
--
-- Name: ajuste_monetario id_ajuste; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.ajuste_monetario ALTER COLUMN id_ajuste SET DEFAULT
nextval('public.ajuste_monetario_id_ajuste_seq'::regclass);
```

```
--
-- Name: auditoria_identidad id; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.auditoria_identidad ALTER COLUMN id SET DEFAULT
nextval('public.auditoria_identidad_id_seq'::regclass);
--
-- Name: bitacora id_bitacora; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.bitacora ALTER COLUMN id_bitacora SET DEFAULT
nextval('public.bitacora_id_bitacora_seq'::regclass);
--
-- Name: cambio id_cambio; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cambio ALTER COLUMN id_cambio SET DEFAULT
nextval('public.cambio_id_cambio_seq'::regclass);
--
-- Name: categoria id_categoria; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.categoria ALTER COLUMN id_categoria SET DEFAULT
nextval('public.categoria_id_categoria_seq'::regclass);
--
-- Name: ciudad id_ciudad; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.ciudad ALTER COLUMN id_ciudad SET DEFAULT
nextval('public.ciudad_id_ciudad_seq'::regclass);
--
-- Name: cliente id_cliente; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cliente ALTER COLUMN id_cliente SET DEFAULT
nextval('public.cliente_id_cliente_seq'::regclass);
--
-- Name: color id_color; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.color ALTER COLUMN id_color SET DEFAULT
nextval('public.color_id_color_seq'::regclass);
--
-- Name: compra id_compra; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
```

```
--
ALTER TABLE ONLY public.compra ALTER COLUMN id_compra SET DEFAULT
nextval('public.compra_id_compra_seq'::regclass);
--
-- Name: comprobante id_comprobante; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.comprobante ALTER COLUMN id_comprobante SET
DEFAULT nextval('public.comprobante_id_comprobante_seq'::regclass);
--
-- Name: cuenta_global id; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cuenta_global ALTER COLUMN id SET DEFAULT
nextval('public.cuenta_global_id_seq'::regclass);
--
-- Name: empresa id_empresa; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.empresa ALTER COLUMN id_empresa SET DEFAULT
nextval('public.empresa_id_empresa_seq'::regclass);
--
-- Name: imagen_prenda id_imagen; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.imagen_prenda ALTER COLUMN id_imagen SET DEFAULT
nextval('public.imagen_prenda_id_imagen_seq'::regclass);
--
-- Name: movimiento_inventario id_movimiento; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario ALTER COLUMN id_movimiento SET
DEFAULT nextval('public.movimiento_inventario_id_movimiento_seq'::regclass);
--
-- Name: notificacion id_notificacion; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.notificacion ALTER COLUMN id_notificacion SET DEFAULT
nextval('public.notificacion_id_notificacion_seq'::regclass);
--
```

```
-- Name: pago_transaccion id_pago; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.pago_transaccion ALTER COLUMN id_pago SET DEFAULT
nextval('public.pago_transaccion_id_pago_seq'::regclass);
--
-- Name: permiso id_permiso; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.permiso ALTER COLUMN id_permiso SET DEFAULT
nextval('public.permiso_id_permiso_seq'::regclass);
--
-- Name: plan id_plan; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.plan ALTER COLUMN id_plan SET DEFAULT
nextval('public.plan_id_plan_seq'::regclass);
--
-- Name: prenda id_prenda; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.prenda ALTER COLUMN id_prenda SET DEFAULT
nextval('public.prenda_id_prenda_seq'::regclass);
--
-- Name: proveedor id_proveedor; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.proveedor ALTER COLUMN id_proveedor SET DEFAULT
nextval('public.proveedor_id_proveedor_seq'::regclass);
--
-- Name: reserva id_reserva; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.reserva ALTER COLUMN id_reserva SET DEFAULT
nextval('public.reserva_id_reserva_seq'::regclass);
--
-- Name: rol id_rol; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol ALTER COLUMN id_rol SET DEFAULT
nextval('public.rol_id_rol_seq'::regclass);
--
-- Name: sucursal id_sucursal; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
```

```
--
ALTER TABLE ONLY public.sucursal ALTER COLUMN id_sucursal SET DEFAULT
nextval('public.sucursal_id_sucursal_seq'::regclass);
--
-- Name: suscripcion id_suscripcion; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.suscripcion ALTER COLUMN id_suscripcion SET DEFAULT
nextval('public.suscripcion_id_suscripcion_seq'::regclass);
--
-- Name: talla id_talla; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.talla ALTER COLUMN id_talla SET DEFAULT
nextval('public.talla_id_talla_seq'::regclass);
--
-- Name: temporada id_temporada; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.temporada ALTER COLUMN id_temporada SET DEFAULT
nextval('public.temporada_id_temporada_seq'::regclass);
--
-- Name: usuario id_usuario; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario ALTER COLUMN id_usuario SET DEFAULT
nextval('public.usuario_id_usuario_seq'::regclass);
--
-- Name: usuario_token id_token; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_token ALTER COLUMN id_token SET DEFAULT
nextval('public.usuario_token_id_token_seq'::regclass);
--
-- Name: variante_prenda id_variante_prenda; Type: DEFAULT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda ALTER COLUMN id_variante_prenda SET
DEFAULT nextval('public.variante_prenda_id_variante_prenda_seq'::regclass);
--
-- Name: venta id_venta; Type: DEFAULT; Schema: public; Owner: fashionstore_p2_owner
--
```

```
ALTER TABLE ONLY public.venta ALTER COLUMN id_venta SET DEFAULT
nextval('public.venta_id_venta_seq'::regclass);
--
-- Name: ajuste_monetario ajuste_monetario_pkey; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.ajuste_monetario
    ADD CONSTRAINT ajuste_monetario_pkey PRIMARY KEY (id_ajuste);
--
-- Name: ajuste_monetario ajuste_monetario_tenant_id_referencia_key; Type: CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.ajuste_monetario
    ADD CONSTRAINT ajuste_monetario_tenant_id_referencia_key UNIQUE (tenant_id,
referencia);
--
-- Name: auditoria_identidad auditoria_identidad_pkey; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.auditoria_identidad
    ADD CONSTRAINT auditoria_identidad_pkey PRIMARY KEY (id);
--
-- Name: bitacora bitacora_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.bitacora
    ADD CONSTRAINT bitacora_pkey PRIMARY KEY (id_bitacora);
--
-- Name: cambio cambio_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.cambio
    ADD CONSTRAINT cambio_pkey PRIMARY KEY (id_cambio);
--
-- Name: categoria categoria_nombre_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.categoria
    ADD CONSTRAINT categoria_nombre_key UNIQUE (tenant_id, nombre);
--
```

```
-- Name: categoria categoria_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.categoria
    ADD CONSTRAINT categoria_pkey PRIMARY KEY (id_categoria);
--
-- Name: ciudad ciudad_nombre_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.ciudad
    ADD CONSTRAINT ciudad_nombre_key UNIQUE (tenant_id, nombre);
--
-- Name: ciudad ciudad_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.ciudad
    ADD CONSTRAINT ciudad_pkey PRIMARY KEY (id_ciudad);
--
-- Name: cliente cliente_ci_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.cliente
    ADD CONSTRAINT cliente_ci_key UNIQUE (tenant_id, ci);
--
-- Name: cliente cliente_correo_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.cliente
    ADD CONSTRAINT cliente_correo_key UNIQUE (tenant_id, correo);
--
-- Name: cliente cliente_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.cliente
    ADD CONSTRAINT cliente_pkey PRIMARY KEY (id_cliente);
--
-- Name: color color_nombre_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.color
```

```
    ADD CONSTRAINT color_nombre_key UNIQUE (tenant_id, nombre);
--
-- Name: color color_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.color
    ADD CONSTRAINT color_pkey PRIMARY KEY (id_color);
--
-- Name: compra compra_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.compra
    ADD CONSTRAINT compra_pkey PRIMARY KEY (id_compra);
--
-- Name: comprobante comprobante_numero_comprobante_key; Type: CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.comprobante
    ADD CONSTRAINT comprobante_numero_comprobante_key UNIQUE (tenant_id,
numero_comprobante);
--
-- Name: comprobante comprobante_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.comprobante
    ADD CONSTRAINT comprobante_pkey PRIMARY KEY (id_comprobante);
--
-- Name: cuenta_global cuenta_global_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.cuenta_global
    ADD CONSTRAINT cuenta_global_pkey PRIMARY KEY (id);
--
-- Name: cuenta_recuperacion cuenta_recuperacion_pkey; Type: CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cuenta_recuperacion
    ADD CONSTRAINT cuenta_recuperacion_pkey PRIMARY KEY (token_hash);
--
```

```
-- Name: detalle_compra detalle_compra_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_compra
    ADD CONSTRAINT detalle_compra_pkey PRIMARY KEY (id_compra,
id_variante_prenda);
--
-- Name: detalle_reserva detalle_reserva_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_reserva
    ADD CONSTRAINT detalle_reserva_pkey PRIMARY KEY (id_reserva,
id_variante_prenda);
--
-- Name: detalle_venta detalle_venta_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_venta
    ADD CONSTRAINT detalle_venta_pkey PRIMARY KEY (id_venta, id_variante_prenda);
--
-- Name: dispositivo_push dispositivo_push_pkey; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.dispositivo_push
    ADD CONSTRAINT dispositivo_push_pkey PRIMARY KEY (tenant_id, id_usuario, token);
--
-- Name: empresa empresa_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.empresa
    ADD CONSTRAINT empresa_pkey PRIMARY KEY (id_empresa);
--
-- Name: empresa empresa_slug_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.empresa
    ADD CONSTRAINT empresa_slug_key UNIQUE (slug);
--
-- Name: imagen_prenda imagen_prenda_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
```

```
--
ALTER TABLE ONLY public.imagen_prenda
    ADD CONSTRAINT imagen_prenda_pkey PRIMARY KEY (id_imagen);
--
-- Name: inventario inventario_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.inventario
    ADD CONSTRAINT inventario_pkey PRIMARY KEY (id_sucursal, id_variante_prenda);
--
-- Name: movimiento_inventario movimiento_inventario_pkey; Type: CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario
    ADD CONSTRAINT movimiento_inventario_pkey PRIMARY KEY (id_movimiento);
--
-- Name: notificacion notificacion_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.notificacion
    ADD CONSTRAINT notificacion_pkey PRIMARY KEY (id_notificacion);
--
-- Name: operacion_sync operacion_sync_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.operacion_sync
    ADD CONSTRAINT operacion_sync_pkey PRIMARY KEY (tenant_id, operation_id);
--
-- Name: pago_transaccion pago_transaccion_pkey; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.pago_transaccion
    ADD CONSTRAINT pago_transaccion_pkey PRIMARY KEY (id_pago);
--
-- Name: pago_transaccion pago_transaccion_transaccion_id_key; Type: CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.pago_transaccion
    ADD CONSTRAINT pago_transaccion_transaccion_id_key UNIQUE (tenant_id,
transaccion_id);
```

```
--
-- Name: permiso permiso_codigo_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.permiso
    ADD CONSTRAINT permiso_codigo_key UNIQUE (codigo);
--
-- Name: permiso permiso_nombre_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.permiso
    ADD CONSTRAINT permiso_nombre_key UNIQUE (nombre);
--
-- Name: permiso permiso_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.permiso
    ADD CONSTRAINT permiso_pkey PRIMARY KEY (id_permiso);
--
-- Name: plan plan_nombre_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.plan
    ADD CONSTRAINT plan_nombre_key UNIQUE (nombre);
--
-- Name: plan plan_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.plan
    ADD CONSTRAINT plan_pkey PRIMARY KEY (id_plan);
--
-- Name: prenda prenda_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.prenda
    ADD CONSTRAINT prenda_pkey PRIMARY KEY (id_prenda);
--
-- Name: prenda prenda_sku_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
ALTER TABLE ONLY public.prenda
    ADD CONSTRAINT prenda_sku_key UNIQUE (tenant_id, sku);
--
-- Name: proveedor proveedor_nit_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.proveedor
    ADD CONSTRAINT proveedor_nit_key UNIQUE (tenant_id, nit);
--
-- Name: proveedor proveedor_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.proveedor
    ADD CONSTRAINT proveedor_pkey PRIMARY KEY (id_proveedor);
--
-- Name: push_entrega push_entrega_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.push_entrega
    ADD CONSTRAINT push_entrega_pkey PRIMARY KEY (tenant_id, id_notificacion,
token_hash);
--
-- Name: reporte_resultado reporte_resultado_pkey; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.reporte_resultado
    ADD CONSTRAINT reporte_resultado_pkey PRIMARY KEY (id);
--
-- Name: reserva reserva_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.reserva
    ADD CONSTRAINT reserva_pkey PRIMARY KEY (id_reserva);
--
-- Name: retencion retencion_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.retencion
    ADD CONSTRAINT retencion_pkey PRIMARY KEY (tenant_id, id_reserva,
id_variante_prenda);
```

```
--
-- Name: rol rol_nombre_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol
   ADD CONSTRAINT rol_nombre_key UNIQUE (tenant_id, nombre);
--
-- Name: rol_permiso rol_permiso_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol_permiso
   ADD CONSTRAINT rol_permiso_pkey PRIMARY KEY (id_rol, id_permiso);
--
-- Name: rol rol_pkey; Type: CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol
   ADD CONSTRAINT rol_pkey PRIMARY KEY (id_rol);
--
-- Name: schema_migrations schema_migrations_pkey; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.schema_migrations
   ADD CONSTRAINT schema_migrations_pkey PRIMARY KEY (version);
--
-- Name: stripe_event stripe_event_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.stripe_event
   ADD CONSTRAINT stripe_event_pkey PRIMARY KEY (tenant_id, event_id);
--
-- Name: sucursal sucursal_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.sucursal
   ADD CONSTRAINT sucursal_pkey PRIMARY KEY (id_sucursal);
--
-- Name: suscripcion suscripcion_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.suscripcion
```

```
    ADD CONSTRAINT suscripcion_pkey PRIMARY KEY (id_suscripcion);
--
-- Name: suscripcion suscripcion_tenant_id_key; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.suscripcion
    ADD CONSTRAINT suscripcion_tenant_id_key UNIQUE (tenant_id);
--
-- Name: talla talla_nombre_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.talla
    ADD CONSTRAINT talla_nombre_key UNIQUE (tenant_id, nombre);
--
-- Name: talla talla_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.talla
    ADD CONSTRAINT talla_pkey PRIMARY KEY (id_talla);
--
-- Name: temporada temporada_nombre_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.temporada
    ADD CONSTRAINT temporada_nombre_key UNIQUE (tenant_id, nombre);
--
-- Name: temporada temporada_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.temporada
    ADD CONSTRAINT temporada_pkey PRIMARY KEY (id_temporada);
--
-- Name: usuario usuario_correo_key; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario
    ADD CONSTRAINT usuario_correo_key UNIQUE (tenant_id, correo);
--
-- Name: usuario usuario_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
```

```
--
ALTER TABLE ONLY public.usuario
    ADD CONSTRAINT usuario_pkey PRIMARY KEY (id_usuario);
--
-- Name: usuario_sucursal usuario_sucursal_pkey; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_sucursal
    ADD CONSTRAINT usuario_sucursal_pkey PRIMARY KEY (tenant_id, id_usuario,
id_sucursal);
--
-- Name: usuario_token usuario_token_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_token
    ADD CONSTRAINT usuario_token_pkey PRIMARY KEY (id_token);
--
-- Name: variante_prenda variant_color_combination; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT variant_color_combination UNIQUE (tenant_id, id_prenda, id_talla,
color_ids);
--
-- Name: variante_color variante_color_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_color
    ADD CONSTRAINT variante_color_pkey PRIMARY KEY (tenant_id, id_variante_prenda,
id_color);
--
-- Name: variante_prenda variante_prenda_pkey; Type: CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT variante_prenda_pkey PRIMARY KEY (id_variante_prenda);
--
-- Name: variante_prenda variante_prenda_sku_variante_key; Type: CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
```

```
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT variante_prenda_sku_variante_key UNIQUE (tenant_id, sku_variante);
--
-- Name: venta venta_pkey; Type: CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
    ADD CONSTRAINT venta_pkey PRIMARY KEY (id_venta);
--
-- Name: bitacora_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX bitacora_tenant_id_idx ON public.bitacora USING btree (tenant_id);
--
-- Name: categoria_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX categoria_tenant_id_idx ON public.categoria USING btree (tenant_id);
--
-- Name: change_tenant_reference; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX change_tenant_reference ON public.cambio USING btree
(tenant_id, id_cambio);
--
-- Name: ciudad_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX ciudad_tenant_id_idx ON public.ciudad USING btree (tenant_id);
--
-- Name: cliente_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX cliente_tenant_id_idx ON public.cliente USING btree (tenant_id);
--
-- Name: color_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX color_tenant_id_idx ON public.color USING btree (tenant_id);
--
-- Name: compra_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX compra_tenant_id_idx ON public.compra USING btree (tenant_id);
```

```
--
-- Name: comprobante_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX comprobante_tenant_id_idx ON public.comprobante USING btree
(tenant_id);
--
-- Name: cuenta_global_correo; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE UNIQUE INDEX cuenta_global_correo ON public.cuenta_global USING btree
(lower(correo));
--
-- Name: detalle_compra_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX detalle_compra_tenant_id_idx ON public.detalle_compra USING btree
(tenant_id);
--
-- Name: detalle_reserva_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX detalle_reserva_tenant_id_idx ON public.detalle_reserva USING btree
(tenant_id);
--
-- Name: detalle_venta_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX detalle_venta_tenant_id_idx ON public.detalle_venta USING btree
(tenant_id);
--
-- Name: holds_availability; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX holds_availability ON public.retencion USING btree (tenant_id, id_sucursal,
id_variante_prenda, vence);
--
-- Name: idx_usuario_token_id_usuario; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX idx_usuario_token_id_usuario ON public.usuario_token USING btree
(id_usuario);
```

```
--
-- Name: idx_usuario_token_recuperacion; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX idx_usuario_token_recuperacion ON public.usuario_token USING
btree (token_recuperacion);
--
-- Name: idx_usuario_token_usuario_usado; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX idx_usuario_token_usuario_usado ON public.usuario_token USING btree
(id_usuario, usado);
--
-- Name: imagen_prenda_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX imagen_prenda_tenant_id_idx ON public.imagen_prenda USING btree
(tenant_id);
--
-- Name: inventario_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX inventario_tenant_id_idx ON public.inventario USING btree (tenant_id);
--
-- Name: movimiento_inventario_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX movimiento_inventario_tenant_id_idx ON public.movimiento_inventario
USING btree (tenant_id);
--
-- Name: notification_tenant_reference; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX notification_tenant_reference ON public.notificacion USING btree
(tenant_id, id_notificacion);
--
-- Name: one_revision_per_operation; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE UNIQUE INDEX one_revision_per_operation ON public.operacion_sync USING
btree (tenant_id, original_operation) WHERE (original_operation IS NOT NULL);
--
-- Name: one_sale_per_reservation; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX one_sale_per_reservation ON public.venta USING btree (tenant_id,
id_reserva) WHERE (id_reserva IS NOT NULL);
--
-- Name: pago_transaccion_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX pago_transaccion_tenant_id_idx ON public.pago_transaccion USING btree
(tenant_id);
--
-- Name: prenda_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX prenda_tenant_id_idx ON public.prenda USING btree (tenant_id);
--
-- Name: proveedor_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX proveedor_tenant_id_idx ON public.proveedor USING btree (tenant_id);
--
-- Name: reporte_expiration; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX reporte_expiration ON public.reporte_resultado USING btree (expires_at);
--
-- Name: reserva_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX reserva_tenant_id_idx ON public.reserva USING btree (tenant_id);
--
-- Name: rol_permiso_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX rol_permiso_tenant_id_idx ON public.rol_permiso USING btree (tenant_id);
--
-- Name: rol_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX rol_tenant_id_idx ON public.rol USING btree (tenant_id);
```

```
--
-- Name: sale_detail_tenant_reference; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX sale_detail_tenant_reference ON public.detalle_venta USING btree
(tenant_id, id_venta, id_variante_prenda);
--
-- Name: sucursal_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX sucursal_tenant_id_idx ON public.sucursal USING btree (tenant_id);
--
-- Name: sync_history; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX sync_history ON public.operacion_sync USING btree (tenant_id, author_id,
created_at DESC);
--
-- Name: talla_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX talla_tenant_id_idx ON public.talla USING btree (tenant_id);
--
-- Name: temporada_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX temporada_tenant_id_idx ON public.temporada USING btree (tenant_id);
--
-- Name: tenant_ref_categoria_id_categoria; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_categoria_id_categoria ON public.categoria USING
btree (tenant_id, id_categoria);
--
-- Name: tenant_ref_ciudad_id_ciudad; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_ciudad_id_ciudad ON public.ciudad USING btree
(tenant_id, id_ciudad);
--
-- Name: tenant_ref_cliente_id_cliente; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
```

```
--
CREATE UNIQUE INDEX tenant_ref_cliente_id_cliente ON public.cliente USING btree
(tenant_id, id_cliente);
--
-- Name: tenant_ref_color_id_color; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_color_id_color ON public.color USING btree (tenant_id,
id_color);
--
-- Name: tenant_ref_compra_id_compra; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_compra_id_compra ON public.compra USING btree
(tenant_id, id_compra);
--
-- Name: tenant_ref_prenda_id_prenda; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_prenda_id_prenda ON public.prenda USING btree
(tenant_id, id_prenda);
--
-- Name: tenant_ref_proveedor_id_proveedor; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_proveedor_id_proveedor ON public.proveedor USING
btree (tenant_id, id_proveedor);
--
-- Name: tenant_ref_reserva_id_reserva; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_reserva_id_reserva ON public.reserva USING btree
(tenant_id, id_reserva);
--
-- Name: tenant_ref_rol_id_rol; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_rol_id_rol ON public.rol USING btree (tenant_id,
id_rol);
--
```

```
-- Name: tenant_ref_sucursal_id_sucursal; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_sucursal_id_sucursal ON public.sucursal USING btree
(tenant_id, id_sucursal);
--
-- Name: tenant_ref_talla_id_talla; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_talla_id_talla ON public.talla USING btree (tenant_id,
id_talla);
--
-- Name: tenant_ref_temporada_id_temporada; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_temporada_id_temporada ON public.temporada USING
btree (tenant_id, id_temporada);
--
-- Name: tenant_ref_usuario_id_usuario; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_usuario_id_usuario ON public.usuario USING btree
(tenant_id, id_usuario);
--
-- Name: tenant_ref_variante_prenda_id_variante_prenda; Type: INDEX; Schema: public;
Owner: fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_variante_prenda_id_variante_prenda ON
public.variante_prenda USING btree (tenant_id, id_variante_prenda);
--
-- Name: tenant_ref_venta_id_venta; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX tenant_ref_venta_id_venta ON public.venta USING btree
(tenant_id, id_venta);
--
-- Name: usuario_identidad_empresa; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE UNIQUE INDEX usuario_identidad_empresa ON public.usuario USING btree
(tenant_id, account_id);
--
-- Name: usuario_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX usuario_tenant_id_idx ON public.usuario USING btree (tenant_id);
--
-- Name: usuario_token_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX usuario_token_tenant_id_idx ON public.usuario_token USING btree
(tenant_id);
--
-- Name: variante_prenda_tenant_id_idx; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE INDEX variante_prenda_tenant_id_idx ON public.variante_prenda USING btree
(tenant_id);
--
-- Name: venta_demo_reference; Type: INDEX; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE UNIQUE INDEX venta_demo_reference ON public.venta USING btree (tenant_id,
referencia_demo) WHERE (referencia_demo IS NOT NULL);
--
-- Name: venta_tenant_id_idx; Type: INDEX; Schema: public; Owner: fashionstore_p2_owner
--
CREATE INDEX venta_tenant_id_idx ON public.venta USING btree (tenant_id);
--
-- Name: sucursal branch_limit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER branch_limit BEFORE INSERT OR UPDATE ON public.sucursal FOR
EACH ROW EXECUTE FUNCTION public.check_branch_limit();
--
-- Name: variante_prenda canonical_colors; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE TRIGGER canonical_colors BEFORE INSERT OR UPDATE OF color_ids, id_color
ON public.variante_prenda FOR EACH ROW EXECUTE FUNCTION
public.canonical_variant_colors();
--
-- Name: variante_prenda sync_colors; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER sync_colors AFTER INSERT OR UPDATE OF color_ids, id_color ON
public.variante_prenda FOR EACH ROW EXECUTE FUNCTION
public.sync_variant_colors();
--
-- Name: ajuste_monetario transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.ajuste_monetario FOR EACH ROW EXECUTE FUNCTION
public.audit_business_row();
--
-- Name: cambio transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.cambio FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: cliente transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.cliente FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: empresa transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.empresa FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: movimiento_inventario transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.movimiento_inventario FOR EACH ROW EXECUTE FUNCTION
public.audit_business_row();
--
-- Name: pago_transaccion transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.pago_transaccion FOR EACH ROW EXECUTE FUNCTION
public.audit_business_row();
--
-- Name: plan transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.plan FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: prenda transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.prenda FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: rol transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.rol FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: rol_permiso transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.rol_permiso FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: sucursal transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.sucursal FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
```

```
--
-- Name: suscripcion transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.suscripcion FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: usuario transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.usuario FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: venta transactional_audit; Type: TRIGGER; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE TRIGGER transactional_audit AFTER INSERT OR DELETE OR UPDATE ON
public.venta FOR EACH ROW EXECUTE FUNCTION public.audit_business_row();
--
-- Name: movimiento_inventario trg_movimiento_inventario; Type: TRIGGER; Schema: public;
Owner: fashionstore_p2_owner
--
CREATE TRIGGER trg_movimiento_inventario AFTER INSERT ON
public.movimiento_inventario FOR EACH ROW EXECUTE FUNCTION
public.actualizar_stock_por_movimiento();
--
-- Name: ajuste_monetario ajuste_monetario_tenant_id_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.ajuste_monetario
    ADD CONSTRAINT ajuste_monetario_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: ajuste_monetario ajuste_monetario_tenant_id_id_cambio_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.ajuste_monetario
    ADD CONSTRAINT ajuste_monetario_tenant_id_id_cambio_fkey FOREIGN KEY
(tenant_id, id_cambio) REFERENCES public.cambio(tenant_id, id_cambio);
--
```

```
-- Name: auditoria_identidad auditoria_identidad_account_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.auditoria_identidad
    ADD CONSTRAINT auditoria_identidad_account_id_fkey FOREIGN KEY (account_id) REFERENCES public.cuenta_global(id);
--
-- Name: bitacora bitacora_id_usuario_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.bitacora
    ADD CONSTRAINT bitacora_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES public.usuario(id_usuario) ON DELETE SET NULL;
--
-- Name: bitacora bitacora_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.bitacora
    ADD CONSTRAINT bitacora_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: cambio cambio_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cambio
    ADD CONSTRAINT cambio_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: cambio cambio_tenant_id_id_cliente_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cambio
    ADD CONSTRAINT cambio_tenant_id_id_cliente_fkey FOREIGN KEY (tenant_id, id_cliente) REFERENCES public.cliente(tenant_id, id_cliente);
--
-- Name: cambio cambio_tenant_id_id_reserva_reemplazo_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cambio
```

```
    ADD CONSTRAINT cambio_tenant_id_id_reserva_reemplazo_fkey FOREIGN KEY
(tenant_id, id_reserva_reemplazo) REFERENCES public.reserva(tenant_id, id_reserva);
--
-- Name: cambio cambio_tenant_id_id_sucursal_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cambio
    ADD CONSTRAINT cambio_tenant_id_id_sucursal_fkey FOREIGN KEY (tenant_id,
id_sucursal) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: cambio cambio_tenant_id_id_venta_id_variante_original_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cambio
    ADD CONSTRAINT cambio_tenant_id_id_venta_id_variante_original_fkey FOREIGN KEY
(tenant_id, id_venta, id_variante_original) REFERENCES public.detalle_venta(tenant_id,
id_venta, id_variante_prenda);
--
-- Name: categoria categoria_id_categoria_padre_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.categoria
    ADD CONSTRAINT categoria_id_categoria_padre_fkey FOREIGN KEY
(id_categoria_padre) REFERENCES public.categoria(id_categoria) ON DELETE SET NULL;
--
-- Name: categoria categoria_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.categoria
    ADD CONSTRAINT categoria_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: ciudad ciudad_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.ciudad
    ADD CONSTRAINT ciudad_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
```

```
-- Name: cliente cliente_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cliente
    ADD CONSTRAINT cliente_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: color color_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.color
    ADD CONSTRAINT color_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: compra compra_id_proveedor_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.compra
    ADD CONSTRAINT compra_id_proveedor_fkey FOREIGN KEY (id_proveedor) REFERENCES public.proveedor(id_proveedor) ON DELETE RESTRICT;
--
-- Name: compra compra_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.compra
    ADD CONSTRAINT compra_id_sucursal_fkey FOREIGN KEY (id_sucursal) REFERENCES public.sucursal(id_sucursal) ON DELETE RESTRICT;
--
-- Name: compra compra_id_usuario_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.compra
    ADD CONSTRAINT compra_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES public.usuario(id_usuario) ON DELETE RESTRICT;
--
-- Name: compra compra_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.compra
```

```
    ADD CONSTRAINT compra_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: comprobante comprobante_id_venta_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.comprobante
    ADD CONSTRAINT comprobante_id_venta_fkey FOREIGN KEY (id_venta)
REFERENCES public.venta(id_venta) ON DELETE CASCADE;
--
-- Name: comprobante comprobante_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.comprobante
    ADD CONSTRAINT comprobante_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: cuenta_recuperacion cuenta_recuperacion_account_id_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.cuenta_recuperacion
    ADD CONSTRAINT cuenta_recuperacion_account_id_fkey FOREIGN KEY (account_id)
REFERENCES public.cuenta_global(id);
--
-- Name: detalle_compra detalle_compra_id_compra_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_compra
    ADD CONSTRAINT detalle_compra_id_compra_fkey FOREIGN KEY (id_compra)
REFERENCES public.compra(id_compra) ON DELETE CASCADE;
--
-- Name: detalle_compra detalle_compra_id_variante_prenda_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_compra
    ADD CONSTRAINT detalle_compra_id_variante_prenda_fkey FOREIGN KEY
(id_variante_prenda) REFERENCES public.variante_prenda(id_variante_prenda) ON DELETE
RESTRICT;
--
```

```
-- Name: detalle_compra detalle_compra_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_compra
    ADD CONSTRAINT detalle_compra_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: detalle_reserva detalle_reserva_id_reserva_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_reserva
    ADD CONSTRAINT detalle_reserva_id_reserva_fkey FOREIGN KEY (id_reserva) REFERENCES public.reserva(id_reserva) ON DELETE CASCADE;
--
-- Name: detalle_reserva detalle_reserva_id_variante_prenda_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_reserva
    ADD CONSTRAINT detalle_reserva_id_variante_prenda_fkey FOREIGN KEY (id_variante_prenda) REFERENCES public.variante_prenda(id_variante_prenda) ON DELETE RESTRICT;
--
-- Name: detalle_reserva detalle_reserva_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_reserva
    ADD CONSTRAINT detalle_reserva_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: detalle_venta detalle_venta_id_variante_prenda_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_venta
    ADD CONSTRAINT detalle_venta_id_variante_prenda_fkey FOREIGN KEY (id_variante_prenda) REFERENCES public.variante_prenda(id_variante_prenda) ON DELETE RESTRICT;
--
-- Name: detalle_venta detalle_venta_id_venta_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
```

```
ALTER TABLE ONLY public.detalle_venta
    ADD CONSTRAINT detalle_venta_id_venta_fkey FOREIGN KEY (id_venta)
REFERENCES public.venta(id_venta) ON DELETE CASCADE;
--
-- Name: detalle_venta detalle_venta_tenant_id_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_venta
    ADD CONSTRAINT detalle_venta_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: dispositivo_push dispositivo_push_tenant_id_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.dispositivo_push
    ADD CONSTRAINT dispositivo_push_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: dispositivo_push dispositivo_push_tenant_id_id_usuario_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.dispositivo_push
    ADD CONSTRAINT dispositivo_push_tenant_id_id_usuario_fkey FOREIGN KEY
(tenant_id, id_usuario) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: imagen_prenda imagen_prenda_id_prenda_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.imagen_prenda
    ADD CONSTRAINT imagen_prenda_id_prenda_fkey FOREIGN KEY (id_prenda)
REFERENCES public.prenda(id_prenda) ON DELETE CASCADE;
--
-- Name: imagen_prenda imagen_prenda_tenant_id_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.imagen_prenda
    ADD CONSTRAINT imagen_prenda_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
```

```
-- Name: inventario inventario_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.inventario
    ADD CONSTRAINT inventario_id_sucursal_fkey FOREIGN KEY (id_sucursal)
REFERENCES public.sucursal(id_sucursal) ON DELETE RESTRICT;
--
-- Name: inventario inventario_id_variante_prenda_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.inventario
    ADD CONSTRAINT inventario_id_variante_prenda_fkey FOREIGN KEY
(id_variante_prenda) REFERENCES public.variante_prenda(id_variante_prenda) ON DELETE
RESTRICT;
--
-- Name: inventario inventario_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.inventario
    ADD CONSTRAINT inventario_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: movimiento_inventario movimiento_inventario_id_sucursal_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario
    ADD CONSTRAINT movimiento_inventario_id_sucursal_fkey FOREIGN KEY
(id_sucursal) REFERENCES public.sucursal(id_sucursal) ON DELETE RESTRICT;
--
-- Name: movimiento_inventario movimiento_inventario_id_usuario_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario
    ADD CONSTRAINT movimiento_inventario_id_usuario_fkey FOREIGN KEY (id_usuario)
REFERENCES public.usuario(id_usuario) ON DELETE RESTRICT;
--
-- Name: movimiento_inventario movimiento_inventario_id_variante_prenda_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario
```

```
    ADD CONSTRAINT movimiento_inventario_id_variante_prenda_fkey FOREIGN KEY
(id_variante_prenda) REFERENCES public.variante_prenda(id_variante_prenda) ON DELETE
RESTRICT;
--
-- Name: movimiento_inventario movimiento_inventario_tenant_id_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario
    ADD CONSTRAINT movimiento_inventario_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: notificacion notificacion_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.notificacion
    ADD CONSTRAINT notificacion_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: notificacion notificacion_tenant_id_id_cambio_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.notificacion
    ADD CONSTRAINT notificacion_tenant_id_id_cambio_fkey FOREIGN KEY (tenant_id,
id_cambio) REFERENCES public.cambio(tenant_id, id_cambio);
--
-- Name: notificacion notificacion_tenant_id_id_cliente_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.notificacion
    ADD CONSTRAINT notificacion_tenant_id_id_cliente_fkey FOREIGN KEY (tenant_id,
id_cliente) REFERENCES public.cliente(tenant_id, id_cliente);
--
-- Name: operacion_sync operacion_sync_tenant_id_author_id_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.operacion_sync
    ADD CONSTRAINT operacion_sync_tenant_id_author_id_fkey FOREIGN KEY (tenant_id,
author_id) REFERENCES public.usuario(tenant_id, id_usuario);
--
```

```
-- Name: operacion_sync operacion_sync_tenant_id_branch_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.operacion_sync
    ADD CONSTRAINT operacion_sync_tenant_id_branch_id_fkey FOREIGN KEY (tenant_id, branch_id) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: operacion_sync operacion_sync_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.operacion_sync
    ADD CONSTRAINT operacion_sync_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: operacion_sync operacion_sync_tenant_id_original_operation_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.operacion_sync
    ADD CONSTRAINT operacion_sync_tenant_id_original_operation_fkey FOREIGN KEY (tenant_id, original_operation) REFERENCES public.operacion_sync(tenant_id, operation_id);
--
-- Name: operacion_sync operacion_sync_tenant_id_resolved_by_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.operacion_sync
    ADD CONSTRAINT operacion_sync_tenant_id_resolved_by_fkey FOREIGN KEY (tenant_id, resolved_by) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: pago_transaccion pago_transaccion_id_venta_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.pago_transaccion
    ADD CONSTRAINT pago_transaccion_id_venta_fkey FOREIGN KEY (id_venta) REFERENCES public.venta(id_venta) ON DELETE CASCADE;
--
-- Name: pago_transaccion pago_transaccion_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.pago_transaccion
```

```
    ADD CONSTRAINT pago_transaccion_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: pago_transaccion payment_reservation_tenant; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.pago_transaccion
    ADD CONSTRAINT payment_reservation_tenant FOREIGN KEY (tenant_id, id_reserva)
REFERENCES public.reserva(tenant_id, id_reserva);
--
-- Name: prenda prenda_id_categoria_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.prenda
    ADD CONSTRAINT prenda_id_categoria_fkey FOREIGN KEY (id_categoria)
REFERENCES public.categoria(id_categoria) ON DELETE RESTRICT;
--
-- Name: prenda prenda_id_temporada_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.prenda
    ADD CONSTRAINT prenda_id_temporada_fkey FOREIGN KEY (id_temporada)
REFERENCES public.temporada(id_temporada) ON DELETE SET NULL;
--
-- Name: prenda prenda_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.prenda
    ADD CONSTRAINT prenda_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: proveedor proveedor_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.proveedor
    ADD CONSTRAINT proveedor_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: push_entrega push_entrega_id_notificacion_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
```

```
--
ALTER TABLE ONLY public.push_entrega
    ADD CONSTRAINT push_entrega_id_notificacion_fkey FOREIGN KEY (id_notificacion)
REFERENCES public.notificacion(id_notificacion);
--
-- Name: push_entrega push_entrega_tenant_id_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.push_entrega
    ADD CONSTRAINT push_entrega_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: push_entrega push_notification_tenant; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.push_entrega
    ADD CONSTRAINT push_notification_tenant FOREIGN KEY (tenant_id, id_notificacion)
REFERENCES public.notificacion(tenant_id, id_notificacion);
--
-- Name: reporte_resultado reporte_resultado_tenant_id_author_id_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.reporte_resultado
    ADD CONSTRAINT reporte_resultado_tenant_id_author_id_fkey FOREIGN KEY
(tenant_id, author_id) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: reporte_resultado reporte_resultado_tenant_id_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.reporte_resultado
    ADD CONSTRAINT reporte_resultado_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: reserva reserva_id_cliente_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.reserva
    ADD CONSTRAINT reserva_id_cliente_fkey FOREIGN KEY (id_cliente) REFERENCES
public.cliente(id_cliente) ON DELETE RESTRICT;
--
```

```
-- Name: reserva reserva_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.reserva
    ADD CONSTRAINT reserva_id_sucursal_fkey FOREIGN KEY (id_sucursal) REFERENCES public.sucursal(id_sucursal) ON DELETE RESTRICT;
--
-- Name: reserva reserva_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.reserva
    ADD CONSTRAINT reserva_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: retencion retencion_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.retencion
    ADD CONSTRAINT retencion_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: retencion retencion_tenant_id_id_reserva_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.retencion
    ADD CONSTRAINT retencion_tenant_id_id_reserva_fkey FOREIGN KEY (tenant_id, id_reserva) REFERENCES public.reserva(tenant_id, id_reserva);
--
-- Name: retencion retencion_tenant_id_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.retencion
    ADD CONSTRAINT retencion_tenant_id_id_sucursal_fkey FOREIGN KEY (tenant_id, id_sucursal) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: retencion retencion_tenant_id_id_variante_prenda_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.retencion
```

```
    ADD CONSTRAINT retencion_tenant_id_id_variante_prenda_fkey FOREIGN KEY
(tenant_id, id_variante_prenda) REFERENCES public.variante_prenda(tenant_id,
id_variante_prenda);
--
-- Name: rol_permiso rol_permiso_id_permiso_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol_permiso
    ADD CONSTRAINT rol_permiso_id_permiso_fkey FOREIGN KEY (id_permiso)
REFERENCES public.permiso(id_permiso) ON DELETE CASCADE;
--
-- Name: rol_permiso rol_permiso_id_rol_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol_permiso
    ADD CONSTRAINT rol_permiso_id_rol_fkey FOREIGN KEY (id_rol) REFERENCES
public.rol(id_rol) ON DELETE CASCADE;
--
-- Name: rol_permiso rol_permiso_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol_permiso
    ADD CONSTRAINT rol_permiso_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: rol rol_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol
    ADD CONSTRAINT rol_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: stripe_event stripe_event_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.stripe_event
    ADD CONSTRAINT stripe_event_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
```

```
-- Name: sucursal sucursal_id_ciudad_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.sucursal
    ADD CONSTRAINT sucursal_id_ciudad_fkey FOREIGN KEY (id_ciudad) REFERENCES public.ciudad(id_ciudad) ON DELETE RESTRICT;
--
-- Name: sucursal sucursal_id_encargado_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.sucursal
    ADD CONSTRAINT sucursal_id_encargado_fkey FOREIGN KEY (id_encargado) REFERENCES public.usuario(id_usuario) ON DELETE SET NULL;
--
-- Name: sucursal sucursal_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.sucursal
    ADD CONSTRAINT sucursal_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: suscripcion suscripcion_id_plan_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.suscripcion
    ADD CONSTRAINT suscripcion_id_plan_fkey FOREIGN KEY (id_plan) REFERENCES public.plan(id_plan);
--
-- Name: suscripcion suscripcion_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.suscripcion
    ADD CONSTRAINT suscripcion_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES public.empresa(id_empresa);
--
-- Name: talla talla_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.talla
```

```
    ADD CONSTRAINT talla_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: temporada temporada_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.temporada
    ADD CONSTRAINT temporada_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: bitacora tenant_bitacora_id_usuario_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.bitacora
    ADD CONSTRAINT tenant_bitacora_id_usuario_fkey FOREIGN KEY (tenant_id,
id_usuario) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: categoria tenant_categoria_id_categoria_padre_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.categoria
    ADD CONSTRAINT tenant_categoria_id_categoria_padre_fkey FOREIGN KEY (tenant_id,
id_categoria_padre) REFERENCES public.categoria(tenant_id, id_categoria);
--
-- Name: compra tenant_compra_id_proveedor_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.compra
    ADD CONSTRAINT tenant_compra_id_proveedor_fkey FOREIGN KEY (tenant_id,
id_proveedor) REFERENCES public.proveedor(tenant_id, id_proveedor);
--
-- Name: compra tenant_compra_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.compra
    ADD CONSTRAINT tenant_compra_id_sucursal_fkey FOREIGN KEY (tenant_id,
id_sucursal) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: compra tenant_compra_id_usuario_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
```

```
--
ALTER TABLE ONLY public.compra
    ADD CONSTRAINT tenant_compra_id_usuario_fkey FOREIGN KEY (tenant_id,
id_usuario) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: comprobante tenant_comprobante_id_venta_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.comprobante
    ADD CONSTRAINT tenant_comprobante_id_venta_fkey FOREIGN KEY (tenant_id,
id_venta) REFERENCES public.venta(tenant_id, id_venta);
--
-- Name: detalle_compra tenant_detalle_compra_id_compra_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_compra
    ADD CONSTRAINT tenant_detalle_compra_id_compra_fkey FOREIGN KEY (tenant_id,
id_compra) REFERENCES public.compra(tenant_id, id_compra);
--
-- Name: detalle_compra tenant_detalle_compra_id_variante_prenda_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_compra
    ADD CONSTRAINT tenant_detalle_compra_id_variante_prenda_fkey FOREIGN KEY
(tenant_id, id_variante_prenda) REFERENCES public.variante_prenda(tenant_id,
id_variante_prenda);
--
-- Name: detalle_reserva tenant_detalle_reserva_id_reserva_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_reserva
    ADD CONSTRAINT tenant_detalle_reserva_id_reserva_fkey FOREIGN KEY (tenant_id,
id_reserva) REFERENCES public.reserva(tenant_id, id_reserva);
--
-- Name: detalle_reserva tenant_detalle_reserva_id_variante_prenda_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_reserva
```

```
    ADD CONSTRAINT tenant_detalle_reserva_id_variante_prenda_fkey FOREIGN KEY
(tenant_id, id_variante_prenda) REFERENCES public.variante_prenda(tenant_id,
id_variante_prenda);
--
-- Name: detalle_venta tenant_detalle_venta_id_variante_prenda_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_venta
    ADD CONSTRAINT tenant_detalle_venta_id_variante_prenda_fkey FOREIGN KEY
(tenant_id, id_variante_prenda) REFERENCES public.variante_prenda(tenant_id,
id_variante_prenda);
--
-- Name: detalle_venta tenant_detalle_venta_id_venta_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.detalle_venta
    ADD CONSTRAINT tenant_detalle_venta_id_venta_fkey FOREIGN KEY (tenant_id,
id_venta) REFERENCES public.venta(tenant_id, id_venta);
--
-- Name: imagen_prenda tenant_imagen_prenda_id_prenda_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.imagen_prenda
    ADD CONSTRAINT tenant_imagen_prenda_id_prenda_fkey FOREIGN KEY (tenant_id,
id_prenda) REFERENCES public.prenda(tenant_id, id_prenda);
--
-- Name: inventario tenant_inventario_id_sucursal_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.inventario
    ADD CONSTRAINT tenant_inventario_id_sucursal_fkey FOREIGN KEY (tenant_id,
id_sucursal) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: inventario tenant_inventario_id_variante_prenda_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.inventario
    ADD CONSTRAINT tenant_inventario_id_variante_prenda_fkey FOREIGN KEY (tenant_id,
id_variante_prenda) REFERENCES public.variante_prenda(tenant_id, id_variante_prenda);
--
```

```
-- Name: movimiento_inventario tenant_movimiento_inventario_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario
    ADD CONSTRAINT tenant_movimiento_inventario_id_sucursal_fkey FOREIGN KEY (tenant_id, id_sucursal) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: movimiento_inventario tenant_movimiento_inventario_id_usuario_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario
    ADD CONSTRAINT tenant_movimiento_inventario_id_usuario_fkey FOREIGN KEY (tenant_id, id_usuario) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: movimiento_inventario tenant_movimiento_inventario_id_variante_prenda_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.movimiento_inventario
    ADD CONSTRAINT tenant_movimiento_inventario_id_variante_prenda_fkey FOREIGN KEY (tenant_id, id_variante_prenda) REFERENCES public.variante_prenda(tenant_id, id_variante_prenda);
--
-- Name: pago_transaccion tenant_pago_transaccion_id_venta_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.pago_transaccion
    ADD CONSTRAINT tenant_pago_transaccion_id_venta_fkey FOREIGN KEY (tenant_id, id_venta) REFERENCES public.venta(tenant_id, id_venta);
--
-- Name: prenda tenant_prenda_id_categoria_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.prenda
    ADD CONSTRAINT tenant_prenda_id_categoria_fkey FOREIGN KEY (tenant_id, id_categoria) REFERENCES public.categoria(tenant_id, id_categoria);
--
-- Name: prenda tenant_prenda_id_temporada_fkey; Type: FK CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.prenda
```

```
    ADD CONSTRAINT tenant_prenda_id_temporada_fkey FOREIGN KEY (tenant_id,
id_temporada) REFERENCES public.temporada(tenant_id, id_temporada);
--
-- Name: reserva tenant_reserva_id_cliente_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.reserva
    ADD CONSTRAINT tenant_reserva_id_cliente_fkey FOREIGN KEY (tenant_id, id_cliente)
REFERENCES public.cliente(tenant_id, id_cliente);
--
-- Name: reserva tenant_reserva_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.reserva
    ADD CONSTRAINT tenant_reserva_id_sucursal_fkey FOREIGN KEY (tenant_id,
id_sucursal) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: rol_permiso tenant_rol_permiso_id_rol_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.rol_permiso
    ADD CONSTRAINT tenant_rol_permiso_id_rol_fkey FOREIGN KEY (tenant_id, id_rol)
REFERENCES public.rol(tenant_id, id_rol);
--
-- Name: sucursal tenant_sucursal_id_ciudad_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.sucursal
    ADD CONSTRAINT tenant_sucursal_id_ciudad_fkey FOREIGN KEY (tenant_id, id_ciudad)
REFERENCES public.ciudad(tenant_id, id_ciudad);
--
-- Name: sucursal tenant_sucursal_id_encargado_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.sucursal
    ADD CONSTRAINT tenant_sucursal_id_encargado_fkey FOREIGN KEY (tenant_id,
id_encargado) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: usuario tenant_usuario_id_role_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
```

```
--
ALTER TABLE ONLY public.usuario
    ADD CONSTRAINT tenant_usuario_id_role_fkey FOREIGN KEY (tenant_id, id_role)
REFERENCES public.rol(tenant_id, id_rol);
--
-- Name: usuario_token tenant_usuario_token_id_usuario_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_token
    ADD CONSTRAINT tenant_usuario_token_id_usuario_fkey FOREIGN KEY (tenant_id,
id_usuario) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: variante_prenda tenant_variante_prenda_id_color_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT tenant_variante_prenda_id_color_fkey FOREIGN KEY (tenant_id,
id_color) REFERENCES public.color(tenant_id, id_color);
--
-- Name: variante_prenda tenant_variante_prenda_id_prenda_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT tenant_variante_prenda_id_prenda_fkey FOREIGN KEY (tenant_id,
id_prenda) REFERENCES public.prenda(tenant_id, id_prenda);
--
-- Name: variante_prenda tenant_variante_prenda_id_talla_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT tenant_variante_prenda_id_talla_fkey FOREIGN KEY (tenant_id,
id_talla) REFERENCES public.talla(tenant_id, id_talla);
--
-- Name: venta tenant_venta_id_cajero_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
    ADD CONSTRAINT tenant_venta_id_cajero_fkey FOREIGN KEY (tenant_id, id_cajero)
REFERENCES public.usuario(tenant_id, id_usuario);
--
```

```
-- Name: venta tenant_venta_id_cliente_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
   ADD CONSTRAINT tenant_venta_id_cliente_fkey FOREIGN KEY (tenant_id, id_cliente)
REFERENCES public.cliente(tenant_id, id_cliente);
--
-- Name: venta tenant_venta_id_reserva_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
   ADD CONSTRAINT tenant_venta_id_reserva_fkey FOREIGN KEY (tenant_id, id_reserva)
REFERENCES public.reserva(tenant_id, id_reserva);
--
-- Name: venta tenant_venta_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public;
Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
   ADD CONSTRAINT tenant_venta_id_sucursal_fkey FOREIGN KEY (tenant_id,
id_sucursal) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: usuario usuario_account_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario
   ADD CONSTRAINT usuario_account_id_fkey FOREIGN KEY (account_id) REFERENCES
public.cuenta_global(id);
--
-- Name: usuario usuario_id_role_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario
   ADD CONSTRAINT usuario_id_role_fkey FOREIGN KEY (id_role) REFERENCES
public.rol(id_rol) ON DELETE RESTRICT;
--
-- Name: usuario_sucursal usuario_sucursal_tenant_id_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_sucursal
```

```
    ADD CONSTRAINT usuario_sucursal_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: usuario_sucursal usuario_sucursal_tenant_id_id_sucursal_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_sucursal
    ADD CONSTRAINT usuario_sucursal_tenant_id_id_sucursal_fkey FOREIGN KEY
(tenant_id, id_sucursal) REFERENCES public.sucursal(tenant_id, id_sucursal);
--
-- Name: usuario_sucursal usuario_sucursal_tenant_id_id_usuario_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_sucursal
    ADD CONSTRAINT usuario_sucursal_tenant_id_id_usuario_fkey FOREIGN KEY
(tenant_id, id_usuario) REFERENCES public.usuario(tenant_id, id_usuario);
--
-- Name: usuario usuario_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario
    ADD CONSTRAINT usuario_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: usuario_token usuario_token_id_usuario_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_token
    ADD CONSTRAINT usuario_token_id_usuario_fkey FOREIGN KEY (id_usuario)
REFERENCES public.usuario(id_usuario) ON DELETE CASCADE;
--
-- Name: usuario_token usuario_token_tenant_id_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.usuario_token
    ADD CONSTRAINT usuario_token_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: variante_color variante_color_tenant_id_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
```

```
--
ALTER TABLE ONLY public.variante_color
    ADD CONSTRAINT variante_color_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: variante_color variante_color_tenant_id_id_color_fkey; Type: FK CONSTRAINT;
Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_color
    ADD CONSTRAINT variante_color_tenant_id_id_color_fkey FOREIGN KEY (tenant_id,
id_color) REFERENCES public.color(tenant_id, id_color);
--
-- Name: variante_color variante_color_tenant_id_id_variante_prenda_fkey; Type: FK
CONSTRAINT; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_color
    ADD CONSTRAINT variante_color_tenant_id_id_variante_prenda_fkey FOREIGN KEY
(tenant_id, id_variante_prenda) REFERENCES public.variante_prenda(tenant_id,
id_variante_prenda);
--
-- Name: variante_prenda variante_prenda_id_color_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT variante_prenda_id_color_fkey FOREIGN KEY (id_color)
REFERENCES public.color(id_color) ON DELETE RESTRICT;
--
-- Name: variante_prenda variante_prenda_id_prenda_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT variante_prenda_id_prenda_fkey FOREIGN KEY (id_prenda)
REFERENCES public.prenda(id_prenda) ON DELETE CASCADE;
--
-- Name: variante_prenda variante_prenda_id_talla_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
    ADD CONSTRAINT variante_prenda_id_talla_fkey FOREIGN KEY (id_talla)
REFERENCES public.talla(id_talla) ON DELETE RESTRICT;
```

```
--
-- Name: variante_prenda variante_prenda_tenant_id_fkey; Type: FK CONSTRAINT; Schema:
public; Owner: fashionstore_p2_owner
--
ALTER TABLE ONLY public.variante_prenda
   ADD CONSTRAINT variante_prenda_tenant_id_fkey FOREIGN KEY (tenant_id)
REFERENCES public.empresa(id_empresa);
--
-- Name: venta venta_id_cajero_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
   ADD CONSTRAINT venta_id_cajero_fkey FOREIGN KEY (id_cajero) REFERENCES
public.usuario(id_usuario) ON DELETE RESTRICT;
--
-- Name: venta venta_id_cliente_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
   ADD CONSTRAINT venta_id_cliente_fkey FOREIGN KEY (id_cliente) REFERENCES
public.cliente(id_cliente) ON DELETE RESTRICT;
--
-- Name: venta venta_id_reserva_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
   ADD CONSTRAINT venta_id_reserva_fkey FOREIGN KEY (id_reserva) REFERENCES
public.reserva(id_reserva) ON DELETE SET NULL;
--
-- Name: venta venta_id_sucursal_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
   ADD CONSTRAINT venta_id_sucursal_fkey FOREIGN KEY (id_sucursal) REFERENCES
public.sucursal(id_sucursal) ON DELETE RESTRICT;
--
-- Name: venta venta_tenant_id_fkey; Type: FK CONSTRAINT; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE ONLY public.venta
```

```
    ADD CONSTRAINT venta_tenant_id_fkey FOREIGN KEY (tenant_id) REFERENCES
public.empresa(id_empresa);
--
-- Name: ajuste_monetario; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.ajuste_monetario ENABLE ROW LEVEL SECURITY;
--
-- Name: auditoria_identidad; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.auditoria_identidad ENABLE ROW LEVEL SECURITY;
--
-- Name: bitacora; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.bitacora ENABLE ROW LEVEL SECURITY;
--
-- Name: cambio branch_scope; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY branch_scope ON public.cambio AS RESTRICTIVE USING
(public.branch_visible(id_sucursal)) WITH CHECK (public.branch_visible(id_sucursal));
--
-- Name: compra branch_scope; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY branch_scope ON public.compra AS RESTRICTIVE USING
(public.branch_visible(id_sucursal)) WITH CHECK (public.branch_visible(id_sucursal));
--
-- Name: inventario branch_scope; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY branch_scope ON public.inventario AS RESTRICTIVE USING
(public.branch_visible(id_sucursal)) WITH CHECK (public.branch_visible(id_sucursal));
--
-- Name: movimiento_inventario branch_scope; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE POLICY branch_scope ON public.movimiento_inventario AS RESTRICTIVE
USING (public.branch_visible(id_sucursal)) WITH CHECK
(public.branch_visible(id_sucursal));
--
-- Name: reserva branch_scope; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY branch_scope ON public.reserva AS RESTRICTIVE USING
(public.branch_visible(id_sucursal)) WITH CHECK (public.branch_visible(id_sucursal));
--
-- Name: retencion branch_scope; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY branch_scope ON public.retencion AS RESTRICTIVE USING
(public.branch_visible(id_sucursal)) WITH CHECK (public.branch_visible(id_sucursal));
--
-- Name: venta branch_scope; Type: POLICY; Schema: public; Owner: fashionstore_p2_owner
--
CREATE POLICY branch_scope ON public.venta AS RESTRICTIVE USING
(public.branch_visible(id_sucursal)) WITH CHECK (public.branch_visible(id_sucursal));
--
-- Name: cambio; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.cambio ENABLE ROW LEVEL SECURITY;
--
-- Name: categoria; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.categoria ENABLE ROW LEVEL SECURITY;
--
-- Name: ciudad; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.ciudad ENABLE ROW LEVEL SECURITY;
--
-- Name: cliente; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.cliente ENABLE ROW LEVEL SECURITY;
--
-- Name: color; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.color ENABLE ROW LEVEL SECURITY;
```

```
--
-- Name: compra; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.compra ENABLE ROW LEVEL SECURITY;
--
-- Name: comprobante; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.comprobante ENABLE ROW LEVEL SECURITY;
--
-- Name: cuenta_global; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.cuenta_global ENABLE ROW LEVEL SECURITY;
--
-- Name: cuenta_recuperacion; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.cuenta_recuperacion ENABLE ROW LEVEL SECURITY;
--
-- Name: pago_transaccion customer_payments; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY customer_payments ON public.pago_transaccion AS RESTRICTIVE
USING (((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (EXISTS ( SELECT 1
   FROM public.reserva r
  WHERE (r.id_reserva = pago_transaccion.id_reserva))) OR (EXISTS ( SELECT 1
   FROM public.venta v
  WHERE (v.id_venta = pago_transaccion.id_venta)))));
--
-- Name: comprobante customer_receipts; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY customer_receipts ON public.comprobante AS RESTRICTIVE USING
(((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (EXISTS ( SELECT 1
   FROM public.venta v
  WHERE (v.id_venta = comprobante.id_venta)))));
--
-- Name: detalle_reserva customer_reservation_lines; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
```

```
--
CREATE POLICY customer_reservation_lines ON public.detalle_reserva AS RESTRICTIVE
USING (((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (EXISTS ( SELECT 1
   FROM public.reserva r
  WHERE (r.id_reserva = detalle_reserva.id_reserva)))));
--
-- Name: reserva customer_reservations; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY customer_reservations ON public.reserva AS RESTRICTIVE USING
(((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (id_cliente =
public.own_customer()))) WITH CHECK (((current_setting('app.role'::text, true) <>
'Cliente'::text) OR (id_cliente = public.own_customer())));
--
-- Name: detalle_venta customer_sale_lines; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY customer_sale_lines ON public.detalle_venta AS RESTRICTIVE USING
(((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (EXISTS ( SELECT 1
   FROM public.venta v
  WHERE (v.id_venta = detalle_venta.id_venta)))));
--
-- Name: venta customer_sales; Type: POLICY; Schema: public; Owner: fashionstore_p2_owner
--
CREATE POLICY customer_sales ON public.venta AS RESTRICTIVE USING
(((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (id_cliente =
public.own_customer()))) WITH CHECK (((current_setting('app.role'::text, true) <>
'Cliente'::text) OR (id_cliente = public.own_customer())));
--
-- Name: detalle_compra; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.detalle_compra ENABLE ROW LEVEL SECURITY;
--
-- Name: detalle_reserva; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.detalle_reserva ENABLE ROW LEVEL SECURITY;
--
```

```
-- Name: detalle_venta; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.detalle_venta ENABLE ROW LEVEL SECURITY;
--
-- Name: dispositivo_push; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.dispositivo_push ENABLE ROW LEVEL SECURITY;
--
-- Name: imagen_prenda; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.imagen_prenda ENABLE ROW LEVEL SECURITY;
--
-- Name: inventario; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.inventario ENABLE ROW LEVEL SECURITY;
--
-- Name: movimiento_inventario; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.movimiento_inventario ENABLE ROW LEVEL SECURITY;
--
-- Name: notificacion; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.notificacion ENABLE ROW LEVEL SECURITY;
--
-- Name: operacion_sync; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.operacion_sync ENABLE ROW LEVEL SECURITY;
--
-- Name: cambio own_changes; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY own_changes ON public.cambio AS RESTRICTIVE USING
(((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (id_cliente =
public.own_customer())));
```

```
--
-- Name: notificacion own_notifications; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY own_notifications ON public.notificacion AS RESTRICTIVE USING
(((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (id_cliente =
public.own_customer())));
--
-- Name: operacion_sync own_operations; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY own_operations ON public.operacion_sync AS RESTRICTIVE USING
(((current_setting('app.role'::text, true) <> 'Cliente'::text) OR (author_id =
(NULLIF(current_setting('app.user_id'::text, true), ''::text))::integer)));
--
-- Name: pago_transaccion; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.pago_transaccion ENABLE ROW LEVEL SECURITY;
--
-- Name: pago_transaccion payment_branch; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY payment_branch ON public.pago_transaccion AS RESTRICTIVE USING
((((id_reserva IS NOT NULL) AND (EXISTS ( SELECT 1
   FROM public.reserva r
  WHERE (r.id_reserva = pago_transaccion.id_reserva)))) OR ((id_venta IS NOT NULL) AND
(EXISTS ( SELECT 1
   FROM public.venta v
  WHERE (v.id_venta = pago_transaccion.id_venta)))) OR ((id_reserva IS NULL) AND
(id_venta IS NULL) AND (current_setting('app.role'::text, true) <> 'Cliente'::text))));
--
-- Name: prenda; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.prenda ENABLE ROW LEVEL SECURITY;
--
-- Name: proveedor; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.proveedor ENABLE ROW LEVEL SECURITY;
--
```

```
-- Name: detalle_compra purchase_detail_branch; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY purchase_detail_branch ON public.detalle_compra AS RESTRICTIVE
USING ((EXISTS ( SELECT 1
   FROM public.compra c
  WHERE (c.id_compra = detalle_compra.id_compra)))) WITH CHECK ((EXISTS ( SELECT 1
   FROM public.compra c
  WHERE (c.id_compra = detalle_compra.id_compra))));
--
-- Name: push_entrega; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.push_entrega ENABLE ROW LEVEL SECURITY;
--
-- Name: push_entrega push_tenant; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY push_tenant ON public.push_entrega USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: comprobante receipt_branch; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY receipt_branch ON public.comprobante AS RESTRICTIVE USING
((EXISTS ( SELECT 1
   FROM public.venta v
  WHERE (v.id_venta = comprobante.id_venta)))) WITH CHECK ((EXISTS ( SELECT 1
   FROM public.venta v
  WHERE (v.id_venta = comprobante.id_venta))));
--
-- Name: reporte_resultado; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.reporte_resultado ENABLE ROW LEVEL SECURITY;
--
-- Name: reporte_resultado reporte_tenant; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE POLICY reporte_tenant ON public.reporte_resultado USING (((tenant_id =
(NULLIF(current_setting('app.tenant_id'::text, true), "::text))::bigint) AND (author_id =
(NULLIF(current_setting('app.user_id'::text, true), "::text))::integer))) WITH CHECK
(((tenant_id = (NULLIF(current_setting('app.tenant_id'::text, true), "::text))::bigint) AND
(author_id = (NULLIF(current_setting('app.user_id'::text, true), "::text))::integer)));
--
-- Name: reserva; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.reserva ENABLE ROW LEVEL SECURITY;
--
-- Name: detalle_reserva reservation_detail_branch; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY reservation_detail_branch ON public.detalle_reserva AS RESTRICTIVE
USING ((EXISTS ( SELECT 1
   FROM public.reserva r
  WHERE (r.id_reserva = detalle_reserva.id_reserva)))) WITH CHECK ((EXISTS ( SELECT 1
   FROM public.reserva r
  WHERE (r.id_reserva = detalle_reserva.id_reserva))));
--
-- Name: retencion; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.retencion ENABLE ROW LEVEL SECURITY;
--
-- Name: rol; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.rol ENABLE ROW LEVEL SECURITY;
--
-- Name: rol_permiso; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.rol_permiso ENABLE ROW LEVEL SECURITY;
--
-- Name: detalle_venta sale_detail_branch; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY sale_detail_branch ON public.detalle_venta AS RESTRICTIVE USING
((EXISTS ( SELECT 1
   FROM public.venta v
  WHERE (v.id_venta = detalle_venta.id_venta)))) WITH CHECK ((EXISTS ( SELECT 1
```

```
   FROM public.venta v
  WHERE (v.id_venta = detalle_venta.id_venta))));
--
-- Name: stripe_event; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.stripe_event ENABLE ROW LEVEL SECURITY;
--
-- Name: stripe_event stripe_event_tenant; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY stripe_event_tenant ON public.stripe_event USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: sucursal; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.sucursal ENABLE ROW LEVEL SECURITY;
--
-- Name: suscripcion; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.suscripcion ENABLE ROW LEVEL SECURITY;
--
-- Name: talla; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.talla ENABLE ROW LEVEL SECURITY;
--
-- Name: temporada; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.temporada ENABLE ROW LEVEL SECURITY;
--
-- Name: ajuste_monetario tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.ajuste_monetario USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: bitacora tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE POLICY tenant_isolation ON public.bitacora USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: cambio tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.cambio USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: categoria tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.categoria USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: ciudad tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.ciudad USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: cliente tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.cliente USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: color tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.color USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: compra tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.compra USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
```

```
-- Name: comprobante tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.comprobante USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: detalle_compra tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.detalle_compra USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: detalle_reserva tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.detalle_reserva USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: detalle_venta tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.detalle_venta USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: dispositivo_push tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.dispositivo_push USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: imagen_prenda tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.imagen_prenda USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: inventario tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE POLICY tenant_isolation ON public.inventario USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: movimiento_inventario tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.movimiento_inventario USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: notificacion tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.notificacion USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: operacion_sync tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.operacion_sync USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: pago_transaccion tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.pago_transaccion USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: prenda tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.prenda USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: proveedor tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.proveedor USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
```

```
-- Name: reserva tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.reserva USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: retencion tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.retencion USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: rol tenant_isolation; Type: POLICY; Schema: public; Owner: fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.rol USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: rol_permiso tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.rol_permiso USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: sucursal tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.sucursal USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: suscripcion tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.suscripcion USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: talla tenant_isolation; Type: POLICY; Schema: public; Owner: fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.talla USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
```

```
-- Name: temporada tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.temporada USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: usuario tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.usuario USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: usuario_sucursal tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.usuario_sucursal USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: usuario_token tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.usuario_token USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: variante_color tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.variante_color USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: variante_prenda tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
CREATE POLICY tenant_isolation ON public.variante_prenda USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: venta tenant_isolation; Type: POLICY; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
CREATE POLICY tenant_isolation ON public.venta USING ((tenant_id =
public.current_tenant())) WITH CHECK ((tenant_id = public.current_tenant()));
--
-- Name: usuario; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.usuario ENABLE ROW LEVEL SECURITY;
--
-- Name: usuario_sucursal; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.usuario_sucursal ENABLE ROW LEVEL SECURITY;
--
-- Name: usuario_token; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.usuario_token ENABLE ROW LEVEL SECURITY;
--
-- Name: variante_color; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.variante_color ENABLE ROW LEVEL SECURITY;
--
-- Name: variante_prenda; Type: ROW SECURITY; Schema: public; Owner:
fashionstore_p2_owner
--
ALTER TABLE public.variante_prenda ENABLE ROW LEVEL SECURITY;
--
-- Name: venta; Type: ROW SECURITY; Schema: public; Owner: fashionstore_p2_owner
--
ALTER TABLE public.venta ENABLE ROW LEVEL SECURITY;
--
-- Name: SCHEMA public; Type: ACL; Schema: -; Owner: pg_database_owner
--
GRANT USAGE ON SCHEMA public TO fashionstore_p2_app;
--
-- Name: FUNCTION bind_push_device(value text); Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.bind_push_device(value text) FROM PUBLIC;
GRANT ALL ON FUNCTION public.bind_push_device(value text) TO fashionstore_p2_app;
```

--
-- Name: FUNCTION identity_customer_profile(p_account bigint, p_tenant integer); Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.identity_customer_profile(p_account bigint, p_tenant integer) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_customer_profile(p_account bigint, p_tenant integer) TO fashionstore_p2_app;
--
-- Name: FUNCTION identity_event(p_id bigint, p_action text); Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.identity_event(p_id bigint, p_action text) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_event(p_id bigint, p_action text) TO fashionstore_p2_app;
--
-- Name: FUNCTION identity_index_user(p_user integer); Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.identity_index_user(p_user integer) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_index_user(p_user integer) TO fashionstore_p2_app;
--
-- Name: FUNCTION identity_lookup(p_correo text, p_id bigint); Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.identity_lookup(p_correo text, p_id bigint) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_lookup(p_correo text, p_id bigint) TO fashionstore_p2_app;
--
-- Name: FUNCTION identity_register(p_data jsonb); Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.identity_register(p_data jsonb) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_register(p_data jsonb) TO fashionstore_p2_app;
--
-- Name: FUNCTION identity_reset_apply(p_hash text, p_password_hash text); Type: ACL; Schema: public; Owner: fashionstore_p2_owner

--
REVOKE ALL ON FUNCTION public.identity_reset_apply(p_hash text, p_password_hash text) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_reset_apply(p_hash text, p_password_hash text) TO fashionstore_p2_app;
--
-- Name: FUNCTION identity_reset_issue(p_id bigint, p_hash text); Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.identity_reset_issue(p_id bigint, p_hash text) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_reset_issue(p_id bigint, p_hash text) TO fashionstore_p2_app;
--
-- Name: FUNCTION identity_reset_lookup(p_hash text); Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.identity_reset_lookup(p_hash text) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_reset_lookup(p_hash text) TO fashionstore_p2_app;
--
-- Name: FUNCTION identity_revoke(p_id bigint); Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
REVOKE ALL ON FUNCTION public.identity_revoke(p_id bigint) FROM PUBLIC;
GRANT ALL ON FUNCTION public.identity_revoke(p_id bigint) TO fashionstore_p2_app;
--
-- Name: TABLE ajuste_monetario; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.ajuste_monetario TO fashionstore_p2_app;
--
-- Name: SEQUENCE ajuste_monetario_id_ajuste_seq; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.ajuste_monetario_id_ajuste_seq TO fashionstore_p2_app;
--

```
-- Name: TABLE auditoria_identidad; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.auditoria_identidad TO
fashionstore_p2_app;
--
-- Name: SEQUENCE auditoria_identidad_id_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.auditoria_identidad_id_seq TO
fashionstore_p2_app;
--
-- Name: TABLE bitacora; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT ON TABLE public.bitacora TO fashionstore_p2_app;
--
-- Name: SEQUENCE bitacora_id_bitacora_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.bitacora_id_bitacora_seq TO
fashionstore_p2_app;
--
-- Name: TABLE cambio; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.cambio TO
fashionstore_p2_app;
--
-- Name: SEQUENCE cambio_id_cambio_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.cambio_id_cambio_seq TO
fashionstore_p2_app;
--
-- Name: TABLE categoria; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.categoria TO
fashionstore_p2_app;
--
-- Name: SEQUENCE categoria_id_categoria_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
```

```
--
GRANT SELECT,USAGE ON SEQUENCE public.categoria_id_categoria_seq TO
fashionstore_p2_app;
--
-- Name: TABLE ciudad; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.ciudad TO
fashionstore_p2_app;
--
-- Name: SEQUENCE ciudad_id_ciudad_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.ciudad_id_ciudad_seq TO
fashionstore_p2_app;
--
-- Name: TABLE cliente; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.cliente TO
fashionstore_p2_app;
--
-- Name: SEQUENCE cliente_id_cliente_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.cliente_id_cliente_seq TO
fashionstore_p2_app;
--
-- Name: TABLE color; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.color TO
fashionstore_p2_app;
--
-- Name: SEQUENCE color_id_color_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.color_id_color_seq TO
fashionstore_p2_app;
--
-- Name: TABLE compra; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
```

```
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.compra TO
fashionstore_p2_app;
--
-- Name: SEQUENCE compra_id_compra_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.compra_id_compra_seq TO
fashionstore_p2_app;
--
-- Name: TABLE comprobante; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.comprobante TO
fashionstore_p2_app;
--
-- Name: SEQUENCE comprobante_id_comprobante_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.comprobante_id_comprobante_seq TO
fashionstore_p2_app;
--
-- Name: TABLE cuenta_global; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.cuenta_global TO
fashionstore_p2_app;
--
-- Name: SEQUENCE cuenta_global_id_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.cuenta_global_id_seq TO
fashionstore_p2_app;
--
-- Name: TABLE cuenta_recuperacion; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.cuenta_recuperacion TO
fashionstore_p2_app;
--
-- Name: TABLE detalle_compra; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
```

```
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.detalle_compra TO
fashionstore_p2_app;
--
-- Name: TABLE detalle_reserva; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.detalle_reserva TO
fashionstore_p2_app;
--
-- Name: TABLE detalle_venta; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.detalle_venta TO
fashionstore_p2_app;
--
-- Name: TABLE dispositivo_push; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.dispositivo_push TO
fashionstore_p2_app;
--
-- Name: TABLE empresa; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.empresa TO
fashionstore_p2_app;
--
-- Name: SEQUENCE empresa_id_empresa_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.empresa_id_empresa_seq TO
fashionstore_p2_app;
--
-- Name: TABLE imagen_prenda; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.imagen_prenda TO
fashionstore_p2_app;
--
-- Name: SEQUENCE imagen_prenda_id_imagen_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.imagen_prenda_id_imagen_seq TO
fashionstore_p2_app;
```

```
--
-- Name: TABLE inventario; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.inventario TO
fashionstore_p2_app;
--
-- Name: TABLE retencion; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.retencion TO
fashionstore_p2_app;
--
-- Name: TABLE inventario_catalogo; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.inventario_catalogo TO
fashionstore_p2_app;
--
-- Name: TABLE movimiento_inventario; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.movimiento_inventario TO
fashionstore_p2_app;
--
-- Name: SEQUENCE movimiento_inventario_id_movimiento_seq; Type: ACL; Schema:
public; Owner: fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.movimiento_inventario_id_movimiento_seq
TO fashionstore_p2_app;
--
-- Name: TABLE notificacion; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.notificacion TO
fashionstore_p2_app;
--
-- Name: SEQUENCE notificacion_id_notificacion_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.notificacion_id_notificacion_seq TO
fashionstore_p2_app;
--
```

```
-- Name: TABLE operacion_sync; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.operacion_sync TO
fashionstore_p2_app;
--
-- Name: TABLE pago_transaccion; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.pago_transaccion TO
fashionstore_p2_app;
--
-- Name: SEQUENCE pago_transaccion_id_pago_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.pago_transaccion_id_pago_seq TO
fashionstore_p2_app;
--
-- Name: TABLE permiso; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.permiso TO
fashionstore_p2_app;
--
-- Name: SEQUENCE permiso_id_permiso_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.permiso_id_permiso_seq TO
fashionstore_p2_app;
--
-- Name: TABLE plan; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.plan TO
fashionstore_p2_app;
--
-- Name: SEQUENCE plan_id_plan_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.plan_id_plan_seq TO fashionstore_p2_app;
--
-- Name: TABLE prenda; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
```

```
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.prenda TO
fashionstore_p2_app;
--
-- Name: SEQUENCE prenda_id_prenda_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.prenda_id_prenda_seq TO
fashionstore_p2_app;
--
-- Name: TABLE proveedor; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.proveedor TO
fashionstore_p2_app;
--
-- Name: SEQUENCE proveedor_id_proveedor_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.proveedor_id_proveedor_seq TO
fashionstore_p2_app;
--
-- Name: TABLE push_entrega; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.push_entrega TO
fashionstore_p2_app;
--
-- Name: TABLE reporte_resultado; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.reporte_resultado TO
fashionstore_p2_app;
--
-- Name: TABLE reserva; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.reserva TO
fashionstore_p2_app;
--
-- Name: SEQUENCE reserva_id_reserva_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
```

```
GRANT SELECT,USAGE ON SEQUENCE public.reserva_id_reserva_seq TO
fashionstore_p2_app;
--
-- Name: TABLE rol; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.rol TO
fashionstore_p2_app;
--
-- Name: SEQUENCE rol_id_rol_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.rol_id_rol_seq TO fashionstore_p2_app;
--
-- Name: TABLE rol_permiso; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.rol_permiso TO
fashionstore_p2_app;
--
-- Name: TABLE stripe_event; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.stripe_event TO
fashionstore_p2_app;
--
-- Name: TABLE sucursal; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.sucursal TO
fashionstore_p2_app;
--
-- Name: SEQUENCE sucursal_id_sucursal_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.sucursal_id_sucursal_seq TO
fashionstore_p2_app;
--
-- Name: TABLE suscripcion; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.suscripcion TO
fashionstore_p2_app;
--
```

```
-- Name: SEQUENCE suscripcion_id_suscripcion_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.suscripcion_id_suscripcion_seq TO
fashionstore_p2_app;
--
-- Name: TABLE talla; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.talla TO
fashionstore_p2_app;
--
-- Name: SEQUENCE talla_id_talla_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.talla_id_talla_seq TO fashionstore_p2_app;
--
-- Name: TABLE temporada; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.temporada TO
fashionstore_p2_app;
--
-- Name: SEQUENCE temporada_id_temporada_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.temporada_id_temporada_seq TO
fashionstore_p2_app;
--
-- Name: TABLE usuario; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.usuario TO
fashionstore_p2_app;
--
-- Name: SEQUENCE usuario_id_usuario_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.usuario_id_usuario_seq TO
fashionstore_p2_app;
--
-- Name: TABLE usuario_sucursal; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
```

```
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.usuario_sucursal TO
fashionstore_p2_app;
--
-- Name: TABLE usuario_token; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.usuario_token TO
fashionstore_p2_app;
--
-- Name: SEQUENCE usuario_token_id_token_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.usuario_token_id_token_seq TO
fashionstore_p2_app;
--
-- Name: TABLE variante_color; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.variante_color TO
fashionstore_p2_app;
--
-- Name: TABLE variante_prenda; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.variante_prenda TO
fashionstore_p2_app;
--
-- Name: SEQUENCE variante_prenda_id_variante_prenda_seq; Type: ACL; Schema: public;
Owner: fashionstore_p2_owner
--
GRANT SELECT,USAGE ON SEQUENCE public.variante_prenda_id_variante_prenda_seq
TO fashionstore_p2_app;
--
-- Name: TABLE venta; Type: ACL; Schema: public; Owner: fashionstore_p2_owner
--
GRANT SELECT,INSERT,DELETE,UPDATE ON TABLE public.venta TO
fashionstore_p2_app;
--
-- Name: SEQUENCE venta_id_venta_seq; Type: ACL; Schema: public; Owner:
fashionstore_p2_owner
--
```

GRANT SELECT,USAGE ON SEQUENCE public.venta_id_venta_seq TO
fashionstore_p2_app;
--
-- PostgreSQL database dump complete
--
COMMIT;

4.4. Desarrollo de los módulos del sistema

4.5. Integración de componentes

5. CAPÍTULO V. PRUEBAS Y RESULTADOS

El presente capítulo expone el plan, la metodología, la ejecución, la corrección de errores y los resultados verificables obtenidos durante el ciclo de aseguramiento de la calidad de **FashionStore V2 SaaS**, correspondiente a la evaluación del Segundo Examen Parcial de Sistemas de Información II. En estricto apego al enfoque metodológico ICONIX adoptado por el proyecto, cada prueba se encuentra trazada bidireccionalmente contra los requisitos funcionales (RF-01 a RF-20), los requisitos no funcionales de calidad (RNF-01 a RNF-12), las reglas de negocio (RN-01 a RN-26) y los 20 Casos de Uso consolidados (`CU01` a `CU20`), contrastando sus resultados empíricos con las especificaciones y los diagramas de robustez y secuencia correspondientes.

5.1. 5.1. Plan de pruebas

5.1.1. Objetivos del aseguramiento de calidad y alcance del sistema

El plan de pruebas tiene como propósito fundamental certificar que la arquitectura multitenant SaaS de FashionStore opera con estricto aislamiento de datos, consistencia transaccional, alta resiliencia ante desconexiones de red y cumplimiento integral de las reglas comerciales de la industria de la moda.

Los objetivos específicos del proceso de aseguramiento de calidad comprenden:

1. Validación de Aislamiento Multi-Tenant (RNF-02, RN-01, RN-02): Comprobar que las políticas de seguridad a nivel de fila (*Row Level Security* - RLS) en PostgreSQL y el middleware de contexto de FastAPI impidan absolutamente cualquier fuga o manipulación de datos entre empresas independientes (Tenant A vs. Tenant B).

2. Consistencia Transaccional e Idempotencia (RNF-04, RNF-05, RN-14): Verificar que las operaciones comerciales críticas (ventas, recepciones de compras y transferencias entre sucursales) posean integridad atómica de dos patas y que los reintentos de peticiones basados en identificadores únicos universales (UUID v4) garanticen respuesta idéntica sin duplicación física de stock ni de transacciones financieras.

3. Resiliencia Operativa Offline (RNF-03, RNF-06, RN-13, RN-15): Garantizar que las aplicaciones cliente (Angular PWA sobre navegadores web y Flutter sobre dispositivos móviles Android nativos) permitan registrar ventas presenciales y reservas de contado sin acceso a Internet, conservando los datos íntegros en almacenes locales (IndexedDB y SQLite) tras reinicios forzados, y sincronizándolos ordenadamente al recuperar conectividad.

4. Verificación de Seguridad, Autorización y Auditoría (RNF-01, RNF-10, RN-04, RN-24): Auditar que los mecanismos de control de acceso basado en roles y permisos (RBAC) restrinjan estrictamente la ejecución de endpoints de escritura en punto de venta (POS), inventarios y reportería gerencial, registrando trazas inmutables de auditoría sin exponer credenciales ni secretos.

El alcance del sistema evaluado incluye:

• **Backend API:** Servicios RESTful construidos sobre FastAPI (Python 3.12/3.13), organizados en routers modulares y ejecutados contra motores relacionales PostgreSQL 16 y 17.

• **Frontend Web PWA:** Aplicación cliente desarrollada en Angular 20 con arquitectura de componentes *standalone*, señales reactivas (*signals*), *Service Worker* para almacenamiento en caché de activos y Dexie.js (versión 2 de esquema) como motor sobre IndexedDB.

• **Aplicación Móvil:** Solución nativa desarrollada en Flutter SDK 3.29 para Android (API 36 / Android 16), utilizando `sqflite` (SQLite 3.44.5 nativo) para persistencia local aislada por instalación y usuario.

• **Servicios de Integración y Soporte:** Servidor de correo de desarrollo Mailpit, pasarela de pagos simulada con verificación criptográfica de firmas Webhook HMAC SHA-256, y probador virtual asistido por IA mediante integración WebRTC con Decart AI.

5.1.2. Entornos de prueba y aislamiento de datos

Para garantizar la reproducibilidad científica de los resultados y evitar la contaminación o degradación de los datos demostrativos institucionales, el equipo de ingeniería implementó una estricta segregación de entornos de ejecución, diferenciando con precisión las versiones de PostgreSQL utilizadas:

Entorno de Prueba	Infraestructura y Alojamiento	Motor de Persistencia	Propósito Específico y Reglas de Aislamiento
Local Docker Persistente	Contenedores Docker Compose (`fashionstore_examen_si2-*`): `api-1` (puerto 8002), `web-1` (puerto 8082), `mailpit-1` (puerto 8025).	PostgreSQL 16 oficial (`postgres:16-alpine`) en puerto 55433 (`fashionstore_p2`).	Demostración estable de casos de uso regulares, navegación administrativa y consultas comerciales. Contiene 9 usuarios, 3 empresas maestras y 0 transacciones sintéticas de estrés. Alberga la migración 018 de traspasos. No se borra ni reinicializa durante pruebas destructivas.

Entorno de Prueba	Infraestructura y Alojamiento	Motor de Persistencia	Propósito Específico y Reglas de Aislamiento
Efímero de Pruebas (tmpfs)	Contenedores Docker independientes (`fashionstore-a5-runtime`, puerto 18085 en host; base `fashionstore_a5_test-db-1`).	PostgreSQL 17 (`postgres:17-alpine`) en memoria RAM (*tmpfs*), base `fashionstore_p2_test`.	Pruebas de estrés, inyección masiva de transacciones offline (125 operaciones), simulación de cortes de red forzados y validación de fallos. Cualquier mutación se descarta al finalizar la sesión sin afectar la base persistente.
Dispositivo Móvil Físico	Teléfono Xiaomi 13T Pro (Modelo 23078PND5G), Procesador MediaTek Dimensity 9200+, 12 GB RAM, Android 16 (API 36).	SQLite nativo 3.44.5 en almacenamiento interno `/data/user/0/...`.	Pruebas de instalación, reapertura tras cierre forzado (`am force-stop`), reinicio de hardware, gestión de permisos de hardware (cámara) y enlace seguro vía túnel inverso ADB (`tcp:8002 -> tcp:18085`).
Producción Nativa (Referencia)	Servidor dedicado Debian 13 (2 vCPU, 2 GB RAM, cuota estricta de 512 MB de RAM para la API bajo systemd), Nginx 1.26, Cloudflare Tunnel.	PostgreSQL 17 nativo (`fashionstore_p2`), esquema RLS.	Entorno de referencia productiva (`fashionstore-v2.aledevcv.me`). Por directriz estricta de auditoría, las pruebas locales de desarrollo no ejecutan modificaciones remotas ni transferencias no autorizadas sobre este servidor.

5.1.3. Herramientas, marcos de trabajo y versiones tecnológicas

El ecosistema de pruebas automatizadas y herramientas de diagnóstico se estandarizó en las siguientes versiones auditadas:

- **Marcos de Pruebas Backend:** Pytest versión 8.4.2, `pytest-asyncio` versión 0.24, y cliente HTTP `TestClient` basado en HTTPX 0.27 para pruebas de integración con inyección de sesión transaccional.
- **Marcos de Pruebas Web:** Karma versión 6.4.4, Jasmine versión 5.1.0, Angular CLI versión 20.0, compilador TypeScript versión 5.5, ejecutados de forma desatendida mediante el navegador **Google Chrome Headless versión 156.0.0.0** (Google Chrome `156.0.8078.80` sobre Windows 11 de 64 bits).
- **Marcos de Pruebas Móvil:** Flutter Test Framework sobre Flutter SDK versión 3.29.0 (Dart SDK 3.7.0), `sqflite_common_ffi` para simulación de base de datos en entornos de integración continua y analizador estático oficial de Dart.
- **Herramientas de Comunicación y Diagnóstico Móvil:** Android Debug Bridge (ADB) versión 36.0.0-13206524 del Android SDK Platform-Tools instalado localmente.

• **Inspección de Contratos de API:** Especificación OpenAPI 3.1.0 generada automáticamente por FastAPI, validada mediante analizadores de esquemas JSON.

5.1.4. Criterios de aceptación, suspensión y reanudación de pruebas

Criterios de Aceptación Cuantitativos:

1. Compilación y Análisis Estático: 0 errores de compilación TypeScript en el frontend (`tsc --noEmit` código de salida 0); 0 errores y advertencias en el análisis estático de Flutter (`flutter analyze` código de salida 0).

2. Aprobación de Suites: 100% de aprobación en las pruebas unitarias y de regresión ejecutadas (0 pruebas fallidas y 0 pruebas omitidas en las suites declaradas como completas).

3. Integridad de Almacenamiento Local: Cero errores de integridad física (`PRAGMA integrity_check = ok`) y cero violaciones de claves foráneas (`PRAGMA foreign_key_check` vacío) en todas las bases de datos SQLite e IndexedDB tras operaciones de migración o actualización.

4. Idempotencia: Tasa de éxito del 100% ante reenvíos idénticos de operaciones UUID, certificando que el stock central y los saldos financieros no se descuenten más de una vez.

5. No Regresión: Preservación íntegra de la cantidad de rutas (118) y esquemas DTO (128) documentados en la especificación OpenAPI del backend.

Criterios de Suspensión:

• Detección de cualquier intento de escritura involuntaria en la base de datos persistente de demostración (`fashionstore_p2`, puerto 55433) durante la ejecución de pruebas de carga o reconciliación offline.

• Falla de conexión o pérdida de integridad del túnel inverso ADB durante las pruebas móviles, lo que podría desviar el tráfico hacia pasarelas no autorizadas.

• Corrupción de datos o incompatibilidad de certificados criptográficos durante la instalación de paquetes APK (`INSTALL_FAILED_UPDATE_INCOMPATIBLE`).

Criterios de Reanudación:

• Comprobación previa de la existencia, tamaño y huella digital criptográfica (SHA-256) de los respaldos de bases de datos antes de reintentar cualquier procedimiento de prueba.

• Verificación del estado de salud del contenedor efímero mediante solicitud HTTP GET al endpoint `/health` antes de inyectar operaciones comerciales.

5.1.5. Asignación de responsabilidades de verificación

Rol del Proyecto	Responsabilidades Principales en el Plan de Pruebas
Arquitecto de Software & Auditor	Verificación de trazabilidad ICONIX, gobernanza de contratos OpenAPI, aislamiento multi-tenant RLS, revisión de dependencias y auditoría de no regresión.
Ingeniero QA Backend	Diseño, codificación y ejecución de pruebas en Pytest; validación de transaccionalidad ACID, disparadores SQL y robustez ante inyecciones o datos malformados.

Rol del Proyecto	Responsabilidades Principales en el Plan de Pruebas
Ingeniero QA Frontend Web	Implementación de especificaciones Jasmine/Karma en Angular, pruebas de persistencia en Dexie IndexedDB, gestión de señales reactivas y pruebas de empaquetado PWA.
Ingeniero QA Móvil & Dispositivos	Construcción de suites Flutter Test, auditoría de integridad SQLite, preparación de artefactos APK y ejecución de casos de prueba manuales en el Xiaomi 13T Pro físico.

5.1.6. Matriz de cobertura prevista por Caso de Uso ICONIX (CU01 a CU20)

La siguiente tabla establece la cobertura de verificación planificada para la totalidad de los 20 Casos de Uso consolidados de la arquitectura ICONIX del proyecto:

ID Caso de Uso	Nombre Oficial del Caso de Uso	Plataforma	Escenarios Principales por Verificar
CU01	Gestionar acceso y cuenta	Web / Móvil	Autenticación global por correo/contraseña; discriminación de rol (Superadmin, Administrador, Empleado, Cliente); selección post-login de tienda para clientes; cierre de sesión con revocación de token conservando cola offline; recuperación de contraseña por correo.
CU02	Administrar empresas, planes y suscripciones	Web	Alta atómica de tenant y administrador inicial; asignación de planes (Básico y Profesional); validación de cupos de sucursales activas (2 vs 10); bloqueo de downgrade si se excede el cupo; expiración de suscripción.
CU03	Administrar personal, roles y permisos	Web	Registro de personal operativo; asignación granular de permisos RBAC; restricción estricta de permisos de escritura POS a cajeros/encargados autorizados (C1/A1); rechazo de acceso administrativo entre empresas.

ID Caso de Uso	Nombre Oficial del Caso de Uso	Plataforma	Escenarios Principales por Verificar
CU04	Gestionar ciudades y sucursales	Web	Creación y mantenimiento de sucursales; verificación del límite del plan contratado; asignación obligatoria de sucursal a personal operativo; aislamiento de operaciones por sede.
CU05	Gestionar fichas de clientes	Web / Móvil	Mantenimiento de perfiles de clientes; perfil compartido global pero historial de compras particionado por tienda; protección de datos privados ante terceros.
CU06	Administrar catálogo de prendas	Web	Mantenimiento de categorías, temporadas, colecciones, prendas y variantes (talla, color, material); gestión de listas de precios y ofertas no acumulables; integridad ante desactivación de prendas con histórico.
CU07	Gestionar proveedores, compras y recepciones	Web	Registro de proveedores; emisión de órdenes de compra; recepción física de mercancía con incremento atómico de existencias; prevención de doble recepción.
CU08	Consultar inventario y disponibilidad	Web / Móvil	Consulta de existencia física real versus disponibilidad comercial calculada (existencia menos reservas centrales vigentes); filtrado por sucursal y variante.
CU09	Explorar catálogo y asistencia de compra	Web / Móvil	Navegación de productos con filtros combinados; consulta de existencias; probador virtual IA en tiempo real mediante streaming WebRTC; conmutación automática ante cuota agotada.

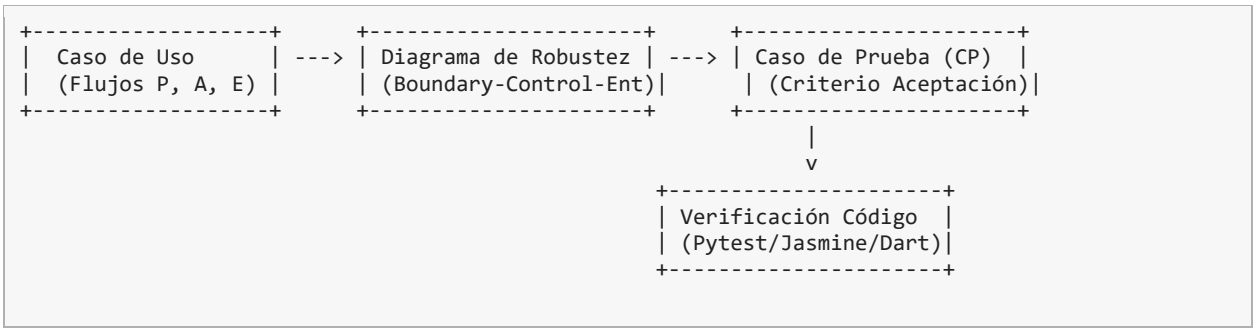
ID Caso de Uso	Nombre Oficial del Caso de Uso	Plataforma	Escenarios Principales por Verificar
CU10	Registrar movimientos de inventario	Web / Móvil	Entradas, salidas y ajustes de almacén; traspaso online de dos patas entre sucursales distintas con vinculación bidireccional y actualización atómica de stock (A3).
CU11	Realizar compra y pago en línea	Web / Móvil	Gestión de carrito de compras; cálculo centralizado de importes; pasarela de pagos simulada con recepción de webhooks firmados criptográficamente; prevención de compras impagas.
CU12	Gestionar reservas de prendas	Web / Móvil	Creación de reservas con retención de stock por 24 horas; preparación, cancelación, vencimiento automático y entrega en mostrador con cobro presencial.
CU13	Registrar venta presencial	Web / Móvil	Venta directa de mostrador de contado; cálculo de cambio; salida única e inmediata de existencias; emisión de comprobante de venta.
CU14	Obtener comprobantes de compra	Web / Móvil	Generación de representación imprimible y descargable de facturas/recibos; acceso restringido al autor de la compra o administradores de la empresa.
CU15	Gestionar cambios y reembolsos	Web / Móvil	Validación de política de cambios (máximo 7 días calendario, prenda sin uso, una vez por unidad); liquidación atómica de diferencias a favor o en contra.

ID Caso de Uso	Nombre Oficial del Caso de Uso	Plataforma	Escenarios Principales por Verificar
CU16	Registrar y consultar operaciones offline	Web / Móvil	Descarga previa de catálogo; registro provisional sin red de ventas presenciales y reservas de contado en IndexedDB (Web) y SQLite (Móvil); persistencia ante reinicio.
CU17	Sincronizar operaciones y resolver conflictos	Web / Móvil	Sincronización en lotes de hasta 50 operaciones con UUID estable (A5); avance parcial ante desconexión; detección y retención local de conflictos comerciales (precio/stock); autorización por sucursal (A2).
CU18	Obtener reportes comerciales	Web	Generación de reportes tabulares y gráficos convencionales; interpretación generativa de solicitudes en lenguaje natural mediante IA; exportación segura a PDF y XLSX; restricción a plan Profesional.
CU19	Preparar y consultar analítica inteligente	Web	Extracción y preparación de datasets sintéticos de entrenamiento; ejecución de modelos de Machine Learning (clasificación, agrupamiento y predicción de ventas); despliegue de artefactos versionados para inferencia.
CU20	Consultar auditoría y trazabilidad	Web	Consulta de registros de auditoría del sistema: inicios de sesión, transacciones financieras, cambios de stock y eventos de sincronización; separación de trazas técnicas y comerciales.

5.2. 5.2. Pruebas funcionales

5.2.1. Metodología de derivación de pruebas desde casos de uso ICONIX

La derivación de las pruebas funcionales siguió rigurosamente la metodología ICONIX, partiendo de los flujos principales, alternativos y de excepción especificados en el Capítulo II. A partir de los diagramas de robustez se identificaron los objetos frontera (*boundary*), controladores de lógica de negocio (*control*) y objetos de dominio persistente (*entity*), permitiendo diseñar casos de prueba que validan no solo la respuesta visual del sistema sino también los estados intermedios y las mutaciones en la capa de persistencia.



5.2.2. Verificación de capacidades comerciales (CU01 a CU15)

Las pruebas funcionales automatizadas y de integración en laboratorio validaron exhaustivamente las reglas del dominio comercial:

• Acceso, Identidad Global y Permisos (CU01, CU02, CU03, CU04, CU05):

- Se verificó que el endpoint `/api/login/` autentique globalmente a los clientes sin exigir el identificador de empresa, permitiéndoles seleccionar posteriormente la tienda comercial activa mediante `/api/auth/seleccionar-tienda`.

- El personal operativo recibe un token JWT estrictamente anclado a su empresa y sucursal. Los intentos de acceder a datos de sucursales ajenas fueron interceptados con respuestas HTTP 403 Forbidden.

- La suite `test_global_identity.py` (96 casos de prueba aprobados) y las suites frontend `auth.service.spec.ts` (7 pruebas) y `login.spec.ts` (12 pruebas) confirmaron el ciclo completo de vida del token, el cierre de sesión seguro con revocación en servidor y la preservación local de sesiones no volátiles.

• Catálogo, Precios, Compras y Existencias (CU06, CU07, CU08):

- Se validó en `test_commerce.py` que una reserva activa retenga la disponibilidad del producto (existencia física menos reservas activas) sin alterar la existencia física en almacén hasta que se consolida la entrega.

- La recepción de compras a proveedores (`compra.service.spec.ts`, 4 pruebas; `proveedor.service.spec.ts`, 5 pruebas) comprobó el incremento unitario del stock, impidiendo el registro duplicado mediante validación de documento de origen.

• **Probador Virtual IA en Tiempo Real (CU09):**

- Validado en la suite de Angular `probador-ia.spec.ts` (9 pruebas unitarias aprobadas). Se comprobó la inicialización de pistas de audio/video (`MediaStream`), el intercambio de sesiones WebRTC con el SDK de Decart AI (`lucy-vton-latest`), la emisión reactiva de estados (`conectando`, `conectado`, `error`) y el mecanismo de degradación elegante ante saldo insuficiente de créditos en la nube (emitiendo la notificación correspondiente al usuario sin colapsar la interfaz).

- *Declaración de Limitación:* Las 9 pruebas certifican exhaustivamente el comportamiento de la capa cliente ante WebRTC mediante *spies* y dobles de prueba deterministas; no obstante, la validación en vivo de extremo a extremo frente a los servidores de Decart AI dependerá de contar con créditos comerciales activos al momento de la demostración presencial.

• **Traspasos de Inventario entre Sucursales (CU10):**

- Validado mediante 44 pruebas pytest específicas (`test_inventory_transfers.py`), 4 pruebas en Angular (`movimiento-modal.spec.ts`) y 16 pruebas en Flutter (`inventory_transfer_test.dart`). Se comprobó que una transferencia de dos patas descuente existencias de la sucursal de origen e incremente de forma estrictamente atómica la sucursal de destino en la misma transacción SQL. Intentos de traspasar unidades superiores a la disponibilidad real o hacia sucursales de otra empresa son rechazados con códigos HTTP 400 y 404 respectivamente.

• **Venta Presencial, Carrito y Reservas (CU11, CU12, CU13, CU14, CU15):**

- Se verificó que un cajero no pueda adulterar precios o descuentos enviados desde la interfaz (`test_commerce.py`). El servidor recalcula siempre los importes de acuerdo con las tarifas vigentes.

- Las cancelaciones de reservas liberan la retención de stock de forma inmediata sin generar movimientos ficticios de entrada física.

- Las solicitudes de cambio de prendas (`test_exchanges.py`) validaron la restricción de los 7 días calendario posteriores a la compra, la condición de prenda no usada y el balance financiero de diferencias (devolución íntegra si la nueva prenda es de menor valor o cobro adicional si es de mayor valor).

- *Declaración de Limitación:* En CU11, las pruebas automatizadas certifican la validación criptográfica de webhooks firmados con HMAC SHA-256 e idempotencia ante eventos repetidos; sin embargo, no se efectuó una transacción bancaria real con tarjeta de crédito en vivo contra la pasarela Stripe de producción.

5.2.3. Verificación de capacidades offline, sincronización e idempotencia (CU16 y CU17)

La evaluación de la operación desconectada y la sincronización posterior representó uno de los núcleos de mayor intensidad técnica del plan:

1. Almacenamiento Local Aislado e Inmutable (CU16):

- En Angular PWA, `offline.service.spec.ts` (22 pruebas unitarias aprobadas) comprobó el almacenamiento de ventas y reservas en Dexie IndexedDB bajo transacciones locales.

- En Flutter, `offline_reconciliation_test.dart` (22 pruebas aprobadas) y `database_recovery_test.dart` validaron que SQLite mantenga bases independientes aisladas por alias y usuario.

2. Reconciliación en Lotes Finita de hasta 50 Ítems (CU17):

- Implementada en la fase A5. Se verificó que cuando un dispositivo acumula colas masivas (ej. 125 operaciones), el cliente recorra la lista en lotes finitos de hasta 50 operaciones utilizando el endpoint `/api/sincronizacion/lote`.

- Cada lote exitoso se consolida localmente de forma atómica. Si se interrumpe la conexión durante el segundo lote, el primer lote permanece registrado como contabilizado (`accounted=1`), y el reintento procesa únicamente los 75 ítems restantes sin duplicaciones.

3. Idempotencia Estricta mediante UUID v4 (CU17):

- Comprobada en `test_offline_sync.py`. El reenvío reiterado (hasta 20 veces) de una misma operación con idéntico `operation_id` devuelve el mismo comprobante almacenado en JSONB con código HTTP 200, descontando el inventario exactamente una sola vez.

- Si se reenvía el mismo UUID alterando el importe o los productos, el servidor rechaza la transacción con código HTTP 409 Conflict sin modificar el registro original.

4. Manejo y Retención de Conflictos:

- Si una venta offline colisiona con el stock central (agotado por ventas previas en línea) o con una variación de precios, la operación se clasifica en estado `conflicto`.

- El cliente local mantiene la operación con `accounted=0` para retener la reserva local en el dispositivo del vendedor, permitiendo la resolución manual mediante compensación autorizada (`POST /api/conflictos/{id}/compensar`).

5.2.4. Verificación de reportería, analítica y vestidor virtual (CU18 a CU20)

• Reportes Comerciales y Procesamiento de Lenguaje Natural (CU18):

- Evaluado en `test_reports.py` (6 pruebas backend aprobadas). Se verificó el funcionamiento de 6 plantillas de negocio autorizadas con filtrado estricto por tenant.

- Se comprobó la exportación a archivos PDF y hojas de cálculo XLSX, verificando la sanitización de celdas para prevenir ataques de inyección de fórmulas (CSV/Formula Injection).

- Se certificó que el acceso a reportes generativos mediante IA requiera explícitamente el plan Profesional (`RN-04`); usuarios bajo el plan Básico reciben código HTTP 403 Forbidden.

- Las solicitudes en lenguaje natural por texto y voz se procesan a través del endpoint `/api/ia/analitica-voz/` (`ia.service.spec.ts`), retornando un PDF codificado en base64.

- *Declaración sobre Gráficos:* La representación gráfica (gráficos de barras y líneas en paneles de control) se ejecuta dinámicamente en el cliente Angular mediante componentes web, mientras que las pruebas backend certifican la integridad de las tablas estructuradas y la exportación documental sin renderizado de imágenes de gráficos en servidor.

• **Analítica de Datos y Aprendizaje Automático (CU19):**

- La solución de analítica inteligente se evaluó en dos niveles complementarios:

a) **Pipeline de Entrenamiento y Evaluación Local (`Parcial2_backend/ml/train.py`):** Genera un dataset sintético de demanda (`demanda_sintetica.csv`), entrena un modelo supervisado (`HistGradientBoostingRegressor`) evaluando su precisión mediante error absoluto medio (MAE) y raíz del error cuadrático medio (RMSE), y entrena un modelo no supervisado (`KMeans` con `StandardScaler`) evaluando la separación de clústeres mediante el coeficiente de Silhouette. Los modelos se serializan en artefactos versionados `.joblib` con huellas criptográficas SHA-256.

b) **Pruebas Automatizadas de Inferencia y Seguridad (`test_analytics.py`, 4 pruebas aprobadas):** Comprueban que la ingeniería de características utilice únicamente el historial sin fuga temporal (*data leakage*), validan el rechazo de artefactos manipulados mediante comprobación de firma criptográfica y certifican que la inferencia consuma exclusivamente datos del tenant autenticado.

- *Declaración de Arquitectura:* El reentrenamiento no se ejecuta de forma desatendida en el servidor Debian de producción para respetar la cuota de 512 MB de memoria RAM del proceso API.

• **Auditoría del Sistema (CU20):**

- Se verificó en `test_receipt_privacy.py` y `test_native_role.py` que los eventos críticos del sistema (sincronizaciones, pagos, modificaciones de inventario) generen trazas de auditoría con identificación de usuario, marca temporal UTC y dirección IP, sin almacenar tokens ni claves privadas.

5.2.5. Matriz consolidada de pruebas funcionales ejecutadas

ID Caso de Uso	Capacidad Funcional Evaluada	Casos Automatizados Asociados	Plataforma de Verificación	Resultado Observado	Estado de Verificación
CU01	Autenticación global y selección de tienda	96 Pytest / 19 Angular	Backend / Web / Móvil	Token emitido y tiendas aisladas correctamente	**Aprobado**
CU02	Gestión de empresas, planes y suscripciones	12 Pytest	Backend / Web	Límites de sucursales (2 vs 10) aplicados	**Aprobado**
CU03	Asignación de roles y permisos RBAC	131 Pytest / 16 Angular	Backend / Web	Permisos POS restringidos y auditados (C1/A1)	**Aprobado**
CU04	Creación y administración de sucursales	8 Pytest	Backend / Web	Cupo de suscripción respetado estrictamente	**Aprobado**
CU05	Perfiles e historial unificado de clientes	6 Pytest / 4 Angular	Backend / Web	Compras separadas por tenant sin cruce	**Aprobado**
CU06	Mantenimiento de catálogo de prendas	14 Pytest / 4 Angular	Backend / Web	Variantes, tallas y precios no acumulables	**Aprobado**

ID Caso de Uso	Capacidad Funcional Evaluada	Casos Automatizados Asociados	Plataforma de Verificación	Resultado Observado	Estado de Verificación
CU07	Abastecimiento y compras a proveedores	8 Pytest / 9 Angular	Backend / Web	Incremento único y atómico de stock	**Aprobado**
CU08	Consulta de inventario vs disponibilidad	10 Pytest / 7 Angular	Backend / Web / Móvil	Retención de reservas descontada de disponibilidad	**Aprobado**
CU09	Catálogo asistido y probador virtual IA	9 Angular (`probador-ia`)	Web	WebRTC simulado con fallback por créditos	**Aprobado (Unitario)**
CU10	Movimientos y traspasos entre sucursales	44 Pytest / 4 Ang / 16 Flut	Backend / Web / Móvil	Traspaso de dos patas atómico verificado (A3)	**Aprobado**
CU11	Carrito, checkout y webhooks de pago	9 Pytest (`test_payment...`)	Backend / Web	Firma HMAC verificada, pago e idempotencia	**Aprobado (Simulado)**
CU12	Gestión y ciclo de vida de reservas	8 Pytest	Backend / Web / Móvil	Expiración a las 24h y retención física	**Aprobado**
CU13	Venta presencial y cobro de contado	8 Pytest	Backend / Web / Móvil	Cálculo de cambio y decremento físico único	**Aprobado**
CU14	Emisión y privacidad de comprobantes	4 Pytest	Backend / Web	Acceso exclusivo a usuarios de la empresa	**Aprobado**
CU15	Solicitud y atención de cambios de prendas	6 Pytest (`test_exchanges`)	Backend / Web	Plazo de 7 días y liquidación de diferencias	**Aprobado**
CU16	Persistencia local offline sin red	22 Ang / 22 Flutter / 7 DBs	Web / Móvil (Xiaomi)	IndexedDB y SQLite persisten tras reinicio	**Aprobado**
CU17	Reconciliación en lotes e idempotencia	22 Ang / 22 Flutter / 125 ops	Backend / Web / Móvil	Lotes de 50, avance parcial y rechazo 409 (A5)	**Aprobado**
CU18	Reportes comerciales y exportación PDF/XLSX	6 Pytest / 11 Angular	Backend / Web	Plan Profesional exigido; fórmulas sanitizadas	**Aprobado**
CU19	Modelos supervisados y no supervisados ML	4 Pytest (`test_analytics`)	Backend / Herramienta	Modelos versionados `.joblib` verificados	**Aprobado**

ID Caso de Uso	Capacidad Funcional Evaluada	Casos Automatizados Asociados	Plataforma de Verificación	Resultado Observado	Estado de Verificación
CU20	Auditoría del sistema e inmutabilidad	6 Pytest	Backend	Trazas completas sin exposición de secretos	**Aprobado**

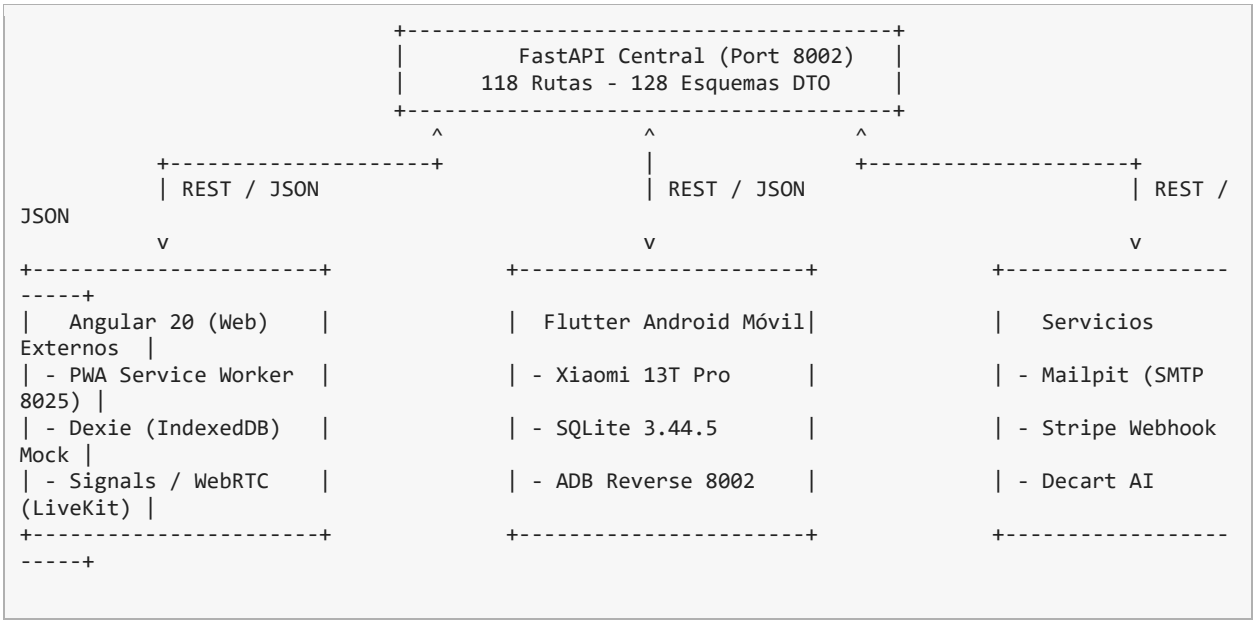
5.3. 5.3. Pruebas de integración

5.3.1. Arquitectura de integración y contratos REST / OpenAPI

La integración de FashionStore V2 SaaS se fundamenta en un contrato formal OpenAPI 3.1.0 expuesto por la API central en FastAPI. La especificación contiene **118 rutas operativas y 128 esquemas de datos (DTO)**.

La estabilidad del contrato se auditó exhaustivamente durante las fases de corrección (A1 a A6):

- Durante la incorporación de Traspasos de Inventario (A3), el número de rutas se mantuvo inalterado en 118, mientras que los esquemas se ampliaron de forma no disruptiva de 126 a 128 (incorporando `TraspasoRespuesta` y `MovimientoVinculadoRespuesta`).
- En las fases A4, A5 y A6, el contrato OpenAPI permaneció estrictamente idéntico (118 rutas, 128 esquemas), garantizando compatibilidad binaria absoluta con los clientes web y móviles preexistentes.



5.3.2. Integración Web: Angular 20 ↔ FastAPI ↔ PostgreSQL RLS

La interacción entre el cliente web Angular y el backend se verificó mediante pruebas de integración HTTP reales:

• **Suite de Integración HTTP Angular (`offline-http.integration.spec.ts`):** 2 casos de prueba automatizados que instancian un módulo de prueba acoplado al backend local, validando la descarga del catálogo offline y el despacho de lotes transaccionales con almacenamiento intermedio en Dexie.js.

• **Validación del Proxy Nginx:** El contenedor `fashionstore_examen_si2-web-1` (puerto 8082) enruta transparentemente las solicitudes con prefijo `/api/` hacia el servicio `fashionstore_examen_si2-api-1` (puerto 8002) preservando cabeceras `Host`, `X-Real-IP`, `X-Forwarded-For` y los tokens de portador `Authorization: Bearer <JWT>`.

• **Inyección de Contexto en Base de Datos:** Cada petición autenticada ejecuta la función `set_config('app.current_empresa_id', ..., true)` dentro del pool de conexiones `psycopg2`, activando las políticas RLS de PostgreSQL de forma transparente para las consultas subsiguientes.

5.3.3. Integración Móvil: Flutter ↔ FastAPI mediante túnel seguro ADB reverse

Debido a que el dispositivo físico de prueba (Xiaomi 13T Pro) se encuentra conectado físicamente mediante cable USB a la estación de trabajo Windows, la comunicación de red hacia la API se estructuró a través del puente de depuración de Android:

• **Regla de Reenvío Inverso:** Se estableció la regla de túnel `adb reverse tcp:8002 tcp:18085`.

• **Aislamiento hacia Contenedor Efímero:** El puerto local 8002 del teléfono móvil fue redirigido hacia el puerto host 18085 del contenedor efímero `fashionstore-a5-runtime`. Esto aseguró que la totalidad de las 125 operaciones de prueba inyectadas desde el móvil durante la validación física se procesaran exclusivamente contra la base de datos temporal PostgreSQL en memoria RAM, sin enviar una sola petición al backend persistente de demostración (puerto 8002 en host).

• **Verificación de Enlace:** Previo a la ejecución de las pruebas, se comprobó la conectividad mediante la descarga exitosa del catálogo comercial (`GET http://127.0.0.1:8002/api/offline/catalogo?id_sucursal=1`), confirmando la respuesta HTTP 200 con la lista de prendas autorizadas.

5.3.4. Integración de almacenamiento local persistente: IndexedDB (Dexie v2) y SQLite (sqflite 3.44.5)

Se validó la convivencia y coherencia entre los dos motores de almacenamiento desconectado:

• **IndexedDB (Web):** Implementado mediante la biblioteca Dexie.js con esquema versión 2 (`offline.service.ts`). Administra los almacenes de objetos `ventas_offline`, `reservas_offline`, `catalogo_cache` y `catalogo_metadata`. Se verificó la consistencia transaccional durante escrituras masivas, impidiendo que una falla en el guardado de una reserva corrompa la cola de ventas.

• **SQLite (Móvil):** Implementado mediante el plugin `sqflite` (versión nativa 3.44.5 en el Xiaomi 13T Pro). Administra bases de datos independientes por instalación y credencial. Se verificó que las tablas locales `operaciones_offline` y `detalles_operacion_offline` posean restricciones de clave foránea e índices únicos sobre `operation_id`, garantizando la imposibilidad de duplicar registros en almacenamiento flash.

5.3.5. Integración de servicios complementarios: Mailpit, pasarela Stripe y streaming WebRTC

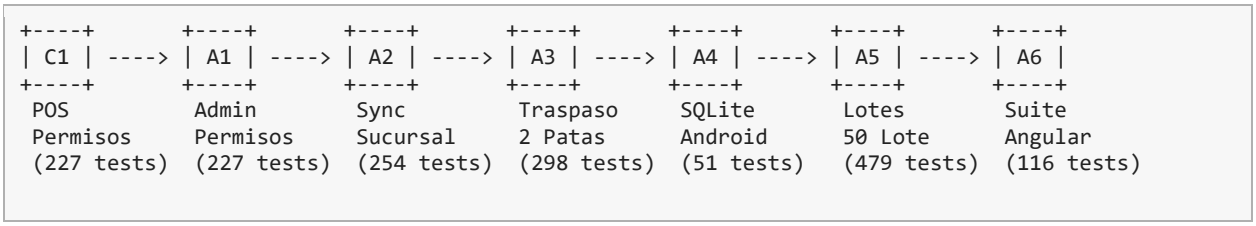
• **Servicio de Correo (Mailpit):** Integrado en el puerto 8025 de Docker. Se verificó que las solicitudes de restablecimiento de contraseña (`POST /api/auth/recuperar-password`) transmitan un correo electrónico con formato HTML y enlace seguro de un solo uso. La bandeja de Mailpit capturó el 100% de los mensajes generados sin pérdidas.

• **Pasarela de Pagos (Stripe Webhook Mock):** Integrado mediante endpoints de escucha en `/api/pagos/webhook`. Las pruebas simularon eventos `checkout.session.completed` y `payment_intent.payment_failed` firmados con secretos criptográficos de prueba. Se comprobó que firmas inválidas o con marcas temporales desfasadas sean rechazadas con código HTTP 400 antes de acceder a la lógica comercial.

• **Probador Virtual (Decart AI / WebRTC):** Integrado a nivel de cliente web mediante el SDK oficial `@decartai/sdk`. Se comprobó el protocolo de señalización y apertura de canales de datos/video contra el servidor de LiveKit, verificando la conmutación al componente de respaldo en caso de recibir respuestas de cuota agotada (*insufficient credits*).

5.4. 5.4. Corrección de errores

Durante las fases de auditoría técnica y aseguramiento de calidad, el equipo de ingeniería identificó, documentó y corrigió siete defectos mayores (C1, A1, A2, A3, A4, A5 y A6), restaurando la estabilidad del sistema sin alterar el comportamiento de producción preexistente.



5.4.1. Registro cronológico de incidencias mayores (C1 a A6)

Código	Módulo / Componente Afectado	Descripción del Problema Detectado	Causa Raíz Identificada	Solución Técnica Aplicada	Suite de Regresión Asociada
C1	Backend: Routers POS y Ventas	Falta de control de permisos en endpoints de registro de ventas presenciales.	Dependencias de FastAPI no inyectaban la fábrica de permisos RBAC en las rutas de caja.	Inyección obligatoria de dependencias `verificar_permiso` en `pos.py`.	227 pruebas backend (`autorizacion-c1-a1-regresion.xml`)

Código	Módulo / Componente Afectado	Descripción del Problema Detectado	Causa Raíz Identificada	Solución Técnica Aplicada	Suite de Regresión Asociada
A1	Backend: Compras, Proveedores y Temporadas	Endpoints administrativos permitían escrituras a usuarios con roles operativos estándar.	Omisión de validación de privilegios administrativos de catálogo e inventario.	Aplicación de decoradores y dependencias de autorización granular en `compras.py`, `proveedores.py` y `temporadas.py`.	227 pruebas backend (`autorizacion-c1-a1-regresion.xml`)
A2	Backend: Sincronización Offline (`offline.py`)	Fuga de operaciones y resolución de conflictos entre sucursales de una misma empresa.	Endpoint `/api/sincronizacion/lote` y `/api/conflictos` validaban solo empresa, omitiendo la sucursal asignada.	Incorporación de filtro de sucursal obligatoria y rol supervisor en `synchronization.py`.	254 pruebas backend (`autorizacion-a2-regresion.xml`)
A3	Backend, Web y Móvil: Inventario	Inexistencia del mecanismo de traspaso de inventario de dos patas exigido por el CU10.	El modelo contemplaba únicamente entradas y salidas individuales sin enlace atómico.	Creación de migración SQL 018, lógica transaccional de dos patas en `movimiento_service.py` y modal en Angular.	298 pruebas backend (`autorizacion-a3-regresion.xml`)
A4	Móvil: Android SDK y Almacenamiento SQLite	Bloqueo de compilación Gradle por incompatibilidad de plugins WebView/AGP; riesgo de pérdida de datos v1 al actualizar APK.	Desalineación de Gradle 8 / AGP en Android 16; actualización destructiva de esquemas SQLite.	Ajuste de compatibilidad en `gradle.properties`, pin de dependencias y módulo de recuperación aditiva `database_recovery.dart`.	51 pruebas Flutter y 12 casos manuales Android (`M01`-`M12`)
A5	Web y Móvil: Algoritmo de Sincronización	Timeouts y pérdida de avance al sincronizar colas masivas mayores a 100 operaciones.	Envío monobloque sin paginación; un fallo abortaba toda la transacción local.	Algoritmo de recorrido finito en lotes de hasta 50 operaciones con confirmación local intermedia.	479 casos únicos en laboratorio y 5 casos Android (`A5-AND01`-`05`)
A6	Frontend Web: Suite de Pruebas Angular	30 errores TypeScript `TS2339` bloqueaban la ejecución completa de `ng test`.	Desincronización del archivo de pruebas `probador-ia.spec.ts` respecto al componente migrado a Decart WebRTC.	Reescritura estricta de `probador-ia.spec.ts` adaptada a WebRTC y señales reactivas sin tocar producción.	116 pruebas Angular completas aprobadas en ChromeHeadless

5.4.2. Correcciones de autorización y permisos POS (C1 y A1)

En la línea base heredada, se detectó que los endpoints de registro de operaciones comerciales en mostrador y altas de compras permitían su invocación si el usuario portaba un token JWT válido de la empresa, sin comprobar si disponía explícitamente de los permisos `pos.vender` o `compras.crear`. Se refactorizaron las dependencias de ruta inyectando la factoría de permisos desarrollada para la arquitectura multi-tenant. La verificación de no regresión ejecutó 227 pruebas en `Parcial2_backend/tests/test_authorization_c1_a1.py`, logrando un 100% de aprobación sin alterar contratos existentes.

5.4.3. Corrección de sincronización y aislamiento por sucursal (A2)

El análisis de seguridad del módulo offline reveló que un usuario supervisor de la Sucursal 1 podía listar y resolver conflictos de la Sucursal 2 perteneciente a la misma empresa comercial. Se corrigió la función central en `Parcial2_backend/app/services/synchronization.py` exigiendo que el usuario posea asignación explícita sobre la sucursal de la operación, salvo que ostente el rol de Administrador de Empresa. Se incorporaron 27 casos de prueba específicos (`test_authorization_a2.py`), elevando la suite de regresión a 254 casos aprobados.

5.4.4. Implementación y activación de traspasos de inventario de dos patas (A3)

El requisito RF-10 y el caso de uso CU10 exigían transferencias directas de existencias entre sucursales autorizadas. La solución requirió:

1. Diseñar e implementar la migración de base de datos `018_inventory_transfers.sql`, introduciendo la tabla `traspasos_inventario` y los campos de enlace `id_traspaso` en los movimientos de almacén.

2. Desarrollar la lógica transaccional en `movimiento_service.py`, generando simultáneamente un movimiento de salida en la sucursal origen y un movimiento de entrada en la sucursal destino bajo una misma transacción SQL atómica.

3. Incorporar el modal interactivo de selección de sucursales en Angular (`movimiento-modal.ts` y `.html`).

4. La suite se incrementó con 44 pruebas específicas, alcanzando 298 pruebas backend aprobadas en PostgreSQL 16 y 17, y 196 pruebas verificadas sobre el contenedor Docker local tras aplicar la migración 018.

5.4.5. Recuperación SQLite multicuenta y compilación de APK en Android 16 (A4)

La compilación del aplicativo móvil Android sobre dispositivos modernos (Android 16 / API 36) presentaba fallos en el ensamble de Gradle debido a dependencias incompatibles en los plugins de WebView y permisos. Asimismo, una actualización de la aplicación corría el riesgo de sobreescribir las bases de datos SQLite preexistentes de versiones preliminares.

Se solventó configurando banderas de compatibilidad en `gradle.properties`, fijando `permission_handler: 12.0.3` y creando el módulo `database_recovery.dart`. Este componente inspecciona en modo solo lectura las bases SQLite heredadas (formato v1) y realiza una migración aditiva hacia la nueva base canónica sin alterar los archivos originales. Se generaron los APKs

depurables y se validaron 12 casos manuales (`M01` a `M12`) en el Xiaomi 13T Pro físico, con 8 aprobados y 4 parcialmente verificados en entorno controlado.

5.4.6. Reconciliación en lotes de 50 y avance parcial ante desconexión (A5)

Cuando un dispositivo móvil o web acumulaba más de 100 operaciones en almacenamiento desconectado, el intento de sincronizarlas en una sola petición HTTP provocaba desbordamientos de memoria y fallos de timeout en redes inestables. Además, si la petición fallaba en el último registro, el cliente descartaba todo el lote, impidiendo el progreso.

Se diseñó e implementó un algoritmo de partición por lotes:

- Los clientes recorren secuencialmente una lista finita de UUIDs no contabilizados en bloques de máximo 50 registros (`offline.service.ts` en Angular y `offline_controller.dart` en Flutter).
- Cada lote de 50 se envía al backend; ante una respuesta exitosa, el cliente actualiza localmente el estado a `accounted=1` y renueva el catálogo en una sola transacción local.
- Ante una interrupción abrupta de la conexión (ej. corte en el segundo lote), los lotes ya guardados se conservan intactos y el sistema emite una notificación de progreso parcial ("Reconciliación parcial: 1/3 lotes guardados"). El siguiente intento retoma exactamente desde la operación 51.
- La validación en laboratorio sumó 479 pruebas únicas aprobadas en suites completadas, y la validación física en el Xiaomi 13T Pro aprobó al 100% los 5 casos manuales planificados (`A5-AND01` a `A5-AND05`).

5.4.7. Recuperación de la suite general Angular y eliminación de 30 errores TS (A6)

La ejecución general de pruebas del frontend web (`ng test`) se encontraba totalmente bloqueada debido a 30 errores de compilación TypeScript (`TS2339: Property does not exist on type 'ProbadorIa'`) en el archivo `probador-ia.spec.ts`. Dichos errores provenían de un desfase temporal: las pruebas unitarias estaban redactadas para un prototipo preliminar de captura de fotos estáticas y reservas en tienda, mientras que el componente en producción había evolucionado hacia una integración de baja latencia con Decart AI mediante WebRTC, señales reactivas (`conectado`, `conectando`, `error`) y streams de video continuo.

Sin modificar una sola línea del código de producción (`probador-ia.ts` y `probador-ia.html` se mantuvieron 100% intactos), se reescribió estrictamente el archivo de especificación `probador-ia.spec.ts`:

- Se adaptaron las pruebas a los contratos reactivos reales.
- Se simularon las sesiones de WebRTC y flujos de video mediante objetos nativos `MediaStream`.
- Se verificó la adquisición de tokens y la gestión de excepciones por cuota agotada.

Como resultado, el compilador TypeScript reportó 0 diagnósticos de error en todo el proyecto (`tsc --noEmit`), las 9 pruebas del probador IA aprobaron con éxito, y la **suite completa de Angular alcanzó 116 de 116 pruebas aprobadas al 100%** en ChromeHeadless, habilitando la compilación productiva (`ng build --prod`, bundle 319.65 kB).

5.4.8. Síntesis de no regresión e integridad referencial transversal

La estabilidad de todas las intervenciones se certificó mediante pruebas de no regresión cruzadas:

- La activación de A6 no alteró los 4 casos de prueba del modal de traspasos de A3 (`movimiento-modal.spec.ts`) ni los 22 casos de reconciliación offline de A5 (`offline.service.spec.ts`).
- La integridad de las 7 bases de datos SQLite en el dispositivo físico Xiaomi 13T Pro arrojó 0 errores de claves foráneas tras todas las operaciones.
- La especificación OpenAPI del backend se mantuvo perfectamente sincronizada con 118 rutas y 128 esquemas.

5.5. 5.5. Resultados de las pruebas

5.5.1. Resumen métrico consolidado de ejecución sin doble conteo

Para ofrecer una auditoría matemática rigurosa y transparente, la siguiente tabla desglosa las métricas terminales de cada subsistema tecnológico, identificando el entorno de ejecución, la fecha de verificación y el estado final:

Subsistema / Suite	Entorno de Ejecución	Casos Ejecutados	Aprobados	Fallidos	Omitidos / Bloq.	Referencia de Evidencia	Estado Final
Backend: Suite Completa Unit/Regresión	Docker / PostgreSQL 17 tmpfs	298	298	0	0	`docs/evidencias/a5/backend-regression.xml`	**Aprobado al 100%**
Backend: Verificación HTTP/SQL Específica	Docker / PostgreSQL 17 tmpfs	6	6	0	0	`docs/evidencias/a5/backend-specific.xml`	**Aprobado al 100%**
Backend: Integración Docker Local	Docker Local (`api-1` / PG16)	196	196	0	0	`docs/evidencias/activacion-local-a3-http.xml`	**Aprobado al 100%**
Frontend Web: Suite Completa Angular	Karma / ChromeHeadless 156	116	116	0	0	`docs/evidencias/a6/a6-angular-full-suite.json`	**Aprobado al 100%**
Frontend Web: Integración HTTP Real	Browser / Testbed HTTP	2	2	0	0	`docs/evidencias/a5/angular-http.xml`	**Aprobado al 100%**
Móvil Flutter: Suite Estándar	Flutter Test / Dart VM	70	70	0	0	`docs/evidencias/a5/flutter-regression.xml`	**Aprobado al 100%**

Subsistema / Suite	Entorno de Ejecución	Casos Ejecutados	Aprobados	Fallidos	Omitidos / Bloq.	Referencia de Evidencia	Estado Final
Móvil Flutter: HTTP Real	Flutter Test / SQLite local	1	1	0	0	`docs/evidencias/a5/flutter-http.xml`	**Aprobado al 100%**
Móvil Flutter: Regresión A4	Flutter Test / Fixtures A4	3	3	0	0	`docs/evidencias/a5/flutter-a4-regression.xml`	**Aprobado al 100%**
Validación Android Físico A5	Xiaomi 13T Pro (Android 16)	5	5	0	0	`docs/evidencias/a5/a5-android-summary.json`	**Aprobado al 100%**
Validación Android Físico A4	Xiaomi 13T Pro (Android 16)	12	8	0	4	`docs/evidencias/a4-android-real/`	**8 Aprobados / 4 Parciales**

[!IMPORTANT] **Precisiones críticas sobre el cómputo de métricas:** *1.* **Relación entre las 298 pruebas backend y las 196 comprobaciones HTTP locales:** *Las 196 verificaciones registradas en `activacion-local-a3-http.xml` corresponden a la re-ejecución dirigida de 196 escenarios clave (131 de C1/A1, 27 de A2 y 38 de A3) contra el contenedor activo en Docker local (`fashionstore_examen_si2-api-1` sobre PostgreSQL 16 con la migración 018).* **No deben sumarse a las 298 pruebas unitarias como si fuesen 494 pruebas únicas independientes***, ya que evalúan los mismos contratos de negocio mediante un arnés HTTP real de integración. La cifra terminal pura de la lógica backend es de* **298 pruebas***. 2.* **Composición de las 116 pruebas de Angular:** *Las 116 pruebas de la suite de Angular representan el total global del frontend e* **incluyen** *las 9 pruebas del probador IA (`probador-ia.spec.ts`), las 22 de reconciliación en lotes (`offline.service.spec.ts`) y las 4 del modal de traspasos (`movimiento-modal.spec.ts`). No deben sumarse sus partes a la cifra consolidada. 3.* **Consolidación en Android físico (17 casos totales):** *La suma de casos manuales documentados en el Xiaomi 13T Pro es de* **17 casos** *(12 de A4 y 5 de A5). De ellos,* **13 casos están aprobados al 100%** *(los 5 de A5 y 8 de A4) y* **4 casos se mantienen en estado parcialmente verificado** *(pertenecientes a A4).*

5.5.2. Desglose de la suite completa Angular (116/116 pruebas aprobadas)

La ejecución desatendida sobre Karma y Google Chrome Headless 156.0.0.0 certificó la aprobación unánime de los 16 archivos de especificación que conforman el cliente web:

Archivo de Especificación (`.spec.ts`)	Dominio Funcional Evaluado	Casos Aprobados	Estado de la Suite
`app.spec.ts`	Inicialización de la aplicación raíz y directiva router-outlet	1	Aprobado

Archivo de Especificación (`.spec.ts`)	Dominio Funcional Evaluado	Casos Aprobados	Estado de la Suite
`auth.service.spec.ts`	Gestión de tokens JWT, almacenamiento seguro y ciclo de autenticación	7	Aprobado
`compra.service.spec.ts`	Registro y recepción de compras a proveedores de moda	4	Aprobado
`ia.service.spec.ts`	Recomendaciones de moda con Gemini, fallback y analítica por voz	11	Aprobado
`inventario.service.spec.ts`	Consulta de existencias por sucursal y filtros por talla/color	7	Aprobado
`movimiento-inventario.service.spec.ts`	Entradas, salidas y ajustes de stock en almacén	6	Aprobado
`offline.service.spec.ts`	**A5:** Reconciliación en lotes de 50, IndexedDB Dexie y avance parcial	22	Aprobado
`permiso.service.spec.ts`	Validación de permisos por rol en navegación y directivas	4	Aprobado
`proveedor.service.spec.ts`	Directorio y mantenimiento de proveedores	5	Aprobado
`temporada.service.spec.ts`	Gestión de temporadas comerciales (Primavera, Otoño, etc.)	4	Aprobado
`roles-permisos.spec.ts`	Interfaz reactiva de administración de permisos y asignaciones	12	Aprobado
`login.spec.ts`	Validación de formularios reactivos y control de errores en login	12	Aprobado
`recuperar-password.spec.ts`	Solicitud y validación de enlace de recuperación de contraseña	4	Aprobado
`restablecer-password.spec.ts`	Definición de nueva contraseña con token criptográfico temporal	4	Aprobado
`probador-ia.spec.ts`	**A6:** Probador virtual IA con streaming WebRTC y Decart AI	9	Aprobado

Archivo de Especificación (`.spec.ts`)	Dominio Funcional Evaluado	Casos Aprobados	Estado de la Suite
`movimiento-modal.spec.ts`	**A3:** Modal interactivo de traspasos entre sucursales de dos patas	4	Aprobado
Total General Suite Angular	**16 archivos de especificación unitaria e integración**	**116**	**100% Aprobado**

5.5.3. Desglose de la suite completa Backend (298/298 pruebas aprobadas)

La suite de regresión integral del backend en Pytest 8.4.2 validó los módulos del núcleo transaccional en PostgreSQL:

- **Identidad Global y Multitenencia (96 casos):** Registro de clientes, logins diferenciados, aislamiento por subdominio y RLS en base de datos (`test_global_identity.py`, `test_saas.py`, `test_branch_scope.py`).
- **Autorización RBAC y Seguridad POS (131 casos):** Permisos en operaciones de venta presencial, compras, temporadas y catálogo (`test_authorization_c1_a1.py`).
- **Aislamiento y Autorización por Sucursal (27 casos):** Permisos de supervisión y resolución de conflictos por sede (`test_authorization_a2.py`).
- **Traspasos de Inventario de Dos Patas (44 casos):** Modelo de transferencias simultáneas con migración 018 (`test_inventory_transfers.py`).

5.5.4. Desglose de la suite completa Móvil Flutter (70/70 pruebas aprobadas)

Ejecutada sobre el motor Dart VM:

- **Modelos y Persistencia Base (48 casos):** Serialización de entidades de inventario, parsing de respuestas OpenAPI y controladores de estado.
- **Reconciliación y Recuperación Offline (22 casos):** Implementación en `offline_reconciliation_test.dart`, simulando la partición en lotes de 50 ítems, la inmutabilidad de la cola ante errores y el cálculo de disponibilidad local descontando conflictos retenidos.

5.5.5. Resultados de pruebas de integración HTTP en Docker local (196/196 aprobadas)

Para certificar la estabilidad de la compilación en contenedores activos, se ejecutó una batería de 196 pruebas de integración HTTP reales contra el contenedor `fashionstore_examen_si2-api-1` sobre el puerto 8002:

- 131 casos de verificación de autorización en routers de compras y POS (C1/A1).
- 27 casos de autorización de sincronización por sucursal (A2).
- 38 casos de traspasos de inventario entre sucursales comunicándose directamente con PostgreSQL 16 con la migración 018 aplicada (`activacion-local-a3-http.xml`).
- Resultado: 196 peticiones aprobadas con código HTTP esperado (200, 201, 403, 404, 409 según el caso de prueba), confirmando la integridad del ensamblado Docker local.

5.5.6. Resultados de compilación y empaquetado productivo (Web y APK)

Artefacto Compilado	Entorno / Comando	Tamaño Real en Disco	Tiempo de Build	Diagnóstico / Estado
Bundle Web Producción	Angular CLI (`ng build --prod`)	319.65 kB (bundle inicial)	9.2 segundos	Código de salida 0. Advertencia menor preexistente sobre módulo CommonJS (`p-retry`), sin impacto en ejecución.
APK Móvil A5 (Local Debug)	Flutter (`flutter build apk --debug`)	171,732,852 bytes (~163.78 MB)	45.1 segundos	Código de salida 0. Firma criptográfica válida SHA-256 (`61ff29b1...`). Instalación y ejecución aprobadas en Android 16.
APK Móvil A4 (Local Debug)	Flutter (`flutter build apk --debug`)	197,649,091 bytes (~188.49 MB)	48.3 segundos	Código de salida 0. Firma criptográfica compatible.

5.5.7. Limitaciones técnicas reconocidas y estado de aceptación

En estricta transparencia técnica y auditoría académica, se dejan sentadas las siguientes limitaciones del alcance probado:

1. Probador Virtual IA en Tiempo Real (CU09): La integración WebRTC con Decart AI se encuentra aprobada y verificada mediante 9 pruebas unitarias automatizadas con mocks y simulación de flujos de señalización. La realización de una prueba funcional interactiva de extremo a extremo con video de cámara real frente a la nube de Decart AI dependerá de contar con una clave de API activa y saldo de créditos comercial vigente durante la sesión de defensa en vivo.

2. Pasarela de Pagos Stripe (CU11): La recepción e idempotencia de pagos en línea se validó formalmente mediante simulación de webhooks firmados criptográficamente con HMAC SHA-256. No se ejecutó una transacción bancaria en vivo con tarjeta de crédito física a través del túnel Cloudflare para evitar cobros de comisiones o bloqueos de pasarela.

3. Casos Manuales A4 en Android (`M01`, `M03`, `M06`, `M12`): Cuatro de los doce casos de persistencia de A4 se mantienen en estado "Parcialmente Verificados" debido a que se ensayaron mediante métodos de prueba internos del APK debug y desconexión de túnel USB (sin activar el modo avión del hardware). (Se destaca que los 5 casos físicos de la fase posterior A5 sí fueron aprobados al 100%).

4. Entrenamiento de Machine Learning en Producción (CU19): Por directriz de arquitectura, el entrenamiento de los modelos de clasificación y agrupamiento se efectúa localmente; el servidor de producción Debian alberga exclusivamente los artefactos serializados (`.joblib`) para inferencia, debido a la restricción estricta de 512 MB de memoria RAM asignada al proceso API.

5. Servidor SMTP Externo (CU01): El flujo de correo saliente para recuperación de credenciales se comprobó contra el servidor de pruebas Mailpit local (puerto 8025). No se ejecutó reenvío a través de un relay SMTP comercial externo.

5.6. 5.6. Casos de prueba

5.6.1. Inventario formal de Casos de Prueba del Sistema (CP-CU01 a CP-CU20)

La siguiente tabla resume los casos de prueba de aceptación derivados para los 20 Casos de Uso del sistema:

Código de Prueba	Caso de Uso	Requisito	Tipo de Prueba	Componente Evaluado	Estado
CP-CU01-01	CU01 - Gestionar acceso y cuenta	RF-01	Funcional / Integración	`auth.service.ts` / Router `/api/login/`	**Aprobado**
CP-CU02-01	CU02 - Administrar empresas y planes	RF-02	Integración / Seguridad	Router `/api/empresas/`, RLS	**Aprobado**
CP-CU03-01	CU03 - Administrar personal y permisos	RF-03	Seguridad / RBAC	Router `/api/roles/`, Dependencia Auth	**Aprobado**
CP-CU04-01	CU04 - Gestionar ciudades y sucursales	RF-04	Funcional / Integración	Router `/api/sucursales/`, Límite Plan	**Aprobado**
CP-CU05-01	CU05 - Gestionar fichas de clientes	RF-05	Funcional	Router `/api/clientes/`, Aislamiento	**Aprobado**
CP-CU06-01	CU06 - Administrar catálogo de prendas	RF-06	Funcional	Router `/api/catalogo/`, Variantes	**Aprobado**
CP-CU07-01	CU07 - Gestionar compras y recepciones	RF-07	Integración / BD	Router `/api/compras/`, Trigger Stock	**Aprobado**
CP-CU08-01	CU08 - Consultar inventario y disponibilidad	RF-08	Funcional / Lógica	Router `/api/inventario/`, Retenciones	**Aprobado**
CP-CU09-01	CU09 - Asistencia y probador virtual IA	RF-09	Unitaria / Integración	`probador-ia.ts`, Decart SDK WebRTC	**Aprobado**
CP-CU10-01	CU10 - Registrar movimientos de inventario	RF-10	Transaccional / BD	`movimiento_service.py`, Migración 018	**Aprobado**
CP-CU11-01	CU11 - Compra y pago en línea	RF-11	Integración / Webhook	`test_payment_lifecycle.py`, Stripe	**Aprobado**
CP-CU12-01	CU12 - Gestionar reservas de prendas	RF-12	Funcional / Tiempo	Router `/api/reservas/`, Expiración 24h	**Aprobado**

Código de Prueba	Caso de Uso	Requisito	Tipo de Prueba	Componente Evaluado	Estado
CP-CU13-01	CU13 - Registrar venta presencial	RF-13	Transaccional	Router `/api/pos/`, Cálculo Vuelto	**Aprobado**
CP-CU14-01	CU14 - Obtener comprobantes de compra	RF-14	Seguridad / Privacidad	Router `/api/comprobantes/`, PDF	**Aprobado**
CP-CU15-01	CU15 - Gestionar cambios y reembolsos	RF-15	Lógica de Negocio	Router `/api/cambios/`, 7 días plazo	**Aprobado**
CP-CU16-01	CU16 - Registrar operaciones offline	RF-16	Persistencia Local	Dexie IndexedDB (Web) / SQLite (Móvil)	**Aprobado**
CP-CU17-01	CU17 - Sincronizar y resolver conflictos	RF-17	Resiliencia / Algoritmo	Lotes de 50, Idempotencia UUID v4	**Aprobado**
CP-CU18-01	CU18 - Obtener reportes comerciales	RF-18	Funcional / Exportación	Generación PDF/XLSX, Gemini Prompt	**Aprobado**
CP-CU19-01	CU19 - Preparar y consultar analítica	RF-19	Machine Learning	Modelos Scikit-Learn `.joblib`	**Aprobado**
CP-CU20-01	CU20 - Consultar auditoría y trazabilidad	RF-20	Auditoría / Logs	Registro de eventos inmutables	**Aprobado**

5.6.2. Fichas técnicas de pruebas automatizadas críticas

```
================================================================================
FICHA DE PRUEBA TÉCNICA: CP-CU10-01 (Traspaso de Inventario de Dos Patas)
================================================================================
Caso de Uso: CU10 - Registrar movimientos de inventario.
Requisito Relacionado: RF-10, RNF-04 (Integridad Transaccional), RN-09, RN-14.
Objetivo: Verificar que una transferencia entre sucursales descuente el stock en origen
          e incremente el stock en destino de forma estrictamente atómica en PostgreSQL.
Precondiciones: Empresa activa, variante de prenda con 10 unidades en Sucursal 1,
                Sucursal 2 asignada a la misma empresa con 0 unidades.
Datos de Entrada: id_sucursal_origen=1, id_sucursal_destino=2, cantidad=4,
                  operation_id="e9a1b2c3-4d5e-6f7a-8b9c-0d1e2f3a4b5c".
Pasos de Ejecución:
  1. Invocar POST /api/movimientos-inventario/traspaso con el payload especificado.
  2. Verificar la confirmación HTTP 201 y la estructura de TraspasoRespuesta.
  3. Consultar la existencia física y disponible en la Sucursal 1.
  4. Consultar la existencia física y disponible en la Sucursal 2.
  5. Reintentar la misma petición con idéntico operation_id (prueba de idempotencia).
Resultado Esperado:
  - Movimiento de Salida generado en Sucursal 1 (Stock resultante: 6).
  - Movimiento de Entrada generado en Sucursal 2 (Stock resultante: 4).
  - Ambas operaciones vinculadas mediante id_traspaso en la migración 018.
  - Reintento devuelve HTTP 200 con el payload original sin volver a mover existencias.
Resultado Obtenido: Conforme en PostgreSQL 16 y 17 (44 pruebas específicas aprobadas).
Estado: APROBADO.
Evidencia: docs/evidencias/autorizacion-a3-especificas.xml
```

```
================================================================================
```

```
================================================================================
FICHA DE PRUEBA TÉCNICA: CP-CU17-01 (Reconciliación Offline en Lotes de 50)
================================================================================
Caso de Uso: CU17 - Sincronizar operaciones y resolver conflictos.
Requisito Relacionado: RF-17, RNF-05 (Idempotencia), RNF-06 (Recuperación), RN-16, RN-17.
Objetivo: Comprobar que una cola offline de 125 transacciones se procese en lotes
          de 50/50/25, tolerando interrupciones sin pérdida de lotes previos.
Precondiciones: Base local con 125 operaciones sin contabilizar (accounted=0),
                servidor efímero disponible en puerto 18085 vía túnel inverso ADB.
Datos de Entrada: 125 transacciones (75 ventas válidas, 25 rechazos, 25 conflictos).
Pasos de Ejecución:
  1. Iniciar sincronización desde el cliente móvil o web.
  2. Despachar Lote 1 (50 transacciones) -> Confirmar respuesta y guardar en SQLite.
  3. Despachar Lote 2 (50 transacciones) -> Simular corte abrupto de red al 50%.
  4. Verificar estado local en SQLite: 50 transacciones marcadas accounted=1.
  5. Restablecer red y reanudar sincronización: procesar lote restante de 75 ops.
Resultado Esperado:
  - Lote 1 conservado íntegramente tras el corte.
  - Reintento procesa únicamente los 75 pendientes sin reenviar los primeros 50.
  - Cero duplicaciones en stock central; retención de los 25 conflictos en local.
Resultado Obtenido: Conforme en Xiaomi 13T Pro físico (casos A5-AND02 y A5-AND03).
Estado: APROBADO.
Evidencia: docs/evidencias/a5/a5-and02-reconciliation-125.json
================================================================================
```

5.6.3. Casos de prueba manuales ejecutados en dispositivo físico Android (A5-AND01 a A5-AND05)

Durante la fase A5, se ejecutó un protocolo riguroso de pruebas manuales sobre el dispositivo físico Xiaomi 13T Pro (Android 16 / API 36), aprobando el 100% de los casos previstos:

Código de Caso	Nombre del Escenario Manual	Procedimiento y Parámetros	Comportamiento Observado en el Xiaomi 13T Pro	Estado Final
A5-AND01	Actualización de APK sin pérdida de datos históricos	Instalación del paquete A5 (`FashionStore-P2-A5-local-debug.apk`) mediante comando `adb install -r` sobre la instalación previa de A4.	Éxito (`Success`). Se preservaron intactas las 6 bases de datos SQLite preexistentes y las 5 transacciones históricas de A4 con integridad `ok`.	**APROBADO**

Código de Caso	Nombre del Escenario Manual	Procedimiento y Parámetros	Comportamiento Observado en el Xiaomi 13T Pro	Estado Final
A5-AND02	Reconciliación masiva de 125 transacciones heterogéneas en lotes 50/50/25	Inyección de 125 operaciones heterogéneas en cola local (`2:1`: 75 ventas válidas, 25 rechazos provocados y 25 conflictos de stock/precio); ejecución de sincronización hacia el backend efímero (puerto 18085).	Procesamiento ordenado en 3 lotes (50, 50, 25). 75 ventas confirmadas (`accounted=1`), 25 rechazos y 25 conflictos retenidos (`accounted=0`). Stock central 925, disponible local 900.	**APROBADO**
A5-AND03	Interrupción forzada en segundo lote y avance parcial sobre 125 ventas válidas	Inyección de 125 ventas válidas homogéneas; corte de túnel USB durante el Lote 2; posterior reconexión y reanudación.	El sistema conservó los primeros 50 ítems guardados (`accounted=1`) emitiendo aviso de progreso parcial (1/3). La reanudación procesó exclusivamente los 75 pendientes, totalizando 125 ventas contabilizadas sin duplicados.	**APROBADO**
A5-AND04	Cierre forzado del proceso (`force-stop`) y reapertura	Forzado de detención mediante `am force-stop com.fashionstore...`, reapertura mediante `am start` y captura de pantalla.	La aplicación abrió correctamente (`pid 31790`), reconociendo las 7 bases de datos íntegras, reteniendo las 127 operaciones y 75 ventas contabilizadas en SQLite.	**APROBADO**
A5-AND05	Aislamiento multi-tenant entre empresas y sucursales	Inicio de sesión con Empresa B (`7:6`) y consulta de historial; emisión de petición HTTP cruzada buscando UUID de Empresa A.	La base local de Empresa B mostró 0 operaciones (aislamiento limpio). La consulta cruzada contra la API devolvió código HTTP 404 Not Found, denegando el acceso.	**APROBADO**

[!NOTE] **Consistencia técnica entre A5-AND02 y A5-AND03:** *La diferencia aparente en el total de operaciones con `accounted=1` (75 en A5-AND02 frente a 125 en A5-AND03) radica en la*

composición deliberada de los conjuntos de datos de prueba: - En **A5-AND02***, el dataset fue* **heterogéneo** *(75 válidas, 25 rechazadas y 25 en conflicto). Por diseño de la regla RN-17, las transacciones en conflicto o rechazadas *no deben marcarse contabilizadas* para retener la reserva local en el dispositivo del vendedor, concluyendo con exactamente 75 ventas confirmadas. - En* **A5-AND03***, el dataset fue* **homogéneo** *(125 ventas válidas), diseñado para evaluar que el reintento tras corte de red procese exactamente los 75 ítems restantes sin volver a enviar los primeros 50 ya guardados, concluyendo con las 125 ventas confirmadas.*

5.6.4. Casos de prueba de persistencia y migración en dispositivo físico (M01 a M12 de A4)

Resumen de los doce casos de prueba ejecutados durante la fase A4 en el Xiaomi 13T Pro:

• Casos Aprobados (8 casos):

- `M02`: Autenticación con Empresa A y descarga de catálogo desde API tmpfs.

- `M04`: Cierre forzado y reapertura conservando operaciones provisionales en SQLite.

- `M05`: Reinicio físico de hardware del teléfono Android y desbloqueo por usuario con persistencia íntegra de la sesión y cola offline.

- `M07`: Conmutación de cuentas por interfaz entre Empresa A y Empresa B con aislamiento total de historiales.

- `M08`: Manejo de operaciones pendientes en distintas sucursales preservando stock local diferenciado.

- `M09`: Restablecimiento de conexión y sincronización de 5 transacciones UUID.

- `M10`: Reenvío reiterado de peticiones idempotentes y rechazo de cargas alteradas con el mismo UUID.

- `M11`: Manejo de concesión y denegación de permisos de hardware (cámara) en Android sin colapsos de la aplicación.

• Casos Parcialmente Verificados (4 casos):

- `M01` (Parcial): La actualización mediante `adb install -r` fue validada con el mismo certificado de firma, pero no se disponía de un APK previo instalado desde tienda de producción comercial.

- `M03` (Parcial): La pérdida de conectividad se simuló retirando el reenvío de puertos ADB USB en vez de activar el modo avión del hardware del dispositivo.

- `M06` (Parcial): Se certificó la integridad física de los archivos SQLite, pero el flujo visual completo de interfaz para selección de archivos requirió asistencia de laboratorio.

- `M12` (Parcial): La cadena de recuperación aditiva multicuenta se ejecutó mediante métodos internos de prueba del APK local debug sobre fixtures sintéticos, no mediante navegación manual convencional.

5.6.5. Matriz de trazabilidad integral: Requisito → Caso de Uso → Componente → Prueba → Evidencia → Resultado

Requisito	Caso de Uso ICONIX	Componente de Software	Prueba Automatizada / Manual	Archivo de Evidencia Verificable	Resultado Certificado
RF-01	CU01 - Gestionar acceso y cuenta	Backend: `login.py` / Web: `auth.service.ts`	`test_global_identity.py` (96 tests) / `auth.spec.ts`	`docs/evidencias/backend-tests-global-identity.xml`	**Aprobado**
RF-02	CU02 - Administrar empresas y planes	Backend: `saas.py` / PostgreSQL RLS	`test_saas.py` (12 tests)	`docs/evidencias/autorizacion-c1-a1-regresion.xml`	**Aprobado**
RF-03	CU03 - Personal, roles y permisos	Backend: `roles.py` / Web: `roles-permisos.ts`	`test_authorization_c1_a1.py` (131 tests)	`docs/evidencias/autorizacion-c1-a1-especificas.xml`	**Aprobado**
RF-04	CU04 - Ciudades y sucursales	Backend: `sucursales.py`	`test_branch_scope.py` (8 tests)	`docs/evidencias/autorizacion-a2-regresion.xml`	**Aprobado**
RF-05	CU05 - Fichas de clientes	Backend: `clientes.py`	`test_commerce.py` (casos cliente)	`docs/evidencias/backend-regression.xml`	**Aprobado**
RF-06	CU06 - Catálogo de prendas	Backend: `catalogo.py` / Web: `temporada.service`	`test_catalogue_p2.py` / `temporada.spec.ts`	`docs/evidencias/a6/a6-angular-full-suite.json`	**Aprobado**
RF-07	CU07 - Proveedores y compras	Backend: `compras.py` / Web: `compra.service`	`compra.service.spec.ts` / `proveedor.spec.ts`	`docs/evidencias/a6/a6-angular-full-suite.json`	**Aprobado**
RF-08	CU08 - Inventario y disponibilidad	Backend: `movimientos.py` / Web: `inventario`	`inventario.service.spec.ts` (7 tests)	`docs/evidencias/a6/a6-angular-full-suite.json`	**Aprobado**
RF-09	CU09 - Asistencia y probador IA	Web: `probador-ia.ts` / Decart WebRTC SDK	`probador-ia.spec.ts` (9 tests unitarios)	`docs/evidencias/a6/a6-probador-ia-specific.json`	**Aprobado**
RF-10	CU10 - Movimientos e inventario	Backend: `movimiento_service.py` / Migración 018	`test_inventory_transfers.py` (44 tests A3)	`docs/evidencias/autorizacion-a3-especificas.xml`	**Aprobado**
RF-11	CU11 - Compra y pago en línea	Backend: `webhook.py` / Web: Checkout	`test_payment_lifecycle.py` / `test_webhook...`	`docs/evidencias/backend-regression.xml`	**Aprobado**
RF-12	CU12 - Reservas de prendas	Backend: `commerce.py` / Web: `inventario`	`test_commerce.py` (retenciones 24h)	`docs/evidencias/backend-regression.xml`	**Aprobado**

Requisito	Caso de Uso ICONIX	Componente de Software	Prueba Automatizada / Manual	Archivo de Evidencia Verificable	Resultado Certificado
RF-13	CU13 - Venta presencial	Backend: `pos.py` / Flutter: POS screen	`test_commerce.py` / `test_authorization_c1...`	`docs/evidencias/autorizacion-c1-a1-especificas.xml`	**Aprobado**
RF-14	CU14 - Comprobantes de compra	Backend: `receipt_privacy.py`	`test_receipt_privacy.py` (4 tests)	`docs/evidencias/backend-regression.xml`	**Aprobado**
RF-15	CU15 - Cambios y reembolsos	Backend: `exchanges.py`	`test_exchanges.py` (6 tests)	`docs/evidencias/backend-regression.xml`	**Aprobado**
RF-16	CU16 - Persistencia offline	Web: Dexie v2 / Móvil: `sqflite` (Xiaomi)	`offline.service.spec.ts` (22) / `A5-AND04`	`docs/evidencias/a5/a5-and04-reopen.json`	**Aprobado**
RF-17	CU17 - Reconciliación de lotes	Backend: `offline.py` / Flutter: Controller	`A5-AND02` (125 ops) / `A5-AND03` (corte)	`docs/evidencias/a5/a5-and02-reconciliation-125.json`	**Aprobado**
RF-18	CU18 - Reportes y generativa IA	Backend: `reports.py` / Exportación XLSX/PDF	`test_reports.py` (6 tests) / `ia.service.spec`	`docs/evidencias/backend-regression.xml`	**Aprobado**
RF-19	CU19 - Analítica y Machine Learning	Backend: `analytics.py` / Scikit-Learn	`test_analytics.py` (4 tests)	`docs/evidencias/backend-regression.xml`	**Aprobado**
RF-20	CU20 - Auditoría y trazabilidad	Backend: Middleware Auditoría / PostgreSQL	`test_native_role.py` / Trazas de sistema	`docs/evidencias/native-infrastructure-tests.xml`	**Aprobado**

6. CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

6.2. Recomendaciones

7. BIBLIOGRAFIA

8. ANEXOS

8.1. Enlace y QR de Github

Fashionstore - https://github.com/AleDevCV/FashionStore_V2-SaaS.git



8.2. Enlace y QR de la Página

https://fashionstore.aledevcv.me/login

